

Mariusz Duka

php8 i SQL

Programowanie dla początkujących
w 43 lekcjach



Helion 

Wszelkie prawa zastrzeżone. Nieautoryzowane rozpowszechnianie całości lub fragmentu niniejszej publikacji w jakiejkolwiek postaci jest zabronione. Wykonywanie kopii metodą kserograficzną, fotograficzną, a także kopiowanie książki na nośniku filmowym, magnetycznym lub innym powoduje naruszenie praw autorskich niniejszej publikacji.

Wszystkie znaki występujące w tekście są zastrzeżonymi znakami firmowymi bądź towarowymi ich właścicieli.

Autor oraz wydawca dołożyli wszelkich starań, by zawarte w tej książce informacje były kompletne i rzetelne. Nie biorą jednak żadnej odpowiedzialności ani za ich wykorzystanie, ani za związane z tym ewentualne naruszenie praw patentowych lub autorskich. Autor oraz wydawca nie ponoszą również żadnej odpowiedzialności za ewentualne szkody wynikłe z wykorzystania informacji zawartych w książce.

Redaktor prowadzący: Małgorzata Kulik

Projekt okładki: Studio Gravite / Olsztyn

Obarek, Pokoński, Pazdrijowski, Zaprucki

Grafika na okładce została wykorzystana za zgodą Shutterstock.com

Helion S.A.

ul. Kościuszki 1c, 44-100 Gliwice

tel. 32 231 22 19, 32 230 98 63

e-mail: helion@helion.pl

WWW: <http://helion.pl> (księgarnia internetowa, katalog książek)

Drogi Czytelniku!

Jeżeli chcesz ocenić tę książkę, zajrzyj pod adres

http://helion.pl/user/opinie/ph8sql_ebook

Możesz tam wpisać swoje uwagi, spostrzeżenia, recenzję.

Kody źródłowe wybranych przykładów dostępne są pod adresem:

<ftp://ftp.helion.pl/przyklady/ph8sql.zip>

ISBN: 978-83-283-8848-2

Copyright © Helion S.A. 2021

- [Poleć książkę na Facebook.com](#)
- [Kup w wersji papierowej](#)
- [Oceń książkę](#)

- [Księgarnia internetowa](#)
- [Lubię to! » Nasza społeczność](#)

*Dla mojej cudownej żony Indry,
miłości mojego życia*

Spis treści

	Przedmowa	7
LEKCJA 01	Co nowego w PHP 8	14
LEKCJA 02	Konfiguracja	24
LEKCJA 03	Klasyka, czyli pierwszy program	40
LEKCJA 04	Komentarz mile widziany	46
LEKCJA 05	Co oferują PHP i MySQL	49
LEKCJA 06	PHP Composer	58
LEKCJA 07	Zmienne i stałe	66
LEKCJA 08	Łącuchy znaków	71
LEKCJA 09	Co, jeśli? Konstrukcje warunkowe	76
LEKCJA 10	Co, jeśli? Inaczej też się da	82
LEKCJA 11	Co, jeśli? Trochę logiki	87
LEKCJA 12	Znajdź coś w tekście	94
LEKCJA 13	Kombinacje z tekstem	101
LEKCJA 14	Operacje na liczbach	105
LEKCJA 15	Trochę matmy dla opornych	112
LEKCJA 16	Wiele matmy dla ambitnych	116
LEKCJA 17	Nareszcie własna funkcja	122
LEKCJA 18	Funkcje mogą coś zwracać	127
LEKCJA 19	Funkcja a zasięg zmiennych	132
LEKCJA 20	Pętla, czyli cykliczne powtarzanie instrukcji	135
LEKCJA 21	Pętla w akcji	139
LEKCJA 22	Tablice i klucze	143
LEKCJA 23	Wymiary tablic	150
LEKCJA 24	Odczyt i zapis plików	159

LEKCJA 25	Sytuacje wyjątkowe	170
LEKCJA 26	Dołącz kod źródłowy z pliku	177
LEKCJA 27	Data i czas	182
LEKCJA 28	Zrób coś z klasą	190
LEKCJA 29	Jest czy ma? Oto jest pytanie	198
LEKCJA 30	Trochę więcej o klasach	205
LEKCJA 31	Pierwsza baza danych	214
LEKCJA 32	Proste tabele z danymi	228
LEKCJA 33	Bezpieczeństwo w SQL-u	242
LEKCJA 34	Arytmetyka w SQL-u	252
LEKCJA 35	Rozmówki klienta z serwerem	260
LEKCJA 36	Serwer webowy w PHP	268
LEKCJA 37	HTML dla zielonych	275
LEKCJA 38	Czas na pierwszy projekt	285
LEKCJA 39	Dwa światy, logika i widok	290
LEKCJA 40	Uruchomienie pierwszej strony	298
LEKCJA 41	Kontakt mile widziany	315
LEKCJA 42	A może trochę rozrywki?	323
LEKCJA 43	Trochę porad na zakończenie	332
	Skorowidz	335

Przedmowa

Minął już ponad rok od wydania przez wydawnictwo Helion mojej pierwszej książki na temat podstaw programowania w języku PHP 7. W międzyczasie twórcy języka udostępnili wersję PHP 7.4 oraz od dawna oczekiwaną wersję PHP 8. Dla wielu programistów na co dzień wykorzystujących język PHP nowe funkcjonalności powinny ułatwić precyzyjne pisanie kodu przy jednoczesnym zwiększeniu jego wydajności. Wiele emocji wzbudza nowy mechanizm JIT (ang. *Just In Time*), który służy do kompilowania programu do postaci kodu maszynowego przed jego wykonaniem. Zgodnie z zapowiedziami twórców języka mechanizm JIT ma zapewnić znaczne przyspieszenie wykonywania programu w porównaniu do tradycyjnej metody, czyli programu wykonywanego przez interpreter. PHP 8 to kolejna duża aktualizacja języka, podobnie jak w przypadku wersji 7, dlatego jest to dobry moment na jego naukę i wykorzystanie w nowych projektach.

Niniejsza książka, podobnie jak poprzednia¹, ma w prosty sposób nauczyć Cię programować w języku PHP, który jest od wielu lat wykorzystywany do projektowania stron internetowych oraz budowy skomplikowanych aplikacji działających po stronie serwera. O potężnych możliwościach PHP niech świadczy to, że jest on używany przez takich gigantów, jak Google, Facebook, Yahoo! czy Wikipedia. Warto dodać, że zgodnie ze statystykami serwisu W3Techs² prawie 80% stron internetowych na świecie działa na serwerach obsługujących język PHP. I choć nie wszystkie serwery udostępniają taką informację, to i tak oznacza to, że 8 na 10 stron internetowych wykorzystuje język PHP. Jestem przekonany, że tak wysoki poziom będzie się długo utrzymywał, zwłaszcza że twórcy języka PHP cały czas pracują nad jego rozwojem.

Od ponad 20 lat PHP dynamicznie się rozwija — powstają kolejne, wydajniejsze wersje, wzbogacone o nowe funkcje. Pod koniec lat 90. ubiegłego wieku programiści do budowy serwisów internetowych często stosowali język Perl, czyli skrypty CGI. Korzystano też z technologii SSI (ang. *Server Side Includes*), a więc mechanizmu skryptowego służącego do dynamicznego generowania stron WWW. Rewolucja nastąpiła, kiedy został udostępniony i spopularyzował się język PHP 3. I tak to się zaczęło.

¹ M. Duka, *PHP7 i SQL. Programowanie dla początkujących w 40 lekcjach*, Helion, Gliwice 2020.

² W3Techs, <https://w3techs.com/technologies/details/pl-php> [dostęp: 26.05.2021].

PHP/FI, PHP 3, PHP 4, PHP 5, PHP 7, PHP 8 to historia rozwoju wspaniałego języka programowania, będąca wynikiem wyłożonej pracy wielu programistów z całego świata. PHP nadaje się nie tylko do programowania stron internetowych, ale również do budowy skomplikowanych aplikacji użytkowych, opartych na mnogości dostępnych rozszerzeń i modułów. Już od wersji PHP 7.4 istotnie poprawiono wydajność przetwarzania kodu przy jednoczesnym zmniejszeniu zapotrzebowania zasobów systemu. PHP 8 z mechanizmem JIT jest po prostu o wiele szybszy od swoich poprzedników, co ma znaczenie choćby dla firm hostingowych, które mogą oferować usługi na jeszcze wyższym poziomie. PHP może być wykorzystywany przez programistów do ogromnej liczby zastosowań, a dostępność wielu dodatkowych bibliotek liczbę tę znacznie zwiększa.

Począwszy od wersji PHP 7.2, dostępne są dla programistów wieloplatformowa biblioteka Sodium oraz nowy algorytm haszujący Argon2, co sprawia, że język PHP stał się bezpiecznym środowiskiem implementującym najnowsze rozwiązania kryptografii. PHP udostępnia bibliotekę OpenSSL, którą programista może wykorzystać do budowy bezpiecznego systemu opartego na kryptografii krzywych eliptycznych. Fascynacja możliwościami, jakie oferuje język PHP, skłoniła mnie do napisania publikacji naukowej³, w której przedstawiłem wyniki badań związanych ze wskazaniem optymalnej krzywej eliptycznej do budowy kryptosystemu.

Adaptacja nowej wersji języka w już istniejących projektach zajmie programistom dużo czasu, natomiast nowe projekty wykorzystujące moc PHP 8 dopiero są tworzone, dlatego jest to dobry moment dla Ciebie na rozpoczęcie nauki programowania w PHP. Wiedzę wyniesioną z lektury tej książki będziesz mógł przez następne lata rozwijać i wykorzystywać w wielu swoich projektach.

Książka ta ma Ci uświadomić, że nauka programowania wcale nie jest taka trudna, wymaga jedynie chęci, cierpliwości, determinacji i czasu. Przerobienie wszystkich lekcji będzie dobrą podstawą do tego, abyś w przyszłości mógł sięgnąć po bardziej zaawansowane tematycznie książki, dotyczące nie tylko PHP, lecz również języków SQL, HTML, CSS i JavaScript.

³ M. Duka, *Elliptic-curve cryptography (ECC) and Argon2 algorithm in PHP using OpenSSL and Sodium libraries*, „IAPGOŚ” 2020, 10 (3), s. 91 – 94, <https://doi.org/10.35784/iapgos.897>.

Zawartość książki

Książkę podzieliłem na praktyczne lekcje i uzupełniłem o wskazówki, ćwiczenia do samodzielnego wykonania i odpowiedzi na pytania, które mogą się pojawić. Kod zamieszczony w książce oznaczyłem dodatkową informacją o wersji PHP, która jest wymagana do prawidłowego działania.

Informacje, na które powinieneś zwrócić uwagę, oznaczyłem takim stylem.

W pierwszej kolejności przygotujesz środowisko programistyczne, w którym będziesz pisał swoje pierwsze programy. Na rynku dostępnych jest wiele ciekawych rozwiązań, my zastanowimy się nad wyborem pomiędzy Eclipse, Apache NetBeans, Visual Studio Code i Notepad++.

Następnie krok po kroku dowiesz się, jak wygląda składnia i struktura języka PHP, w tym nowe elementy PHP 8, a także nauczysz się używać języka SQL do obsługi baz danych SQLite i MySQL. W następnych lekcjach zapoznasz się ze sposobem prawidłowego łączenia kodu PHP i HTML, a na zakończenie wykorzystasz popularny framework Bootstrap oraz bibliotekę Smarty do budowy swojej pierwszej strony internetowej.

Ćwiczenia praktyczne

Każda lekcja zawiera ćwiczenia, które powinieneś przerobić samodzielnie. Do ich rozwiązania wykorzystasz już zdobyte wiadomości. Uzupełnieniem są ćwiczenia dodatkowe, które wymagają samodzielnego poszerzenia wiedzy. Wszystkie przykłady staraj się przepisywać, a nie kopiować i wklejać. Ma to ogromne znaczenie, gdyż tylko w taki sposób nabierzesz prawidłowego nawyku kodowania w PHP.

Jak zapewne wiesz, matematyka to królowa nauk, dlatego w ćwiczeniach praktycznych pojawiają się również zagadnienia związane z logiką, algebrą czy geometrią. Mam nadzieję, że podstawy matematyki są Ci znane i chętnie zastosujesz je w programowaniu.

Pamiętaj, tylko praktyka czyni mistrza!

W jakim środowisku programować?

Bardzo dobre pytanie! Każdy programista ma swoją teorię na ten temat i nie ukrywam, że ja również. Tak naprawdę każdy stara się wykorzystywać najwygodniejsze dla siebie narzędzie i jest święcie przekonany, że jest ono najlepsze. Na przestrzeni ostatnich lat do programowania wykorzystywałem wiele różnych narzędzi, jednak najczęściej ograniczałem się do kilku moich ulubionych, mianowicie Eclipse, Apache NetBeans i Notepad++. Na uwagę zasługuje również Visual Studio Code, które świetnie nadaje się do kodowania.

Dobry programista powinien umiejętnie używać dowolnego narzędzia umożliwiającego zrealizowanie konkretnego zadania. Niewielkie programy często szybciej się pisze za pomocą prostego edytora tekstowego, natomiast aktualizacja istotnego elementu w dużym projekcie, nad którym pracuje wiele osób, musi być wykonana w profesjonalnym środowisku programistycznym.

W poprzedniej książce opisałem procedurę instalacji i konfiguracji środowiska programistycznego Eclipse, w tej dodam opis konfiguracji oprogramowania Apache NetBeans i Visual Studio Code. Warto nauczyć się wykorzystywać je do kodowania, gdyż są to środowiska bardzo popularne wśród programistów i umożliwiają programowanie w wielu językach, takich jak Java, C/C++, Python, JavaScript, Ruby oraz PHP, a przy tym, co najważniejsze, są bezpłatne i cały czas rozwijane. Przyswojenie obsługi tych narzędzi z pewnością ułatwi Ci naukę innych języków i pozwoli bez większych problemów tworzyć interesujące i nowoczesne projekty.

Zachęcam Cię, abyś wykorzystał do nauki programowania również swój ulubiony edytor tekstowy. Polecam Notepad++; nabierzesz wówczas wprawy w poruszaniu się w konsoli. Bardzo często korzystam z tego edytora. W pierwszej lekcji pokażę Ci, jak go w prosty sposób zainstalować.

Dobłą wiadomością jest to, że wszystkie programy zawarte w tej książce były przetestowane w każdym z wyżej wymienionych narzędzi programistycznych, a kody źródłowe można pobrać ze strony wydawnictwa Helion.

Dla kogo przeznaczona jest ta książka?

Książka przeznaczona jest dla początkujących programistów, jak również dla tych, którzy chcieliby odświeżyć swoją wiedzę. W porównaniu do mojej poprzedniej pozycji o programowaniu w języku PHP część przykładów w tej książce wybiega poza jego podstawy, aby zmotywować Cię do jeszcze większego wysiłku.

Książka przyda się również osobom prowadzącym warsztaty z informatyki, chcącym uzupełnić swoje zajęcia. Przy każdej lekcji znajdziesz informację o czasie, w jakim powinienś sobie z nią poradzić. Nie martw się, jeśli go przekroczysz, i tak nikt tego nie sprawdzi.

PHP jest bardzo rozbudowanym językiem programowania. Programy mają często po kilka lub nawet kilkanaście tysięcy linii kodu i wykorzystują ogromną liczbę funkcji, bibliotek i modułów. Nie musisz się jednak martwić — w tej książce ograniczyłem się do podstaw, abyś mógł krok po kroku rozwijać swoje umiejętności.

Czy dasz radę?

Pewnie zastanawiasz się, czy dasz radę i czy uda Ci się pomyślnie zrealizować wszystkie lekcje. Zastanawiasz się, jak masz to zrobić, przecież nie masz o tym zielonego pojęcia! Czytaj i ćwicz, a już po pierwszych lekcjach zauważysz postęp!

Moja historia z programowaniem zaczęła się pod koniec lat 80. ubiegłego wieku, kiedy jedynym językiem programowania, jaki wówczas był dostępny na moim komputerze, był język maszynowy. Do dyspozycji miałem książkę w języku niemieckim (sic!) i kartka po kartce uczyłem się programować. Tak zapoznałem się z asemblerem, moim pierwszym językiem programowania. Z innymi językami było już znacznie łatwiej. Miło wspominał Turbo Pascal firmy Borland i pierwsze programy w Perlu.

Do dzisiaj mam zachowane swoje pierwsze programy napisane w PHP. Są już od dawna pełnoletnie!

Dzisiaj masz o wiele większe możliwości nauki programowania, niż miałem ja. Właśnie dlatego jestem przekonany, że dasz sobie świetnie radę!

Czy na PHP można dobrze zarabiać?

To pytanie często się pojawia na wielu forach dla programistów i jest stawiane przez osoby dopiero co zaczynające przygodę z kodowaniem. Odpowiedź jest jak zwykle taka sama — „to zależy”.

PHP jest skorelowany z technologią webową i w tym kierunku powinieneś podążać, stawiając właśnie na ten język. Nie będę ukrywał, że znajomość języków HTML5, JavaScript, CSS (w tym Sass) i SQL będzie pomocna przy realizacji własnych projektów, które będziesz mógł dobrze sprzedać. Bardzo często popełniany jest błąd, jakim jest porównywanie wprost zarobków programistów Javy, C#, C++, Pythona i PHP, bez uwzględniania ich środowiska pracy. W mojej ocenie dobry programista znajdzie wiele sposobów na to, aby dobrze zmaterializować swój potencjał, zatrudniając się w nowoczesnej firmie technologicznej czy zakładając własną działalność.

Programista PHP znający dodatkowo języki HTML5, JS, CSS i SQL ma ogromne możliwości na rynku usług związanych z projektowaniem stron internetowych. Możesz śmiało założyć jednoosobową działalność i szukać klientów wśród wielu milionów firm na całym świecie. Znajomość języka angielskiego z pewnością to ułatwi.

Strony internetowe to ledwie jeden aspekt wykorzystania języka PHP. W związku z coraz częstszym wykorzystywaniem systemów zarządzania treścią (ang. *Content Management System*, CMS) do budowy stron pojawiła się też dodatkowa możliwość zarabkowania, a mianowicie pisanie rozszerzeń (wtyczek) do takich systemów, jak choćby Wordpress, Drupal czy Moodle. Firmy oferujące takie usługi z reguły nie są zbyt duże, często zatrudniają jednego lub dwóch programistów.

A teraz trochę liczb — nowoczesny moduł do WordPressa o nazwie The7⁴, z którego chętnie korzystam, kosztuje 39 dolarów za jedną licencję. Do dzisiaj sprzedanych jest ponad 230 tysięcy sztuk. Kolejny przykład to nakładka do popularnego systemu e-learningowego Moodle o nazwie Lambda⁵. Koszt to 59 dolarów za licencję — do dzisiaj sprzedanych jest 8 tysięcy sztuk. Weź do ręki kalkulator i policz, jakie to są kwoty! I teraz najlepsze! Robiąc raz udaną wtyczkę czy nakładkę, przez lata będziesz mógł ją rozwijać i czerpać zyski z jej sprzedaży.

⁴ Envato Market, <https://themeforest.net/item/the7/5556590> [dostęp: 26.05.2021].

⁵ Envato Market, <https://themeforest.net/item/lambda/9442816> [dostęp: 26.05.2021].

Niech do rozpoczęcia nauki programowania w PHP przekona Cię również to, że sam zajmuję się zawodowo PHP od ponad 20 lat, czyli od jego początków, i ciągle jest coś nowego i inspirującego do zrealizowania.

Osiągnięcie sukcesu i satysfakcji z pracy jest ściśle związane z nabywanym na przestrzeni wielu lat doświadczeniem. To właśnie zdobyte doświadczenie ułatwi Ci znalezienie świetnej pracy czy założenie własnej firmy. Powodzenia!

Podziękowania

Chciałbym podziękować swojej żonie Idrze za wsparcie i motywację. Dzięki niej praca nad książką była ogromnie satysfakcjonująca.

Chciałbym także podziękować wydawnictwu Helion, a w szczególności Pani Małgorzacie Kulik za świetną i owocną współpracę.

Dziękuję Czytelnikom za cenne uwagi i komentarze dotyczące mojej ostatniej książki o programowaniu w języku PHP 7, wydanej przez wydawnictwo Helion. Wszystkich Czytelników chcących się ze mną skontaktować zapraszam na moją stronę internetową: <https://duka.pl>.

Bardzo Wam dziękuję.



Co nowego w PHP 8

Długo oczekiwana aktualizacja języka PHP pojawiła się 26 listopada 2020 roku i od razu wywołała wiele dyskusji na temat nowych funkcjonalności. Nowy mechanizm JIT (ang. *Just In Time*), który polega na kompilowaniu programu do postaci kodu maszynowego przed jego wykonaniem, ma wpłynąć na zwiększenie wydajności serwisów internetowych. Ale czy faktycznie tak będzie? Już aktualizacja z wersji PHP 5 do PHP 7 przyniosła znaczny wzrost wydajności, a PHP 7.4 działa naprawdę szybko. Przesiadka z wersji 7.4 na 8 najprawdopodobniej nie przyniesie aż tak dużego przyrostu wydajności dla klasycznych aplikacji webowych, może być jednak inspiracją do projektowania zaawansowanych programów matematycznych czy kryptograficznych. O tym jak JIT wpływa na szybkość działania aplikacji, możemy się przekonać, oglądając film⁶ Zeewa Suraskiego, czołowego programisty zaangażowanego w rozwój języka PHP, który przedstawia, jak mechanizm ten może poprawić wydajność generowania fraktali w czasie rzeczywistym. Krótko mówiąc, technologia JIT to nowe wyzwanie dla programistów z takich dziedzin jak modelowanie 3D, uczenie maszynowe (ang. *machine learning*, ML), uczenie głębokie (ang. *deep learning*) czy sieci neuronowe.

Na stronie twórców PHP znajduje się dokładny opis zmian⁷, jakie zostały wprowadzone w wersji PHP 8, wraz z porównaniem ich do wersji PHP 7. Poniżej przedstawię te najważniejsze, które warto poznać, rozpoczynając naukę programowania w PHP 8.

⁶ YouTube, <https://www.youtube.com/watch?v=dWH65pmnsrl> [dostęp: 26.05.2021].

⁷ PHP, <https://www.php.net/releases/8.0/en.php> [dostęp: 26.05.2021].

Przekazywanie wartości z nazwą argumentu

Nowa technika przekazywania do funkcji wartości razem z nazwą argumentu (ang. *named arguments*)⁸ znacznie ułatwia pracę programisty, który nie musi się teraz zastanawiać, czy kolejność parametrów jest prawidłowa. Technika ta umożliwia również pominięcie parametrów opcjonalnych.

W praktyce, kiedy aktualizujesz kod funkcji, często musisz pamiętać o kolejności jej argumentów. Zdarza się, że argumenty opcjonalne nie zawsze znajdują się na końcu, co generuje dużo dodatkowej pracy przy aktualizacji kodu. W PHP 8 nie musisz już o tym pamiętać — wystarczy, że przekażesz wartość z nazwą argumentu.

Listing 1.1 przedstawia przykład przekazywania wartości z nazwą argumentu.

LISTING 1.1	PHP 8	code_01_01.php
01	<?php	
02	function user(string \$imie, string \$nazwisko, ?int \$wiek = 18) : string	
03	{	
04	\$string =	
05	\$imie.PHP_EOL.	
06	\$nazwisko.PHP_EOL.	
07	\$wiek.PHP_EOL;	
08		
09	return \$string;	
10	}	
11	print user(	
12	nazwisko: 'Kowalski',	
13	imie: 'Tadek',	
14	);	
\$ php code_01_01.php ↵		KONSOLA
Tadek		
Kowalski		
18		

Zauważ, że w linii 02 zdefiniowałem funkcję `user` z trzema parametrami, z których jeden jest opcjonalny. Zadeklarowałem również, że oczekiwanym typem zwracanym przez funkcję będzie typ `string`. W wierszach od 11 do 14 wywołuję funkcję, gdzie jasno określám (nazywam) argumenty, które są przekazywane do funkcji.

⁸ PHP, https://wiki.php.net/rfc/named_params [dostęp: 26.05.2021].

Pomimo że w definicji funkcji pierwszym argumentem jest \$imie, wywołanie funkcji będzie prawidłowe. Nie przeszkadza nawet przecinek za ostatnim argumentem.

Konstrukcja warunkowa match

W PHP 8 dostępna jest nowa konstrukcja warunkowa `match`⁹, będąca alternatywą dla obecnej konstrukcji `switch`. W praktyce wykorzystanie konstrukcji `switch` powodowało konieczność tworzenia dodatkowych zmiennych, które były niezbędne do dalszego działania programu.

Konstrukcja `match` jest znacznie krótsza i nie wymaga używania instrukcji `break`, pozwala na łączenie warunków oddzielonych od siebie przecinkiem, a jej wartość jest przypisywana do zmiennej.

Listing 1.2 przedstawia kod testowania zmiennej `$test` przez konstrukcję `switch`. Jest to klasyczne wykorzystanie tej konstrukcji przez programistów.

LISTING 1.2	PHP 7/8	code_01_02.php
01	<?php	
02	\$test = 2;	
03	switch (\$test) {	
04	case 1:	
05	\$x = 'jeden';	
06	break;	
07	case 2:	
08	\$x = 'dwa';	
09	break;	
10	case 3:	
11	\$x = 'trzy';	
12	break;	
13	default:	
14	\$x = 'zero';	
15	}	
16	print \$x;	
\$ php code_01_02.php ↵		KONSOLA
dwa		

⁹ PHP, https://wiki.php.net/rfc/match_expression_v2 [dostęp: 26.05.2021].

Natomiast na listingu 1.3 wykorzystana jest konstrukcja `match`. Zwróć uwagę, że w porównaniu do listingu 1.2 liczba linii znacznie się zmniejszyła, dzięki czemu cały kod stał się czytelniejszy.

LISTING 1.3	PHP 8	code_01_03.php
01	<?php	
02	\$test = 2;	
03	print match (\$test) {	
04	1 => 'jeden',	
05	2 => 'dwa',	
06	3 => 'trzy',	
07	default => 'zero'	
08	};	
\$ php code_01_03.php ↵		KONSOLA
dwa		

Typy łączone

W PHP 8 możemy zadeklarować zbiór dwóch lub większej liczby typów (tak zwanych typów unii)¹⁰ dla parametrów (na wejściu) oraz dla wartości zwracanych (na wyjściu). Możliwość ta ułatwia pisanie bardziej uniwersalnego kodu, który może być później wielokrotnie wykorzystywany — w zależności od typu parametrów wejściowych.

Na listingu 1.4 w liniach od 02 do 15 zdefiniowana jest klasa `Number`, a w niej właściwość `$number` oraz dwie metody `setNumber` i `getNumber`. Zauważ, że zarówno na wejściu, jak i na wyjściu wartości mogą być typu `int` lub `float`. Aby zobaczyć, jak klasa działa w praktyce, w wierszu 17 został utworzony jej egzemplarz i przypisany do zmiennej `$number`. Następnie w liniach 19, 22 i 25 wywoływana jest metoda `setNumber`, za każdym razem z innym typem parametru. W związku z tym, że zadeklarowaliśmy tylko dwa możliwe typy, czyli `int` i `float`, wywołanie metody z parametrem typu `string` spowoduje wystąpienie błędu.

¹⁰ PHP, https://wiki.php.net/rfc/union_types_v2 [dostęp: 26.05.2021].

LISTING 1.4	PHP 8	code_01_04.php
01	<?php	
02	class Number	
03	{	
04	private int float \$number;	
05		
06	public function setNumber(int float \$number): void	
07	{	
08	\$this->number = \$number;	
09	}	
10		
11	public function getNumber(): int float	
12	{	
13	return \$this->number;	
14	}	
15	}	
16		
17	\$number = new Number();	
18		
19	\$number->setNumber(5);	
20	print \$number->getNumber().PHP_EOL;	
21		
22	\$number->setNumber(7.5);	
23	print \$number->getNumber().PHP_EOL;	
24		
25	\$number->setNumber('A');	
\$ php code_01_04.php ↵		KONSOLA
5		
7.5		
PHP Fatal error: [...]		

Deklaracja właściwości klasy

Dotychczas deklaracja właściwości klasy wymagała trzykrotnego ich powtórzenia, co znacznie zwiększało prawdopodobieństwo wystąpienia błędu, nie mówiąc już o dodatkowych liniach kodu programu.

W najnowszej wersji PHP 8 rozwiązano ten problem, wprowadzając uproszczony zapis deklaracji właściwości klasy przy użyciu jej konstruktora¹¹.

¹¹ PHP, https://wiki.php.net/rfc/constructor_promotion [dostęp: 26.05.2021].

Zarówno dotychczasowa, jak i uproszczona metoda deklaracji właściwości klasy są prawidłowe, programista może więc sam zdecydować, z której z nich skorzysta. Na listingu 1.5 znajduje się definicja klasy według dotychczasowej metody. Jak można zauważyć, właściwość `$x` występuje trzykrotnie, w liniach 04, 09 i 13.

LISTING 1.5	PHP 7/8	code_01_05.php
01	<?php	
02	class Point	
03	{	
04	public float \$x;	
05	public float \$y;	
06	public float \$z;	
07		
08	public function __construct(	
09	float \$x = 0.0,	
10	float \$y = 0.0,	
11	float \$z = 0.0	
12	) {	
13	\$this->x = \$x;	
14	\$this->y = \$y;	
15	\$this->z = \$z;	
16	}	
17	}	

Natomiast na listingu 1.6 znajduje się definicja klasy zgodnie z nową metodą (przez konstruktor), dostępną od wersji PHP 8. Jak można zauważyć, kod jest bardziej przejrzysty i zajmuje mniej miejsca.

LISTING 1.6	PHP 8	code_01_06.php
01	<?php	
02	class Point	
03	{	
04	public function __construct(	
05	public float \$x = 0.0,	
06	public float \$y = 0.0,	
07	public float \$z = 0.0,	
08	) {}	
09	}	

Operator zerowy null

Do sprawdzenia, czy wartość jest zerowa (null), wykorzystuje się składnię `!==null`. Od wersji PHP 8 dostępna jest nowa składnia `?->`, która znacznie upraszcza pisanie kodu¹².

Listing 1.7 przedstawia tradycyjne wykorzystanie składni, czyli `!==null`, gdzie krok po kroku — w liniach 03, 06 i 09 — sprawdzane są kolejne warunki przy użyciu konstrukcji `if`. Taki zapis spotkasz w prawie każdym projekcie.

LISTING 1.7	PHP 7/8	code_01_07.php
01	<?php	
02	\$country = null;	
03	if (\$session !== null) {	
04	\$user = \$session->user;	
05		
06	if (\$user !== null) {	
07	\$address = \$user->getAddress();	
08		
09	if (\$address !== null) {	
10	\$country = \$address->country;	
11	}	
12	}	
13	}	

Na listingu 1.8 znajdziesz z kolei nową składnię, czyli `?->`. Mam nadzieję, że dostrzeżasz różnicę, nawet jeśli uwzględnić tylko długość samego kodu.

LISTING 1.8	PHP 8	code_01_08.php
01	<?php	
02	\$country = \$session?->user?->getAddress()?->country;	

Analizując powyższe dwa listingi, zwróć uwagę na to, że pierwsza konstrukcja `?->` na listingu 1.8 jest odpowiednikiem `if ($session !== null) {...}` z listingu 1.7, kolejna `if ($user !== null) {...}`, a ostatnia `if ($address !== null) {...}`. Początkowo może się to wydawać skomplikowane, ale po przeanalizowaniu całości wszystko staje się zrozumiałe.

¹² PHP, https://wiki.php.net/rfc/nullsafe_operator [dostęp: 26.05.2021].

Atrybuty

Język PHP 8 wprowadza atrybuty¹³ umożliwiające dodanie do kodu metadanych, zastępując dotychczasową konstrukcję opartą na adnotacjach PHPDoc. Atrybuty są definiowane za pomocą składni `#[Attribute]`.

Listing 1.9 przedstawia dotychczasową adnotację PHPDoc.

LISTING 1.9	PHP 7/8	code_01_09.php
01	<code><?php</code>	
02	<code>class PostsController</code>	
03	<code>{</code>	
04	<code> /**</code>	
05	<code> * @Route("/api/posts/{id}", methods={"GET"})</code>	
06	<code> */</code>	
07	<code> public function get(\$id) { /* ... */ }</code>	
08	<code>}</code>	

Listing 1.10 przedstawia zaś przykład definicji atrybutów w PHP 8.

LISTING 1.10	PHP 8	code_01_10.php
01	<code><?php</code>	
02	<code>class PostsController</code>	
03	<code>{</code>	
04	<code> #[Route("/api/posts/{id}", methods: ["GET"])]</code>	
05	<code> public function get(\$id) { /* ... */ }</code>	
06	<code>}</code>	

Operacje na łańcuchach

W najnowszej wersji języka PHP zostały wprowadzone trzy funkcje operujące na łańcuchach znaków: `str_contains`, `str_starts_with` i `str_ends_with`.

Dotychczas, aby sprawdzić, czy dany łańcuch znaków zawiera się w drugim lub czy łańcuch rozpoczyna się bądź kończy konkretnym ciągiem znaków, należało używać silnego porównania `===` (trzy znaki równości) lub wyrażeń regularnych.

Listing 1.11 przedstawia dotychczasową metodę na sprawdzenie, czy dany łańcuch znaków zawiera się w drugim łańcuchu, za pomocą funkcji `strpos` i `preg_match` (linie 03 i 07). Natomiast do sprawdzenia, czy łańcuch rozpoczyna się konkretnym

¹³ PHP, https://wiki.php.net/rfc/attributes_v2 [dostęp: 26.05.2021].

ciągami znaków, wykorzystana jest funkcja `strpos` (wiersz 11), a funkcjami `substr` i `strlen` (linia 15) sprawdzamy, czy łańcuch kończy się szukanym ciągiem.

LISTING 1.11	PHP 7/8	code_01_11.php
01	<?php	
02	\$string = 'Jak się masz?';	
03	if (strpos (\$string, 'się') !== false) {	
04	echo 'OK (zawiera się)'.PHP_EOL;	
05	}	
06	\$search = 'się';	
07	if (preg_match ("/{ \$search }/i", \$string)) {	
08	echo 'OK (zawiera się)'.PHP_EOL;	
09	}	
10	\$search = 'Jak';	
11	if(strpos (\$string, \$search) === 0) {	
12	echo 'OK (rozpoczyna się)'.PHP_EOL;	
13	}	
14	\$search = 'masz?';	
15	if (substr (\$string, - strlen (\$search)) === \$search) {	
16	echo 'OK (kończy się)'.PHP_EOL;	
17	}	
\$ php code_01_11.php ↵		KONSOLA
OK (zawiera się)		
OK (zawiera się)		
OK (rozpoczyna się)		
OK (kończy się)		

Listing 1.12 przedstawia wykorzystanie nowych funkcji języka PHP 8 operujących na łańcuchach. Zauważ, że w porównaniu do listingu 1.11 kod jest czytelniejszy, a nazwy nowych funkcji jednoznacznie wskazują, jakie jest ich znaczenie.

LISTING 1.12	PHP 8	code_01_12.php
01	<?php	
02	\$string = 'Jak się masz?';	
03	if (str_contains (\$string, 'się')) {	
04	echo 'OK (zawiera się)'.PHP_EOL;	
05	}	
06	if (str_starts_with (\$string, 'Jak')) {	
07	echo 'OK (rozpoczyna się)'.PHP_EOL;	
08	}	
09	if (str_ends_with (\$string, 'masz?')) {	
10	echo 'OK (kończy się)'.PHP_EOL;	
11	}	

```
$ php code_01_12.php ↵
```

KONSOLA

```
OK (zawiera się)  
OK (rozpoczyna się)  
OK (kończy się)
```

Podsumowanie

Najnowsza wersja języka PHP wprowadza wiele zmian i nowości, najważniejsze z nich starałem się w miarę przystępnie przedstawić na poprzednich stronach.

Warto jeszcze wspomnieć o takich nowościach, jak:

- przechwytywanie wyjątków bez konieczności przypisania do zmiennej;
- mechanizm WeakMaps do przechowywania referencji do obiektów, który pozwala na ich usuwanie przez automatyczną dealokację, jeśli obiekt zostanie usunięty przez naszą aplikację;
- funkcja `fdiv` pozwalająca na dzielenie przez zero bez generowania błędu;
- funkcja `get_debug_type` (będąca rozszerzeniem funkcji `gettype`) oraz funkcja `get_resource_id` zwracająca identyfikator obiektu.

Pełna lista nowości dostępna jest na stronie <https://wiki.php.net/rfc>.

Wiele wprowadzonych zmian w PHP 8 ma na celu uporządkowanie składni języka i dostosowanie jej do wymagań programistów. Dla początkującego koder PHP, czyli dla Ciebie, powyższe informacje mają charakter głównie informacyjny.

Pamiętaj, język PHP jest zestawem wielu narzędzi, z których będziesz korzystał podczas realizacji ciekawych projektów. Na początku poznasz te najważniejsze, ale z czasem nauczysz się wykorzystywać też te bardziej zaawansowane. Nie musisz znać na pamięć wszystkich funkcji, ważne, abyś wiedział, gdzie możesz znaleźć więcej informacji na ich temat. Na stronie www.php.net znajdziesz dokumentację języka i wiele przykładów, z których śmiało możesz korzystać.

Przejście z wersji PHP 7 na PHP 8 jest nieuniknione. Wsparcie techniczne dla PHP 7.3 wygasa w grudniu 2021 roku, natomiast dla PHP 7.4 — w listopadzie 2022 roku. Informacyjnie dodam, że na koniec 2021 roku zaplanowana jest premiera kolejnej wersji języka PHP o numerze 8.1.



Konfiguracja

Na początek, krok po kroku, zainstalujesz środowisko programistyczne, a następnie skonfigurujesz je pod kątem programowania w najnowszej wersji języka PHP. Zaczniiesz od instalacji zintegrowanego pakietu XAMPP, który składa się z serwera Apache, bazy danych MariaDB (MySQL) i interpreterów skryptów napisanych w językach PHP i Perl. Pakiet ten jest darmowy i świetnie nadaje się do testowania projektów webowych, zanim zostaną uruchomione na serwerach trybucyjnych.

Po zainstalowaniu XAMPP wybierzesz, z jakiego środowiska programistycznego będziesz chciał skorzystać. Przygotowałem dla Ciebie cztery warianty, a mianowicie: Eclipse, Apache NetBeans, Visual Studio Code i Notepad++, zatem masz wybór.

Pakiety Eclipse i Apache NetBeans wymagają dodatkowo zainstalowania Javy, używam ich do kodowania nie tylko w języku PHP, ale również w językach C i C++. Eclipse, choć jest przeznaczony dla programistów Javy, nadaje się dobrze do pisania programów w językach Python, JavaScript, a nawet Ruby. Jeśli oprócz PHP planujesz programować także w innych językach, możesz śmiało wybrać Eclipse lub Apache NetBeans.

Visual Studio Code to bardzo popularny edytor kodu źródłowego stworzony przez firmę Microsoft, który za pomocą licznych wtyczek umożliwia kodowanie w wielu językach. Program pozwala na debugowanie kodu, pomaga w zarządzaniu wersjami kodu źródłowego za pośrednictwem systemu kontroli wersji Git, a także wykorzystuje mechanizm IntelliSense do automatycznego uzupełniania kodu. Visual Studio Code może być trafnym wyborem dla początkującego koderka.

Notepad++ to rozbudowany edytor z kolorowaniem składni dla wielu języków. Świetnie nadaje się do edycji plików tekstowych i konfiguracyjnych. Po prostu warto mieć go pod ręką.

Opis instalacji i konfiguracji przygotowałem dla użytkowników systemu Windows, natomiast użytkownicy innych systemów, tj. macOS lub Linux, powinni bez większych problemów przygotować sobie środowisko na podstawie poniższych wskazówek. Zanim zaczniesz cokolwiek robić, czytaj uważnie każdą instrukcję, którą masz wykonać. Musisz być dokładny, szczególnie gdy będziesz pracował w konsoli. Najlepiej, jeśli **teraz** zapoznasz się ze wskazówkami, które umieściłem dla Ciebie na końcu lekcji, i wrócisz do tego miejsca.

Instalacja pakietu XAMPP

Pakiet XAMPP służy do szybkiej instalacji podstawowych narzędzi, takich jak serwer Apache¹⁴ z obsługą PHP oraz serwer baz danych MariaDB¹⁵ (kompatybilny z MySQL), które mają usprawnić pracę web dewelopera. Oprogramowanie zostało wstępnie skonfigurowane, dlatego zaraz po instalacji możesz śmiało rozpocząć pracę nad nowym projektem, wykorzystując zarówno język PHP, jak i HTML, CSS czy JavaScript.

Otwórz stronę <https://www.apachefriends.org>, pobierz najnowszą wersję pakietu XAMPP dla Twojego systemu operacyjnego i zainstaluj. Podczas instalacji zalecam wybrać katalog domyślny `c:\xampp` lub `d:\xampp`, ponieważ ułatwi to później konfigurację środowiska programistycznego.

Panel kontrolny aplikacji XAMPP został przedstawiony na rysunku 2.1. Zauważ, że uruchomione są już moduły (ang. *module*) serwera Apache i bazy danych MySQL.

Moduły możesz włączyć i wyłączyć za pomocą przycisków *Start* i *Stop*. Przycisk *Admin* w przypadku bazy danych MySQL uruchamia aplikację phpMyAdmin.

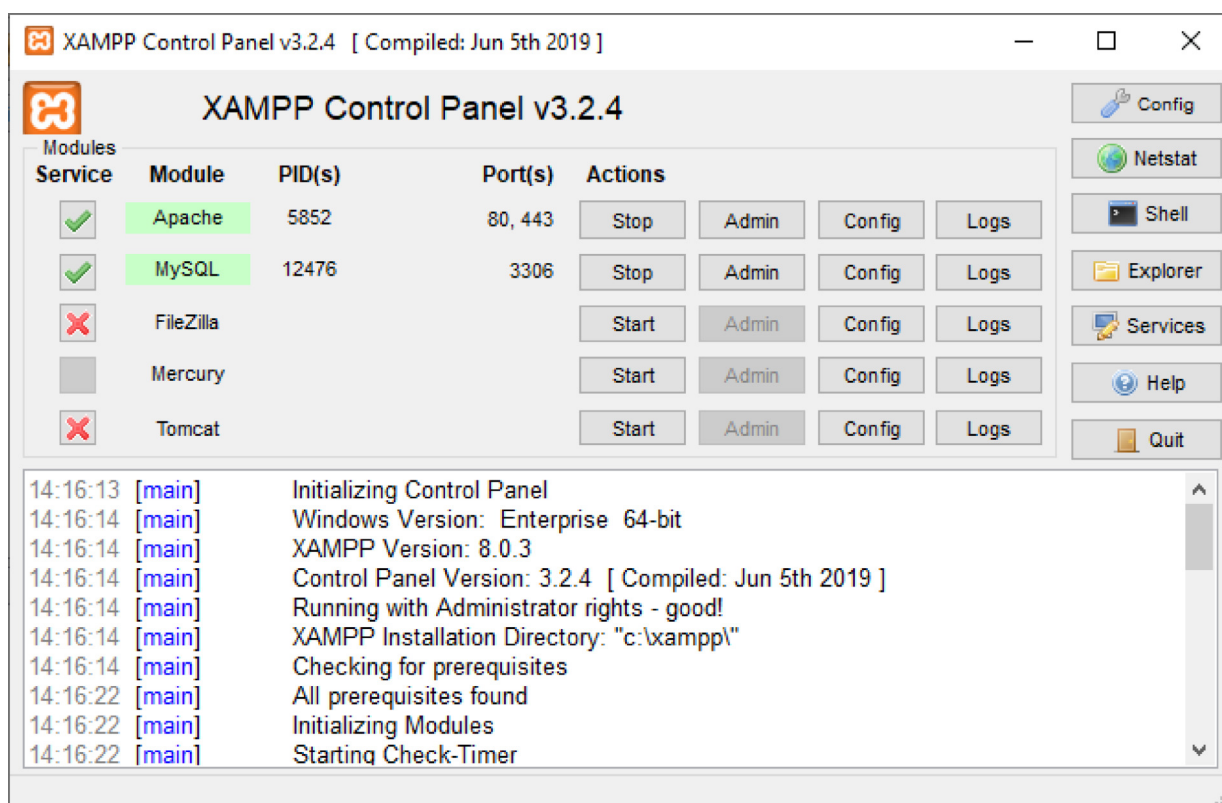
Serwer działa pod adresem <http://localhost/> — możesz to teraz sprawdzić.

Konfiguracja serwera wirtualnego

Domyślnie pliki serwera `localhost` znajdują się w katalogu `c:\xampp\htdocs`, dlatego dobrym pomysłem jest przygotowanie sobie oddzielnego serwera na potrzeby nauki programowania z niniejszą książką.

¹⁴ Apache HTTP Server Project, <https://httpd.apache.org> [dostęp: 26.05.2021].

¹⁵ MariaDB Server, <https://mariadb.org> [dostęp: 26.05.2021].



Rysunek 2.1. Panel kontrolny aplikacji XAMPP uruchomiony w trybie administratora

Proponuję, aby dodatkowy (testowy) serwer miał adres `php8.localhost`, dlatego utwórz na dysku pusty katalog `c:\xampp\apps\php8\htdocs`.

Następnie otwórz w dowolnym edytorze plik konfiguracyjny `httpd-vhosts.conf` znajdujący się w katalogu `c:\xampp\apache\conf\extra` (zakładam, że pakiet XAMPP jest zainstalowany w katalogu domyślnym `c:\xampp`; jeśli nie, odpowiednio zmodyfikuj poniższą konfigurację).

Na końcu pliku `httpd-vhosts.conf` dodaj taką konfigurację:

```
<VirtualHost *:80>
    DocumentRoot "/xampp/htdocs/"
    ServerName localhost
</VirtualHost>
<VirtualHost *:80>
    DocumentRoot "/xampp/apps/php8/htdocs"
    ServerName php8.localhost
    <Directory "/xampp/apps/php8/htdocs">
        DirectoryIndex index.php
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

Zapisz plik i zamknij. Zrestartuj moduł Apache w panelu kontrolnym XAMPP.

Dodatkowo w systemie Windows należy zmapować nowy adres serwera z adresem IP hosta lokalnego. Otwórz w edytorze plik `C:\windows\system32\drivers\etc\hosts` i dodaj następujący wpis:

```
127.0.0.1      php8.localhost
```

Teraz możesz sprawdzić, czy wszystko działa poprawnie. W tym celu utwórz w katalogu `c:\xampp\apps\php8\htdocs` plik o nazwie `index.php` z następującą zawartością (listing 2.1):

LISTING 2.1	PHP 7/8	index.php
01	<?php	
02	print "XAMPP gotowy do pracy!";	

Gotowe! Otwórz w przeglądarce internetowej adres `http://php8.localhost/` i ciesz się ze świetnie wykonanej pracy!

Konfiguracja PHP

Po instalacji pakietu XAMPP interpreter PHP jest już wstępnie skonfigurowany, a dostępne biblioteki w zupełności wystarczą do nauki programowania i budowy pierwszych projektów webowych. W praktyce (na serwerze dystrybucyjnym u operatora hostingowego) większość bibliotek jest aktywna i prawie wszystkie Twoje aplikacje będą działać bez problemu.

Niniejsza książka obejmuje wykorzystanie bazy danych SQLite, zatem aktywujemy jej obsługę w pliku konfiguracyjnym `php.ini`. Ćwiczenie to ułatwi Ci później skonfigurować interpreter PHP według własnych potrzeb.

Otwórz więc w dowolnym edytorze plik konfiguracyjny `c:\xampp\php\php.ini`, odśledź następującą linię i usuń z niej znak średnika (u mnie jest to wiersz 954):

```
;extension=sqlite3
```

Po zmianie powinno być tak:

```
extension=sqlite3
```

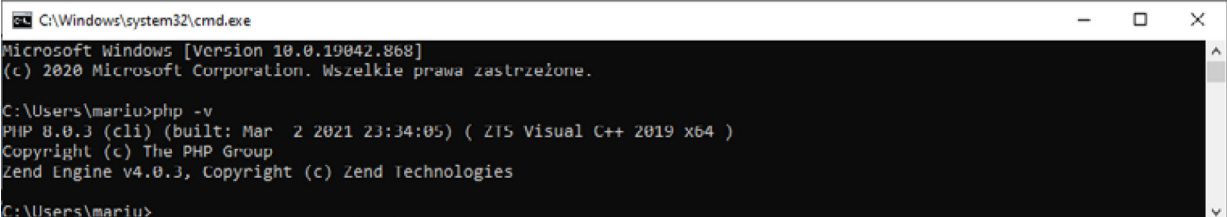
Gotowe! Nauczyłeś się aktywować dodatkowe biblioteki programistyczne w PHP. Teraz, kiedy będziesz potrzebował nowych funkcji, będziesz wiedział, jak to zrobić.

Aktualizacja zmiennej środowiskowej PATH

Zalecam dodanie ścieżki `c:\xampp\php` w zmiennej środowiskowej PATH. Ułatwi to dostęp do interpretera PHP w linii poleceń (konsoli) po wpisaniu tylko nazwy `php`.

Procedura dodania ścieżki do interpretera PHP w zmiennej środowiskowej PATH w systemie Windows:

- W polu wyszukiwania wpisz *Zaawansowane ustawienia systemu*, po czym wybierz *Otwórz*.
- Kliknij przycisk *Zmienne środowiskowe* i w obszarze *Zmienne systemowe* odśledź zmienną środowiskową *Path*. Wybierz ją i kliknij przycisk *Edytuj*.
- W oknie *Edycja zmiennej środowiskowej* kliknij przycisk *Nowa* i dodaj ścieżkę do katalogu z interpreterem języka PHP, u Ciebie będzie to `c:\xampp\php`.
- Naciśnij przycisk *OK* i zamknij pozostałe okna.
- Wpisz w konsoli `php -v`, a dowiesz się, z której wersji języka PHP korzystasz (rysunek 2.2).



```

C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19042.868]
(c) 2020 Microsoft Corporation. Wszelkie prawa zastrzeżone.

C:\Users\mariu>php -v
PHP 8.0.3 (cli) (built: Mar 2 2021 23:34:05) ( ZTS Visual C++ 2019 x64 )
Copyright (c) The PHP Group
Zend Engine v4.0.3, Copyright (c) Zend Technologies
C:\Users\mariu>

```

Rysunek 2.2. Informacja o zainstalowanej wersji języka PHP w systemie Windows

Instalacja pakietu Eclipse

Pakiet Eclipse jest profesjonalnym środowiskiem programistycznym przeznaczonym do tworzenia programów w Javie. Poprzez obsługę wtyczek umożliwia również programowanie w innych językach, takich jak C/C++, PHP, Python czy JavaScript. Poznanie tego środowiska może być pomocne, kiedy uznasz, że poza kodowaniem w języku PHP chcesz też pisać aplikacje, np. w języku Java.

Otwórz stronę <https://www.eclipse.org>, pobierz najnowszą wersję instalatora pakietu Eclipse i uruchom go. Zobaczysz listę dostępnych pakietów IDE. Odszukaj *Eclipse IDE for PHP Developers* i rozpocznij proces instalacji. Jeśli wszystko przebiegnie poprawnie, to w menu i na pulpicie pojawi się ikona programu.

Konfiguracja Eclipse z PHP

Podczas pierwszego uruchomienia zostaniesz zapytany o katalog, w którym będziesz przechowywał swoje projekty. Katalog możesz również wybrać, klikając przycisk *Browse*. Wybór katalogu zatwierdzasz przyciskiem *Launch*. Program będzie Cię pytał o ten katalog przy każdym uruchomieniu, dlatego aby jego nazwa została zapamiętana, zaznacz pole wyboru *Use this as the default and do not ask again*.

Po pierwszym uruchomieniu Eclipse wykonaj następujące czynności:

1. Zamknij ekran powitalny. Najlepiej, jeśli usuniesz zaznaczenie pola wyboru *Always show Welcome at start up*, znajdujące się w prawym dolnym rogu okna.
2. Z menu *Window* wybierz opcję *Preferences*. Pojawi się nowe okno.
3. Wybierz z listy *General/Workspace* i po prawej stronie w sekcji *Text file encoding* ustaw kodowanie na UTF-8.
4. Wybierz z listy *PHP/Installed PHPs*. Po prawej stronie pojawi się sekcja do konfiguracji PHP.
5. Kliknij przycisk *Add*. Pojawi się okno *New PHP Executable*, w którym podasz lokalizację interpretera języka PHP. W polu *Name* wpisz np. *PHP 8*, a w polu *Executable path* wpisz *c:\xampp\php\php.exe* lub kliknij przycisk *Browse* i wybierz plik *php.exe* z katalogu *c:\xampp\php*. Potwierdzasz przyciskiem *Finish*.
6. Po powrocie do okna *Preferences* zauważysz, że została odświeżona lista z dostępnymi wersjami PHP, z którymi Eclipse będzie współpracował. Kliknij przycisk *Apply and Close*.

Następnym krokiem jest skonfigurowanie serwera PHP, na którym będziesz uruchamiał swoje programy. Wykonaj następujące czynności:

1. Z menu wybierz polecenie *Window/Show View/Console*.
2. W dolnym oknie Eclipse znajdują się cztery zakładki: *Problems*, *Tasks*, *Console* i *Servers*. Wybierz ostatnią — *Servers*.
3. W zakładce *Servers* kliknij odnośnik *No servers are available [...]*. Uruchomi się okno *Define a New Server*.
4. Wybierz z listy *PHP Built-in Server*, a pole *Server's host name* pozostaw na *localhost*. Następnie kliknij przycisk *Next*.

5. Wskaż nazwę dla serwera (możesz pozostawić *PHP*) oraz wybierz z rozwijanej listy wersję PHP, którą przed chwilą dodałeś do Eclipse. Potwierdzasz przyciskiem *Finish*.
6. W dolnym oknie programu Eclipse wybierz zakładkę *Console*.

Ostatnim krokiem jest sprawdzenie, czy wszystko działa prawidłowo. Wykonaj następujące czynności:

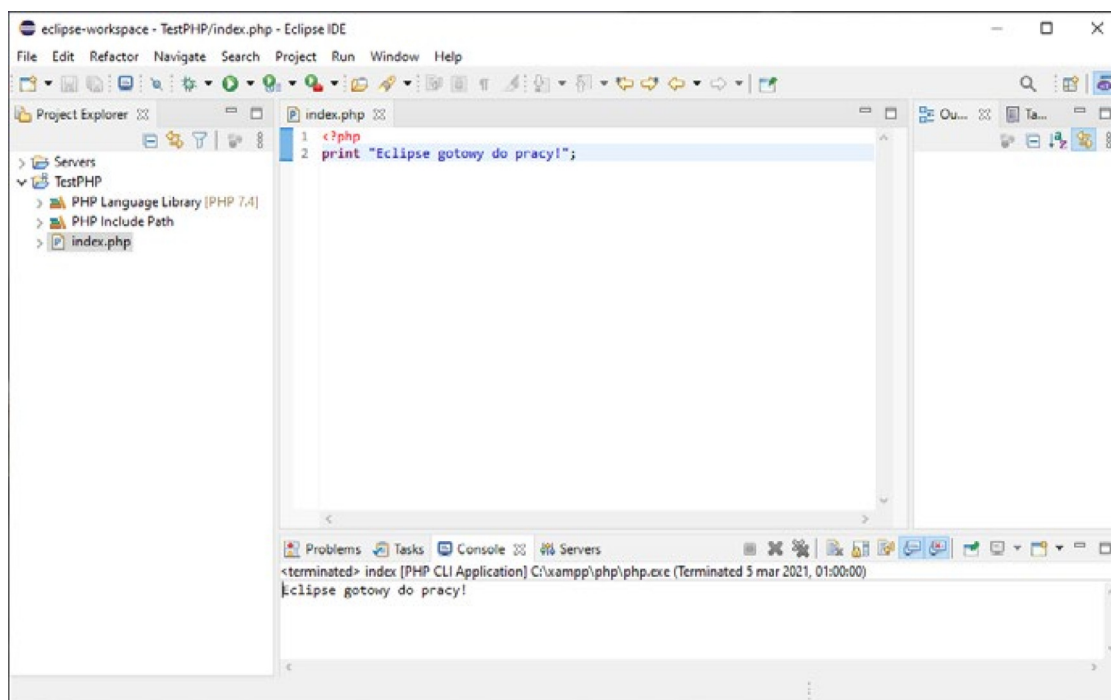
1. Z menu wybierz polecenie *File/New/PHP Project*. Pojawi się nowe okno.
2. W polu *Project name* wpisz `TestPHP` i kliknij przycisk *Finish*. Eclipse utworzy katalog *TestPHP* w Twoim katalogu z projektami.
3. Wybierz z menu polecenie *File/New/PHP File*. Pojawi się nowe okno.
4. W polu *File name* wpisz `index.php` (Eclipse domyślnie proponuje nazwę `newfile.php`), zaznacz nazwę projektu, który przed chwilą utworzyłeś — *TestPHP*, a na koniec kliknij przycisk *Finish*. Plik `index.php` zostanie utworzony w katalogu projektu *TestPHP*.
5. W głównym oknie programu Eclipse będziesz mógł edytować plik `index.php`. Wpisz poniższy kod, a następnie zapisz plik (*Ctrl+S*).

```
<?php  
print "Eclipse gotowy do pracy!";
```

6. Wybierz z menu polecenie *Run/Run as/PHP CLI Application*.
7. W zakładce *Console* zobaczysz wynik działania programu, czyli tekst *Eclipse gotowy do pracy!*

Instalacja zakończona!

Rysunek 2.3 przedstawia kompletne środowisko programistyczne Eclipse. Jak widać, w głównym oknie możesz edytować pliki źródłowe PHP, z lewej strony dostępny jest eksplorator projektów (*Project Explorer*), a dolne okno zawiera cztery zakładki: *Problems*, *Tasks*, *Console* i *Servers*. Wynik działania Twoich programów będzie prezentowany w konsoli (*Console*), dlatego tę zakładkę będziesz miał aktywną najczęściej.



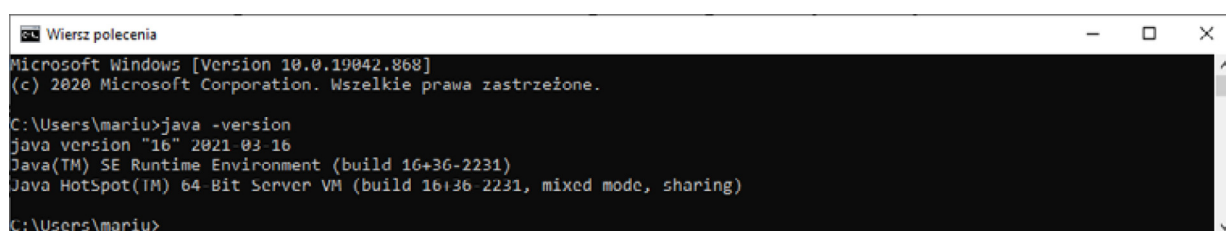
Rysunek 2.3. Środowisko programistyczne Eclipse po instalacji i konfiguracji z PHP

Instalacja pakietu Apache NetBeans

Apache NetBeans jest kolejnym profesjonalnym środowiskiem programistycznym przeznaczonym do tworzenia programów w Javie i, tak jak w przypadku Eclipse, sprawdza się również przy pisaniu programów w językach C/C++, PHP i JavaScript. Apache NetBeans używam od lat i napisałem w nim wiele projektów.

Na początku zainstaluj Javę JDK ze strony <https://www.oracle.com/downloads>. Pamiętaj, aby nie była to wersja JRE, tylko JDK. Do pobrania oprogramowania może być wymagane założenie konta w serwisie Oracle. Po zainstalowaniu Javy naciśnij klawisze **Win + R** i wpisz `cmd`, a po uruchomieniu się konsoli wpisz `java -version`.

Jeśli na Twoim komputerze wyświetla się informacja o zainstalowanej wersji Javy (rysunek 2.4), możesz zamknąć konsolę przez wpisanie `exit`. Jeśli nie, sprawdź poprawność instalacji.



Rysunek 2.4. Informacja o zainstalowanej wersji Javy w systemie Windows

Następnie otwórz stronę <https://netbeans.org>, pobierz najnowszą wersję pakietu Apache NetBeans dla Twojego systemu operacyjnego i zainstaluj. Jeśli wszystko przebiegnie poprawnie, w menu i na pulpicie pojawi się ikona programu.

Konfiguracja Apache NetBeans z PHP

Po pierwszym uruchomieniu Apache NetBeans wykonaj następujące czynności:

1. Zamknij ekran powitalny. Najlepiej, jeśli usuniesz zaznaczenie pola wyboru *Show On Startup*, znajdujące się w prawym górnym rogu okna.
2. Z menu *Tools* wybierz opcję *Options*. Pojawi się nowe okno.
3. Odszukaj i kliknij logo PHP. Pojawi się informacja o rozpoczęciu aktywacji obsługi języka PHP w programie.
4. W zakładce *General* w polu *PHP Interpreter* wpisz `c:\xampp\php\php.exe` lub kliknij przycisk *Browse* i wybierz plik `php.exe` z katalogu `c:\xampp\php`. Potwierdzasz przyciskiem *OK*.

Następnym krokiem jest sprawdzenie, czy wszystko działa prawidłowo. Wykonaj następujące czynności:

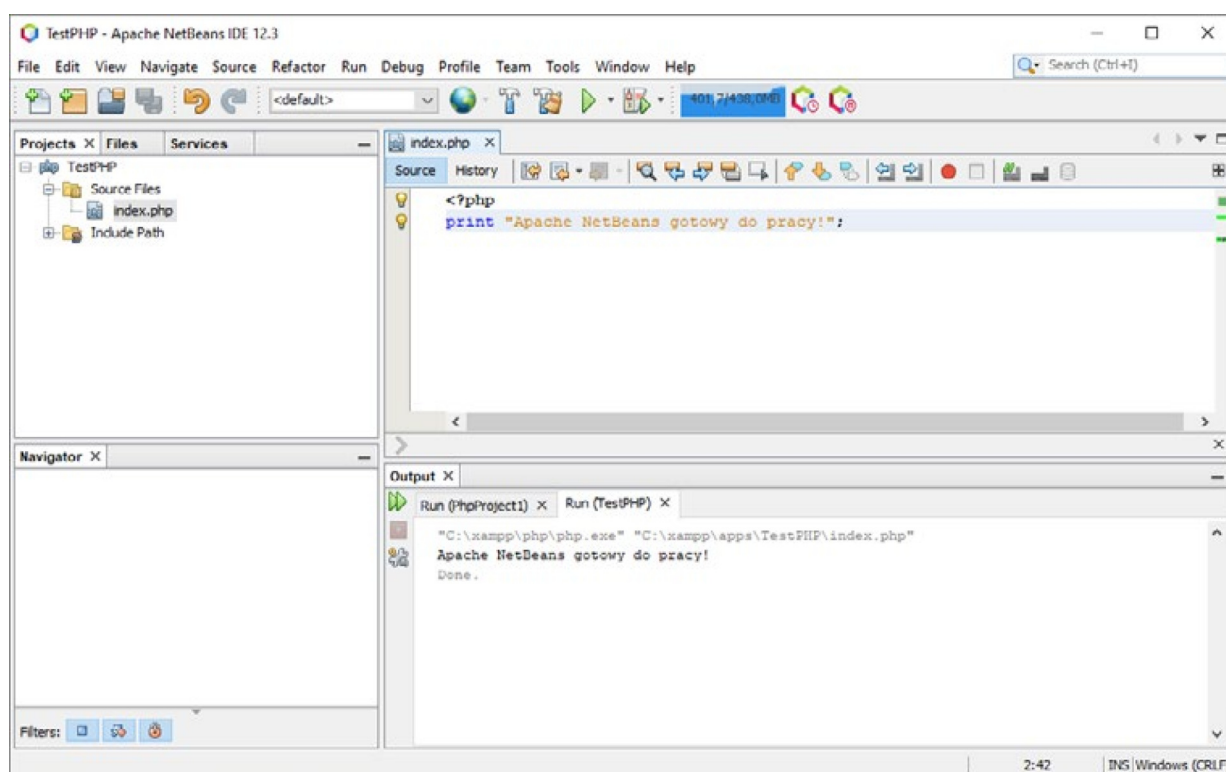
1. Wybierz z menu polecenie *File/New Project*. Pojawi się nowe okno.
2. W sekcji *Choose Project* w obszarze *Categories* wybierz *PHP*, a następnie *PHP Application* w obszarze *Projects*. Potwierdzasz przyciskiem *Next*.
3. W sekcji *Name and Location* w polu *Project Name* wpisz `TestPHP`. Pole *Source Folder* wskazuje na katalog z Twoimi projektami — możesz je pozostawić lub dowolnie zmienić. Potwierdzasz przyciskiem *Next*.
4. W sekcji *Run Configuration* z listy *Run As* wybierz *Script (run in command line)*. Pole *PHP Interpreter* powinno wskazywać na plik wykonywalny interpretera języka PHP, który podałeś wcześniej w zakładce *General*. Na koniec kliknij przycisk *Finish*. Plik `index.php` zostanie utworzony w katalogu projektu *TestPHP*.
5. W głównym oknie programu Apache NetBeans będziesz mógł edytować plik `index.php`. Wpisz poniższy kod, a następnie zapisz plik (*Ctrl+S*).

```
<?php
print "Apache NetBeans gotowy do pracy!";
```

6. Z menu wybierz polecenie *Run/Run Project* lub naciśnij klawisze *Shift+F6*.
7. W oknie *Output* zobaczysz wynik działania programu, czyli tekst *Apache NetBeans gotowy do pracy!*

Instalacja zakończona!

Rysunek 2.5 przedstawia kompletne środowisko programistyczne Apache NetBeans. Jak widać, w głównym oknie możesz edytować pliki źródłowe PHP, z lewej strony dostępna jest lista projektów (*Projects*), plików (*Files*) i usług (*Services*), a dolne okno (*Output*) zawiera wynik działania Twoich programów.



Rysunek 2.5. Środowisko programistyczne Apache NetBeans po instalacji i konfiguracji z PHP

Instalacja Visual Studio Code

Visual Studio Code to profesjonalny edytor kodu źródłowego, stworzony przez firmę Microsoft, z którym warto się zapoznać, by wiedzieć, jak wykorzystywać go w codziennej pracy. Program umożliwia kodowanie w najbardziej popularnych językach programowania i, co ważne, jest bardzo prosty w obsłudze. Bez problemu możesz zarządzać swoimi repozytoriami za pośrednictwem systemu kontroli wersji Git,

a mechanizm IntelliSense pomoże Ci w pisaniu kodu. Co ciekawe, z badań przeprowadzonych przez serwis społecznościowy *Stack Overflow*¹⁶ wynika, że Visual Studio Code był najpopularniejszym wyborem programistów w 2019 roku.

Otwórz stronę <https://code.visualstudio.com>, pobierz najnowszą wersję pakietu Visual Studio Code dla Twojego systemu operacyjnego i zainstaluj. Jeśli wszystko przebiegnie poprawnie, w menu i na pulpicie pojawi się ikona programu.

Konfiguracja Visual Studio Code z PHP

Po pierwszym uruchomieniu Visual Studio Code wykonaj następujące czynności:

1. W aktywnej karcie *Welcome* w sekcji *Customize/Tools and languages* (po prawej stronie) zobaczysz aktualną informację o wspieranych językach programowania. Po kliknięciu *PHP* w zdaniu *Install support for JavaScript, Python, Java, PHP...* zostaną zainstalowane moduły *PHP IntelliSense* oraz *PHP Debug*, po czym Visual Studio Code zostanie uruchomiony ponownie. Od teraz mechanizm IntelliSense pomoże Ci w pisaniu kodu w języku PHP.
2. Po zainstalowaniu dodatkowego modułu *Code Runner* będziesz mógł uruchamiać swoje programy w języku PHP bezpośrednio w Visual Studio Code. W tym celu wybierz z menu *View/Extensions*. W polu wyszukiwania odszukaj pakiet *Code Runner* i zainstaluj go. Kod w PHP możesz uruchomić, klikając w prawym rogu okna ikonę *Run Code* lub wybierając tę opcję z menu kontekstowego wywołanego prawym przyciskiem myszki.
3. Opcjonalnie możesz zainstalować dodatkowy moduł *PHP Server*, umożliwiający zaawansowaną obsługę serwera PHP w programie Visual Studio Code. W tym celu wybierz z menu *View/Extensions*. W polu wyszukiwania odszukaj pakiet *PHP Server* i zainstaluj go.

Następnym krokiem jest sprawdzenie, czy wszystko działa prawidłowo. Wykonaj następujące czynności:

1. Wybierz z menu polecenie *File/New File*.
2. W oknie *Untitled-1* wpisz następujący kod, a następnie zapisz plik (*Ctrl+S*). Pamiętaj, aby plik był zapisany z rozszerzeniem *.php*.

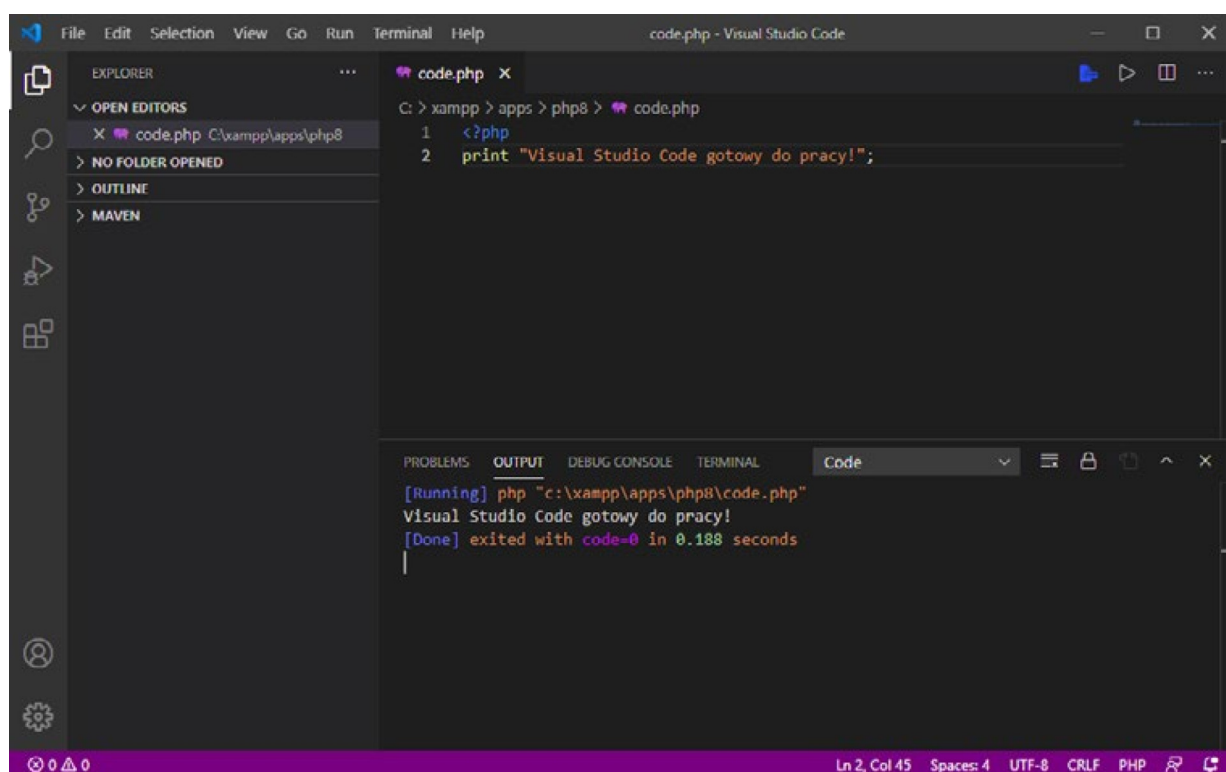
```
<?php  
print "Visual Studio Code gotowy do pracy!";
```

¹⁶ Stack Overflow, <https://insights.stackoverflow.com/survey/2019> [dostęp: 26.05.2021].

3. Kliknij ikonę *Run Code* (w prawym górnym rogu) lub wykorzystaj menu kontekstowe dostępne po kliknięciu prawego przycisku myszki.
4. W oknie *Output* zobaczysz wynik działania programu, czyli tekst *Visual Studio Code gotowy do pracy!*.

Instalacja zakończona!

Rysunek 2.6 przedstawia oprogramowanie Visual Studio Code skonfigurowane do programowania w języku PHP. Jak widać, w głównym oknie możesz edytować pliki źródłowe PHP, z lewej strony dostępna jest lista z narzędziami (*Explorer*), a dolne okno (*Output*) zawiera wynik działania Twoich programów.



Rysunek 2.6. Środowisko programistyczne Visual Studio Code po instalacji i konfiguracji z PHP

Instalacja Notepad++

Edytor tekstowy Notepad++ to bardzo popularne narzędzie stosowane przez wielu programistów. Program ma wbudowany mechanizm kolorowania składni, który ułatwia szybką modyfikację kodu źródłowego. Mnogość dostępnych wtyczek znacznie zwiększa jego funkcjonalność i poprawia komfort pracy.

Otwórz stronę <https://notepad-plus-plus.org> i zainstaluj program — wystarczy kliknąć przycisk *Download*. Jeśli wszystko przebiegnie poprawnie, w menu i na pulpicie pojawi się ikona programu (o ile zaznaczyłeś tę opcję podczas instalacji).

Wykonaj następujące czynności:

1. Uruchom program Notepad++ i wpisz poniższy kod źródłowy:

```
<?php
print "Notepad++ gotowy do pracy!";
```

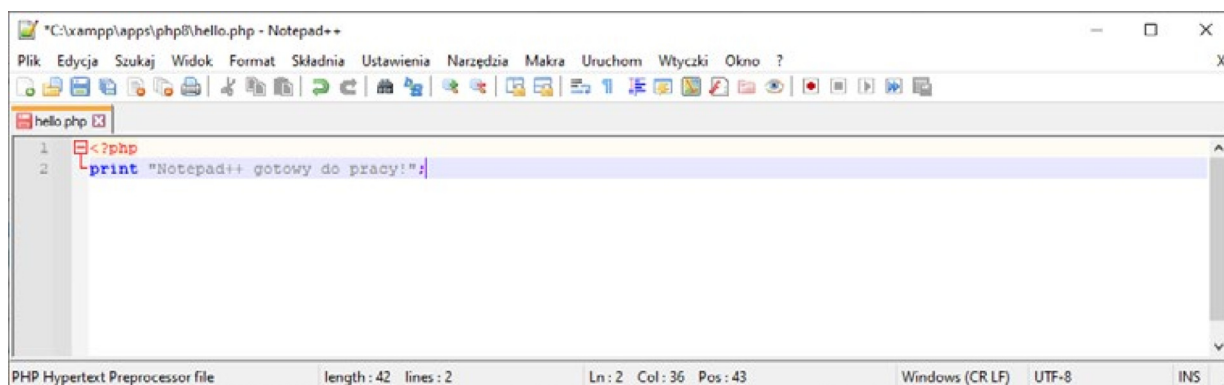
2. Następnie zapisz go w pliku na dysku pod dowolną nazwą, jak choćby `hello.php`. Pliki możesz zapisywać w katalogu, do którego będziesz miał szybki dostęp z konsoli, np.: `c:\xampp\apps\php8\htdocs`.
3. Po wpisaniu w konsoli poniższego polecenia wydrukuje się napis *Notepad++ gotowy do pracy!*.

```
cd /d c:\xampp\apps\php8\htdocs ↵
php hello.php ↵
```

4. Jeśli nie dodałeś katalogu z interpreterem PHP do zmiennej środowiskowej PATH, musisz wpisać pełną ścieżkę, np.: `c:\xampp\php\php.exe hello.php ↵`.

Instalacja zakończona!

Rysunek 2.7 przedstawia gotowy do pracy program Notepad++.



Rysunek 2.7. Edytor Notepad++ gotowy do pisania programów w języku PHP

Wskazówki

Możliwe, że większość czynności w trakcie tej lekcji będziesz wykonywał po raz pierwszy. Wbrew pozorom jest to niezwykle ważna lekcja, dlatego postaraj się być bardzo precyzyjny.

Pamiętaj zatem:

- czytaj dokładnie, najlepiej ze zrozumieniem, jeśli trzeba kilka razy;
- adresy internetowe i polecenia przepisuj bardzo dokładnie, każdy znak ma znaczenie;
- prawy ukośnik (slash) / to nie to samo co lewy ukośnik \ (backslash);
- pracując w konsoli, nie próbuj używać myszki, nic nią nie zrobisz;
- *katalog* oznacza to samo co *folder*;
- ↵ oznacza: wciśnij klawisz *Enter*;
- jeśli `php -v ↵` nie działa, sprawdź `c:\xampp\php\php.exe -v ↵`;
- konsolę uruchamiasz klawiszami **⌘ + R** i poleceniem `cmd ↵`.

Podczas instalacji i konfiguracji oprogramowania mogą wystąpić problemy związane z prawami użytkownika, a wtedy niezbędna będzie pomoc administratora. Nie staraj się instalować programów na siłę, lepiej zrób to pod okiem osoby bardziej doświadczonej.

Zalecam dodanie ścieżki `c:\xampp\php` w zmiennej środowiskowej `PATH`. Ułatwi to dostęp do interpretera PHP w linii poleceń (konsoli) po wpisaniu tylko nazwy `php`. Wpisz w konsoli `php -v ↵`, a dowiesz się, z której wersji PHP korzystasz.

Ćwiczenia

1. Zainstaluj i skonfiguruj środowiska programistyczne Eclipse, Apache NetBeans i Visual Studio Code, a następnie przeklikaj wszystkie dostępne opcje. Jeśli coś zepsujesz, nie martw się, zawsze możesz zainstalować programy ponownie.
2. Zapoznaj się z pakietem XAMPP, uruchom serwer baz danych MySQL i otwórz w przeglądarce panel administracyjny phpMyAdmin. Spróbuj założyć nową bazę danych i przypisać do niej nowego użytkownika. Jeśli coś zepsujesz, nie martw się, zawsze możesz zainstalować pakiet ponownie.
3. Aktywuj w pliku konfiguracyjnym `c:\xampp\php\php.ini` następujące biblioteki programistyczne: `gd`, `gmp`, `openssl`, `sockets`.
4. W przedmowie nie wspomniałem o PHP 6. Jak myślisz dlaczego? Poszukaj więcej informacji na ten temat.

Pytania

Dlaczego muszę się rejestrować w serwisie Oracle, pobierając Javę?

Firma Oracle, począwszy od 16 kwietnia 2019 roku, zmieniła zasady licencjonowania technologii Java — sprecyzowała, który typ użytkownika będzie nadal bezpłatny, a który objęty zostanie opłatą.

W Twoim przypadku, a więc do osobistego użytku, oprogramowanie Java jest dostępne bezpłatnie. Dlatego możesz śmiało zainstalować Javę w wersji od 8 do 16. Pamiętaj, aby po zakończeniu instalacji sprawdzić, czy ścieżka do Javy znajduje się w zmiennej środowiskowej PATH.

Aby sprawdzić, czy wszystko działa poprawnie, naciśnij klawisze  + R i wpisz `cmd`, a po uruchomieniu się konsoli wpisz `java -version` ↵. Jeśli na Twoim komputerze wyświetla się informacja o zainstalowanej wersji Javy (rysunek 2.4), możesz zamknąć konsolę przez wpisanie `exit`. Jeśli nie, sprawdź poprawność instalacji.

Jak uruchamiać programy w języku PHP?

W programie Eclipse kod PHP uruchamiasz przez wybranie z menu *Run/Run as/PHP CLI Application*. Pamiętaj, aby wcześniej utworzyć plik, wybierając z menu *File/New/PHP File*. Wynik działania programu zobaczysz w konsoli (zakładka *Console* w dolnym oknie Eclipse).

W programie Apache NetBeans kod PHP uruchamiasz przez naciśnięcie klawiszy *Shift+F6* lub wybranie z menu *Run/Run Project*. Wynik działania programu zobaczysz w oknie *Output* w dolnej części Apache NetBeans.

W programie Visual Studio Code kod PHP uruchamiasz, klikając ikonę *Run Code*, widoczną w prawym górnym rogu programu, lub z menu kontekstowego dostępnego po kliknięciu prawego przycisku myszki. W oknie *Output* w dolnej części Visual Studio Code zobaczysz wynik działania programu.

W programie Notepad++ zapisz plik z rozszerzeniem *.php* na dysku. Uruchom konsolę i przejdź do katalogu z zapisanym plikiem. Uruchom program poleceniem `php nazwa_pliku.php` ↵.

Pamiętaj o dodaniu katalogu z interpreterem PHP do zmiennej środowiskowej PATH. Ilekroć będę wspominał o tym, abyś utworzył plik z kodem źródłowym, uruchomił go i zobaczył wynik w konsoli, będę miał na myśli właśnie takie działania jak te, które opisałem powyżej.

W Eclipse nie mogę wpisać polskiej litery „ś”. Co mam zrobić?

Z tym problemem często muszą się mierzyć programiści używający narzędzi, w których dostęp do wielu funkcji bazuje na skrótach klawiszowych. Najprawdopodobniej skrót *Alt+S* ma już przypisaną funkcję, która blokuje możliwość obsługi polskich znaków.

W Eclipse wykonaj następujące czynności:

1. Wybierz z menu *Window/Preferences* i w nowo otwartym oknie wybierz *General/Keys*.
2. W polu tekstowym (*type filter text*) wpisz tekst `ctrl+alt+s php` (nie naciskaj tych klawiszy, tylko wpisz tekst). Powinna się pojawić tylko jedna pozycja — *Show Source Quick Menu*.
3. Kliknij przycisk *Unbind Command*, a następnie *Apply and Close*.
4. Spróbuj teraz wpisać w Eclipse śśśśmiiig — i gotowe!
5. Upewnij się, czy możesz wpisać wszystkie polskie znaki.

Podobne problemy mogą wystąpić w innych programach, dlatego aby je rozwiązać, należy zmienić przypisanie skrótów klawiszowych.

W programie Apache NetBeans skróty klawiszowe zmienisz, wybierając z menu *Tools/Options/KeyMap*. Natomiast w programie Visual Studio Code wybierz z menu *File/Preferences/Keyboard Shortcuts*.



Klasyka, czyli pierwszy program

Tak naprawdę pierwszy program już napisałeś i uruchomiłeś, konfigurując swoje środowisko programistyczne. Przepisz i uruchom kod z listingu 3.1. Z pewnością zauważyłeś, że kod PHP rozpoczyna się znacznikiem `<?php` i kończy `?>`, a pomiędzy nimi znajduje się `print`, pierwsza funkcja w PHP, jaką poznałeś.

LISTING 3.1	PHP 7/8	code_03_01.php
01	<code><?php</code>	
02	<code>print "Zabieram się do pracy!";</code>	
03	<code>?></code>	
<code>\$ php code_03_01.php ↵</code>		KONSOLA
Zabieram się do pracy!		

Ogólna zasada jest taka, że kod PHP należy umieszczać pomiędzy znacznikiem otwierającym `<?php` a znacznikiem zamykającym `?>`. Cała reszta będzie traktowana jako zwykły tekst i drukowana na wyjściu, czyli np. w konsoli czy przeglądarce internetowej. Jest to bardzo ważna funkcja interpretera PHP, którą wykorzystasz podczas łączenia kodu PHP z kodem HTML.

W przypadku gdy plik z programem będzie zawierał tylko kod PHP, w zupełności wystarczy znacznik otwierający `<?php`.

W języku PHP istotną rolę odgrywa znak średnika, który oznacza koniec instrukcji.

Wszystkie listingi przepisuj bez numerów linii; umieściłem je w kodzie po to, abym mógł się do nich później w opisach łatwo odnieść. Pliki z kodem źródłowym możesz zapisywać pod dowolną nazwą, możesz też wykorzystać moje propozycje, które znajdują się nad listingiem, z prawej strony.

Pamiętaj o rozszerzeniu `.php` dla plików.

Pod listingiem umieściłem wynik jego działania, czyli to, co powinieneś zobaczyć w konsoli lub w przeglądarce internetowej. Możesz już zacząć kodować!

Ćwiczenie 1. Drukowanie

Rozszerzymy trochę program z listingu 3.1. Zwróć uwagę na kilka wariantów wykorzystania funkcji `print` — każdy z nich jest prawidłowy. Pamiętaj, że tekst musisz umieszczać pomiędzy cudzysłowami tego samego typu.

LISTING 3.2	PHP 7/8	code_03_02.php
01	<?php	
02	print "Uśmiechaj ";	
03	print 'się ';	
04	print ("każdego ");	
05	print ('dnia!');	
\$ php code_03_02.php ↵		KONSOLA
Uśmiechaj się każdego dnia!		

Ćwiczenie 2. Znaki ucieczki

Teraz poznasz znaki ucieczki i wykorzystasz je do formatowania tekstu. Jak zauważyłeś, funkcja `print` drukuje tekst jeden za drugim, co nie zawsze jest pożądane. Do bardziej eleganckiego formatowania tekstu możesz wykorzystać poniższe znaki ucieczki. Wypisałem te, których podczas kodowania w języku PHP będziesz używał najczęściej.

Znak ucieczki	Znaczenie
\'	pojedynczy cudzysłów
\"	podwójny cudzysłów
\n	przejsie do nowej linii
\r	powrót karetki (powrót kursora do początku linii)
\t	tabulator

Przepisz kod z listingu 3.3 i uruchom.

LISTING 3.3	PHP 7/8	code_03_03.php
01	<?php	
02	print "Uczestnicy szkolnego konkursu \"Mistrz Programowania\":\n";	
03	print "\tklasy 1-4:\t5 uczniów\n";	
04	print "\tklasy 5-8:\t12 uczniów\n";	
05	print 'Wszyscy \'koderzy\' świetnie się spisali!';	

```
$ php code_03_03.php ↵
```

KONSOLA

```
Uczestnicy szkolnego konkursu "Mistrz Programowania":
    klasy 1-4:      5 uczniów
    klasy 5-8:      12 uczniów
Wszyscy 'koderzy' świetnie się spisali!
```

Ćwiczenie 3. Tekst poza kodem PHP

A teraz zobaczysz, co się stanie, kiedy wstawisz tekst poza kodem PHP, czyli pomiędzy znacznikiem otwierającym `<?php` a znacznikiem zamykającym `?>` (listing 3.4). Zauważ, że interpreter PHP potraktuje linie 01 i 05 jako zwykły tekst i wydrukuje go w konsoli, a wiersze od 02 do 04 potraktuje jako kod w języku PHP i postara się wykonać. Tę funkcjonalność wykorzystasz później do łączenia kodu PHP z HTML-em przy budowie swojej pierwszej strony internetowej.

LISTING 3.4

PHP 7/8

code_03_04.php

```
01 Znaczniki języka HTML, które interpreter PHP potraktuje jak zwykły tekst.
02 <?php
03 print "Tekst wydrukowany przez funkcję print.".PHP_EOL;
04 ?>
05 print "A tutaj chyba nie :(";
$ php code_03_04.php ↵
```

KONSOLA

```
Znaczniki języka HTML, które interpreter PHP potraktuje jak zwykły tekst.
Tekst wydrukowany przez funkcję print.
print "A tutaj chyba nie :(";
```

W wierszu 03 dodałem stałą `PHP_EOL`, która oznacza znak końca linii (ang. *End Of Line*). W zależności od systemu operacyjnego znak przejścia do nowej linii może być różny, dlatego wykorzystanie `PHP_EOL` ułatwia pisanie uniwersalnego kodu.

Ćwiczenie 4. Błędy z cudzysłowami

No dobrze, ale co się stanie, jeśli się pomylisz? W kodzie z listingu 3.5 umieściłem błąd. Czy potrafisz go odnaleźć? Nie martw się, interpreter PHP powiadomi Cię o tym stosownym komunikatem. Pamiętaj, aby z komunikatu wywnioskować, w której linii kodu jest problem, i szybko go naprawić!

LISTING 3.5	PHP 7/8	code_03_05.php
01	<?php	
02	print "Houston, mamy problem";	
\$ php code_03_05.php ↵		KONSOLA
PHP Parse error: syntax error, unexpected end of file, expecting variable or "\${" or "{\$" in C:\xampp\apps\php8\htdocs\code_03_05.php on line 3		

Ćwiczenie 5. Drukowanie sformatowane

Drukowanie tekstu opanowałeś już w ćwiczeniu 1, teraz czas na coś bardziej zaawansowanego, a mianowicie na drukowanie sformatowanych ciągów znaków. W języku PHP dostępna jest funkcja `printf`, drukująca tekst w różnych formatach, w zależności od argumentów formatujących. Specyfikacja formatu zaczyna się od znaku `%` i typu, np. format `%b` oznacza format binarny (dwójkowy).

Możliwości funkcji `printf` są bardzo duże, a ich opis zająłby pewnie kilka następnych stron, dlatego na potrzeby tego ćwiczenia przedstawię jedynie najważniejsze formaty:

Typ	Format
<code>%b</code>	binarny (dwójkowy)
<code>%e</code>	naukowy
<code>%f</code>	rzeczywisty (zmiennoprzecinkowy)
<code>%o</code>	ósemkowy
<code>%s</code>	tekstowy
<code>%x, %X</code>	szesnastkowy (heksadecymalny)

Przepisz kod z listingu 3.6 i uruchom. Nowością jest pojawienie się zmiennej `$liczba` w linii 02. W następnych rozdziałach dowiesz się więcej o zastosowaniu zmiennych.

LISTING 3.6	PHP 7/8	code_03_06.php
01	<?php	
02	\$liczba = 2021;	
03		
04	printf("%b = '%b'".PHP_EOL, \$liczba); //format binarny (dwójkowy)	
05	printf("%e = '%e'".PHP_EOL, \$liczba); //format naukowy	
06	printf("%f = '%f'".PHP_EOL, \$liczba); //format zmiennoprzecinkowy	
07	printf("%o = '%o'".PHP_EOL, \$liczba); //format ósemkowy	

```
08 printf("%s = '%s'".PHP_EOL, $liczba); //format tekstowy
09 printf("%x = '%x'".PHP_EOL, $liczba); //format szesnastkowy (heksadecymalny)
10
11 printf("Rok %d w formacie szesnastkowym to %X.", $liczba, $liczba);
```

```
$ php code_03_06.php ↵
```

KONSOLA

```
%b = '11111100101'
%e = '2.021000e+3'
%f = '2021.000000'
%o = '3745'
%s = '2021'
%x = '7e5'
Rok 2021 w formacie szesnastkowym to 7E5.
```

Ćwiczenia dodatkowe

1. Otwórz stronę <http://php.net> i wyszukaj opis funkcji print i echo. Spróbuj przepisać i uruchomić przykładowe listingi. Jaka jest różnica między tymi funkcjami?
2. Napisz program składający się z 10 linii kodu. Wykorzystaj printf i poznane znaki ucieczki.

Wskazówki

Znaków ucieczki \ (pojedynczy cudzysłów) i \" (podwójny cudzysłów) użyjesz, kiedy będziesz musiał wydrukować w tekście cudzysłów, np. *Pismo „Pies i kot” jest ciekawe*. Prawidłowe wykorzystanie funkcji print przedstawia listing 3.7.

LISTING 3.7

PHP 7/8

code_03_07.php

```
01 <?php
02 print "Pismo \"Pies i kot\" jest ciekawe.";
```

```
$ php code_03_07.php ↵
```

KONSOLA

```
Pismo "Pies i kot" jest ciekawe.
```

Znaki ucieczki \n, \r, \t będziesz wykorzystywał przede wszystkim do drukowania w konsoli. W przeglądarce internetowej do prezentacji tekstu będziesz stosował znaczniki HTML.

W dokumentacji PHP znajdziesz informację, że `print` nie należy traktować jako prawdziwej funkcji, lecz jako konstrukcję języka, dlatego też nie musisz używać nawiasów. Pomimo to dla uproszczenia w tej książce konstrukcję `print` będę nazywał funkcją.

Pytania

O co chodzi w ćwiczeniu 4?

Tekst musi być zawarty pomiędzy cudzysłowami tego samego typu. Aby naprawić błąd, należy poprawić cudzysłów otwierający (zmienić na pojedynczy) lub zamykający (zmienić na podwójny). Więcej informacji na ten temat znajdziesz w ćwiczeniu 1.

Czym jest listing?

Listing to inaczej kod źródłowy.



Komentarz mile widziany

Podczas pisania kodu często umieszcza się dodatkowe informacje (komentarze) dla osób, które chciałyby się dowiedzieć, co autor miał na myśli. Sam często umieszczam w swoim kodzie dodatkowe informacje dla siebie, zwłaszcza kiedy kod jest skomplikowany, abym w przyszłości mógł go szybko zrozumieć.

Zasada umieszczania komentarzy w kodzie jest bardzo prosta. Nie komentuj, co dany kod robi, tylko jaka jest przyczyna, że tak jest skonstruowany. Kiedy będziesz chciał wykorzystać swój kod w nowym projekcie, komentarze mogą być bardzo pomocne.

W języku PHP masz do dyspozycji komentarze liniowe (znak # lub dwa prawe ukośniki //) i blokowe (/*...*/). Liniowe zajmują przestrzeń jednej linii, natomiast blokowe mogą się składać z wielu. Pamiętaj, że komentarze są przez interpreter języka PHP ignorowane i nie wpływają na działanie programu.

Ćwiczenie 1. Komentarze blokowy i liniowy

Poniższy listing demonstruje poprawne wykorzystanie komentarzy.

LISTING 4.1	PHP 7/8	code_04_01.php
01	<?php	
02	/*	
03	<i>To jest komentarz blokowy, w którym</i>	
04	<i>możesz umieścić wiele cennych informacji</i>	
05	<i>dla następnych pokoleń programistów.</i>	
06	*/	
07	print "Lubię programować!"; # To jest komentarz liniowy dla krótkich informacji.	
\$ php code_04_01.php ↵		KONSOLA
Lubię programować!		

Zwróć uwagę, że na listingu 4.1 komentarz blokowy jest ujęty pomiędzy liniami 02 i 06, natomiast komentarz liniowy jest umieszczony w wierszu 07.

Ćwiczenie 2. Sterowanie wykonywaniem kodu

Zauważ, że na listingu 4.2 wszystko, co znajduje się za znakami # i //, jest ignorowane. Znaki te często stosuje się do tymczasowego oznaczenia kodu, który nie ma być wykonywany.

LISTING 4.2	PHP 7/8	code_04_02.php
01	<?php	
02	#print "Lubię programować!"; // To się nie wydrukuje. Dlaczego?	
03	print "A to już się wydrukuje.".PHP_EOL;	
04	/* Teraz nastąpi kolejne drukowanie. */	
05	print "I to też.";	
\$ php code_04_02.php ↵		KONSOLA
A to już się wydrukuje. I to też.		

A teraz mała ciekawostka. Dawno temu, kiedy programiści pisali programy w języku BASIC, komentarze poprzedzało się słowem REM (od angielskiego słowa *Remark*), stąd popularne określenie „zaremuj kod”, które możesz czasem usłyszeć u starych, przepraszam, u dojrzałych programistów. Słowo REM nadal występuje np. w języku Visual Basic. Dodam tylko, że nauka języków niskiego poziomu, np. assemblera, bez odpowiednich komentarzy jest praktycznie niemożliwa.

Ćwiczenie 3. Jak nie komentować kodu

Nadmiarowe i zbędne komentarze mogą zagmatwać kod programu i utrudnić jego analizę. Na listingu 4.3 pokazałem, jak nie należy komentować kodu.

LISTING 4.3	PHP 7/8	code_04_03.php
01	<?php	
02	/*	
03	<i>Drukowanie tekstu funkcją print.</i>	
04	<i>Poniższy kod wydrukuje dwie linie tekstu, wykorzystując funkcję print.</i>	
05	<i>Program po wydrukowaniu dwóch linii tekstu zakończy swoją ciężką pracę.</i>	
06	*/	
07	// drukowanie tekstu "Ło matko! Po co te komentarze powyżej!"	
08	print "Ło matko! Po co te komentarze powyżej!".PHP_EOL;	
09	print "Po co ten komentarz z prawej strony?".PHP_EOL; // drukowanie tekstu	

```
$ php code_04_03.php ↵
```

KONSOLA

```
Ło matko! Po co te komentarze powyżej!  
Po co ten komentarz z prawej strony?
```

Ćwiczenia dodatkowe

1. Sprawdź w sieci, jakie nazwy mogą występować dla znaku kratki (#).
2. Wykorzystaj komentarz blokowy do wyłączenia funkcji print w programie z ćwiczenia 1.
3. Usuń komentarze w programie z ćwiczenia 2, które według Ciebie są zbędne. Czy bez nich program jest dla Ciebie nadal zrozumiały?

Wskazówki

Nie ma reguły, jaki styl komentowania jest najlepszy, to zależy tylko od Ciebie. Komentarzem blokowym bardzo szybko możesz wyłączyć znaczną część kodu, jednak możesz to zrobić również przez wstawienie znaku kratki na początku każdej linii.

Pamiętaj, dobrze napisany kod w PHP obroni się bez dodatkowych komentarzy.

Pytania

Ile nazw ma znak kratki?

No tak, najlepiej, gdyby znak ten miał tylko jedną nazwę, jednak nie wiem, czy wyrażenie „znak kratki” byłoby zrozumiałe również w innym języku. Przyjmuje się, że wśród programistów znak ten nazywa się po prostu hashem — i tak właśnie ja go nazywam. Możesz też przeczytać o płotku, jednak najlepiej, jeśli będziesz np. „wstawiał hash na początku linii”.

Czy muszę przepisywać komentarze z listingów?

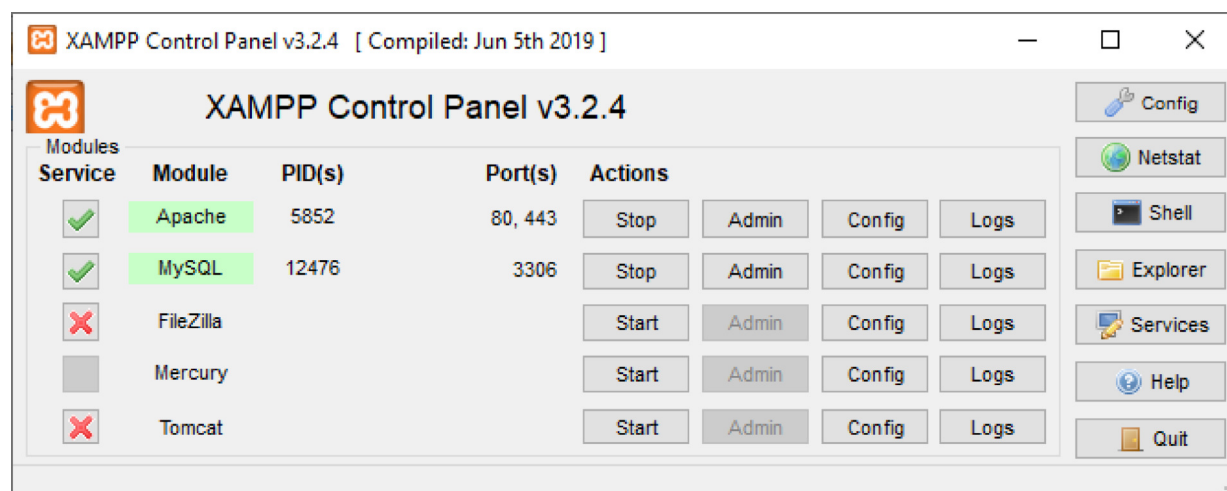
Oczywiście, że nie. Komentarze są tylko dodatkową informacją ode mnie i nie mają żadnego wpływu na działanie programu.



Co oferują PHP i MySQL

Jeśli wykonałeś ćwiczenia z poprzednich dwóch rozdziałów, wiesz już, jak wygląda podstawowa składnia języka PHP oraz jak umieszczać dodatkowe informacje w kodzie programu, ułatwiające jego późniejszą analizę. Teraz wykorzystasz pakiet XAMPP do sprawdzenia, jakie funkcjonalności oferuje zainstalowany interpreter języka PHP, a także utworzysz pierwszą bazę danych MySQL, wykorzystując narzędzie phpMyAdmin¹⁷.

Otwórz panel kontrolny XAMPP, a następnie uruchom moduł serwera Apache i baz danych MySQL. Rysunek 5.1 przedstawia poprawne uruchomienie tych modułów.



Rysunek 5.1. Panel kontrolny aplikacji XAMPP z uruchomionymi modułami Apache i MySQL

¹⁷ phpMyAdmin, <https://www.phpmyadmin.net>.

Ćwiczenie 1. Co kryje się pod maską PHP?

Język PHP oferuje wiele ciekawych i funkcjonalnych bibliotek, z których możesz korzystać w swoich programach. Nie wszystkie wersje PHP zapewniają ten sam zestaw bibliotek, dlatego w tym ćwiczeniu sprawdzisz, z jakich funkcji możesz skorzystać. Naciśnij klawisze  +R i wpisz cmd, a po uruchomieniu się konsoli wpisz polecenie `php -m ↵`.

```
[PHP Modules]
bcmath
calendar
Core
ctype
date
dom
filter
hash
iconv
json
libxml
mbstring
mysqli
mysqlnd
pcre
PDO
Phar
readline
Reflection
session
SimpleXML
SPL
sqlite3
standard
tokenizer
xml
xmlreader
xmlwriter
zip
zlib
```

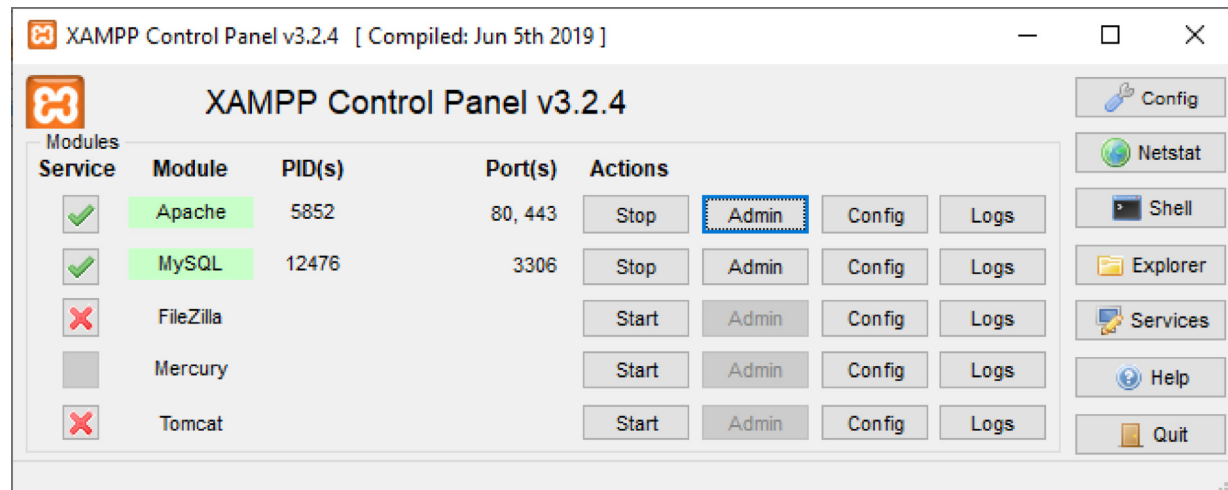
Oczywiście, w Twoim przypadku lista dostępnych bibliotek może być inna (w praktyce dostępnych jest ich więcej), gdyż zależy to od wersji interpretera PHP, systemu, serwera czy reguł bezpieczeństwa. Ważne, żebyś wiedział, jak szybko sprawdzić, co Twoje PHP ma pod maską.

Ćwiczenie 2. Więcej informacji o PHP

W języku PHP dostępna jest specjalna funkcja `phpinfo`, która pozwala uzyskać wiele szczegółowych informacji na temat środowiska pracy na serwerze, takich jak m.in. wersja interpretera języka PHP, lista zmiennych środowiskowych oraz innych parametrów, które są istotne podczas projektowania serwisu WWW. W praktyce pierwszym etapem podczas instalacji serwisu WWW jest sprawdzenie, która wersja języka PHP jest dostępna na serwerze oraz z jakich bibliotek może skorzystać projektant strony. Informacje te można uzyskać w konsoli poleceniem `php -i` lub w przeglądarce za pomocą kodu z listingu 5.1.

LISTING 5.1	PHP 7/8	code_05_01.php
01	<?php	
02	<code>phpinfo();</code>	

Informacje wyświetlone w konsoli dla początkującego programisty PHP mogą nie być do końca zrozumiałe, dlatego wygodniej jest je uzyskać za pomocą przeglądarki internetowej. W panelu kontrolnym XAMPP, przy aktywnym module Apache, kliknij przycisk *Admin* (rysunek 5.2).



Rysunek 5.2. Aktywacja panelu administracyjnego serwera Apache w programie XAMPP

Powinieneś zobaczyć stronę informacyjną pakietu XAMPP, którą rozpoznasz po logotypie i tytule *XAMPP Apache + MariaDB + PHP + Perl*. Z górnego menu wybierz link *PHPInfo* — zobaczysz szczegółową informację techniczną o zainstalowanej wersji interpretera języka PHP (rysunek 5.3).

PHP Version 8.0.3	
	
System	Windows NT MARIUSZ-PC 10.0 build 19042 (Windows 10) AMD64
Build Date	Mar 2 2021 23:26:36
Build System	Microsoft Windows Server 2016 Standard [10.0.14393]
Compiler	Visual C++ 2019
Architecture	x64
Configure Command	<code>cscript /nologo /e:jscript configure.js "--enable-snapshot-build" "--enable-debug-pack" "--with-pdo-oci=c:\php-snap-build\dep-aux\oracle\x64\instantclient_19_9\sdk,shared" "--with-oci8-12c=c:\php-snap-build\dep-aux\oracle\x64\instantclient_12_1\sdk,shared" "--with-oci8-19=c:\php-snap-build\dep-aux\oracle\x64\instantclient_19_9\sdk,shared" "--enable-object-out-dir=.\obj" "--enable-com-dotnet=shared" "--without-analyzer" "--with-pgo"</code>
Server API	Apache 2.0 Handler
Virtual Directory Support	enabled
Configuration File (php.ini) Path	no value
Loaded Configuration File	C:\xampp\php\php.ini
Scan this dir for additional .ini files	(none)
Additional .ini files parsed	(none)
PHP API	20200930
PHP Extension	20200930
Zend Extension	420200930
Zend Extension Build	API420200930,TS,VS16
PHP Extension Build	API20200930,TS,VS16
Debug Build	no
Thread Safety	enabled
Thread API	Windows Threads
Zend Signal Handling	disabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
IPv6 Support	enabled
DTrace Support	disabled
Registered PHP Streams	php, file, glob, data, http, ftp, zip, compress.zlib, compress.bzip2, https, ftps, phar
Registered Stream Socket Transports	tcp, udp, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2, tlsv1.3
Registered Stream Filters	convert.iconv.*, string.rot13, string.toupper, string.tolower, convert.*, consumed, dechunk, zlib.*, bzip2.*

Rysunek 5.3. Informacja o zainstalowanej wersji interpretera języka PHP

Przeglądając informacje o aktualnej konfiguracji, zwróć uwagę, jakie biblioteki są dostępne dla programisty. Nie przejmuj się, jeśli nie wszystkie informacje będą dla Ciebie zrozumiałe, po prostu wiedz, że zawsze możesz zajrzeć pod maskę PHP.

Ćwiczenie 3. Zakres liczb w PHP

W tym ćwiczeniu (listing 5.2) sprawdzisz podstawowe informacje o zainstalowanej wersji PHP. Dowiesz się również, jaki zakres liczbowy możesz nadać zmiennym. Oznaczenie INT dotyczy liczb całkowitych, natomiast oznaczenie FLOAT odnosi się do liczb rzeczywistych. Więcej informacji na ten temat znajdziesz na stronie <https://www.php.net/manual/en/reserved.constants.php>.

LISTING 5.2	PHP 7/8	code_05_02.php
01	<?php	
02	print "Moja wersja PHP: ". PHP_VERSION .PHP_EOL;	
03	print "Mój system operacyjny: ". PHP_OS. " (" . PHP_OS_FAMILY. ") " .PHP_EOL;	
04	print "Minimalna wartość typu INT: ". PHP_INT_MIN .PHP_EOL;	
05	print "Maksymalna wartość typu INT: ". PHP_INT_MAX .PHP_EOL;	
06	print "Minimalna wartość typu FLOAT: ". PHP_FLOAT_MIN .PHP_EOL;	
07	print "Maksymalna wartość typu FLOAT: ". PHP_FLOAT_MAX;	
\$ php code_05_02.php ↵		KONSOLA
Moja wersja PHP: 8.0.3 Mój system operacyjny: WINNT (Windows) Minimalna wartość typu INT: -9223372036854775808 Maksymalna wartość typu INT: 9223372036854775807 Minimalna wartość typu FLOAT: 2.2250738585072E-308 Maksymalna wartość typu FLOAT: 1.7976931348623E+308		

Ćwiczenie 4. Stałe magiczne w PHP

Język PHP dostarcza dziewięć stałych specjalnych: `__LINE__`, `__FILE__`, `__DIR__`, `__FUNCTION__`, `__CLASS__`, `__TRAIT__`, `__METHOD__`, `__NAMESPACE__` i `class::class`, zwanych stałymi magicznymi, których wartość jest uzależniona od konkretnego miejsca ich wykorzystania w kodzie. Wartości stałych magicznych definiowane są w trakcie procesu kompilacji kodu. Dla początkującego programisty znajomość pierwszych trzech stałych magicznych w zupełności wystarczy.

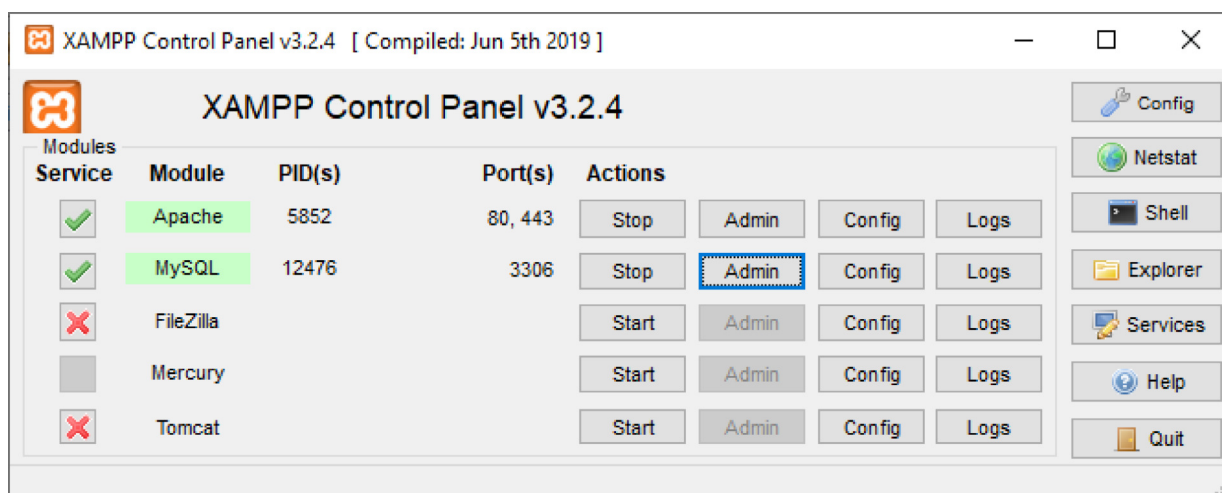
Przykładowo wartość stałej `__LINE__` jest uzależniona od wiersza kodu, w którym stała ta jest wykorzystana. Przepisz i uruchom kod z listingu 5.3. Pamiętaj, że `__` oznacza dwa podkreślenia.

LISTING 5.3	PHP 7/8	code_05_03.php
01	<?php	
02	print "Nazwa pliku z całą ścieżką: ". __FILE__ .PHP_EOL;	
03	print "Nazwa katalogu, w którym plik się znajduje: ". __DIR__ .PHP_EOL;	
04	print "Numer tej linii w kodzie: ". __LINE__;	
\$ php code_05_03.php ↵		KONSOLA
Nazwa pliku z całą ścieżką: C:\xampp\apps\php8\htdocs\code_05_03.php Nazwa katalogu, w którym plik się znajduje: C:\xampp\apps\php8\htdocs Numer tej linii w kodzie: 4		

Ćwiczenie 5. Baza danych MySQL

Przyszłał dobry moment na to, aby zapoznać się z narzędziem phpMyAdmin, za pomocą którego utworzysz nową bazę danych MySQL. Dzisiejsze aplikacje webowe bardzo często wykorzystują bazę danych do składowania wielu różnych informacji, takich jak m.in. dane użytkowników, posty na blogu, produkty i zamówienia w sklepie internetowym, a nawet pliki muzyczne, graficzne i archiwa. Popularne dziś narzędzie phpMyAdmin, które powstało w 1998 roku, jest dostępne u prawie wszystkich operatorów hostingowych. Jeśli nauczysz się teraz jego obsługi, później dasz sobie świetnie radę, konfigurując nowy projekt webowy.

W XAMPP, przy aktywnym module MySQL, kliknij przycisk *Admin* (rysunek 5.4).

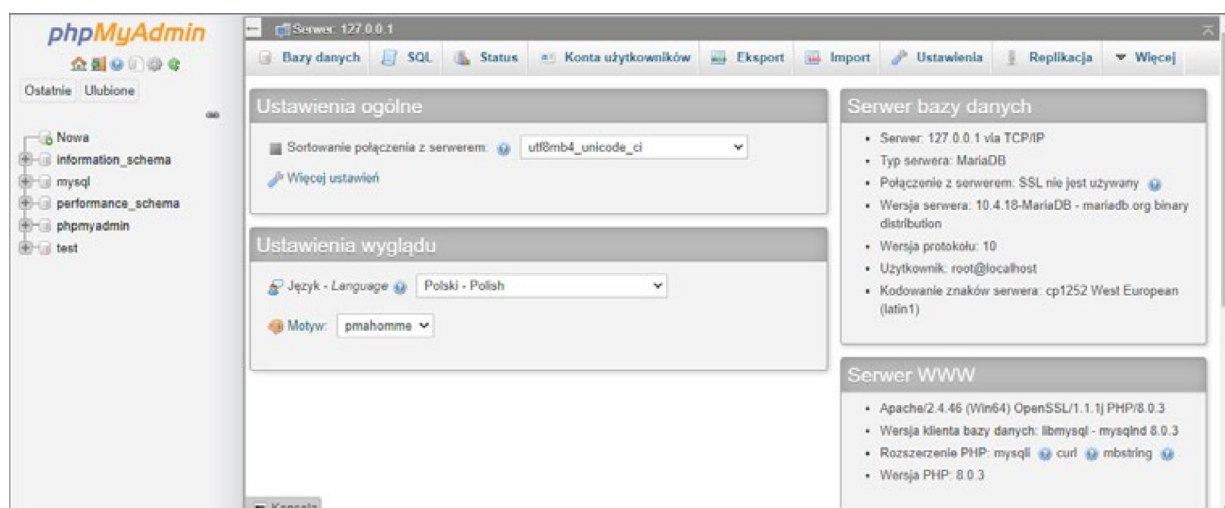


Rysunek 5.4. Uruchomienie narzędzia phpMyAdmin do zarządzania bazami danych MySQL

Powinieneś zobaczyć stronę główną narzędzia phpMyAdmin z widocznym w górnej części menu (rysunek 5.5), które wykorzystasz do zarządzania bazami danych.

Do każdej bazy należy przypisać użytkownika, który będzie miał prawo do odczytu, modyfikacji i zapisu nowych danych. Narzędzie phpMyAdmin umożliwia dodanie użytkownika z jednoczesnym utworzeniem bazy danych, do której użytkownik ten będzie miał pełny dostęp.

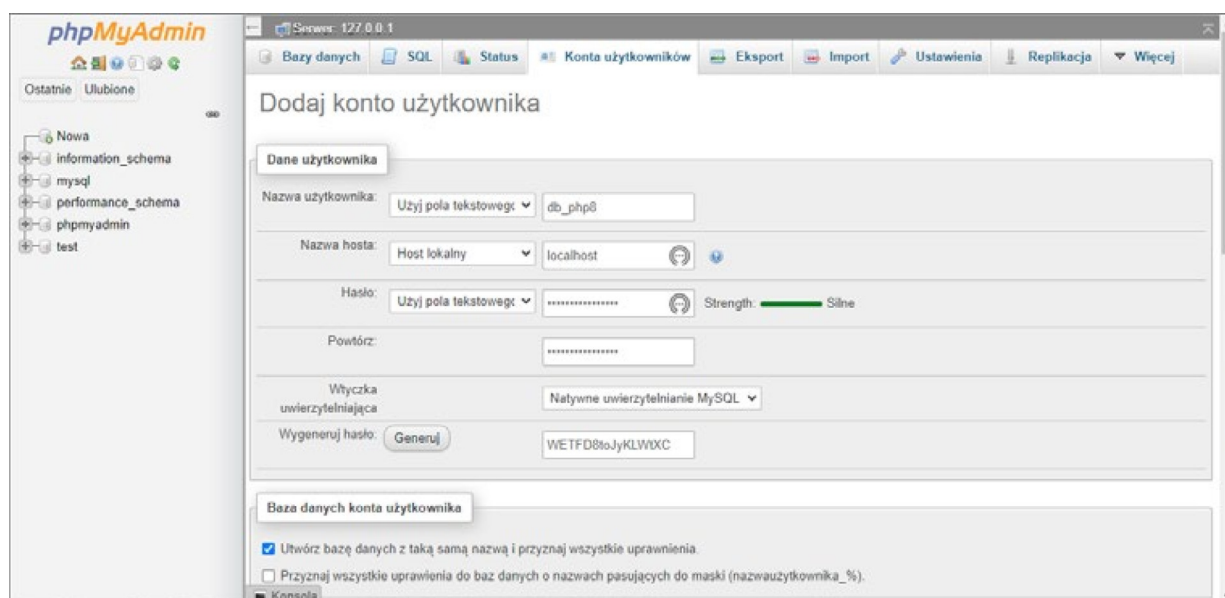
Wybierz z menu zakładkę *Konta użytkowników* i kliknij link *Dodaj konto użytkownika*. W sekcji *Dane użytkownika* wpisz wybrane przez siebie nazwę i hasło, a w nazwie hosta wybierz z listy *Host lokalny*. Następnie w sekcji *Baza danych konta użytkownika* zaznacz opcję *Utwórz bazę danych z taką samą nazwą i przyznaj wszystkie uprawnienia*. Potwierdź, klikając przycisk *Wykonaj* na dole strony.



Rysunek 5.5. Strona główna narzędzia phpMyAdmin

Ze względów bezpieczeństwa dostęp do bazy danych powinien być możliwy tylko z serwera lokalnego (localhost). Część operatorów hostingowych blokuje dostęp do bazy spoza dozwolonych adresów, np. umożliwia połączenie tylko z localhosta.

Rysunek 5.6 przedstawia procedurę dodania nowego użytkownika i bazy danych. Pamiętaj o tym, że wybrana przeze mnie nazwa db_php8 będzie zarówno nazwą użytkownika, jak i nazwą bazy danych.



Rysunek 5.6. Tworzenie nowego użytkownika i bazy danych MySQL w narzędziu phpMyAdmin

Dobrym pomysłem, aby nie zapomnieć nazwy użytkownika i hasła, jest utworzenie w katalogu roboczym pliku db_config.php z konfiguracją MySQL-a (wykorzystaj listing 5.4). Konfiguracja serwera MySQL przyda się w dalszej części niniejszej książki.

LISTING 5.4	PHP 7/8	db_config.php
01	<?php	
02	/* Konfiguracja serwera MySQL */	
03	\$db_server = 'localhost';	
04	\$db_name = 'db_php8';	
05	\$db_user = 'db_php8';	
06	\$db_passwd = 'WETFD8toJyKLWtXC'; // U Ciebie hasło będzie z pewnością inne :)	

A teraz coś trudniejszego. Pokażę Ci, jak utworzyć tabelę w bazie danych. Temat baz danych i języka SQL będzie omówiony w dalszej części książki, dlatego teraz przyjmij do wiadomości, że coś takiego istnieje.

Informacje będziesz umieszczał w tabelach, podstawowych obiektach bazy danych, które z kolei składają się z pól. Pola te mogą przechowywać dane różnego typu, np. tekst, liczby, daty. Jeśli korzystałeś z programu Excel, z pewnością tworzyłeś tabelki, które zapełniałeś danymi. Podobnie jest z tabelami w bazie danych MySQL.

Poniżej znajduje się tabela z przykładowymi danymi. Nagłówek tabeli jest oznaczony kolorem i zawiera następujące pola: *numer*, *imię*, *nazwisko* i *wiek*. Następnie w kolejnych wierszach (rekordach, krotkach) znajdują się dane.

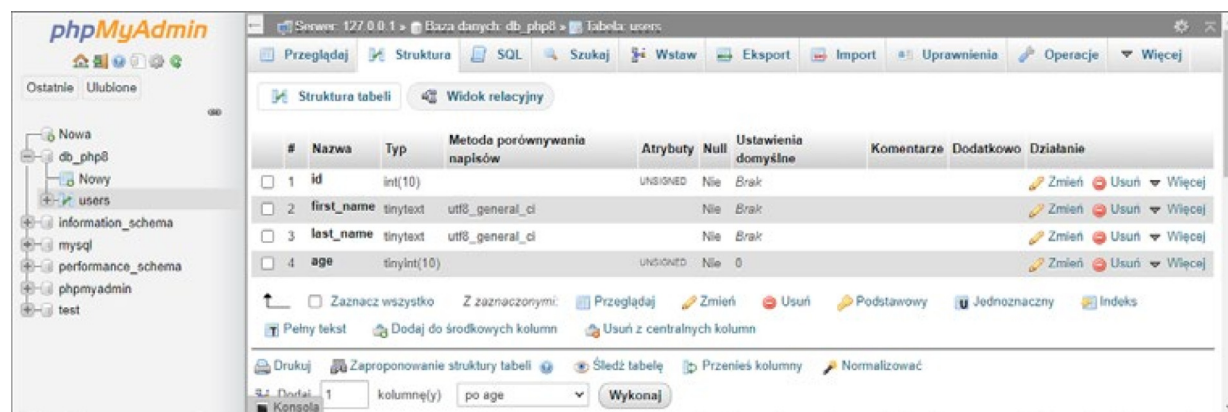
numer	imię	nazwisko	wiek
1	Adam	Kowalski	24
2	Ewa	Nowak	21

Na podstawie powyższych informacji możemy zaprojektować prostą tabelę na dane. Wiemy, że wartość pola *numer* jest liczbą i z każdym kolejnym rekordem zwiększa się o jeden, pola *imię* i *nazwisko* zawierają tekst, natomiast pole *wiek* — liczbę. Do utworzenia tabeli wykorzystasz język SQL (ang. *Structured Query Language*), którego podstawy znajdziesz w dalszej części niniejszej książki.

Z lewej strony panelu phpMyAdmin kliknij nazwę bazy danych, w której utworzysz nową tabelę. W górnej części okna, nad menu, zobaczysz adres serwera i nazwę wybranej bazy (np. *Serwer: 127.0.0.1 » Baza danych: db_php8*). Wybierz zakładkę *SQL*, wpisz następujący kod w języku SQL i kliknij przycisk *Wykonaj*.

```
CREATE TABLE `users` (
  `id` int(10) UNSIGNED NOT NULL,
  `first_name` tinytext NOT NULL,
  `last_name` tinytext NOT NULL,
  `age` tinyint(10) UNSIGNED NOT NULL DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

Gratuluję! Właśnie nauczyłeś się korzystać z narzędzia phpMyAdmin, dodałeś nowego użytkownika i bazę, a także utworzyłeś nową tabelę na dane. Rysunek 5.7 przedstawia strukturę tabeli `users`, którą przed chwilą dodałeś do bazy `db_php8`.



Rysunek 5.7. Prezentacja struktury tabeli w narzędziu phpMyAdmin

Informacyjnie dodam, że w kodzie w języku SQL użyłem angielskich nazw dla pól, ponieważ jest to standard wśród projektantów baz danych.

Szczegółowy opis silnika bazy danych MySQL, jego konfiguracji i optymalizacji pod kątem wydajności znacznie wybiega poza ramy niniejszej książki. Gorąco zachęcam Cię do pogłębienia tematu w tej dziedzinie, ponieważ prawie na pewno spotkasz się z tym zagadnieniem, projektując nowe systemy wykorzystujące bazy danych.

Ćwiczenia dodatkowe

1. Wykorzystując narzędzie phpMyAdmin, utwórz nowego użytkownika i bazę, np. `db_test`, a następnie wykorzystaj kod SQL z ćwiczenia 5 do utworzenia tabeli.
2. Poszukaj informacji na temat dostępnych typów danych w bazie MySQL. Skorzystaj ze strony projektu MariaDB (<https://mariadb.com/kb/en/data-types>) lub MySQL (<https://dev.mysql.com/doc/refman/8.0/en/data-types.html>).
3. Jeśli nadal nie dodałeś ścieżki do swojego interpretera PHP w zmiennej środowiskowej `PATH`, to czas najwyższy, aby to zrobić. To naprawdę nie jest skomplikowane.



PHP Composer

Tworząc projekty, prędzej czy później będziesz korzystał z pakietów napisanych przez innych programistów. Takie rozwiązanie ma wiele zalet. Po pierwsze, zaoszczędzisz czas na pisanie podobnego kodu, a po drugie, wykorzystasz kod przetestowany przez tysiące programistów. Może przyjdzie taki moment, że dojdiesz do wniosku, iż warto podzielić się swoim kodem z innymi, teraz jednak naucz się wykorzystywać pracę innych i szlifować swoje umiejętności. W niniejszym rozdziale nauczysz się instalować narzędzie Composer i pobierzesz pakiet oprogramowania Smarty¹⁸, który później wykorzystasz w swoim projekcie.

Pakiety, o których wyżej wspomniałem, to aplikacje napisane w języku PHP. Za pomocą narzędzia Composer możesz je pobrać i dołączyć do swojego projektu.

Narzędzie Composer zarządza zależnościami wykorzystywanych pakietów, które pobiera z centralnego repozytorium. Zaletą tego narzędzia jest to, że umożliwia ono automatyczne pobranie wszystkich zależności danego pakietu wchodzącego w skład Twojego projektu, a także dokonuje ich aktualizacji do najnowszych wersji.

Instalacja PHP Composer

Composer to aplikacja zapisana w specjalnym formacie `phar`, uruchamiana przez interpreter języka PHP. Narzędzie to jest uruchamiane w konsoli, dlatego ścieżka do niego powinna się znajdować w zmiennej środowiskowej `PATH`.

Otwórz stronę <https://getcomposer.org>, pobierz najnowszą wersję instalacyjną dla systemu Windows i zainstaluj. Podczas instalacji zostaniesz zapytany o lokalizację interpretera języka PHP. Możesz również wskazać ręcznie lokalizację pliku `php.exe`, wybierając przycisk *Browse*. Jeśli po zainstalowaniu pakietu XAMPP dodałeś ścieżkę

¹⁸ Smarty Template Engine, <https://www.smarty.net>.

do interpretera języka PHP, np. `c:\xampp\php`, w zmiennej środowiskowej `PATH`, powinieneś bez problemu zainstalować narzędzie Composer.

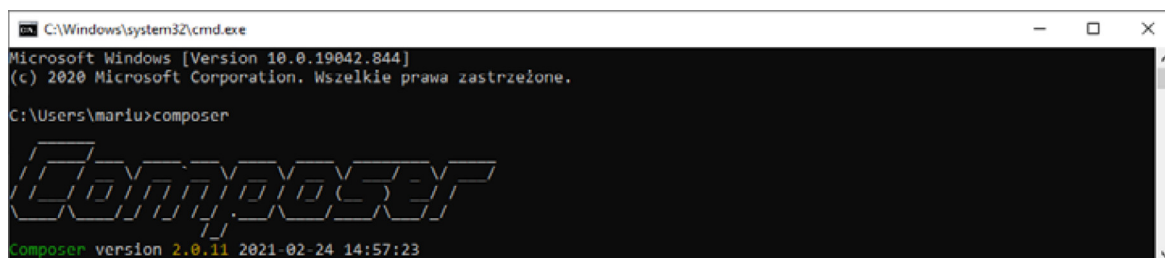
Instalator Composer dla systemu Windows automatycznie zaktualizuje zmienną środowiskową `PATH`, natomiast instalacja wykonana w konsoli wymaga ręcznej aktualizacji tej zmiennej.

Użytkownicy systemów operacyjnych innych niż Windows powinni zainstalować narzędzie Composer zgodnie ze wskazówkami dostępnymi na stronie. W systemie Unix czy Linux instalacja sprowadza się do wykonania następujących poleceń:

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');" ↵
HASH="$(wget -q -O - https://composer.github.io/installer.sig)" ↵
php -r "if (hash_file('SHA384', 'composer-setup.php') === '$HASH') { echo 'Installer verified'; } else { echo 'Installer corrupt'; unlink('composer-setup.php'); } echo PHP_EOL;" ↵
sudo php composer-setup.php --install-dir=/usr/local/bin --filename=composer ↵
```

Teraz czas na weryfikację poprawności instalacji. W systemie Windows naciśnij klawisze **Windows** + **R** i wpisz `cmd`, a po uruchomieniu się konsoli wpisz `composer` ↵.

Informację o zainstalowanej wersji narzędzia Composer przedstawia rysunek 6.1.



Rysunek 6.1. Informacja o dostępnej wersji narzędzia Composer w systemie Windows

Katalog z nowym projektem

Podczas konfiguracji serwera wirtualnego `php8.localhost` założyliśmy, że pliki tego serwera będą się znajdowały w katalogu `c:\xampp\apps\php8\htdocs`. Utwórz w tym katalogu nowy podkatalog o nazwie `projekt` — będziesz tutaj zapisywał pliki projektu i instalował pakiety narzędziem Composer.

Naciśnij klawisze **Windows** + **R** i wpisz `cmd`, a po uruchomieniu się konsoli wpisz polecenia:

```
cd /d c:\xampp\apps\php8\htdocs ↵
mkdir projekt\libs ↵
cd projekt\libs ↵
```

Zwróć uwagę, że jednocześnie zostały utworzone katalog projekt i podkatalog libs, w którym będziesz instalował dodatkowe oprogramowanie. Nowy projekt będzie później dostępny w przeglądarce pod adresem *http://php8.localhost/projekt*.

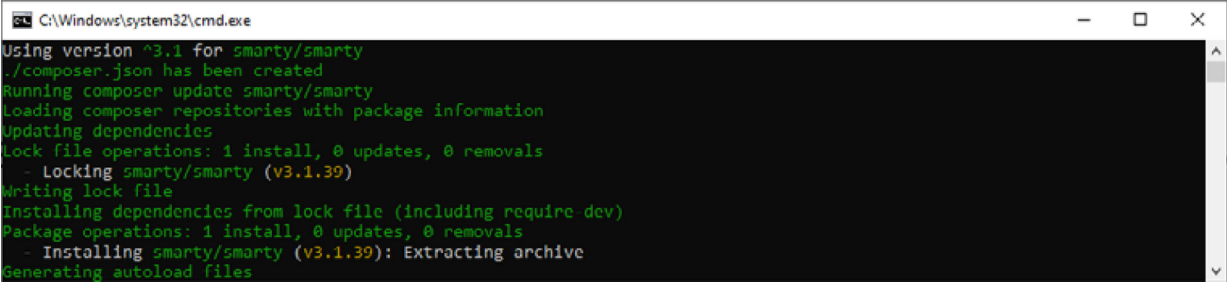
Dodanie pierwszego pakietu do projektu

Teraz przyszedł czas na instalację pakietu narzędziem Composer. Dodam tylko, że listę wszystkich dostępnych pakietów znajdziesz na stronie *https://packagist.org*. Są tam tysiące pakietów, które możesz bezpłatnie wykorzystać.

Na wstępie tego rozdziału wspomniałem, że zainstalujesz oprogramowanie Smarty. Jest to moim zdaniem jeden z najlepszych systemów do tworzenia szablonów dla aplikacji PHP. System Smarty pozwala na separację logiki aplikacji od jej warstwy prezentacyjnej, co ma istotne znaczenie, jeśli nad projektem pracuje więcej osób.

Naciśnij klawisze **Windows** + **R** i wpisz `cmd`, a po uruchomieniu się konsoli wpisz polecenia:

```
cd /d c:\xampp\apps\php8\htdocs\projekt\libs ↵
composer require smarty/smarty ↵
```



```
C:\Windows\system32\cmd.exe
Using version ^3.1 for smarty/smarty
./composer.json has been created
Running composer update smarty/smarty
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
- Locking smarty/smarty (v3.1.39)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
- Installing smarty/smarty (v3.1.39): Extracting archive
Generating autoload files
```

Rysunek 6.2. Instalacja pakietu Smarty narzędziem Composer

Z informacji, które pojawiły się podczas instalacji (rysunek 6.2), możemy odczytać, że został utworzony specjalny plik `composer.json` w formacie JSON, w którym będą przechowywane informacje o zainstalowanych pakietach. Informacyjnie dodam, że Composer utworzy dodatkowy plik `composer.lock` ze swoją konfiguracją.

Zawartość pliku `composer.json` może być taka:

```
{
  "require": {
    "smarty/smarty": "^3.1"
  }
}
```


Zwróć uwagę na klucz *require*, który informuje narzędzie Composer, że pakiet ten jest wymagany przez Twój projekt. Przy nazwie pakietu jest umieszczona dodatkowa informacja o wymaganym numerze wersji. Pełna dokumentacja pliku `composer.json` znajduje się na stronie <https://getcomposer.org/doc/04-schema.md>.

Composer utworzy specjalny katalog `vendor`, w którym będą zapisane informacje konfiguracyjne oraz wszystkie pobrane i zainstalowane pakiety. Pamiętaj, aby nie edytować plików znajdujących się w tym katalogu, ponieważ mogą one zostać automatycznie nadpisane przez Composer podczas instalacji kolejnego pakietu.

Dodanie kolejnego pakietu do projektu

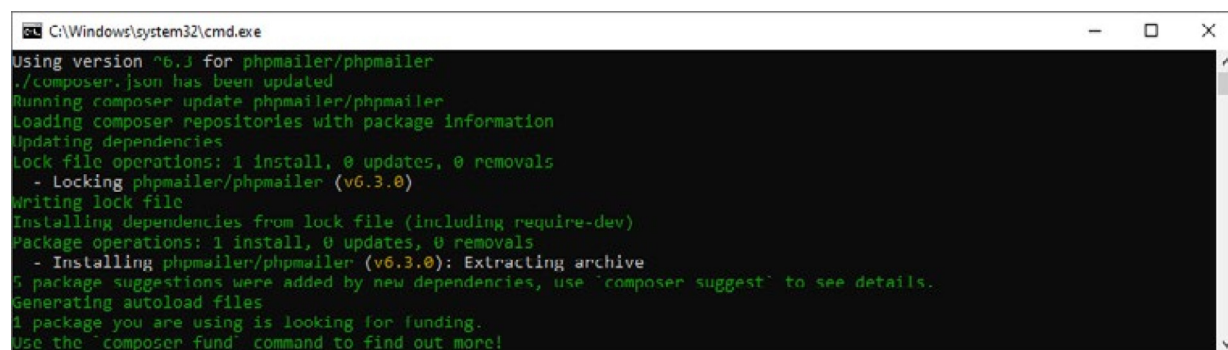
Twój projekt z pewnością będzie się rozwijał, co może wymagać zainstalowania dodatkowych pakietów. Wejdź na stronę <https://packagist.org> i poszukaj pakietu, który będzie odpowiadał Twoim oczekiwaniom. Wybór jest naprawdę bardzo duży, masz do dyspozycji ponad trzysta tysięcy pakietów.

Jeśli chcesz np. zainstalować dodatkowy pakiet do obsługi poczty — PHPMailer, w katalogu, w którym znajduje się plik `composer.json`, powinieneś wykonać polecenie `composer require phpmailer/phpmailer`.

Jeśli nie pamiętasz, jak uruchomić konsolę, naciśnij klawisze **Windows** + **R** i wpisz `cmd`, a po uruchomieniu się konsoli wpisz polecenia:

```
cd /d c:\xampp\apps\php8\htdocs\projekt\libs
composer require phpmailer/phpmailer
```

Z informacji, które pojawiły się podczas instalacji pakietu (rysunek 6.3), możemy odczytać, że plik `composer.json` został zaktualizowany.



```
C:\Windows\system32\cmd.exe
Using version ^6.3 for phpmailer/phpmailer
./composer.json has been updated
Running composer update phpmailer/phpmailer
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking phpmailer/phpmailer (v6.3.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
  - Installing phpmailer/phpmailer (v6.3.0): Extracting archive
5 package suggestions were added by new dependencies, use 'composer suggest' to see details.
Generating autoload files
1 package you are using is looking for funding.
Use the 'composer fund' command to find out more!
```

Rysunek 6.3. Instalacja pakietu PHPMailer narzędziem Composer

Uaktualniona zawartość pliku `composer.json` może być taka:

```
{
  "require": {
    "smarty/smarty": "^3.1",
    "phpmailer/phpmailer": "^6.3"
  }
}
```

Instalacja kolejnych pakietów nie powinna sprawiać Ci już większych kłopotów.

Aktywacja pakietów w projekcie

Możesz być z siebie zadowolony — udało Ci się pomyślnie zainstalować narzędzie Composer oraz pakiety Smarty i PHPMailer. Teraz należy je za pomocą *autoloadera* dołączyć do projektu. Na listingu 6.1 pokazałem, jak można to zrobić. Zapisz kod w pliku `index.php` w katalogu projektu, np. `c:\xampp\apps\php8\htdocs\projekt`, i uruchom. Jeśli nie pojawiły się błędy typu *PHP Fatal error*, oznacza to, że wszystko poprawnie zainstalowałeś i skonfigurowałeś.

LISTING 6.1	PHP 7/8	index.php
01	<?php	
02	use PHPMailer\PHPMailer\{PHPMailer, SMTP, Exception};	
03		
04	require_once __DIR__."/libs/vendor/autoload.php"; // autoloader	
05		
06	\$smarty = new Smarty;	
07	\$mail = new PHPMailer(true);	
08		
09	if (\$smarty && \$mail) {	
10	print "Pakiety zainstalowane!";	
11	}	
\$ php index.php ↵		KONSOLA
Pakiety zainstalowane!		

W linii 04 wskazuję położenie pliku `autoload.php` i dołączam funkcją `require_once`. Stała magiczna `__DIR__` zawiera ścieżkę do katalogu z plikiem `index.php`. Zgodnie z dokumentacją pakietu PHPMailer jego aktywacja wymaga dodania kodu w linii 02. W wierszach 06 i 07 tworzone są egzemplarze klas, które później będziesz wykorzystywał przy projektowaniu strony. W linii 09 sprawdzam, czy klasy zostały poprawnie utworzone.

Wskazówki

Narzędzie Composer jest wykorzystywane przez wielu programistów, ponieważ ułatwia tworzenie projektów i zarządzanie nimi. Composer ma ogromne możliwości, aby jednak poprawnie z niego korzystać, należy nabrać doświadczenia. W rozdziale tym pokazałem Ci, jak w prosty sposób zainstalować pakiety i dodać je do projektu. Composer udostępnia wiele dodatkowych funkcji poza poznaną *require*. Wystarczy, że w linii poleceń wpiszesz `composer ↵`, a zobaczysz pełną listę komend i opcji. Najważniejsze, które powinieneś poznać, to *require*, *install*, *update* i *remove*.

Pamiętaj, aby narzędzie Composer uruchamiać w odpowiednim katalogu. W naszym przypadku będzie to `libs`. W tym katalogu Composer utworzy podkatalog `vendor` oraz pliki `composer.json` i `composer.lock`.

Komenda *require*

Komenda *require* „nazwa” zainstaluje pakiet w katalogu `vendor` i doda go do pliku `composer.json`. Jeśli plik `composer.json` nie istnieje, utworzy go.

```
composer require smarty/smarty ↵
```

Komenda *install*

Komenda *install* odczyta zawartość z pliku `composer.json`, a następnie zainstaluje wszystkie pakiety w katalogu `vendor`. Komenda nadaje się do ponownej instalacji wszystkich pakietów.

```
composer install ↵
```

Komenda *update*

Komenda *update* zaktualizuje wszystkie pakiety zapisane w pliku `composer.json`.

```
composer update ↵
```

Komenda *remove*

Komenda *remove* „nazwa” usuwa pakiet i modyfikuje plik `composer.json`.

```
composer remove smarty/smarty ↵
```

Ćwiczenia

1. Wykorzystując narzędzie Composer, zainstaluj pakiet *guzzlehttp/guzzle*. Sprawdź, ile pakietów zostało rzeczywiście zainstalowanych i dlaczego.
2. Ze strony <https://packagist.org/explore/> wybierz dowolny pakiet i zainstaluj go. Sprawdź w zawartości pliku `composer.json`, czy pakiet ten został dodany. Następnie usuń ten pakiet i ponownie sprawdź zawartość pliku `composer.json`.
3. Usuń zawartość katalogu `vendor`, a następnie w katalogu, gdzie znajduje się plik `composer.json`, wykonaj polecenie `composer install` ↴.
4. Wykonaj w konsoli polecenie `composer self-update` ↴. Uaktualnisz narzędzie Composer do najnowszej wersji.

Pytania

Zrobiłem, co trzeba, ale nie rozumiem kodu z listingu 6.1.

Doskonale Cię rozumiem! Na tym etapie, jeśli poprawnie przepiszesz kod programu do edytora, a następnie uruchomisz go bez problemu, to oznacza, że wykonałeś kawałek dobrej pracy. W następnych rozdziałach dowiesz się, czym są zmienne i konstrukcje warunkowe — wówczas wszystko stanie się zrozumiałe.

Kiedy korzystać z narzędzia Composer?

Przy pisaniu rozbudowanych aplikacji w języku PHP prawdopodobnie wykorzystasz dodatkowe biblioteki (pakiety) o różnych funkcjonalnościach. Aby pakiety działały poprawnie, mogą wymagać zainstalowania innych pakietów.

Wyobraź sobie, że Twój projekt wymaga instalacji dziesięciu pakietów. Możesz to zrobić ręcznie, pobierając je wszystkie z sieci. Potem musisz je jakoś dodać do swojego projektu, co może być nie lada wyzwaniem. Nie wspominam już o tym, że co jakiś czas będziesz musiał sprawdzić, czy nie ma nowszej wersji jakiegoś pakietu.

Narzędzie Composer zdecydowanie ułatwia to zadanie. Wystarczy, że w konsoli wykonasz polecenie `composer update` ↴ i gotowe! Pamiętaj też, że za dołączenie pakietów do projektu odpowiada *autoloader*, czyli jedna dodatkowa linia w kodzie.

Czy mogę pobrać pakiety Smarty i PHPMailer bez narzędzia Composer?

Tak. Otwórz w przeglądarce stronę <https://www.smarty.net> i pobierz najnowszą wersję pakietu Smarty. Na stronie startowej powinieneś zauważyć odnośnik do pobrania najnowszej wersji. Zostaniesz przeniesiony na stronę <https://github.com/>, z której pobierzesz wersję w formacie `.zip`. Natomiast pakiet PHPMailer możesz pobrać ze strony <https://github.com/PHPMailer/PHPMailer>.

W czasie pisania tej książki najnowsza wersja pakietu Smarty miała numer 3.1.39 (wydana w lutym 2021 roku), natomiast pakietu PHPMailer — numer 6.3.0 (wydana w lutym 2021 roku).



Zmienne i stałe

Kiedy będziesz programował — bez znaczenia w jakim języku — z pewnością wykorzystasz zmienne i stałe do przechowywania dowolnych wartości tekstowych, liczbowych, logicznych itp. W swoim programie będziesz mógł w wielu miejscach odwoływać się do raz zdefiniowanej zmiennej i stałej. Jak sama nazwa wskazuje, zmienną możesz w trakcie wykonywania kodu modyfikować, natomiast raz zdefiniowanej stałej zmienić nie można.

Zmienną może być dowolna wartość wprowadzona przez użytkownika: nazwa miasta, liczba przebytych kilometrów i wiele innych. Stałe mają inny charakter. Często są definiowane na początku kodu i nie mogą być później zmieniane. Klasycznym przykładem stałej może być liczba π (pi). W języku PHP zmienne rozpoczynają się od znaku dolara \$ i przypisywana jest do nich konkretna wartość, a stałe są definiowane za pomocą specjalnej deklaracji `define`.

Ćwiczenie 1. Definicja zmiennych

Na listingu 7.1 w liniach od 03 do 07 zdefiniowałem kilka zmiennych, nadając im różne nazwy i wartości. Przyjrzyj się uważnie, jak definiuje się zmienne tekstowe, liczbowe i logiczne. Zauważ, że raz zdefiniowane zmienne można później wiele razy wykorzystać. Przykładowo zmienna `$name` jest zdefiniowana w wierszu 03, a następnie pojawia się w liniach 11, 18 i 21. Wpisz w wierszu 03 swoje imię i sprawdź, jak zmiana wartości zmiennej wpływa na działanie programu. Więcej informacji na temat, jak tworzyć nazwy zmiennych, umieściłem we wskazówkach do tej lekcji.

LISTING 7.1

PHP 7/8

code_07_01.php

```
01 <?php
02 // definicja zmiennych
03 $name = "Zebra"; // zmienna tekstowa
04 $city = "Zwierzyniec"; // zmienna tekstowa
05 $phone = "+48 123-456-789"; // zmienna tekstowa
06 $age = 21; // zmienna liczbowa
```

```

07  $alive = true; //zmienna logiczna — true (1) lub false (0)
08
09  //wykorzystanie zmiennych
10  print "Informacje o gościu:".PHP_EOL;
11  print "imię: $name".PHP_EOL;
12  print "miasto: $city".PHP_EOL;
13  print "telefon: $phone".PHP_EOL;
14  print "wiek: $age".PHP_EOL;
15  print "na tym świecie: $alive".PHP_EOL;
16
17  //ponowne wykorzystanie zmiennych
18  print "Numer telefonu do $name to $phone.".PHP_EOL;
19
20  //i jeszcze raz
21  print "$name ma $age lat.";

```

```
$ php code_07_01.php ↵
```

KONSOLA

```

Informacje o gościu:
imię: Zebra
miasto: Zwierzyniec
telefon: +48 123-456-789
wiek: 21
na tym świecie: 1
Numer telefonu do Zebra to +48 123-456-789.
Zebra ma 21 lat.

```

Ćwiczenie 2. Definicja stałych

Stałą definiujemy za pomocą deklaracji `define`. Raz zdefiniowanej stałej nie można później zmienić w trakcie działania programu. Na listingu 7.2 w liniach od 03 do 05 zdefiniowałem stałe, które wykorzystałem w wierszach 08 i 10. W odróżnieniu od zmiennych nazw stałych nie poprzedzamy znakiem `$` (dolar).

Zwróć uwagę, że również `PHP_EOL` (znak końca linii) jest stałą. Jeśli chciałbyś ją zdefiniować na nowo w programie, zostanie wyświetlony komunikat o błędzie: *PHP Warning: Constant PHP_EOL already defined in [...]*. Aby to sprawdzić, usuń hash (`#`) w linii 06 i ponownie uruchom program.

LISTING 7.2

PHP 7/8

code_07_02.php

```

01  <?php
02  //definicja stałych
03  define('BOOK', 'PHP8 i SQL');
04  define('YEAR', 2021);
05  define('PHP', ['3','4','5','7','8']); //stałą może być też tablica

```

```

06  #define(PHP_EOL, "\n");
07
08  print "Książka: ".BOOK." (".YEAR." rok)";
09  print PHP_EOL;
10  print "Wersja PHP: ".PHP[4];

```

```
$ php code_07_02.php ↵
```

KONSOLA

```

Książka: PHP8 i SQL (2021 rok)
Wersja PHP: 8

```

Ćwiczenie 3. Modyfikacja wartości zmiennych

W dowolnym momencie możesz zmodyfikować wartość zmiennych, co jest dużą zaletą, ale musisz mieć świadomość, jakie są tego konsekwencje. W tym ćwiczeniu na listingu 7.3 zmienna `$tekst1` jest definiowana dwukrotnie, w liniach 02 i 06, dlatego w wierszach 03 i 07 drukowane są zupełnie różne ciągi znaków.

LISTING 7.3

PHP 7/8

code_07_03.php

```

01  <?php
02  $tekst1 = "Lubię programować!"; //ustawienie wartości zmiennej
03  print $tekst1;
04  print PHP_EOL;
05
06  $tekst1 = "Lubię wakacje!"; //nadpisanie (zmiana) wartości zmiennej
07  print $tekst1;

```

```
$ php code_07_03.php ↵
```

KONSOLA

```

Lubię programować!
Lubię wakacje!

```

Ćwiczenia dodatkowe

1. Napisz program, w którym zdefiniujesz pięć zmiennych i pięć stałych związanych z Twoim miejscem zamieszkania. Pamiętaj, że stała to coś niezmiennego, jak np. rok uzyskania praw miejskich, natomiast wartość zmiennej może się w trakcie działania programu zmieniać, jak np. liczba mieszkańców konkretnej dzielnicy.
2. Poszukaj więcej informacji na temat sposobów nadawania nazw zmiennym i wybierz któryś dla siebie lub opracuj własny.

Wskazówki

Poniższe programy drukują identyczne informacje. Jeśli uznasz, że zamiast zwrotu „lubię” lepszym będzie „uwielbiam”, to w którym z poniższych programów będziesz musiał dokonać mniej zmian, czyli szybciej zmienisz tekst do wydrukowania?

Bez zmiennej	Ze zmienną
<pre><?php print "Lubię Adama!".PHP_EOL; print "Lubię Ewę!".PHP_EOL; print "Lubię Magdę!".PHP_EOL; print "Lubię Tadka!".PHP_EOL; print "Lubię Edka!".PHP_EOL;</pre>	<pre><?php \$zmienna = "Lubię"; print "\$zmienna Adama!".PHP_EOL; print "\$zmienna Ewę!".PHP_EOL; print "\$zmienna Magdę!".PHP_EOL; print "\$zmienna Tadka!".PHP_EOL; print "\$zmienna Edka!".PHP_EOL;</pre>

Oczywiście, że w programie ze zmienną! Wystarczy tylko jedna modyfikacja kodu w linii 02, podczas gdy w programie bez zmiennej będziesz musiał wykonać ich aż pięć. I teraz wyobraź sobie, że program ma kilka tysięcy linii kodu.

Tworząc nazwy zmiennych, kieruj się regułą mówiącą, aby dokładnie określić, jakie dane przechowują. Możesz znaleźć dużo ciekawych pomysłów, jak konstruować nazwy zmiennych, z których wiele odnosi się do szczegółowych aspektów w programowaniu. Na początku swojej drogi programisty z pewnością nie będziesz przywiązywał do tego zbyt dużej wagi, zalecam jednak, aby przed wymyśleniem nazwy dla zmiennej przez chwilę się nad tym zastanowić.

Dobre nazwy dla zmiennych wyglądają np. tak:

```
$bodyWeight = 85; //waga ciała
$myName = "Armands"; //imię
$cmtKm = 250; //liczba kilometrów
```

Unikaj nazw rozwlekłych:

```
$theWeightOfMyBeautifulBody = 85; //za długie, trudno to zapamiętać
$myNameOnMyBirth = "Armands"; //przecież można napisać to krócej
$howManyKilometersYouCanDriveACarWithoutAnyBreak = 250; //ojej, co to jest?
```

Do nazywania zmiennych zastosowałem notację camelCase, której często używam, polegającą na łącznym pisaniu wyrazów i rozpoczynaniu każdego wielką literą (oprócz pierwszego wyrazu). Nie musisz używać języka angielskiego do nazywania zmiennych, choć nie ukrywam, że będzie to z pewnością pomocne dla innych programistów analizujących Twój kod.

Kolejne przykłady poprawnych nazw zmiennych:

```
$myText = 'Pierwszy tekst';  
$my_text = 'Drugi tekst';  
$mojTekst = 'Trzeci tekst';  
$moj_tekst = 'Czwarty tekst';  
$myAge = 45;  
$moj_wiek = 45;  
$cnt = 5;  
$licznik = 5;
```

Pytania

Czy mimo wszystko mogę używać długich nazw zmiennych?

Oczywiście, dla programu nie ma to żadnego znaczenia. Wybór należy do Ciebie.

Dlaczego na listingach są puste linie?

Puste linie (odstęp między wierszami kodu) pełnią funkcję dekoracyjną i nie mają znaczenia dla działania programu. Stosuję je, aby ułatwić sobie czytanie kodu.

W ćwiczeniu 2 po usunięciu hasha (#) w linii 6 nie pojawia mi się żaden komunikat o błędzie.

Jeśli korzystasz z programu Eclipse, najprawdopodobniej musisz uaktywnić wyświetlanie informacji o błędach w kodzie. Wykonaj następujące czynności:

1. Wybierz z menu *Window/Preferences* i w nowo otwartym oknie wybierz *PHP/Validation/Errors/Warnings*.
2. Kliknij przycisk *Restore Defaults*, a następnie *Apply and Close*.
3. Wykonaj program jeszcze raz i sprawdź, czy w konsoli pojawi się komunikat o błędzie.
4. Jeśli to nie zadziała, sprawdź plik `php.ini` w katalogu z interpreterem PHP pod kątem wyświetlania komunikatów o błędach.



Łańcuchy znaków

Łańcuchy znaków w języku PHP są reprezentowane za pomocą typu `string`. To po prostu tekst, który możesz z użyciem dostępnych funkcji dowolnie formatować. W praktyce, aby ułatwić sobie to zadanie, tekst umieszcza się w zmiennej.

W tej lekcji nauczysz się łączyć i drukować ciągi znaków z wykorzystaniem funkcji `print` i `echo`, a także poznasz nowe funkcje: `trim`, `ltrim`, `rtrim`.

Nauczysz się rozdzielać tekst na elementy, wykorzystując funkcję `explode`.

Poznasz nowe funkcje, które pojawiły się w najnowszej wersji języka PHP 8, czyli `str_contains`, `str_starts_with` i `str_ends_with`.

Ćwiczenie 1. Drukowanie łańcuchów

W tym ćwiczeniu poznasz różne techniki drukowania tekstu. Zauważ, że razem z tekstem możesz drukować wartości wcześniej zdefiniowanych zmiennych (listing 8.1).

LISTING 8.1	PHP 7/8	code_08_01.php
01	<?php	
02	<i>/* jedna linia */</i>	
03	print "Lubię programować!".PHP_EOL;	
04	echo "Lubię programować!".PHP_EOL;	
05		
06	\$myText = "PHP";	
07		
08	<i>/* wiele linii */</i>	
09	print "Ten tekst zawiera wiele linii,	
10	które wyświetlane są jedna pod drugą.	
11	Możesz umieszczać wewnątrz tekstu zmienne \$myText ".PHP_EOL;	
12		
13	<i>/* znaki specjalne */</i>	
14	echo "Przykład umieszczenia \"cudzysłowu\" w tekście".PHP_EOL;	
15		

```

16  /* łączenie tekstu i zmiennych */
17  print "To "."jest "."przykładowy "."tekst "."w "."języku $myText".PHP_EOL;
18  echo "To ", "jest ", "przykładowy ", "tekst ", "w ", "języku ", $myText, PHP_EOL;
$ php code_08_01.php ↵

```

KONSOLA

```

Lubię programować!
Lubię programować!
Ten tekst zawiera wiele linii,
które wyświetlane są jedna pod drugą.
Możesz umieszczać wewnątrz tekstu zmienne PHP
Przykład umieszczenia "cudzysłowu" w tekście
To jest przykładowy tekst w języku PHP
To jest przykładowy tekst w języku PHP

```

Ćwiczenie 2. Usuwanie białych znaków

W tym ćwiczeniu poznasz trzy nowe funkcje służące do usuwania białych znaków (spacji) z łańcucha znaków. Pierwsza funkcja `trim` usuwa spacje z lewej i z prawej strony łańcucha, natomiast `ltrim` i `rtrim`, odpowiednio, usuwają je z lewej i z prawej strony. Funkcje te będziesz mógł wykorzystać np. do przetwarzania tekstu wprowadzonego przez użytkownika na Twojej stronie internetowej (listing 8.2).

LISTING 8.2 PHP 7/8 code_08_02.php

```

01  <?php
02  /* imię wprowadzone przez użytkownika, możesz wpisać swoje */
03  $name = "  Robert  ";
04  print "Tekst oryginalny: >". $name. "<".PHP_EOL;
05
06  /* usunięcie spacji z lewej i z prawej strony */
07  print "Funkcja trim: >". trim($name). "<".PHP_EOL;
08
09  /* usunięcie spacji z lewej strony */
10  print "Funkcja ltrim: >". ltrim($name). "<".PHP_EOL;
11
12  /* usunięcie spacji z prawej strony */
13  print "Funkcja rtrim: >". rtrim($name). "<".PHP_EOL;

```

```
$ php code_08_02.php ↵
```

KONSOLA

```

Tekst oryginalny: >  Robert  <
Funkcja trim: >Robert<
Funkcja ltrim: >Robert <
Funkcja rtrim: >  Robert<

```

Ćwiczenie 3. Format heredoc

Wykorzystując tak zwane dokumenty wbudowane w formacie *heredoc*, możemy w prosty sposób umieścić w programie łańcuchy zawierające wiele linii tekstu.

Na listingu 8.3 w wierszu 03 zdefiniowałem identyfikator <<< EOF, który informuje interpreter PHP, że tworzymy dokument wbudowany w formacie *heredoc*. W liniach od 04 do 07 znajdują się kolejne wiersze tekstu. Linia 08 zawiera tylko identyfikator oznaczający koniec dokumentu wbudowanego. Nazwa identyfikatora może być dowolna, jednak proponuję ograniczyć się do krótkich ciągów znaków. Zauważ, że zawartość dokumentu wbudowanego przypisałem w wierszu 03 do zmiennej \$inwokacja.

LISTING 8.3	PHP 7/8	code_08_03.php
01	<?php	
02	<i>/* definicja dokumentu wbudowanego w formacie heredoc */</i>	
03	\$inwokacja = <<< EOF	
04	Litwo, Ojczyzno moja! ty jesteś jak zdrowie;	
05	Ile cię trzeba cenić, ten tylko się dowie,	
06	Kto cię stracił. Dziś piękność twą w całej ozdobie	
07	Widzę i opisuję, bo tęsknię po tobie.	
08	EOF;	
09		
10	print \$inwokacja;	
\$ php code_08_03.php ↵		KONSOLA
Litwo, Ojczyzno moja! ty jesteś jak zdrowie; Ile cię trzeba cenić, ten tylko się dowie, Kto cię stracił. Dziś piękność twą w całej ozdobie Widzę i opisuję, bo tęsknię po tobie.		

Ćwiczenie 4. Dzielenie łańcucha

Język PHP udostępnia funkcje umożliwiające podzielenie łańcucha na mniejsze części składowe. Jedną z nich jest `explode`, która rozdziela tekst na elementy na podstawie określonego separatora. Wszystkie elementy umieszczone są w tablicy (w następnych rozdziałach poznasz znaczenie tablic w programowaniu).

Na listingu 8.4 w linii 03 zdefiniowałem zmienną \$tekst. Zauważ, że łańcuch składa się z imion oddzielonych od siebie znakiem przecinka. W wierszu 04 funkcja `explode` rozdziela łańcuch na elementy zgodnie ze wskazanym separatorem. Wynikiem będzie tablica, której zawartość zostanie przypisana do zmiennej \$elementy. W linii 06 wykorzystałem funkcję `print_r` do wydrukowania zawartości tablicy.

LISTING 8.4	PHP 7/8	code_08_04.php
<pre> 01 <?php 02 /* dzielenie łańcucha */ 03 \$tekst = 'Adam,Ewa,Janusz'; 04 \$elementy = explode(',', \$tekst); 05 print \$tekst.PHP_EOL; 06 print_r(\$elementy); </pre>		
\$ php code_08_04.php ↵		KONSOLA
<pre> Adam,Ewa,Janusz Array ([0] => Adam [1] => Ewa [2] => Janusz) </pre>		

Ćwiczenie 5. Przeszukiwanie łańcucha

Język PHP 8 wprowadza nowe funkcje umożliwiające przeszukiwanie łańcuchów znaków, a mianowicie `str_contains`, `str_starts_with` i `str_ends_with`. Jest to z pewnością duże ułatwienie dla programisty, ponieważ eliminuje używanie silnego porównania `===` (trzy znaki równości) lub wyrażeń regularnych.

Funkcja `str_contains` umożliwia łatwiejsze sprawdzenie niż przy wykorzystaniu silnego porównania i funkcji `strpos`, czy dany ciąg znaków znajduje się w łańcuchu. Funkcje `str_starts_with` i `str_ends_with` sprawdzają, czy dany ciąg znaków znajduje się na początku lub na końcu łańcucha.

Przepisz kod z listingu 8.5 i uruchom. W programie pojawiają się konstrukcje warunkowe, które poznasz w dalszej części niniejszej książki.

LISTING 8.5	PHP 8	code_08_05.php
<pre> 01 <?php 02 \$string = 'Jak się masz?'; 03 if (str_contains(\$string, 'się')) { 04 print 'OK (zawiera się).PHP_EOL; 05 } 06 if (str_starts_with(\$string, 'Jak')) { 07 print 'OK (rozpoczyna się).PHP_EOL; 08 } 09 if (str_ends_with(\$string, 'masz?')) { 10 print 'OK (kończy się).PHP_EOL; 11 } </pre>		

```
$ php code_08_05.php ↵
```

KONSOLA

```
OK (zawiera się)  
OK (rozpoczyna się)  
OK (kończy się)
```

Ćwiczenia dodatkowe

1. Napisz program, który rozdzieli tekst „Adam*Ewa*Janusz”. Wykorzystaj kod z ćwiczenia 4.
2. Porównaj kod z listingów 8.5 i 1.11. Jestem przekonany, że w swoich programach do przeszukiwania łańcuchów wykorzystasz nowe funkcje języka PHP 8.

Wskazówki

W praktyce najczęściej będziesz wykorzystywać funkcję `trim`, aby pozbyć się zbędnych pustych znaków z początku i końca tekstu. Ciągi znaków, takie jak `"PROGRAM"` i `" PROGRAM "`, dla Ciebie są identyczne, ponieważ automatycznie optymalizujesz je w głowie, natomiast dla Twojego programu są to dwa różne ciągi znaków. Dobrą praktyką jest optymalizacja informacji bezpośrednio po otrzymaniu ich ze źródła zewnętrznego (np. danych adresowych użytkownika Twojej strony internetowej), nim zostaną zapisane w bazie danych.

Pytania

Czym różni się słowo „tekst” od wyrażenia „ciąg znaków” lub „łańcuch znaków”?

Niczym. Używam ich zamiennie, aby się nie powtarzać.

Do czego przyda mi się funkcja `trim`?

Zajrzyj do wskazówek do tej lekcji.



Co, jeśli? Konstrukcje warunkowe

W języku PHP istnieją funkcje warunkowe, bez których programowanie najprawdopodobniej straciłoby sens. Istotą funkcji warunkowych jest dokonanie wyboru w zależności od spełnienia określonego warunku. Przykładem funkcji warunkowej może być wyświetlenie konkretnej oceny, zależnie od liczby punktów uzyskanych w teście. W konstrukcjach warunkowych będziesz korzystał z operatorów porównania, aby sprawdzić, czy dwie zmienne są ze sobą równe, czy różne, lub która ze zmiennych jest mniejsza bądź większa. Operatory porównania przedstawia poniższa tabela.

Operator	Co sprawdza?
==	czy zmienne są sobie równe co do wartości
!=	czy zmienne są różne co do wartości
===	czy zmienne są identyczne
!==	czy zmienne nie są identyczne
>	czy zmienna z lewej strony jest większa od zmiennej z prawej strony
>=	czy zmienna z lewej strony jest większa od zmiennej z prawej strony bądź jej równa
<	czy zmienna z prawej strony jest większa od zmiennej z lewej strony
<=	czy zmienna z prawej strony jest większa od zmiennej z lewej strony bądź jej równa

Przykładowa konstrukcja warunkowa może mieć postać:

```
01 //kod programu, np. deklaracja zmiennych $a i $b
02 if ($a > $b)
03 { //pierwszy warunek
04     print "a jest większe od b";
05 }
06 elseif ($a < $b)
07 { //drugi warunek
08     print "a jest mniejsze od b";
```



```

09  }
10  else
11  { //żaden warunek nie jest spełniony
12      print "a jest równe b";
13  }
14  //dalszy ciąg programu

```

Jak działa powyższy kod?

1. Sprawdź w linii 02, czy zmienna \$a jest większa od zmiennej \$b; jeśli tak, wykonaj kod w wierszu 04 pomiędzy nawiasami klamrowymi (linie 03 i 05), a następnie opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 14.
2. Jeśli pierwszy warunek nie został spełniony, sprawdź kolejny w linii 06, czyli czy zmienna \$a jest mniejsza od zmiennej \$b. Jeśli warunek jest spełniony, wykonaj kod w wierszu 08 pomiędzy nawiasami klamrowymi (linie 07 i 09), po czym opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 14.
3. W związku z tym, że żaden z warunków nie został spełniony, wykonaj kod w wierszu 12 pomiędzy nawiasami klamrowymi (linie 11 i 13), a następnie opuść konstrukcję warunkową i wykonaj kod, począwszy od linii 14.

Pamiętaj, że w konstrukcji warunkowej wykonywany jest kod znajdujący się pomiędzy nawiasami klamrowymi { i }.

Ćwiczenie 1. Konstrukcja if

Najprostszą konstrukcją warunkową jest `if`, która służy do wykonania kodu ujętego w nawiasy klamrowe tylko wtedy, kiedy określony operatorem warunek jest spełniony (listing 9.1).

LISTING 9.1	PHP 7/8	code_09_01.php
01	<?php	
02	<i>/* definicja zmiennych */</i>	
03	\$a = 25;	
04	\$b = 12;	
05		
06	<i>/* konstrukcja warunkowa */</i>	
07	if (\$a > \$b)	
08	{	
09	print "zmienna a jest większa od zmiennej b";	
10	}	

```
$ php code_09_01.php ↵
```

KONSOLA

```
zmienna a jest większa od zmiennej b
```

Jak działa powyższy kod?

1. W liniach 03 i 04 zdefiniowane są zmienne \$a i \$b.
2. W wierszu 07 pojawia się konstrukcja warunkowa if, która sprawdza, czy zmienna \$a jest większa od zmiennej \$b.
3. Jeśli tak, wykonywany jest kod programu zawarty pomiędzy nawiasami klamrowymi { i }, czyli pomiędzy 08 a 10 wierszem.

Ćwiczenie 2. Konstrukcja if-else

Następną konstrukcją warunkową jest if-else, która służy do wykonania odpowiedniego kodu w zależności od tego, czy warunek określony operatorem jest spełniony (listing 9.2).

LISTING 9.2

PHP 7/8

code_09_02.php

```
01 <?php
02 /* definicja zmiennych */
03 $a = 12;
04 $b = 25;
05
06 /* konstrukcja warunkowa */
07 if ($a > $b)
08 {
09     print "zmienna a jest większa od zmiennej b";
10 }
11 else
12 {
13     print "zmienna a jest równa lub mniejsza od zmiennej b";
14 }
```

```
$ php code_09_02.php ↵
```

KONSOLA

```
zmienna a jest równa lub mniejsza od zmiennej b
```

Jak działa powyższy kod?

1. W liniach 03 i 04 zdefiniowane są zmienne \$a i \$b.
2. W wierszu 07 pojawia się konstrukcja warunkowa if, która sprawdza, czy zmienna \$a jest większa od zmiennej \$b.

3. Jeśli tak, wykonywany jest kod programu zawarty pomiędzy nawiasami klamrowymi { i }, czyli pomiędzy wierszami 08 a 10.
4. Linia 11 informuje, że w razie niespełnienia warunku określonego w linii 07 zostanie wykonany kod pomiędzy wierszami 12 a 14.

Ćwiczenie 3. Konstrukcja if-elseif-else

Bardziej złożoną konstrukcją warunkową jest if-elseif-else, która wykorzystywana jest do obsługi bardziej rozbudowanych warunków (listing 9.3).

LISTING 9.3	PHP 7/8	code_09_03.php
01	<?php	
02	<i>/* definicja zmiennych */</i>	
03	\$a = 18;	
04	\$b = 18;	
05		
06	<i>/* konstrukcja warunkowa */</i>	
07	if (\$a > \$b)	
08	{	
09	print "zmienna a jest większa od zmiennej b";	
10	}	
11	elseif (\$a == \$b)	
12	{	
13	print "zmienna a jest równa zmiennej b";	
14	}	
15	else	
16	{	
17	print "zmienna a jest mniejsza od zmiennej b";	
18	}	
\$ php code_09_03.php ↵		KONSOLA
zmienna a jest równa zmiennej b		

Jak działa powyższy kod?

1. W liniach 03 i 04 zdefiniowane są zmienne \$a i \$b.
2. W wierszu 07 pojawia się konstrukcja warunkowa if, która sprawdza, czy zmienna \$a jest większa od zmiennej \$b.
3. Jeśli tak, wykonywany jest kod programu zawarty pomiędzy nawiasami klamrowymi { i }, czyli pomiędzy wierszami 08 a 10.

4. W linii 11 pojawia się następna konstrukcja warunkowa, która sprawdza, czy zmienna \$a jest równa zmiennej \$b.
5. Jeśli tak, wykonywany jest kod programu zawarty pomiędzy nawiasami klamrowymi { i }, czyli pomiędzy wierszami 12 a 14.
6. Linia 15 informuje, że w razie niespełnienia warunku określonego w wierszach 07 i 11 zostanie wykonany kod pomiędzy liniami 16 a 18.

Ćwiczenie 4. Operator trójargumentowy

Operator trójargumentowy (?:) wykorzystasz do sprawdzenia prostych warunków (listing 9.4).

LISTING 9.4	PHP 7/8	code_09_04.php
01	<?php	
02	\$wiek = 34;	
03		
04	<i>// przypisanie zmiennej określonej wartości w zależności od spełnienia warunku</i>	
05	\$status = (\$wiek >= 18) ? 'jest' : 'nie jest';	
06		
07	print "Mateusz \$status pełnoletni.";	
\$ php code_09_04.php ↵		KONSOLA
Mateusz jest pełnoletni.		

Jak działa powyższy kod?

1. W linii 02 zdefiniowana jest zmienna \$wiek.
2. W wierszu 05 do sprawdzenia warunku, czy zmienna \$wiek jest większa bądź równa 18, został użyty trójargumentowy operator warunkowy ?.

Można oczywiście wykorzystać operator warunkowy if; wówczas kod w linii 05 będzie wyglądał tak:

```
if ($wiek >= 18)
{
    $status = 'jest';
}
else
{
    $status = 'nie jest';
}
```

Oczywiście, obie metody są poprawne, choć najczęściej korzystam z operatora trójargumentowego dla prostych warunków.

Ćwiczenia dodatkowe

1. W każdym ćwiczeniu wstaw różne wartości zmiennych i przetestuj działanie programu.

Wskazówki

Zawsze pamiętaj, że operator porównania to dwa znaki równości ==, a nie jeden. Początkujący programiści bardzo często popełniają ten błąd.

Operator porównania === (trzy znaki równości) służy do sprawdzenia, czy zmienne są równe co do wartości i typu. W następnych lekcjach dowiesz się, jak reprezentowane są liczby całkowite i rzeczywiste w języku PHP.



Co, jeśli? Inaczej też się da

Konstrukcje warunkowe poznane w poprzedniej lekcji bardzo dobrze spełniają swoją funkcję dla kilku warunków. Natomiast nieczęsto spotyka się bardzo rozbudowane konstrukcje z wielokrotnym wykorzystaniem `elseif`, gdyż może to powodować szereg problemów związanych z ich obsługą. W języku PHP do obsługi wielokrotnych warunków możemy użyć konstrukcji warunkowej `switch`, z której często korzystam w swoich programach, lub z konstrukcji `match`, będącej nowością w języku PHP 8. Przećwiczmy to.

Przykładowe konstrukcje warunkowe `match`, `switch` oraz jej odpowiednik `if-elseif-else` mogą mieć postać:

match	switch	if-elseif-else
<pre>\$zmienna = match (\$i) { 1 => 'A', 2 => 'B', 3 => 'C', default => '' };</pre>	<pre>switch (\$i) { case 1: \$zmienna = 'A'; break; case 2: \$zmienna = 'B'; break; case 3: \$zmienna = 'C'; break; default; \$zmienna = ''; }</pre>	<pre>if (\$i == 1) { \$zmienna = 'A'; } elseif (\$i == 2) { \$zmienna = 'B'; } elseif (\$i == 3) { \$zmienna = 'C'; } else { \$zmienna = ''; }</pre>

Podczas pisania programu często rozważa się, która konstrukcja warunkowa jest lepsza: `switch` czy `if-elseif-else`; teraz dochodzi jeszcze `match`, nowa konstrukcja wprowadzona w wersji PHP 8. Każda konstrukcja warunkowa ma swoją logikę i nie zawsze będzie można zamienić ze sobą `switch` i `match`; w tym konkretnym przypadku `match` jest zdecydowanie lepszym wyborem.

Ćwiczenie 1. Konstrukcja switch

W tym ćwiczeniu (listing 10.1) wykorzystasz konstrukcję switch do obsługi zmiennej \$city zdefiniowanej w linii 02. Sprawdzanych jest pięć warunków (w wierszach 05, 08, 09, 12 i 13), a w razie niespełnienia żadnego z nich wykonywany jest domyślny kod w linii 17. Zwróć uwagę, że warunki mogą się przenikać, to znaczy, że kod w wierszach 10 i 11 zostanie wykonany po spełnieniu warunku w linii 08 lub 09.

LISTING 10.1	PHP 7/8	code_10_01.php
01	<?php	
02	\$city = "Kraków"; <i>// informacja od użytkownika</i>	
03		
04	switch (\$city) {	
05	case "Londyn":	
06	print "Stolica Wielkiej Brytanii";	
07	break;	
08	case "Berlin": <i>// przenikanie warunku</i>	
09	case "Monachium":	
10	print "Interesujące miasto w Niemczech";	
11	break;	
12	case "Zakopane": <i>// przenikanie warunku</i>	
13	case "Kraków":	
14	print "\$city to bardzo piękne miasto w Polsce!";	
15	break;	
16	default:	
17	print "Pozostałe miasta też są fajne!";	
18	}	
\$ php code_10_01.php ↵		KONSOLA
Kraków to bardzo piękne miasto w Polsce!		

Jak działa powyższy kod?

1. W linii 02 zdefiniowana jest zmienna \$city.
2. W wierszu 04 instrukcja switch jako argument otrzymuje wartość zmiennej \$city i porównuje do wszystkich przypadków case znajdujących się w konstrukcji.
3. Po dopasowaniu warunku wykonywany jest kod do napotkania instrukcji break. Warunki mogą się przenikać (wiersze 08 i 09 oraz 12 i 13), dopasowanie jednego z nich spowoduje wykonanie kodu.

4. Jeśli dopasowanie nie zostanie znalezione, a został utworzony przypadek default, wykonywany jest kod przewidziany dla tego przypadku aż do napotkania pierwszej instrukcji break.

Ćwiczenie 2. Konstrukcja match

Konstrukcja `match` może przyjąć argument dopasowujący, podobnie jak `switch`, lub sprawdzić warunki na podstawie innych konstrukcji, wówczas argumentem będzie wyrażenie `true`. Różnica jest taka, że w pierwszym przypadku dopasowana jest wartość przekazanego argumentu, natomiast w drugim — operatorów logicznych lub funkcji.

Listing 10.2 zawiera dwa typy konstrukcji `match`: z argumentem (linie od 05 do 11) i bez argumentu dopasowującego (wiersze od 18 do 23). Przepisz go i uruchom.

W pierwszym przypadku w linii 05 przekazywany jest argument do konstrukcji `match`, a w wierszach od 06 do 10 dopasowywane są warunki. Tak jak w przypadku `switch`, tutaj też warunki mogą się przenikać — linie 07 i 08.

W drugim przypadku w linii 18 informujemy konstrukcję `match` argumentem `true`, że wartości dopasowujące będą wynikiem działania funkcji. W naszym przypadku będzie to funkcja `str_contains` — wiersze 19, 20 i 21, której parametrem będzie zmienna `$text` (linia 16) spoza konstrukcji `match`.

LISTING 10.2	PHP 8	code_10_02.php
01	<?php	
02	<i>/* konstrukcja match z argumentem */</i>	
03	<code>\$code = 432;</code>	
04		
05	<code>\$e = match (\$code) {</code>	
06	<code> 400 => 'kwas',</code>	
07	<code> 401, 402, 403 => 'alginian',</code>	
08	<code> 432, 433 => 'monolaurynian',</code>	
09	<code> 483 => 'winian',</code>	
10	<code> default => 'nie mam pojęcia',</code>	
11	<code>};</code>	
12		
13	<code>print "Dodatek do żywności o kodzie \$code to jakiś \$e.".PHP_EOL;</code>	
14		
15	<i>/* konstrukcja match bez argumentu */</i>	
16	<code>\$text = 'Cześć';</code>	
17		
18	<code>\$lang = match(true) {</code>	


```

19     str_contains($text, 'Cześć') => 'polski',
20     str_contains($text, 'Welcome') => 'angielski',
21     str_contains($text, 'Hola') => 'hiszpański',
22     default => '',
23 };
24
25 print "W łańcuchu '$text' wykryłem język $lang.".PHP_EOL;
$ php code_10_02.php ↵

```

KONSOLA

Dodatek do żywności o kodzie 432 to jakiś monolaurynian.
W łańcuchu 'Cześć' wykryłem język polski.

Jak działa powyższy kod?

Wariant z argumentem dopasowującym:

1. W linii 03 zdefiniowana jest zmienna \$code.
2. W wierszu 05 zdefiniowana jest zmienna \$e, której wartość jest uzależniona od wyniku działania konstrukcji warunkowej match. Zwróć uwagę, że konstrukcja match została wywołana z argumentem dopasowującym.
3. W liniach od 06 do 09 sprawdzane są kolejne dopasowania argumentu \$code. Brak dopasowania spowoduje zwrócenie wartości domyślnej, zadeklarowanej instrukcji default, zdefiniowanej w wierszu 10.
4. Wydrukowanie zmiennych \$code i \$e następuje w linii 13.

Wariant bez argumentu dopasowującego:

1. W linii 16 zdefiniowana jest zmienna \$text.
2. W wierszu 18 zdefiniowana jest zmienna \$lang, której wartość jest uzależniona od wyniku działania konstrukcji warunkowej match. Zwróć uwagę, że konstrukcja match została wywołana bez argumentu dopasowującego, co oznacza, że zwracana wartość będzie uzależniona od wyniku działania operatorów logicznych lub funkcji.
3. W liniach od 19 do 21 wywoływana jest funkcja str_contains, której jeden z parametrów (\$text) jest zmienną zdefiniowaną poza konstrukcją match. Brak dopasowania spowoduje zwrócenie wartości domyślnej, zadeklarowanej instrukcji default, zdefiniowanej w linii 22.
4. Wydrukowanie zmiennych \$text i \$lang następuje w linii 25.

Ćwiczenia dodatkowe

1. Otwórz stronę <http://php.net> i wyszukaj opis konstrukcji `switch` i `match`. Spróbuj zrozumieć, jak działają, przepisuj i uruchom przykładowe listingi.
2. Wykorzystując konstrukcje `switch` i `match`, napisz program, który ustali wartość zmiennej `$liczbaDni` w zależności od zmiennej `$numerMiesiaca`.



Co, jeśli? Trochę logiki

No właśnie. Co, jeśli wykonanie konkretnego kodu będzie uzależnione od konkretnego zakresu zmiennych lub od spełnienia kilku warunków jednocześnie? W tej lekcji poznasz operatory logiczne i wykorzystasz je do napisania bardziej rozbudowanych konstrukcji warunkowych.

Operatory logiczne przedstawia poniższa tabela.

Operator	Co oznacza?
&& (AND)	koniunkcję lub iloczyn logiczny
(OR)	alternatywę lub sumę logiczną
!	negację lub zaprzeczenie

Przykładowa konstrukcja warunkowa z operatorem logicznym może mieć postać:

```
01 // kod programu, np. deklaracja zmiennych $a i $b
02 if ($a > $b && $a > 10) // pierwszy warunek — AND
03 {
04     print "a jest większe od b i a jest większe od 10";
05 }
06 elseif ($a == $b || $b == 10) // drugi warunek — OR
07 {
08     print "a jest równe b lub b jest równe 10";
09 }
10 else // pozostałe przypadki
11 {
12     print "a < b, a < 11 i b != 10";
13 }
14 // dalszy ciąg programu
```

Jak działa powyższy kod?

1. Sprawdź następujące warunki w linii 02:

Warunek 1. Czy zmienna \$a jest większa od zmiennej \$b?

Warunek 2. Czy zmienna \$a jest większa od 10?

Jeśli oba warunki są jednocześnie spełnione (operator AND), wykonaj kod w linii 04 pomiędzy nawiasami klamrowymi (wiersze 03 i 05), a następnie opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 14.

2. Jeśli pierwszy warunek nie został spełniony, sprawdź kolejny w linii 06:

Warunek 3. Czy zmienna \$a jest równa zmiennej \$b?

Warunek 4. Czy zmienna \$b jest równa 10?

Jeśli którykolwiek z powyższych warunków został spełniony (operator OR), wykonaj kod w linii 08 pomiędzy nawiasami klamrowymi (wiersze 07 i 09), po czym opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 14.

3. W przypadku kiedy żaden z warunków nie został spełniony, wykonaj kod w linii 12 pomiędzy nawiasami klamrowymi (wiersze 11 i 13), a następnie opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 14.

Pamiętaj, że w konstrukcji warunkowej wykonywany jest kod znajdujący się pomiędzy nawiasami klamrowymi { i }.

Ćwiczenie 1. Koniunkcja (AND)

W tym ćwiczeniu (listing 11.1) nauczysz się wykorzystywać operator AND, czyli koniunkcję. Trudny wyraz, ale w programowaniu oznacza to po prostu jednoczesne występowanie kilku warunków. Aby więc odpowiedni kod został wykonany, wszystkie warunki muszą być spełnione.

W kodzie używaj symbolu && dla operatora AND.

LISTING 11.1	PHP 7/8	code_11_01.php
01	<?php	
02	/* definicja zmiennych */	
03	\$a = 15;	
04	\$b = 8;	
05		
06	/* konstrukcja warunkowa */	

```

07  if ($a >= 10 && $b < 12)
08  {
09      print "Super, wszystkie zmienne spełniają warunek!";
10  }
11  else
12  {
13      print "Niestety, jakaś zmienna nie spełnia warunku :(";
14  }
15  /* dalszy ciąg programu */

```

```
$ php code_11_01.php ↵
```

KONSOLA

```
Super, wszystkie zmienne spełniają warunek!
```

Jak działa powyższy kod?

1. W liniach 03 i 04 zdefiniowane są zmienne \$a i \$b.

2. Sprawdź następujące warunki w linii 07:

Warunek 5. Czy zmienna \$a jest większa bądź równa 10?

Warunek 6. Czy zmienna \$b jest mniejsza od 12?

Jeśli oba warunki są jednocześnie spełnione (operator AND), wykonaj kod w linii 09 pomiędzy nawiasami klamrowymi (wiersze 08 i 10), a następnie opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 15.

3. W przypadku kiedy warunki nie zostały spełnione jednocześnie, wykonaj kod w linii 13 pomiędzy nawiasami klamrowymi (wiersze 12 i 14), po czym opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 15.

Ćwiczenie 2. Alternatywa (OR)

W tym ćwiczeniu (listing 11.2) nauczysz się wykorzystywać operator OR, czyli alternatywę. Chodzi tu o wykonanie kodu, jeśli przynajmniej jeden z warunków jest spełniony.

W kodzie używaj symbolu || (dwie kreski pionowe) dla operatora OR.

LISTING 11.2

PHP 7/8

code_11_02.php

```

01  <?php
02  /* definicja zmiennych */
03  $a = 1973;
04  $b = 2021;
05

```

```

06  /* konstrukcja warunkowa */
07  if ($a >= 1950 || $b < 966)
08  {
09      print "Fajnie, że przynajmniej jedna zmienna spełnia warunek!";
10  }
11  else
12  {
13      print "Szkoda, żadna zmienna nie spełnia warunku :(";
14  }
15  /* dalszy ciąg programu */

```

\$ php code_11_02.php ↵

KONSOLA

Fajnie, że przynajmniej jedna zmienna spełnia warunek!

Jak działa powyższy kod?

1. W liniach 03 i 04 zdefiniowane są zmienne \$a i \$b.
2. Sprawdź następujące warunki w wierszu 07:

Warunek 7. Czy zmienna \$a jest większa bądź równa 1950?

Warunek 8. Czy zmienna \$b jest mniejsza od 966?

Jeśli którykolwiek z warunków został spełniony (operator OR), wykonaj kod w linii 09 pomiędzy nawiasami klamrowymi (wiersze 08 i 10), a następnie opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 15.

3. W przypadku kiedy żaden z warunków nie został spełniony, wykonaj kod w linii 13 pomiędzy nawiasami klamrowymi (wiersze 12 i 14), po czym opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 15.

Ćwiczenie 3. Negacja (!)

W tym ćwiczeniu nauczysz się wykorzystywać negację (zaprzeczenie), co oznacza wykonanie kodu w razie niespełnienia warunku.

Operatorem negacji jest znak ! (wykrzyknik).

Przeanalizuj kod z listingu 11.3 i zastanów się, jak działa.

LISTING 11.3	PHP 7/8	code_11_03.php
01	<?php	
02	/* definicja zmiennej */	
03	\$wiek = 14;	
04		
05	/* konstrukcja warunkowa */	
06	if (!(\$wiek >= 18))	
07	{	
08	print "Niestety, nie jesteś pełnoletni!";	
09	}	
10	else	
11	{	
12	print "Jesteś pełnoletni!";	
13	}	
14	/* dalszy ciąg programu */	
\$ php code_11_03.php ↵		KONSOLA
Niestety, nie jesteś pełnoletni!		

Jak działa powyższy kod?

1. W linii 03 zdefiniowana jest zmienna \$wiek.
2. Sprawdź warunek w wierszu 06:

Krok 1. Czy zmienna \$wiek jest większa bądź równa 18?

Krok 2. Zaneguj (odwróć) wartość zwróconą w kroku 1.

Jeśli warunek został spełniony (krok 2. zwraca wartość true), wykonaj kod w linii 08 pomiędzy nawiasami klamrowymi (wiersze 07 i 09), a następnie opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 14.

3. W przeciwnym przypadku wykonaj kod w linii 12 pomiędzy nawiasami klamrowymi (wiersze 11 i 13), po czym opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 14.

Ćwiczenie 4. Operatory logiczne

W tym ćwiczeniu wykorzystasz wiedzę, jaką zdobyłeś w tej lekcji, do analizy bardziej złożonej konstrukcji warunkowej. W praktyce z takimi konstrukcjami będziesz miał najczęściej do czynienia.

Przeanalizuj szczegółowo konstrukcje warunkowe w tym ćwiczeniu (listing 11.4) i zastanów się, jak działają, a następnie zmodyfikuj wartości zmiennych i przetestuj program.

LISTING 11.4	PHP 7/8	code_11_04.php
01	<?php	
02	<i>/* definicja zmiennych */</i>	
03	\$a = 151;	
04	\$b = 81;	
05	\$c = 55;	
06	\$d = 12;	
07		
08	<i>/* konstrukcja warunkowa */</i>	
09	if ((\$a >= 50 && \$b < 100) (\$c < 80 && \$d != 20))	
10	{	
11	print "Wykonuję sobie kod pierwszy.".PHP_EOL;	
12	}	
13		
14	if (!(\$a >= 50 && \$b > 100) && (\$c < 80 && \$d != 20))	
15	{	
16	print "Wykonuję sobie kod drugi.".PHP_EOL;	
17	}	
\$ php code_11_04.php ↵		KONSOLA
Wykonuję sobie kod pierwszy. Wykonuję sobie kod drugi.		

Ćwiczenia dodatkowe

1. Poszukaj więcej informacji na temat operatorów logicznych.
2. Poszukaj informacji na temat algebry Boole’a.

Wskazówki

W języku PHP możesz zamiennie używać AND z && oraz OR z ||. Choć nie powinno to mieć znaczenia podczas uruchamiania programu, zdecydowanie zalecam używanie w kodzie symboli && i ||. W konstrukcjach warunkowych w pierwszej kolejności obliczane są wartości znajdujące się w nawiasach, następnie kolejno: negacja (!), koniunkcja (AND) i alternatywa (OR). Podstawowe zasady logiczne (wyrażenia boolowskie) przedstawia poniższa tabela.

X	Y	X AND Y	X OR Y	!(X AND Y)	!(X OR Y)
0	0	0	0	1	0
1	0	0	1	1	0
0	1	0	1	1	0
1	1	1	1	0	1

Co to tak naprawdę oznacza? Wyobraź sobie, że sukcesem będzie wspólne pójście do kina z przyjacielem, czyli musi być spełniony warunek AND. Jeśli zaś uznasz, że sukcesem będzie zobaczenie filmu przynajmniej przez jednego z Was, będzie musiał być spełniony warunek OR. Poniższa tabela przedstawia taką sytuację. X i Y to Ty i Twój przyjaciel, a 0 i 1 to, odpowiednio, pójde (pójdzie) lub nie pójde (nie pójdzie) do kina.

X	Y	X AND Y	X OR Y
0	0	0	0
1	0	0	Sukces!
0	1	0	Sukces!
1	1	Sukces!	Sukces!



Znajdź coś w tekście

Podczas tej lekcji poznasz kilka ciekawych funkcji do prostej analizy tekstu. Będziesz je wykorzystywał najczęściej do sprawdzenia, czy w analizowanym łańcuchu znaków występuje konkretna litera, cyfra, czy inny ciąg znaków, a także, co najważniejsze, w którym dokładnie miejscu.

Poznasz takie funkcje jak `mb_strlen`, `mb_strpos`, `mb_stripos`, `mb_strrpos`, `strspn`. Funkcje te zwracają wartość `false`, jeśli ciąg znaków nie został odnaleziony. Musisz również pamiętać o szczególnym przypadku, kiedy szukany ciąg znaków znajduje się na samym początku. Wówczas zostanie zwrócona wartość `0`, która nie będzie oznaczać, że ciąg znaków nie został odnaleziony, ale to, że szukany ciąg znaków znajduje się na samym początku, czyli na pozycji `0`. Do obsługi takiego przypadku możesz wykorzystać operator silnego porównania `===` (trzy znaki równości) lub `!==` (wykrzyknik i dwa znaki równości).

W najnowszej wersji języka PHP 8 pojawiła się nowa funkcja `str_contains`, której zadaniem jest sprawdzenie, czy dany ciąg znaków znajduje się w łańcuchu.

Przetwarzanie tekstu w języku PHP wiąże się z tematem kodowania znaków, dlatego zachęcam do przeczytania wskazówek do tej lekcji.

Ćwiczenie 1. Wielkość liter ma znaczenie

W tym ćwiczeniu (listing 12.1) poznasz prawidłową konstrukcję warunkową, którą wykorzystasz do obsługi funkcji `mb_strpos`. Funkcja ta zwraca położenie pierwszego wystąpienia ciągu znaków (podciągu) w innym ciągu. W przypadku tej funkcji wielkość liter ma znaczenie. Jeśli ciąg nie zostanie odnaleziony, funkcja zwróci wartość `false`.

Zwróć uwagę na silne porównanie `!==` (wykrzyknik i dwa znaki równości) w linii 09. Jeśli zależy Ci tylko na informacji, czy podciąg znajduje się w innym ciągu, wykorzystaj funkcję `str_contains`, którą opisałem w lekcji o łańcuchach.

LISTING 12.1	PHP 7/8	code_12_01.php
01	<?php	
02	<i>// słynne słowa Neila Armstronga</i>	
03	<code>\$myText = "To jest mały krok dla człowieka, ale wielki skok dla ludzkości.";</code>	
04	<code>\$searchText = "dla";</code>	
05		
06	<code>\$pos = mb_strpos(\$myText, \$searchText);</code> <i>// sprawdź pozycję</i>	
07		
08	<i>// tekst został odnaleziony?</i>	
09	<code>if (\$pos !== false)</code> <i>// tak</i>	
10	{	
11	<code>print "Pierwsze wystąpienie ciągu znaków \"\$searchText\" ";</code>	
12	<code>print "zostało odnalezione na pozycji nr \$pos.";</code>	
13	}	
14	<code>else</code> <i>// nie</i>	
15	{	
16	<code>print "Szukany tekst nie został odnaleziony.";</code>	
17	}	
18	<i>// dalszy ciąg programu</i>	
\$ php code_12_01.php ↵		KONSOLA
Pierwsze wystąpienie ciągu znaków "dla" zostało odnalezione na pozycji nr 18.		

Jak działa powyższy kod?

1. W liniach 03 i 04 zdefiniowane są zmienne `$myText` i `$searchText`.
2. W wierszu 06 do zmiennej `$pos` przypisana jest wartość funkcji `mb_strpos`, zwracającej pozycję pierwszego wystąpienia podciągu `$searchText` w ciągu `$myText`, przy czym wielkość liter ma znaczenie. Jeśli podciąg nie został odnaleziony, zwracana jest wartość `false`.
3. Sprawdź warunek w linii 09, czy zmienna `$pos` nie jest równa wartości logicznej `false`, czyli czy podciąg został odnaleziony, a następnie wykonaj kod w wierszach od 11 do 12 pomiędzy nawiasami klamrowymi (linie 10 i 13). Następnie opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 18.
4. W przeciwnym przypadku, jeśli ciąg znaków nie został odnaleziony, wykonaj kod w linii 16 pomiędzy nawiasami klamrowymi (wiersze 15 i 17), po czym opuść konstrukcję warunkową i wykonaj kod, począwszy od linii 18.

Ćwiczenie 2. Pierwsze i ostatnie wystąpienie ciągu

W następnym ćwiczeniu zastosujesz funkcje `mb_stripos` i `mb_strrpos` do wyszukania położenia pierwszego i ostatniego wystąpienia ciągu znaków w tekście. W przypadku funkcji `mb_stripos` wielkość liter nie ma znaczenia. W tym ćwiczeniu zastosowałem bardziej rozbudowaną konstrukcję warunkową `if-else`, a także wykorzystałem koniunkcję (iloczyn logiczny). Przeanalizuj kod z listingu 12.2 i zastanów się, jak działa.

LISTING 12.2	PHP 7/8	code_12_02.php
01	<?php	
02	\$myText = "Jak czytać KSIĄŻKI i dlaczego książki warto czytać?";	
03	\$searchText = "książki";	
04		
05	\$pos1 = mb_stripos(\$myText, \$searchText); <i>// sprawdź pierwsze wystąpienie</i>	
06	\$pos2 = mb_strrpos(\$myText, \$searchText); <i>// sprawdź ostatnie wystąpienie</i>	
07		
08	<i>// tekst został odnaleziony?</i>	
09	if (\$pos1 === false && \$pos2 === false) <i>// nie</i>	
10	{	
11	print "Szukany tekst nie został odnaleziony.";	
12	}	
13	else <i>// tak</i>	
14	{	
15	if (\$pos1 !== false) { <i>// pierwsze</i>	
16	print "Pierwsze wystąpienie ciągu znaków \"\$searchText\" ";	
17	print "zostało odnalezione na pozycji nr \$pos1.";	
18	}	
19	print PHP_EOL;	
20	if (\$pos2 !== false) { <i>// ostatnie</i>	
21	print "Ostatnie wystąpienie ciągu znaków \"\$searchText\" ";	
22	print "zostało odnalezione na pozycji nr \$pos2.";	
23	}	
24	}	
25	<i>// dalszy ciąg programu</i>	
\$ php code_12_02.php ↵		KONSOLA
Pierwsze wystąpienie ciągu znaków "książki" zostało odnalezione na pozycji nr 11. Ostatnie wystąpienie ciągu znaków "książki" zostało odnalezione na pozycji nr 30.		

Jak działa powyższy kod?

1. W liniach 02 i 03 zdefiniowane są zmienne `$myText` i `$searchText`.
2. W wierszu 05 do zmiennej `$pos1` przypisana jest wartość funkcji `mb_stripos`, zwracającej pozycję pierwszego wystąpienia podciągu `$searchText` w ciągu `$myText`, przy czym wielkość liter nie ma znaczenia. Jeśli podciąg nie został odnaleziony, zwracana jest wartość `false`.
3. W linii 06 do zmiennej `$pos2` przypisana jest wartość funkcji `mb_strrpos`, zwracającej pozycję ostatniego wystąpienia podciągu `$searchText` w ciągu `$myText`; tutaj również wielkość liter nie ma znaczenia. Jeśli podciąg nie został odnaleziony, zwracana jest wartość `false`.
4. W wierszu 09 sprawdzamy, czy zmienne `$pos1` i `$pos2` zwracają wartości `false`, czyli czy szukany podciąg został odnaleziony. Jeśli nie, wykonaj kod w linii 11 pomiędzy nawiasami klamrowymi (wiersze 10 i 12), a następnie opuść konstrukcję warunkową i wykonaj kod, począwszy od linii 25.
5. W przeciwnym razie, czyli kiedy podciąg został odnaleziony, sprawdź warunki w wierszach 15 i 20. Na podstawie wcześniejszego ćwiczenia bez problemu zrozumiesz, jak działają.

Ćwiczenie 3. Długość łańcucha

W swoich programach z pewnością będziesz obliczał długość ciągu znaków. W tym ćwiczeniu (listing 12.3) poznasz i wykorzystasz funkcję `mb_strlen`.

Spróbuj zamiast funkcji `mb_strlen` w liniach 05 i 06 wstawić funkcję `strlen` i porównaj drukowane informacje. Potem zajrzyj do wskazówek do niniejszej lekcji.

LISTING 12.3	PHP 7/8	code_12_03.php
01	<?php	
02	\$myText1 = "Życie jest jak pudełko czekoladek - nigdy nie wiesz, co ci się trafi.";	
03	\$myText2 = "Miej serce i patrzaj w serce.";	
04		
05	\$len1 = mb_strlen(\$myText1);	
06	\$len2 = mb_strlen(\$myText2);	
07		
08	if (\$len1 == \$len2)	
09	{	
10	print "Ciągi znaków mają tę samą długość.";	
11	}	

```

12 else
13 {
14     print "Ciągi znaków nie mają tej samej długości.".PHP_EOL;
15     print "Pierwszy ma długość $len1 znaków, a drugi ma długość $len2 znaków.";
16 }
17 // dalszy ciąg programu

```

```
$ php code_12_03.php ↵
```

KONSOLA

```

Ciągi znaków nie mają tej samej długości.
Pierwszy ma długość 69 znaków, a drugi ma długość 29 znaków.

```

Jak działa powyższy kod?

1. W liniach 02 i 03 zdefiniowane są zmienne `$myText1` i `$myText2`.
2. W wierszach 05 i 06 do zmiennych `$len1` i `$len2` przypisana jest wartość funkcji `mb_strlen`, zwracającej długość łańcucha `$myText1` i `$myText2`.
3. W linii 08 sprawdzamy warunek, czy zmienne `$len1` i `$len2` zawierają te same wartości. Jeśli tak, wykonaj kod w linii 10 pomiędzy nawiasami klamrowymi (wiersze 09 i 11), po czym opuść konstrukcję warunkową i wykonaj kod, począwszy od linii 17.
4. W przeciwnym razie, czyli kiedy zmienne `$len1` i `$len2` są różne, wykonaj kod w wierszach 14 i 15 pomiędzy nawiasami klamrowymi (linie 13 i 16), a następnie opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 17.

Ćwiczenie 4. Użycie maski

W tym ćwiczeniu (listing 12.4) wykorzystasz funkcję `strspn`, która informuje, jak dużo znaków na początku łańcucha jest złożonych z określonych znaków. Funkcja ta nadaje się do sprawdzenia, czy łańcuch składa się z konkretnych znaków, np. czy wprowadzony przez użytkownika tekst jest numerem telefonu, czy liczbą binarną. Jeśli liczba zwracana przez funkcję `strspn` jest równa długości łańcucha, to oznacza, że łańcuch ten złożony jest tylko z określonych znaków. Może się to wydawać skomplikowane, ale naprawdę jest bardzo proste.

LISTING 12.4

PHP 7/8

code_12_04.php

```

01 <?php
02 $myText = "01101011"; // łańcuch z zerami i jedynekami
03 $mask = "01"; // tych znaków szukamy
04

```

```

05 if (strpos($myText, $mask) == strlen($myText))
06 {
07     print "łańcuch '$myText' to ciąg znaków binarnych.".PHP_EOL;
08 }
09 // dalszy ciąg programu

```

```
$ php code_12_04.php ↵
```

KONSOLA

```
łańcuch '01101011' to ciąg znaków binarnych.
```

Jak działa powyższy kod?

1. W liniach 02 i 03 zdefiniowane są zmienne `$myText` i `$mask`.
2. W wierszu 05 sprawdzamy warunek, czy liczba wystąpień znaków określonych w zmiennej `$mask` na początku łańcucha `$myText` jest równa jego długości. Jeśli tak, wykonaj kod w linii 07 pomiędzy nawiasami klamrowymi (wiersze 06 i 08), a następnie opuść konstrukcję warunkową i wykonaj kod, począwszy od wiersza 09.

Ćwiczenia dodatkowe

1. Zmień nazwę funkcji w ćwiczeniu 1 z `mb_strpos` na `strpos` i porównaj wyniki. Dlaczego się różnią? Odpowiedź znajdziesz we wskazówkach do tej lekcji.
2. Otwórz stronę <http://php.net> i wyszukaj opis funkcji `mb_strstr` oraz `mb_strrstr`. Spróbuj zrozumieć, jak działają. Przepisz i uruchom przykładowe listingi.
3. Poszukaj informacji na temat wielobajтового kodowania znaków (UTF-8).
4. Znajdź tabelę kodów ASCII i zapoznaj się z nią.

Wskazówki

Pisząc programy w PHP, musisz wiedzieć, że istnieje coś takiego jak 7-bitowy system kodowania znaków (ASCII). Siedem bitów oznacza 128 możliwych do zakodowania znaków — liter, cyfr, znaków przestankowych i innych symboli. Z początku ASCII dobrze sprawdzał się w obsłudze alfabetu języka angielskiego, jednak nie było w nim miejsca na znaki narodowe innych alfabetów. Częściowo rozwiązano ten problem przez wprowadzenie 8-bitowego (1-bajtowego) systemu kodowania znaków (256 możliwych do zakodowania znaków), czego przykładem są systemy kodowania ISO-8859 i Windows-1250, obsługujące polskie znaki narodowe.

Dopiero wprowadzenie Unicode pozwoliło umieścić w jednym kodowaniu wszystkie alfabety używane na całym świecie.

Najczęściej stosowanym standardem kodowania plików i informacji rozpowszechnianych w sieci jest UTF-8, który używa od 1 do 4 bajtów do zakodowania jednego znaku. Kodowanie UTF-8 pozwala zakodować ponad milion różnych znaków. Wykorzystanie więcej niż jednego bajta do zakodowania znaku w praktyce oznacza, że litery narodowe (znaki diakrytyczne) będą zajmowały więcej pamięci. W poniższej tabeli umieściłem przykładowe ciągi znaków z informacją, ile zajmują pamięci.

Tekst	Liczba znaków	ASCII	ISO-8859	UTF-8
A	1	7 bitów	8 bitów (1 bajt)	8 bitów (1 bajt)
PHP	3	21 bitów	24 bity (3 bajty)	24 bity (3 bajty)
Ą	1	-	8 bitów (1 bajt)	16 bitów (2 bajty)
SŁOŃ	4	-	32 bity (4 bajty)	48 bitów (6 bajtów)
日本語の表記体系	8	-	-	192 bity (24 bajty)

Kodowanie w UTF-8 umożliwia obsługę wszystkich alfabetów, co jest jego dużą zaletą. Pamiętaj tylko, że odbywa się to kosztem pamięci, w której znaki są przechowywane. W języku PHP dostępna jest biblioteka Multibyte String (mbstring), która ma funkcje obsługujące wielobajtowe kodowanie znaków. Funkcje z tej biblioteki rozpoczynają się od przedrostka `mb_`. Jeśli nie możesz korzystać z biblioteki Multibyte String w swoich programach, usuń przedrostek w nazwie funkcji (wykonaj ćwiczenie dodatkowe nr 1, aby zobaczyć, jakie będą wówczas różnice w wynikach).

Pytania

Dlaczego w ćwiczeniu 1 słowo „dla” zostało odnalezione tylko raz? Przecież występuje dwa razy.

Dlatego, że funkcja `mb_strpos` zwraca pozycję pierwszego wystąpienia szukanego ciągu znaków. Otwórz stronę <http://php.net> i wyszukaj opis funkcji `mb_strpos`. Zwróć uwagę, że możesz podać również trzeci parametr, określający, od której pozycji funkcja ma rozpocząć przeszukiwanie.



Kombinacje z tekstem

W języku PHP dostępnych jest wiele ciekawych funkcji, które wykorzystasz do przetwarzania tekstu. Zamiana wielkich liter na małe i odwrotnie, zastąpienie fragmentu tekstu innym tekstem czy usunięcie fragmentu tekstu z łańcucha znaków nie będzie już dla Ciebie problemem.

Poznasz funkcje: `mb_strtolower`, `mb_strtoupper`, `mb_substr`, `str_replace`, `ucfirst`, `ucwords` i `mb_convert_case`.

Ćwiczenie 1. Zamiana wielkości liter

W tym ćwiczeniu poznasz nowe funkcje do zmiany wielkości liter w ciągu znaków, a mianowicie: `mb_strtolower`, `mb_strtoupper`, `ucfirst`, `ucwords` i `mb_convert_case`. Jestem przekonany, że doskonale odgadniesz, jak te funkcje działają, analizując listing 13.1. Pamiętaj, że funkcje `ucfirst` i `ucwords` (linie 07 i 10) nie mają odpowiednika w bibliotece `mbstring`, dlatego nie będą one poprawnie działać, jeśli w łańcuchu znajdują się znaki diakrytyczne. Lepszym wariantem jest wykorzystanie funkcji `mb_convert_case` z odpowiednim parametrem (wiersz 11).

LISTING 13.1	PHP 7/8	code_13_01.php
01	<?php	
02	\$myText = "Działaj! Nie jutro, nie za godzinę. Teraz!";	
03	\$myTextLower = mb_strtolower (\$myText); // zamiana na małe litery	
04	\$myTextUpper = mb_strtoupper (\$myText); // zamiana na wielkie litery	
05		
06	// wielka litera na początku łańcucha	
07	\$myTextFirst1 = ucfirst (\$myTextLower);	
08		
09	// wielka litera na początku każdego wyrazu w łańcuchu	
10	\$myTextWords1 = ucwords (\$myTextLower);	
11	\$myTextWords2 = mb_convert_case (\$myTextLower, MB_CASE_TITLE, 'UTF-8');	
12		

```

13 print $myText.PHP_EOL;
14 print "mb_strtolower: $myTextLower".PHP_EOL;
15 print "mb_strtoupper: $myTextUpper".PHP_EOL;
16 print "ucfirst: $myTextFirst1".PHP_EOL;
17 print "ucwords: $myTextWords1".PHP_EOL;
18 print "mb_convert_case: $myTextWords2".PHP_EOL;

```

```
$ php code_13_01.php ↵
```

KONSOLA

```

Działaj! Nie jutro, nie za godzinę. Teraz!
mb_strtolower: działaj! nie jutro, nie za godzinę. teraz!
mb_strtoupper: DZIAŁAJ! NIE JUTRO, NIE ZA GODZINĘ. TERAZ!
ucfirst: Działaj! nie jutro, nie za godzinę. teraz!
ucwords: Działaj! Nie Jutro, Nie Za Godzinę. Teraz!
mb_convert_case: Działaj! Nie Jutro, Nie Za Godzinę. Teraz!

```

Ćwiczenie 2. Wycinanie fragmentów tekstu

W tym ćwiczeniu (listing 13.2) nauczysz się za pomocą funkcji `mb_substr` wycinać fragmenty tekstu (podciągów) z ciągu znaków, w zależności od podanych parametrów. Pierwszy parametr wskazuje na źródłowy ciąg znaków, a następne oznaczają pozycję, od której ma nastąpić wycięcie, i długość (w znakach) wycięcia. Zwróć uwagę na linie od 07 do 09 i zastanów się, jak ta funkcja działa, na podstawie moich komentarzy. Zmień wartości zmiennej `$myCut` i zobacz, co się stanie.

LISTING 13.2

PHP 7/8

code_13_02.php

```

01 <?php
02 $myText = "Z uśmiechem na twarzy człowiek podwaja swoje możliwości.";
03
04 $myTextLen = mb_strlen($myText); // liczba znaków w tekście
05 $myCut = 22; // po którym znaku wycinamy tekst
06
07 $newText1 = mb_substr($myText, 0, 21); // pierwsze 21 znaków
08 $newText2 = mb_substr($myText, $myCut, $myTextLen-$myCut); // od znaku 22 do końca
09 $newText3 = mb_substr($myText, $myCut, 8); // 8 znaków, zaczynając od 22
10
11 print $newText1.PHP_EOL;
12 print $newText2.PHP_EOL;
13 print $newText3.PHP_EOL;

```

```
$ php code_13_02.php ↵
```

KONSOLA

```

Z uśmiechem na twarzy
człowiek podwaja swoje możliwości.
człowiek

```

Ćwiczenie 3. Zamiana fragmentów tekstu

A teraz podmienimy fragment tekstu na inny za pomocą funkcji `str_replace`. Przeanalizuj kod z listingu 13.3 w wierszach 04 i 05, a z pewnością zorientujesz się, jak ta funkcja działa.

LISTING 13.3	PHP 7/8	code_13_03.php
01	<?php	
02	\$myText = "Oczywiście, że dzik jest najlepszym przyjacielem człowieka!";	
03		
04	\$myTextDog = str_replace ('dzik', 'pies', \$myText); <i>// zamiana dzik na pies</i>	
05	\$myTextCat = str_replace ('dzik', 'kot', \$myText); <i>// zamiana dzik na kot</i>	
06		
07	print \$myText.PHP_EOL;	
08	print \$myTextDog.PHP_EOL;	
09	print \$myTextCat.PHP_EOL;	
\$ php code_13_03.php ↵		KONSOLA
Oczywiście, że dzik jest najlepszym przyjacielem człowieka!		
Oczywiście, że pies jest najlepszym przyjacielem człowieka!		
Oczywiście, że kot jest najlepszym przyjacielem człowieka!		

Ćwiczenie 4. Odwracanie i powtarzanie tekstu

A teraz odwrócimy tekst za pomocą funkcji `strrev` i utworzymy łańcuch składający się z ustalonej liczby konkretnego znaku. Przeanalizuj kod z listingu 13.4 w liniach 04 i 05 oraz wynik działania programu, a z pewnością zorientujesz się, jak te funkcje działają.

LISTING 13.4	PHP 7/8	code_13_04.php
01	<?php	
02	\$myText = "PROGRAMOWANIE";	
03		
04	\$myTextRev = strrev (\$myText); <i>// odwróć tekst</i>	
05	\$myTextRepeat = str_repeat ('*', 10); <i>// utwórz łańcuch z 10 znaków *</i>	
06		
07	print \$myTextRev." to odwrócony tekst ".\$myText.PHP_EOL;	
08	print "10 gwiazdek: ".\$myTextRepeat.PHP_EOL;	
\$ php code_13_04.php ↵		KONSOLA
EINAWOMARGORP to odwrócony tekst PROGRAMOWANIE		
10 gwiazdek: **********		

Ćwiczenie 5. Porównanie ciągów znaków

Kiedy będziesz porównywał ze sobą dwa ciągi znaków, a wielkość liter nie będzie miała znaczenia, postaraj się przed porównaniem przetworzyć ciągi funkcjami `mb_strtolower` i `trim`. Unikniesz sytuacji, kiedy program błędnie porówna ze sobą zmienne zawierające np. imię użytkownika. Na listingu 13.5 w linii 06 została wykonana wstępna optymalizacja, a samo porównanie nastąpiło już w wierszu 08.

LISTING 13.5	PHP 7/8	code_13_05.php
01	<?php	
02	\$name = "indra"; //wszystkie małe litery (wzorzec)	
03	\$enter = " Indra "; //imię wpisane przez użytkownika	
04		
05	//pozbycie się spacji na początku i na końcu oraz zamiana na małe litery	
06	\$enter = mb_strtolower(trim(\$enter)); //przypisujemy wynik do tej samej zmiennej	
07		
08	if (\$name == \$enter) {	
09	print "Imiona są identyczne!";	
10	} else {	
11	print "Imiona się różnią!";	
12	}	
\$ php code_13_05.php ↵		KONSOLA
Imiona są identyczne!		

Ćwiczenia dodatkowe

1. Otwórz stronę <http://php.net> i wyszukaj opis funkcji `mb_substr` oraz `str_replace`. Spróbuj zrozumieć, jak działają, przepisz i uruchom przykładowe listingi.

Wskazówki

Pamiętaj, że dla Ciebie ciągi znaków „MyApp” i „myapp” oznaczają to samo, jednak dla programu są to dwa różne ciągi. Właśnie dlatego przed porównaniem takich ciągów warto zastosować funkcje `mb_strtolower` lub `mb_strtoupper`.

	mb_strtolower	mb_strtoupper
(MyApp == myapp) ?	(myapp == myapp) ?	(MYAPP == MYAPP) ?
NIE	TAK	TAK



Operacje na liczbach

Niemal w każdym języku programowania, również w PHP, będziesz mógł wykonać nawet najbardziej skomplikowane operacje matematyczne. W tej lekcji dowiesz się, jak reprezentowane są liczby w języku PHP, i przeprowadzisz na nich proste operacje arytmetyczne.

W języku PHP liczby mogą być typu całkowitego lub rzeczywistego. Mam nadzieję, że wiesz, jaka jest różnica między zbiorem liczb całkowitych a zbiorem liczb rzeczywistych. Występuje też typ logiczny `true` (prawda) i `false` (fałsz). W poniższej tabeli umieściłem typy danych, których będziesz najczęściej używał do obliczeń.

Typ	Opis	Przykład
integer	liczby całkowite	-1920, -500, -34, -8, 0, 1, 2, 99, 2021
float	liczby rzeczywiste	-890.56, -44.12, -20, 0, 1.5, 4, 77.5
boolean	prawda lub fałsz	true (1), false (0)

W celu wykonania określonych obliczeń najczęściej będziesz korzystał z operatorów arytmetycznych, logicznych i bitowych, które w znacznej mierze przypominają oznaczenia stosowane w matematyce. W poniższej tabeli umieściłem te operatory, których będziesz używał najczęściej.

Operatory arytmetyczne

Operacja	Operator	Przykład
dodawanie	+	<code>\$a + \$b</code>
odejmowanie	-	<code>\$a - \$b</code>
mnożenie	*	<code>\$a * \$b</code>
dzielenie	/	<code>\$a / \$b</code>
modulo	%	<code>\$a % \$b</code>
potęgowanie	**	<code>\$a ** \$b</code>

Operacja	Operator	Przykład
inkrementacja	++	<code>\$a++</code>
dekrementacja	--	<code>\$b--</code>

Operatory logiczne

Operacja	Operator	Przykład
iloczyn logiczny	<code>&&</code>	<code>\$a && \$b = true, jeśli \$a i \$b = true</code>
suma logiczna	<code> </code>	<code>\$a \$b = true, jeśli \$a lub \$b = true</code>
logiczna alternatywa wykluczająca	<code>xor</code>	<code>(\$a xor \$b) = true, jeśli \$a lub \$b = true, ale nie jednocześnie</code>
negacja logiczna	<code>!</code>	<code>!\$a = true false, jeśli \$a = false true</code>

Operatory bitowe

Operacja	Operator	Przykład
iloczyn bitowy	<code>&</code>	<code>\$a & \$b</code>
suma bitowa	<code> </code>	<code>\$a \$b</code>
bitowa alternatywa wykluczająca	<code>^</code>	<code>\$a ^ \$b</code>
negacja bitowa	<code>~</code>	<code>~\$a</code>
przesunięcie bitowe w prawo	<code>>></code>	<code>\$a >> \$c</code>
przesunięcie bitowe w lewo	<code><<</code>	<code>\$a << \$c</code>

Ćwiczenie 1. Operatory arytmetyczne

Na początek bardzo proste obliczenia. Wykorzystasz ponownie funkcję `printf` do formatowania tekstu. Przykładowo w linii 06 funkcja przyjmuje dwa argumenty oddzielone znakiem przecinka. Pierwszy ujęty jest w cudzysłów i zawiera tekst, który będzie wydrukowany. Zauważ, że jest w nim zawarty specjalny znak `%d`, pod który będzie podstawiany drugi argument (suma `$x` i `$y`). Dla liczb rzeczywistych użyjesz znaku `%f` (linia 10). Przeanalizuj kod z listingu 14.1 wiersz po wierszu i postaraj się go zrozumieć.

LISTING 14.1	PHP 7/8	code_14_01.php
01	<code><?php</code>	
02	<code>/* definicja zmiennych */</code>	
03	<code>\$x = 18;</code>	

```

04 $y = 32;
05
06 printf("wartości początkowe: x = %d, y = %d".PHP_EOL, $x, $y);
07
08 printf("x + y = %d".PHP_EOL, $x + $y); // suma
09 printf("x - y = %d".PHP_EOL, $x - $y); // różnica
10
11 printf("x * y = %d".PHP_EOL, $x * $y); // iloczyn
12 printf("x / y = %.4f".PHP_EOL, $x / $y); // iloraz (pokaż 4 miejsca po przecinku)
13
14 printf("x * x = %d".PHP_EOL, $x ** 2); // do kwadratu
15 printf("x * x * x * x * x = %d".PHP_EOL, $x ** 5); // do 5 potęgi
16
17 printf("x mod y = %d".PHP_EOL, $x % $y); // reszta z dzielenia (modulo)
18
19 printf("wartości początkowe: x = %d, y = %d".PHP_EOL, $x, $y);
20 $x++; $y++; // inkrementacja (zwiększenie o jeden)
21
22 printf("inkrementacja: x = %d, y = %d".PHP_EOL, $x, $y);
23
24 $x--; $y--; // dekrementacja (zmniejszenie o jeden)
25 printf("dekrementacja: x = %d, y = %d".PHP_EOL, $x, $y);

```

```
$ php code_14_01.php ↵
```

KONSOLA

```

wartości początkowe: x = 18, y = 32
x + y = 50
x - y = -14
x * y = 576
x / y = 0.5625
x * x = 324
x * x * x * x * x = 1889568
x mod y = 18
wartości początkowe: x = 18, y = 32
inkrementacja: x = 19, y = 33
dekrementacja: x = 18, y = 32

```

Ćwiczenie 2. Operatory rzutowania

W języku PHP możesz z góry ustalić typ zmiennej — nazywa się to rzutowaniem lub typowaniem. Na listingu 14.2 w liniach 03 i 04 zdefiniowałem zmienne, jednoznacznie ustalając ich typ. Zmienna `$x` reprezentuje liczby całkowite (`int`), a zmienna `$y` — liczby rzeczywiste (`float`). W wierszach 06 i 12 zmienne są ze sobą porównywane co do wartości (dwa znaki równości) oraz co do wartości i typu (trzy znaki równości). Zauważ, że w drugim porównaniu zmienne nie są identyczne. Jak myślisz, dlaczego?

LISTING 14.2	PHP 7/8	code_14_02.php
01	<?php	
02	<i>/* definicja zmiennych */</i>	
03	\$x = (int)2021; <i>// typu integer</i>	
04	\$y = (float)2021; <i>// typu float</i>	
05		
06	if (\$x == \$y) { <i>// sprawdź tylko wartość</i>	
07	print "Zmienne mają tę samą wartość.".PHP_EOL;	
08	} <i>else {</i>	
09	print "Zmienne są różnej wartości.".PHP_EOL;	
10	}	
11		
12	if (\$x === \$y) { <i>// sprawdź wartość i typ</i>	
13	print "Zmienne mają tę samą wartość i są tego samego typu.".PHP_EOL;	
14	} <i>else {</i>	
15	print "Zmienne są różnego typu.".PHP_EOL;	
16	}	
\$ php code_14_02.php ↵		KONSOLA
Zmienne mają tę samą wartość. Zmienne są różnego typu.		

Ćwiczenie 3. Operatory logiczne

W tym ćwiczeniu wykorzystasz typ logiczny (boolean). Może on przybrać wartość `true` (prawda) lub `false` (fałsz). O co chodzi w kodzie z listingu 14.3? Warunkiem wyjazdu na plażę jest koniec tygodnia, pogoda i pora dnia. Przeanalizuj konstrukcję warunkową (linie od 07 do 13) i zastanów się, jak działa. Uruchom program przy różnych wartościach zmiennych.

LISTING 14.3	PHP 7/8	code_14_03.php
01	<?php	
02	<i>/* definicja zmiennych */</i>	
03	\$weekend = true; <i>// jest weekend?</i>	
04	\$sun = true; <i>// świeci słońce?</i>	
05	\$hour = 10; <i>// która godzina?</i>	
06		
07	if (\$weekend === true) { <i>// tak?</i>	
08	if (\$sun === true && \$hour < 18) { <i>// świeci słońce i nie jest za późno?</i>	
09	print "Hurra! Jedziemy na plażę!";	
10	} <i>else {</i>	
11	print "Znowu siedzimy w domu :(";	
12	}	
13	}	


```
$ php code_14_03.php ↵
```

KONSOLA

```
Hurra! Jedziemy na plażę!
```

Ćwiczenie 4. Operatory bitowe

W tym ćwiczeniu wykorzystasz operatory bitowe do prostych obliczeń. Na listingu 14.4 w liniach 03 i 04 zdefiniowane są dwie zmienne, na których w wierszach od 07 do 11 wykonywane są operacje bitowe. Wykorzystałem funkcję `decbin`, konwertującą wartość w systemie dziesiętnym na system binarny (dwójkowy).

Przykładowo iloczyn bitowy (linia 07) ustawia bit na 1, jeśli oba porównywane bity mają wartość 1, czyli:

```
  1010
& 1100
-----
  1000
```

Zauważ, że tylko pierwsze bity mają wartość 1, dlatego w wyniku również pierwszy bit będzie miał wartość 1, natomiast pozostałe wartość 0.

LISTING 14.4

PHP 7/8

code_14_04.php

```
01 <?php
02 /* definicja zmiennych */
03 $a = 10; // 1010 binarnie
04 $b = 12; // 1100 binarnie
05
06 /* operacje bitowe */
07 $x1 = $a & $b; // iloczyn bitowy (AND)
08 $x2 = $a | $b; // suma bitowa (OR)
09 $x3 = $a ^ $b; // alternatywa wykluczająca (XOR)
10 $x4 = $a >> 2; // przesun o 2 bity w prawo
11 $x5 = $a << 2; // przesun o 2 bity w lewo
12
13 printf("\$a = %06d".PHP_EOL, decbin($a));
14 printf("\$a = %06d".PHP_EOL, decbin($b));
15 printf(str_repeat('-', 11).PHP_EOL);
16 printf("AND: %06d".PHP_EOL, decbin($x1));
17 printf("OR:  %06d".PHP_EOL, decbin($x2));
18 printf("XOR: %06d".PHP_EOL, decbin($x3));
19 printf(">>: %06d".PHP_EOL, decbin($x4));
20 printf("<<:  %06d".PHP_EOL, decbin($x5));
```

```
$ php code_14_04.php ↵
```

KONSOLA

```
$a = 001010  
$a = 001100  
-----  
AND: 001000  
OR: 001110  
XOR: 000110  
>>: 000010  
<<: 101000
```

Ćwiczenia dodatkowe

1. Otwórz stronę <http://php.net> i wyszukaj opis funkcji `printf`, `sprintf`, `round`, `ceil`, `floor` i `number_format`. Spróbuj zrozumieć, jak działają. Przepisz i uruchom przykładowe listingi.

Wskazówki

Podczas obliczeń możesz wpaść w pułapkę liczb zmiennoprzecinkowych. Chodzi o to, że liczby rzeczywiste reprezentowane są przez część całkowitą oraz część ułamkową i, co najważniejsze, są tylko pewnym przybliżeniem konkretnej wartości (bardzo dużej lub bardzo małej). W praktyce oznacza to, że dwie liczby rzeczywiste (zmiennoprzecinkowe) o pozornie tej samej wartości nie będą sobie równe.

Kod z listingu 14.5 przedstawia problem liczb zmiennoprzecinkowych. Po zdefiniowaniu w liniach od 03 do 05 zmiennych `float` następują proste obliczenia w wierszach 07 i 08 oraz przypisanie ich do zmiennych `$c1` i `$c2`. Teoretycznie powinny być sobie równe, jednak w przypadku porównania w liniach 13 i 17 możemy nabrać pewnych wątpliwości. Dopiero po zaokrągleniu tych zmiennych (np. do ośmiu miejsc po przecinku, co w praktyce zupełnie wystarczy) będziemy mogli je ze sobą porównać. Tak, poznałeś właśnie nową funkcję `round`, którą wykorzystasz do precyzyjnego zaokrąglania liczb.

LISTING 14.5	PHP 7/8	code_14_05.php
01	<?php	
02	/* definicja zmiennych */	
03	\$x = (float)1;	
04	\$y = (float)10;	
05	\$z = (float)0.9;	
06		
07	\$c1 = \$x / \$y; // 1 / 10 = 0.1	
08	\$c2 = \$x - \$z; // 1 - 0.9 = 0.1	
09		
10	print "Czy c1 = \$c1 i c2 = \$c2 są sobie równe?".PHP_EOL;	
11	# printf("Czy c1 = %.20f i c2 = %.20f są sobie równe?".PHP_EOL, \$c1, \$c2);	
12		
13	if (\$c1 == \$c2) { //niestety, nie są równe	
14	print "operator ==";	
15	}	
16		
17	if (\$c1 === \$c2) { //a teraz? też nie!	
18	print "operator ===";	
19	}	
20		
21	if (round(\$c1, 8) === round(\$c2, 8)) { // trik z zaokrągleniem	
22	print "Dopiero po zastosowaniu zaokrąglenia do 8 miejsc po przecinku.";	
23	}	
\$ php code_14_05.php ↵		KONSOLA
Czy c1 = 0.1 i c2 = 0.1 są sobie równe?		
Dopiero po zastosowaniu zaokrąglenia do 8 miejsc po przecinku.		

W kodzie ukryłem dla Ciebie niespodziankę. Usuń znak hash'a w linii 11 i uruchom program. Zobaczysz, jaką naprawdę wartość mają zmienne \$c1 i \$c2 po wykonaniu obliczeń w wierszach 07 i 08. Jestem przekonany, że zrozumiesz, dlaczego w liniach 13 i 16 zmienne te nie mogły być sobie równe.



Trochę matmy dla opornych

W tej lekcji zrelaksujesz się przy rozwiązywaniu prostych zadań z matematyki. Przećwiczysz operatory arytmetyczne poznane na wcześniejszej lekcji. A za-tem do dzieła!

Ćwiczenie 1. Pola figur płaskich

W tym ćwiczeniu przypomnisz sobie wzory na pola figur płaskich. Aby nie utrudniać, celowo pominąłem jednostki (mam nadzieję, że wiesz, w jakich jednostkach wyrażamy pole figur). Na listingu 15.1 znajdziesz nowe funkcje, takie jak `sqrt`, obliczającą pierwiastek kwadratowy, oraz `pi`, funkcję zwracającą wartość stałej Archimedesza, czyli π .

LISTING 15.1	PHP 7/8	code_15_01.php
01	<?php	
02	/*	
03	<i>* Dany jest kwadrat o boku $a = 12$</i>	
04	<i>* Oblicz obwód, pole i przekątną kwadratu</i>	
05	<i>*/</i>	
06	<code>\$a = 12;</code>	
07	<code>\$p = \$a**2; //pole kwadratu</code>	
08	<code>\$o = \$a*4; // obwód kwadratu</code>	
09	<code>\$q = \$a*sqrt(2); //przekątna kwadratu, sqrt oznacza pierwiastek</code>	
10		
11	<code>printf("Pole kwadratu o boku %.1f wynosi %.1f \n", \$a, \$p);</code>	
12	<code>printf("Obwód kwadratu o boku %.1f wynosi %.1f \n", \$a, \$o);</code>	
13	<code>printf("Przekątna kwadratu o boku %.1f wynosi %.1f \n", \$a, \$q);</code>	
14		
15	<i>/*</i>	
16	<i>* W trójkącie prostokątnym o bokach abc bok $a = 25$, $b = 12$</i>	
17	<i>* a i b są przyprostokątnymi, c jest przeciwprostokątną</i>	
18	<i>* Oblicz obwód i pole trójkąta</i>	
19	<i>*/</i>	
20	<code>\$a = 25;</code>	
21	<code>\$b = 12;</code>	

```

22 $c = sqrt($a**2 + $b**2); // sqrt oznacza pierwiastek
23 $o = $a + $b + $c; // obwód trójkąta
24 $po = $o / 2; // połowa obwodu trójkąta
25 $p = sqrt($po*($po-$a)*($po-$b)*($po-$c)); // pole trójkąta
26
27 printf("Pole trójkąta o bokach a=%.1f, b=%.1f, c=%.1f wynosi %.1f".PHP_EOL, $a,
    $b, $c, $p);
28 printf("Obwód trójkąta o bokach a=%.1f, b=%.1f, c=%.1f wynosi %.1f".PHP_EOL, $a,
    $b, $c, $o);
29
30 /*
31  * Dane jest koło o średnicy r = 45
32  * Oblicz pole i obwód koła
33  */
34 $r = 45;
35 $p = pi()*($r**2); // pole koła, π pobieramy z funkcji pi()
36 $o = 2*pi()*$r; // obwód koła
37
38 printf("Pole koła o promieniu %.1f wynosi %.1f".PHP_EOL, $r, $p);
39 printf("Obwód koła o promieniu %.1f wynosi %.1f".PHP_EOL, $r, $o);

```

```
$ php code_15_01.php ↵
```

KONSOLA

```

Pole kwadratu o boku 12.0 wynosi 144.0
Obwód kwadratu o boku 12.0 wynosi 48.0
Przekątna kwadratu o boku 12.0 wynosi 17.0
Pole trójkąta o bokach a=25.0, b=12.0, c=27.7 wynosi 150.0
Obwód trójkąta o bokach a=25.0, b=12.0, c=27.7 wynosi 64.7
Pole koła o promieniu 45.0 wynosi 6361.7
Obwód koła o promieniu 45.0 wynosi 282.7

```

Ćwiczenie 2. Pola i objętości brył

A teraz policzysz pola i objętości brył. Odszukaj w pamięci lub w sieci odpowiednie wzory i postaraj się zrozumieć, jak zostały obliczone w kodzie z listingu 15.2. Aby nie utrudniać, celowo pominąłem jednostki (mam nadzieję, że wiesz, w jakich jednostkach wyrażamy pole i objętość brył). W programie zamiast funkcji `pi`, zwracającej wartość stałej Archimedesesa (π), wykorzystałem stałą `M_PI`, która jest zdefiniowana w interpreterze języka PHP.

LISTING 15.2

PHP 7/8

code_15_02.php

```

01 <?php
02 /*
03  * Dany jest sześcian o boku a = 12
04  * Oblicz pole i objętość sześcianu
05  */

```

```

06  $a = 12;
07  $p = 6*($a**2); //pole sześciianu
08  $v = $a**3; //objętość sześciianu
09
10  printf("Pole sześciianu o boku %.1f wynosi %.1f".PHP_EOL, $a, $p);
11  printf("Objętość sześciianu o boku %.1f wynosi %.1f".PHP_EOL, $a, $v);
12
13  /*
14   * Dany jest walec o promieniu  $r = 18$  i wysokości  $h = 32$ 
15   * Oblicz pole i objętość walca
16   */
17  $r = 18;
18  $h = 32;
19  $p = 2*M_PI*$r*($r+$h); //pole walca
20  $v = M_PI*($r**2)*$h; //objętość walca
21
22  printf("Pole walca o promieniu %.1f i wysokości %.1f wynosi %.1f".PHP_EOL, $r,
    $h, $p);
23  printf("Objętość walca o promieniu %.1f i wysokości %.1f wynosi %.1f".PHP_EOL,
    $r, $h, $v);
24
25  /*
26   * Dana jest kula o promieniu  $r = 18$ 
27   * Oblicz pole i objętość kuli
28   */
29  $r = 18;
30  $p = 4*M_PI*($r**2); //pole kuli
31  $v = (4/3)*M_PI*($r**3); //objętość kuli
32
33  printf("Pole kuli o promieniu %.1f wynosi %.1f".PHP_EOL, $r, $p);
34  printf("Objętość kuli o promieniu %.1f wynosi %.1f".PHP_EOL, $r, $p);

```

```
$ php code_15_02.php ↵
```

KONSOLA

```

Pole sześciianu o boku 12.0 wynosi 864.0
Objętość sześciianu o boku 12.0 wynosi 1728.0
Pole walca o promieniu 18.0 i wysokości 32.0 wynosi 5654.9
Objętość walca o promieniu 18.0 i wysokości 32.0 wynosi 32572.0
Pole kuli o promieniu 18.0 wynosi 4071.5
Objętość kuli o promieniu 18.0 wynosi 4071.5

```

Ćwiczenia dodatkowe

1. Napisz program, który obliczy obwód i pole koła o promieniu $r = 14$.
2. Napisz program, który obliczy pole i objętość stożka o promieniu $r = 6$ i wysokości $h = 12$.
3. Otwórz stronę <http://php.net> i wyszukaj opis funkcji `sqrt` i `pow`.

Pytania

Co oznacza `%.1` w funkcji `printf`?

Funkcja `printf` drukuje tekst na podstawie szablonu i zestawu argumentów. Odśledź na stronie <http://php.net> opis funkcji `printf` oraz `sprintf`, a następnie zapoznaj się z informacjami na temat formatowania tekstu pod kątem konkretnych argumentów (liczby całkowite, liczby rzeczywiste itd.). Przykładowo użycie w szablonie `%.1f` będzie oznaczać, że argument będzie reprezentowany jako liczba zmiennoprzecinkowa z dokładnością do jednego miejsca po przecinku.

Dla liczb całkowitych zapis może być następujący:

```
printf ("%d + %d = %d", 3, 4, 3 + 4); // 3 + 4 = 7
```

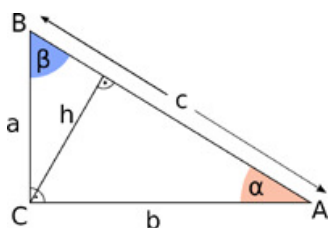
A dla liczb rzeczywistych (zmiennoprzecinkowych) może wyglądać w ten sposób:

```
printf ("%0.1f + %0.1f = %0.2f", 1.6, 2.8, 1.6 + 2.8); // 1.6 + 2.8 = 4.4
```



Więcej matmy dla ambitnych

W tej lekcji nauczysz się stosować w swoich programach funkcje trygonometryczne oraz logarytmy i obliczysz wartość bezwzględną. Możliwe, że ta lekcja będzie pierwszą, na której poznasz takie funkcje jak sinus czy cosinus, ale nie martw się, kiedyś trzeba przez to przebrnąć. Pokażę Ci, jak wykorzystywać funkcje dostępne w języku PHP w trygonometrii.



Rysunek 16.1. Trójkąt prostokątny

Na początek, dla przypomnienia, trochę teorii. Na rysunku 16.1 przedstawiony jest trójkąt prostokątny¹⁹ o trzech bokach: a, b i c. Taki trójkąt ma trzy kąty: jeden kąt prosty (90 stopni) i dwa kąty ostre (w przedziale obustronnie otwartym od 0 do 90 stopni).

Funkcje trygonometryczne w trójkącie prostokątnym oblicza się tak:

$$\sin \alpha = \frac{a}{c} \quad \cos \alpha = \frac{b}{c} \quad \tan \alpha = \frac{a}{b} \quad \cot \alpha = \frac{b}{a}$$

Język PHP ma wiele funkcji trygonometrycznych, jednak prawdopodobnie nie będziesz korzystał ze wszystkich. W tej lekcji poznasz funkcje trygonometryczne $\sin$, $\cos$ i $\tan$ oraz funkcje logarytmiczne $\log$ i $\log_{10}$, a także funkcję abs do obliczania wartości bezwzględnej.

Ćwiczenie 1. Trójkąt prostokątny

W tym ćwiczeniu obliczysz sinus i cosinus trójkąta prostokątnego. Na początku poniższego kodu (listing 16.1) znajdziesz treść zadania. Narysuj trójkąt na kartce papieru i oblicz wynik ręcznie, a następnie porównaj z wynikiem programu. Przeanalizuj program krok po kroku i postaraj się go zrozumieć.

¹⁹ https://pl.wikipedia.org/wiki/Trójkąt_prostokątny#/media/Plik:Pole_Trójkąta_Prostokątnego.svg.

LISTING 16.1	PHP 7/8	code_16_01.php
01	<?php	
02	/*	
03	<i>* W trójkącie prostokątnym przyprostokątne mają</i>	
04	<i>* długość $a = 9$ i $b = 3$, a jeden z kątów ostrych ma miarę α</i>	
05	<i>* Oblicz $\sin \alpha$ * $\cos \alpha$</i>	
06	<i>*/</i>	
07		
08	$\$a = 9;$ // pierwszy bok	
09	$\$b = 3;$ // drugi bok	
10		
11	// z twierdzenia Pitagorasa obliczamy długość trzeciego boku	
12	$\$c = \text{sqrt}(\$a**2 + \$b**2);$	
13		
14	// obliczamy funkcje trygonometryczne	
15	// pierwszy wariant, kąt ostry między a i c	
16	$\$sina = \$a / \$c;$	
17	$\$cosa = \$b / \$c;$	
18		
19	$\$result = \$sina * \$cosa;$	
20	$\text{printf}(\text{"kąt ac: sin}\alpha * \cos\alpha = \%.2f * \%.2f = \%.2f", \text{PHP_EOL}, \$sina, \$cosa, \$result);$	
21		
22	// drugi wariant, kąt ostry między b i c	
23	$\$sina = \$b / \$c;$	
24	$\$cosa = \$a / \$c;$	
25		
26	$\$result = \$sina * \$cosa;$	
27	$\text{printf}(\text{"kąt bc: sin}\alpha * \cos\alpha = \%.2f * \%.2f = \%.2f", \text{PHP_EOL}, \$sina, \$cosa, \$result);$	
\$ php code_16_01.php ↵		KONSOLA

```
kąt ac: sinα * cosa = 0.95 * 0.32 = 0.30
kąt bc: sinα * cosa = 0.32 * 0.95 = 0.30
```

Ćwiczenie 2. Funkcje trygonometryczne

W tym ćwiczeniu obliczysz sinus, cosinus i tangens kąta 45 stopni. Wykorzystasz funkcje `sin`, `cos` i `tan`. W związku z tym, że funkcje te działają na argumentach wyrażonych w radianach, użyjesz funkcji `deg2rad` do zamiany stopni na radiany. Wykonaj kod z listingu 16.2 dla różnych wartości kąta (np. 0, 30, 45, 60, 120 i 180 stopni).

LISTING 16.2	PHP 7/8	code_16_02.php
01	<?php	
02	\$deg = 45; <i>// stopnie</i>	
03	\$rad = deg2rad (\$deg); <i>// zamiana stopni na radiany</i>	
04		
05	<i>// argumentem dla funkcji trygonometrycznych w języku PHP są radiany</i>	
06	\$sin = sin (\$rad);	
07	\$cos = cos (\$rad);	
08	\$tan = tan (\$rad);	
09		
10	printf ("Sinus kąta %.2f stopni (%.2f radiana) wynosi %.2f".PHP_EOL, \$deg, \$rad, \$sin);	
11	printf ("Cosinus kąta %.2f stopni (%.2f radiana) wynosi %.2f".PHP_EOL, \$deg, \$rad, \$cos);	
12	printf ("Tangens kąta %.2f stopni (%.2f radiana) wynosi %.2f".PHP_EOL, \$deg, \$rad, \$tan);	
\$ php code_16_02.php ↵		KONSOLA
Sinus kąta 45.00 stopni (0.79 radiana) wynosi 0.71		
Cosinus kąta 45.00 stopni (0.79 radiana) wynosi 0.71		
Tangens kąta 45.00 stopni (0.79 radiana) wynosi 1.00		

Ćwiczenie 3. Logarytmy

W tym ćwiczeniu wykorzystasz nowe funkcje do obliczenia logarytmu naturalnego $\log$, logarytmu dziesiętnego $\log_{10}$ i wartości bezwzględnej abs . Na listingu 16.3 w linii 02 wykorzystałem stałą M_E wbudowaną w interpreter PHP, zwracającą podstawę logarytmu naturalnego e , zwaną też liczbą Eulera.

LISTING 16.3	PHP 7/8	code_16_03.php
01	<?php	
02	\$arg1 = M_E ; <i>// liczba Eulera 2.7182818284590452354</i>	
03	\$log = log (\$arg1); <i>// logarytm naturalny</i>	
04		
05	\$arg2 = 45;	
06	\$log10 = log10 (\$arg2); <i>// logarytm dziesiętny</i>	
07		
08	\$x = -85;	
09	\$abs = abs (\$x); <i>// wartość bezwzględna</i>	
10		
11	printf ("Logarytm naturalny z liczby %.2f wynosi %.2f".PHP_EOL, \$arg1, \$log);	
12	printf ("Logarytm dziesiętny z liczby %.2f wynosi %.2f".PHP_EOL, \$arg2, \$log10);	
13	printf ("Wartość bezwzględna z liczby %.2f wynosi %.2f".PHP_EOL, \$x, \$abs);	

```
$ php code_16_03.php ↵
```

KONSOLA

```
Logarytm naturalny z liczby 2.72 wynosi 1.00
Logarytm dziesiętny z liczby 45.00 wynosi 1.65
Wartość bezwzględna z liczby -85.00 wynosi 85.00
```

Ćwiczenia dodatkowe

1. Napisz program, który obliczy sumę sinusa i cosinusa dla kąta 45 stopni.
2. Poszukaj, jakie funkcje trygonometryczne są wykorzystywane do obliczania biorytmów, i spróbuj napisać przykładowy program. Nie martw się, jeśli ćwiczenie to będzie za trudne — pod koniec tej lekcji znajdziesz odpowiedź.
3. Otwórz stronę <http://php.net> i wyszukaj opis funkcji `date`, `time` i `mktime`. Spróbuj zrozumieć, jak działają. Przepisz i uruchom przykładowe listingi.
4. Otwórz stronę <https://www.php.net/manual/en/math.constants.php> i zapoznaj się ze stałymi matematycznymi wbudowanymi w interpreter języka PHP.

Pytania

Jak obliczyć swój biorytm na dzisiaj?

Jak zapewne wiesz, każdy z nas ma swoje lepsze i gorsze dni. Przyjęło się, że na nasze życie mają wpływ trzy cykle, tj. intelektualny, emocjonalny i fizyczny. Na listingu 16.4 znajduje się kod programu do wyliczania aktualnego biorytmu dla najpopularniejszego na świecie piłkarza na podstawie jego daty urodzin. Wylicz biorytm dla siebie, zmieniając wartość zmiennych w liniach od 03 do 06. Oczywiście, możesz to potraktować jako dobrą zabawę.

Do obliczenia biorytmów została wykorzystana funkcja trygonometryczna `sin` (sinus). W programie pojawiły się nowe funkcje `time` i `mktime`, a także została utworzona nowa funkcja `t` w liniach od 30 do 33. O tworzeniu własnych funkcji dowiesz się w następnej lekcji.

LISTING 16.4

PHP 7/8

code_16_04.php

```
01 <?php
02 // Data urodzin (możesz wpisać swoją)
03 $person = 'Robert Lewandowski';
04 $d = 21; // dzień
05 $m = 8; // miesiąc
```

```

06 $y = 1988; //rok
07
08 //jak długo żyjesz?
09 $seconds = time() - mktime(0, 0, 1, $m, $d, $y);
10 $days = $seconds / 86400; //86 400 sekund = 1 dzień
11
12 //obliczanie trendów — zakres od -100 do 100
13 //korzystamy ze stałych zdefiniowanych w języku PHP — M_PI i M_PI_2
14 //więcej: http://php.net/manual/en/function.pi.php
15 $pi2 = M_PI + M_PI;
16
17 //cykl intelektualny
18 $i = (($days % 33) / 33) * $pi2;
19 $ii = sin($i) * 100;
20
21 //cykl emocjonalny
22 $e = (($days % 28) / 28) * $pi2;
23 $ee = sin($e) * 100;
24
25 //cykl fizyczny
26 $f = (($days % 23) / 23) * $pi2;
27 $ff = sin($f) * 100;
28
29 //jaka jest tendencja cyklu?
30 function t($t) {
31     $r = (($t >= M_PI_2) && ($t < M_PI_2 + M_PI)) ? 'spadkowa' : 'wzrostowa';
32     return $r;
33 }
34
35 //drukowanie wyników
36 printf("$person żyje już %d sekund, czyli %d dni!".PHP_EOL, $seconds, $days);
37 printf("Dzisiaj jego cykle są następujące:".PHP_EOL);
38 printf("- intelektualny %d [tendencja %s]".PHP_EOL, $ii, t($i));
39 printf("- emocjonalny %d [tendencja %s]".PHP_EOL, $ee, t($e));
40 printf("- fizyczny %d [tendencja %s]".PHP_EOL, $ff, t($f));

```

```
$ php code_16_04.php ↵
```

KONSOLA

```

Robert Lewandowski żyje już 1027693421 sekund, czyli 11894 dni!
Dzisiaj jego cykle są następujące:
- intelektualny 45 [tendencja spadkowa]
- emocjonalny -97 [tendencja wzrostowa]
- fizyczny 73 [tendencja wzrostowa]

```

Co to jest sinus i cosinus? Do czego się tego używa?

Funkcje trygonometryczne odnoszą się z reguły do trójkąta prostokątnego. Wykorzystasz je głównie w zadaniach dotyczących figur płaskich i brył. Jeśli interesujesz się trygonometrią, powinieneś poznać te funkcje i nauczyć się stosować je w praktyce.

Czym są logarytmy? Co to jest liczba Eulera?

Logarytmów używa się na co dzień, choćby przy określaniu wielkości trzęsienia Ziemi (skala Richtera). Więcej o logarytmach i podstawie logarytmu naturalnego (liczbie Eulera) znajdziesz w książkach z matematyki. W tej lekcji pokazałem Ci, że za pomocą języka PHP możesz wykonać wiele skomplikowanych obliczeń.



Nareszcie własna funkcja

Doszedłeś już bardzo daleko. Gratuluję! Masz za sobą instalację środowiska programistycznego, pierwsze programy i powtórkę z matematyki. Poznałeś wiele ciekawych funkcji języka PHP, a to dopiero początek Twojej drogi w programowaniu. Jeśli wszystkie lekcje wykonałeś samodzielnie, to jesteś gotowy na dalszą naukę i na to, aby napisać swoją pierwszą funkcję.

Funkcje tworzymy po to, aby ułatwić sobie programowanie. Czasami często powtarzający się kod umieszczamy wewnątrz funkcji, którą następnie wielokrotnie w swoich programach wykorzystujemy.

Jak zauważyłeś, funkcje mogą przyjmować jakieś argumenty i mogą zwracać jakiś wynik. Przykładem może być funkcja drukująca tekst zgodnie z naszymi wytycznymi czy funkcja licząca pole figury geometrycznej. W tej lekcji nauczysz się konstruować proste funkcje. Do dzieła!

Konstrukcja funkcji wygląda tak:

```
function nazwa_funkcji($argument_1, /* ..., */ $argument_n) : zwracany typ
{
    /* wykonaj kod funkcji */
    return $wartosc;
}
```

Ćwiczenie 1. Definicja funkcji

Na listingu 17.1 znajduje się definicja funkcji drukuj (wiersz 02), której zadaniem jest drukowanie zdefiniowanego tekstu z jednoczesnym przejściem do nowej linii. Działanie funkcji jest bardzo proste. Po jej wywołaniu w wierszach 08, 09 i 10 za każdym razem (dokładnie trzy razy) jest wykonywany kod od linii 03 do 06 (pomiędzy nawiasami klamrowymi).

LISTING 17.1	PHP 7/8	code_17_01.php
01	<?php	
02	function drukuj() <i>// definicja funkcji bez argumentu</i>	
03	{	
04	print "Lubię programować!";	
05	print PHP_EOL;	
06	}	
07		
08	drukuj(); <i>// pierwsze wywołanie funkcji</i>	
09	drukuj(); <i>// kolejne</i>	
10	drukuj(); <i>// i kolejne</i>	
\$ php code_17_01.php ↵		KONSOLA
Lubię programować! Lubię programować! Lubię programować!		

Ćwiczenie 2. Funkcja z argumentem

W tym ćwiczeniu (listing 17.2) rozbudujesz funkcję drukuj (linia 02) o możliwość przyjmowania argumentu. Argument \$tekst dostaje się do wnętrza funkcji w postaci zmiennej o tej samej nazwie. Jak widzisz, w liniach 08, 09 i 10 została wywołana funkcja z argumentem w postaci tekstu. Tekst ten jest za każdym razem poddawiany wewnątrz funkcji pod zmienną \$tekst, a następnie drukowany.

LISTING 17.2	PHP 7/8	code_17_02.php
01	<?php	
02	function drukuj(string \$tekst) <i>// definicja funkcji z jednym argumentem</i>	
03	{	
04	print \$tekst;	
05	print PHP_EOL;	
06	}	
07		
08	drukuj("Lubię programować!");	
09	drukuj("Moja pierwsza funkcja!");	
10	drukuj("Programowanie nie jest trudne!");	
\$ php code_17_02.php ↵		KONSOLA
Lubię programować! Moja pierwsza funkcja! Programowanie nie jest trudne!		

Ćwiczenie 3. Wartość domyślna argumentu

W tym ćwiczeniu (listing 17.3) rozbudujesz funkcję drukuj (linia 04) o możliwość przyjmowania dwóch argumentów. Nowością jest to, że argumenty funkcji mają z góry ustalone wartości początkowe, co znacznie ułatwia ich wykorzystywanie. Jeśli nie podasz w wywołaniu funkcji argumentu, funkcja sama go zdefiniuje, nadawczy mu zdefiniowaną wartość początkową. Jak działa poniższy kod?

LISTING 17.3	PHP 7/8	code_17_03.php
01	<?php	
02	<i>// definicja funkcji z 2 argumentami</i>	
03	<i>// każdy argument ma ustaloną wartość domyślną</i>	
04	function drukuj(?string \$tekst = "", ?bool \$nowa_linia = true)	
05	{	
06	print \$tekst;	
07	if (\$nowa_linia === true)	
08	{	
09	print PHP_EOL;	
10	}	
11	}	
12		
13	drukuj("Lubię ", false);	
14	drukuj("programować!");	
15	drukuj();	
16	drukuj("Ale lubię też wakacje!");	
\$ php code_17_03.php ↵		KONSOLA
Lubię programować!		
Ale lubię też wakacje!		

Jak działa powyższy kod?

1. W liniach od 04 do 11 zdefiniowana jest funkcja drukuj z dwoma argumentami \$tekst i \$nowa_linia, które mają ustalone wartości początkowe. Działanie funkcji polega na drukowaniu zmiennej \$tekst (wiersz 06) i znaku przejścia do nowej linii PHP_EOL (wiersz 09). Przejście do nowej linii jest uzależnione od zmiennej \$nowa_linia (wiersz 07).
2. W linii 13 jest wywoływana funkcja drukuj z dwoma argumentami. Do wnętrza funkcji trafia zmienna \$tekst o wartości "Lubię " i zmienna \$nowa_linia o wartości false. Drukowana jest zatem tylko zmienna \$tekst, bez znaku przejścia do nowej linii.

3. W wierszu 14 wywoływana jest funkcja drukuj z jednym argumentem. Do wnętrza funkcji trafia zmienna \$tekst o wartości "programować!", a zmienna \$nowa_linia przyjmuje automatycznie wartość true. Drukowane są zmienna \$tekst i znak przejścia do nowej linii.
4. W wierszu 15 wywoływana jest funkcja drukuj bez argumentów. Zmienne \$tekst i \$nowa_linia automatycznie przyjmują wartości domyślne. Drukowane są pusty tekst i znak przejścia do nowej linii.
5. Wiersz 16 jest analogiczny do 14.

Ćwiczenie 4. Funkcje anonimowa i zwrotna

W języku PHP możesz tworzyć funkcje, które nie mają nazwy. Działają jak normalne funkcje, które możesz wykonać po przypisaniu do zmiennej. Na listingu 17.4 w liniach od 03 do 06 zdefiniowana jest funkcja anonimowa, która jest przypisana do zmiennej \$drukuj. Zwróć uwagę na sposób wywołania tej funkcji w wierszu 08.

Język PHP umożliwia tworzenie funkcji zwrotnych (tak zwanych callbacków), czyli funkcji, które mogą być przekazane jako argument do innej funkcji i wewnątrz niej uruchomione. Metoda ta jest dobrze znana programistom języka JavaScript. W liniach od 11 do 13 zdefiniowana jest funkcja mojaFunkcja, która w wierszu 12 wywołuje funkcję \$callback przekazaną jako argument. Wywołanie funkcji \$callback jest takie jak w przypadku funkcji anonimowej.

Do funkcji anonimowych możemy przekazać dodatkowe zmienne za pomocą wyrażenia use. W liniach od 20 do 22 zdefiniowana jest funkcja anonimowa, która jest przypisana do zmiennej \$drukujWiek. W wierszu 24 funkcja ta jest wywoływana z argumentem będącym odwołaniem do funkcji zwrotnej zdefiniowanej w zmiennej \$drukuj oraz dodatkowym argumentem \$zmienna.

LISTING 17.4

PHP 7/8

code_17_04.php

```

01 <?php
02 // definicja funkcji anonimowej
03 $drukuj = function(string $tekst) {
04     print $tekst;
05     print PHP_EOL;
06 };
07
08 $drukuj('Funkcja $drukuj po raz pierwszy.');// wywołanie funkcji anonimowej
09

```

```
10 // definicja funkcji zwrotnej (callback)
11 function mojaFunkcja($callback) {
12     $callback('Funkcja $drukuj po raz drugi.');
```

// wywołanie funkcji zwrotnej

```
13 }
14
15 mojaFunkcja($drukuj); // wywołanie funkcji zwrotnej
16
17 // argumenty funkcji anonimowej
18 $zmienna = 'Funkcja $drukuj po raz trzeci.';
19
20 $drukujWiek = function($callback) use ($zmienna) {
21     $callback($zmienna); // wywołanie funkcji zwrotnej
22 };
23
24 $drukujWiek($drukuj); // wywołanie funkcji anonimowej
```

```
$ php code_17_04.php ↵
```

KONSOLA

```
Funkcja $drukuj po raz pierwszy.
Funkcja $drukuj po raz drugi.
Funkcja $drukuj po raz trzeci.
```

Ćwiczenia dodatkowe

1. Napisz program z dwiema funkcjami. Pierwsza funkcja wy drukuje dowolny tekst małymi literami, a druga — ten sam tekst wielkimi literami.
2. Napisz program z jedną funkcją, która będzie drukować tekst małymi lub wielkimi literami, w zależności od podanego argumentu.



Funkcje mogą coś zwracać

Praktyka, praktyka, praktyka. W zasadzie każdy użyteczny program wykorzystuje funkcje. Na początku programista korzysta ze standardowych funkcji dostępnych w języku programowania. Za każdym razem jednak pojawia się potrzeba napisania własnych, indywidualnych, specjalnie dopasowanych funkcji, które coś policzą, przetworzą, wydrukują, zapiszą itd. W tej lekcji zastosujesz zdobytą już wiedzę do napisania nowych funkcji w języku PHP.

W najnowszej wersji języka PHP 8 możliwe jest przekazywanie wartości do funkcji poprzez nazwę argumentu, dzięki czemu nie musimy się zastanawiać, jaka jest ich kolejność. Argumenty przekazywane po nazwie umożliwiają również pominięcie parametrów opcjonalnych.

Ćwiczenie 1. Nazwy argumentów

Na rozgrzewkę przeanalizuj działanie funkcji `calcStatsYears` na listingu 18.1. Jej zadaniem jest obliczenie i wydrukowanie, ile upłynęło dni, godzin, minut i sekund w zadanym okresie. Funkcja zwraca liczbę sekund (w linii 16) za pomocą instrukcji `return`.

Definiując funkcję `calcStatsYears` (wiersz 04), wskazaliśmy, jakiego typu obiektu oczekujemy w argumentach funkcji — dwóch typu `int` i jednego typu `bool`. Dodatkowo wskazaliśmy, że funkcja ta ma zwracać wartość typu `int`. Definiując tak funkcję, dajemy programiście jasno do zrozumienia, jakie są oczekiwane typy parametrów wejściowych i jaki ma być typ wartości zwracanej. Za pomocą instrukcji `declare` możemy wskazać interpreterowi języka PHP, aby dokładnie sprawdzał, czy przekazywane argumenty są zgodne z ich deklaracją w definicji funkcji (linia 02).

Zwróć uwagę na wywołanie funkcji `calcStatsYears` w wierszach 19 i 20. Wykorzystałem nową technikę wprowadzoną w wersji PHP 8, polegającą na przekazywaniu

wartości do funkcji poprzez nazwę argumentu. Porównaj linie 19 i 20. Zauważysz, że kolejność argumentów jest inna, a mimo to funkcja działa tak, jak oczekujemy.

LISTING 18.1	PHP 8	code_18_01.php
01	<?php	
02	declare (strict_types = 1);	
03		
04	function calcStatsYears(int \$start, int \$end, ?bool \$print = true) : int	
05	{	
06	\$days = (\$end - \$start) * 365;	
07	\$hours = \$days * 24;	
08	\$mins = \$hours * 60;	
09	\$secs = \$mins * 60;	
10		
11	if (\$print === true) {	
12	print "Od początku roku \$start do końca roku \$end jest:".PHP_EOL;	
13	print "\$days dni, \$hours godzin, \$mins minut i \$secs sekund!".PHP_EOL;	
14	}	
15		
16	return (int)\$secs;	
17	}	
18		
19	\$sec1 = calcStatsYears(start: 1901, end: 2000, print: true); <i>// XX wiek</i>	
20	\$sec2 = calcStatsYears(end: 2019, start: 2001); <i>// od początku XXI wieku</i>	
21		
22	print "Razem sekund: " . (\$sec1 + \$sec2);	
\$ php code_18_01.php ↵		KONSOLA
Od początku roku 1901 do końca roku 2000 jest: 36135 dni, 867240 godzin, 52034400 minut i 3122064000 sekund! Od początku roku 2001 do końca roku 2019 jest: 6570 dni, 157680 godzin, 9460800 minut i 567648000 sekund! Razem sekund: 3689712000		

Ćwiczenie 2. Obliczenia

Na listingu 18.2 znajduje się funkcja `triangle`, za pomocą której obliczysz pole trójkąta. Funkcja zwraca wynik (w linii 07) z użyciem instrukcji `return`. Tutaj również za pomocą instrukcji `declare` w wierszu 02 wymuszamy na interpreterze PHP dokładne sprawdzanie typów argumentów przekazywanych do funkcji.

Jestem przekonany, że poniższy kod nie jest dla Ciebie trudny. Spróbuj wykorzystać go do napisania funkcji liczącej pole innej figury.

LISTING 18.2	PHP 8	code_18_02.php
01	<?php	
02	declare (strict_types = 1);	
03		
04	function triangle(float \$a, float \$h) : float <i>//funkcja licząca pole trójkąta</i>	
05	{	
06	\$p = (0.5 * \$a * \$h); <i>//obliczanie pola</i>	
07	return (float)\$p; <i>//zwracanie wyniku</i>	
08	}	
09		
10	\$a = 12; <i>//podstawa trójkąta</i>	
11	\$h = 34; <i>//wysokość trójkąta</i>	
12		
13	\$t = triangle(a: \$a, h: \$h); <i>//wywołaj funkcję i przypisz wynik do zmiennej</i>	
14		
15	printf ("Pole trójkąta o podstawie %d i wysokości %d wynosi %d.", \$a, \$h, \$t);	
\$ php code_18_02.php ↵		KONSOLA
Pole trójkąta o podstawie 12 i wysokości 34 wynosi 204.		

Ćwiczenie 3. Kalkulator

Wykorzystując nowe możliwości wprowadzone w PHP 8, możemy stworzyć prostą funkcję zwracającą wynik działań matematycznych. W programie z listingu 18.3 wykorzystałem znaną Ci konstrukcję warunkową `match` oraz nową funkcję `fdiv`. Funkcja `fdiv` zwraca wynik dzielenia dwóch liczb, a w przypadku dzielenia przez zero nie przerywa działania programu, tylko zwraca wartość `INF` (jeśli dzielna jest dodatnia), `-INF` (jeśli dzielna jest ujemna) lub `NAN` (jeśli dzielna jest równa zero).

LISTING 18.3	PHP 8	code_18_03.php
01	<?php	
02	declare (strict_types = 1);	
03		
04	\$kalkulator = function (string \$operacja, float \$x, float \$y) : float	
05	{	
06	return match (\$operacja) {	
07	'+' => \$x + \$y,	
08	'-' => \$x - \$y,	
09	'*' => \$x * \$y,	
10	'/' => fdiv (\$x, \$y),	
11	};	
12	};	
13		

```

14 $x = (float)12;
15 $y = (float)8;
16
17 printf("%.2f + %.2f = %.2f".PHP_EOL, $x, $y, $kalkulator('+', $x, $y));
18 printf("%.2f - %.2f = %.2f".PHP_EOL, $x, $y, $kalkulator('-', $x, $y));
19 printf("%.2f * %.2f = %.2f".PHP_EOL, $x, $y, $kalkulator('*', $x, $y));
20 printf("%.2f / %.2f = %.2f".PHP_EOL, $x, $y, $kalkulator('/', $x, $y));
21 printf("%.2f / 0 = %.2f".PHP_EOL, $x, $kalkulator('/', $x, 0));

```

```
$ php code_18_03.php ↵
```

KONSOLA

```

12.00 + 8.00 = 20.00
12.00 - 8.00 = 4.00
12.00 * 8.00 = 96.00
12.00 / 8.00 = 1.50
12.00 / 0 = INF

```

Jak działa powyższy kod?

1. W linii 02 wskazałem interpreterowi PHP, aby sprawdzał poprawność typów argumentów funkcji oraz typ wartości zwracanej.
2. W wierszach od 04 do 12 zdefiniowana jest funkcja anonimowa przypisana do zmiennej \$kalkulator. Wartość zwracana przez funkcję jest uzależniona od rezultatu działania konstrukcji match w liniach od 06 do 12. Funkcja fdiv w wierszu 10 ma zwrócić wynik dzielenia i jednocześnie zapobiec przerwaniu programu w przypadku dzielenia przez zero.
3. W wierszach 14 i 15 zdefiniowane są zmienne \$x i \$y z jawnie zadeklarowanym typem.
4. Od linii 17 do 21 drukowane są wyniki obliczeń.

Ćwiczenia dodatkowe

1. Napisz program z funkcją liczącą pole prostokąta.
2. Popraw funkcję w ćwiczeniu 1, aby drukowana była również liczba miesięcy.
3. Uzupełnij funkcję kalkulatora z ćwiczenia 3 o możliwość obliczenia reszty z dzielenia (modulo).

Pytania

Dlaczego funkcje tak się nazywają?

To od Ciebie zależy, jak nowe funkcje będą się nazywać. Ja stosuję notację *camel-Case* zarówno do tworzenia nazw zmiennych, jak i nazw funkcji. W ćwiczeniu 1 funkcja `calcStatsYears` mogłaby się również nazywać `calc_stats_years`, `csy`, `StatystykaRoczna`, `takaSobieFunkcjaStatystyczna` itd.

Funkcja a zasięg zmiennych

W programowaniu wyróżnia się zmienne lokalne, globalne i superglobalne. Chodzi o to, aby wyeliminować możliwą kolizję zmiennych, czyli sytuację, kiedy wewnątrz funkcji i poza nią zdefiniowana jest zmienna o identycznej nazwie. W związku z tym wszystkie zmienne, które zdefiniujesz wewnątrz funkcji, będą dostępne tylko dla niej, natomiast zmienne zdefiniowane poza funkcją nie będą dla tej funkcji dostępne (poza szczególnymi przypadkami).

W dużym uproszczeniu jest tak dlatego, aby funkcja mogła bezpiecznie wykonać swój kod i zwrócić odpowiedni wynik. W praktyce, kiedy wykorzystujesz jakąś funkcję w języku PHP (np. `printf` czy `str_replace`), interesuje Cię tylko wynik jej działania, a to, co się dzieje wewnątrz funkcji, jest dla Ciebie mało istotne.

Ćwiczenie 1. Lokalnie i globalnie

Na listingu 19.1 w linii 02 zdefiniowałem zmienną `$name`, która dla programu jest zmienną globalną. To samo zrobiłem w wierszu 06, jednak w tym przypadku zmienna `$name` jest dostępna tylko dla funkcji `printName` — jest to więc zmienna lokalna. Obie zmienne mają tę samą nazwę, jednak działają niezależnie od siebie. Jakakolwiek zmiana wartości tej zmiennej wewnątrz funkcji (czyli lokalnie) nie spowoduje zmiany wartości zmiennej globalnej.

LISTING 19.1	PHP 7/8	code_19_01.php
01	<?php	
02	<code>\$name = "Mistrz Miecza Ken";</code> <i>//zmienna globalna</i>	
03		
04	function printName()	
05	{	
06	<code>\$name = "Agentka Polowa Rio";</code> <i>//zmienna lokalna</i>	
07	print "Zmienna lokalna: <code>\$name</code> ".PHP_EOL;	
08	}	
09		


```

10 print "Zmienna globalna: $name".PHP_EOL; // zmienna z linii 2
11 printName();
12 print "Zmienna globalna: $name".PHP_EOL; // ponownie zmienna z linii 2

```

```
$ php code_19_01.php ↵
```

KONSOLA

```

Zmienna globalna: Mistrz Miecza Ken
Zmienna lokalna: Agentka Polowa Rio
Zmienna globalna: Mistrz Miecza Ken

```

Ćwiczenie 2. Dostęp do zmiennych globalnych

Istnieje możliwość dostępu do zmiennych globalnych wewnątrz funkcji za pomocą instrukcji `global`. W szczególnych przypadkach wprowadzenie zmiennej globalnej do funkcji jest związane z uzyskaniem dostępu do jej wartości.

Na listingu 19.2 po wprowadzeniu do funkcji w linii 06 zmiennej globalnej `$name` zmieniłem jej wartość w wierszu 10. Zmiana ta ma charakter globalny, czyli zmienna `$name` z linii 02 zmieniła swoją wartość. Musisz zawsze o tym pamiętać, a najlepiej unikaj takich sytuacji.

LISTING 19.2

PHP 7/8

code_19_02.php

```

01 <?php
02 $name = "Mistrz Miecza Ken"; // zmienna globalna
03
04 function printName()
05 {
06     global $name; // dostęp do zmiennej globalnej
07
08     print "Globalna przed zmianą: $name".PHP_EOL;
09
10     $name = "Agentka Polowa Rio"; // zmiana zmiennej globalnej
11     print "Globalna po zmianie: $name".PHP_EOL;
12 }
13
14 print "Zmienna globalna: $name".PHP_EOL; // zmienna z linii 2
15 printName();
16 print "Zmienna globalna: $name".PHP_EOL; // nowa wartość zmiennej globalnej

```

```
$ php code_19_02.php ↵
```

KONSOLA

```

Zmienna globalna: Mistrz Miecza Ken
Globalna przed zmianą: Mistrz Miecza Ken
Globalna po zmianie: Agentka Polowa Rio
Zmienna globalna: Agentka Polowa Rio

```

Ćwiczenie 3. Zmienne superglobalne

W języku PHP istnieją specjalne zmienne nazywane zmiennymi superglobalnymi. Są to zmienne dostępne w całym kodzie, a dostęp do nich nie wymaga użycia instrukcji `global`. Lista tych zmiennych jest następująca: `$GLOBALS`, `$_SERVER`, `$_GET`, `$_POST`, `$_FILES`, `$_COOKIE`, `$_SESSION`, `$_REQUEST`, `$_ENV`. Pamiętaj, aby nie tworzyć nowych zmiennych, korzystając z nazw zmiennych superglobalnych.

Zmienne superglobalne przechowują informacje dotyczące serwera (komputera), na którym uruchomiony jest interpreter języka PHP, informacje pochodzące z formularzy wysłanych ze strony internetowej metodą *get* lub *post*, dane sesji użytkownika i wiele innych.

Uruchom kod z listingu 19.3 i przeanalizuj wydrukowane informacje (będzie ich dużo).

LISTING 19.3	PHP 7/8	code_19_03.php
01	<?php	
02	<code>print_r(\$_SERVER);</code>	
\$ php code_19_03.php ↵		KONSOLA
Array ([ALLUSERSPROFILE] => C:\ProgramData [...])		

Ćwiczenia dodatkowe

1. Popraw poniższy kod w taki sposób, aby funkcja `printName` drukowała zmienną globalną `$name` (bez wykorzystania wyrażenia `global`).

```
$name = "Mroźny Zenit";  
function printName()  
{  
}  
printName();
```



Pętla, czyli cykliczne powtarzanie instrukcji

Często istnieje potrzeba wykonania pewnej czynności wiele razy. Trudno mi sobie wyobrazić prosty program drukujący tysiąc kolejnych liczb bez wykorzystania pętli. W PHP masz do dyspozycji trzy podstawowe warianty pętli: `for`, `while` i `do-while`.

Pętla `for`

Pętla `for` jest najpopularniejszą pętlą używaną w programowaniu. Zastosujesz ją wtedy, kiedy z góry będziesz wiedział, ile dokładnie ma wykonać pętli, czyli ile ma zrobić cykli. Przykładowa konstrukcja pętli `for` wygląda tak:

```
for ($i = 1; $i <= 10; $i++) // zwiększenie $i o 1 z każdym cyklem
{
    print $i; // wynik: 12345678910
}
```

Działanie powyższej pętli jest bardzo proste. W pierwszej kolejności zdefiniowana jest zmienna `$i`, której nadano wartość 1. Następnie został określony warunek, kiedy pętla ma się skończyć, czyli kiedy zmienna `$i` osiągnie wartość co najmniej 10. Na końcu została określona zasada (inkrementacja), jak ma się zmieniać zmienna `$i` po zakończeniu każdego cyklu. Z każdym cyklem zmienna zwiększa swoją wartość o 1, zatem po 10 cyklach zakończy działanie.

```
for ($i = 1; $i <= 10; $i += 2) // zwiększenie $i o 2 z każdym cyklem
{
    print $i; // wynik: 13579
}
```

Możesz również zastosować inną regułę zmiany wartości zmiennej `$i` po każdym cyklu pętli. Powyższa pętla wykona się tylko pięć razy, ponieważ wartość zmiennej `$i` wzrasta o 2 z każdym cyklem.

```
for ($i = 10; $i >= 1; $i--) // zmniejszenie $i o 1 z każdym cyklem
{
    print $i; // wynik: 10987654321
}
```

Nic nie stoi na przeszkodzie, aby uruchomić pętlę malejącą. Zauważ, że do zmniejszenia wartości zmiennej o 1 wykorzystałem operator dekrementacji.

Pętla while

Pętla while wykonuje cykle, jeśli określony warunek jest z góry spełniony. Pętla może wykonywać cykle w nieskończoność, ponieważ istotą tej pętli nie jest liczba cykli do wykonania, ale warunek, w którym ma je wykonywać. Przykładowa konstrukcja pętli while wygląda tak:

```
$i = 1;
while ($i <= 10) // podczas gdy zmienna $i <= 10, wykonaj instrukcje
{
    print $i; // wynik: 12345678910
    $i++;
}
```

W pętli while zmienna `$i` nie jest automatycznie zwiększana lub zmniejszana (inkrementowana `++` lub dekrementowana `--`), tak jak to jest w przypadku pętli `for`. Samodzielnie musisz zadbać o taką modyfikację zmiennej `$i`, aby pętla została zakończona zgodnie z oczekiwaniem. Pętlę while wykorzystasz w sytuacjach, kiedy nie będziesz pewien, ile dokładnie cykli należy wykonać, aby uzyskać oczekiwany efekt.

Pętla do-while

Pętla do-while działa podobnie do pętli while, z tą różnicą, że warunek sprawdzany jest dopiero na końcu cyklu. Oznacza to, że pętla zostanie wykonana przynajmniej raz, bez względu na spełnienie warunku. Przykładowa konstrukcja pętli do-while wygląda tak:

```
$i = 1;
do {
    print $i; // wynik: 12345678910
    $i++;
} while ($i <= 10);
```

Ćwiczenie 1. Pętla for

W tym ćwiczeniu (listing 20.1) wykorzystasz pętlę `for` do wydrukowania wyników potęgowania kolejnych liczb z zakresu od 1 do 10. Program nie jest skomplikowany, jednak postaraj się go dokładnie zrozumieć. Zwróć uwagę, że zmienna `$i` z każdym cyklem zwiększa swoją wartość o 1.

LISTING 20.1	PHP 7/8	code_20_01.php
01	<?php	
02	for (\$i = 1; \$i <= 10; \$i++) // inicjacja pętli	
03	{	
04	\$pow = \$i ** 2; // obliczenie kwadratu	
05	print "\$i do potęgi 2 = \$pow".PHP_EOL;	
06	}	
\$ php code_20_01.php ↵		KONSOLA
1	do potęgi 2 = 1	
2	do potęgi 2 = 4	
3	do potęgi 2 = 9	
4	do potęgi 2 = 16	
5	do potęgi 2 = 25	
6	do potęgi 2 = 36	
7	do potęgi 2 = 49	
8	do potęgi 2 = 64	
9	do potęgi 2 = 81	
10	do potęgi 2 = 100	

Ćwiczenie 2. Pętla while

W tym ćwiczeniu (listing 20.2) pętla `while` zakończy działanie, kiedy wartość zmiennej `$sqrt`, przechowującej wynik pierwiastkowania, będzie większa od 20. Zwróć uwagę, że nie ma tutaj z góry ustalonej liczby cykli. Pętla wykonuje kolejne cykle, jeśli warunek jest spełniony.

LISTING 20.2	PHP 7/8	code_20_02.php
01	<?php	
02	\$i = 10;	
03	\$sqrt = 0;	
04		
05	while (\$sqrt <= 20) // inicjacja pętli	
06	{	
07	\$sqrt = sqrt(\$i); // obliczenie pierwiastka	
08	printf("pierwiastek z %d = %.2f".PHP_EOL, \$i, \$sqrt);	
09		
10	\$i += 50;	
11	}	
\$ php code_20_02.php ↵		KONSOLA
pierwiastek z 10 = 3.16		
pierwiastek z 60 = 7.75		
pierwiastek z 110 = 10.49		
pierwiastek z 160 = 12.65		
pierwiastek z 210 = 14.49		

```

pierwiastek z 260 = 16.12
pierwiastek z 310 = 17.61
pierwiastek z 360 = 18.97
pierwiastek z 410 = 20.25

```

Ćwiczenie 3. Pętla do-while

W tym ćwiczeniu (listing 20.3) wykorzystasz pętlę do-while do wydrukowania wyników mnożenia. Zwróć uwagę, że pętla wykona się przynajmniej raz, nawet jeśli warunek od samego początku nie będzie spełniony. W przypadku pętli while byłoby to niemożliwe.

LISTING 20.3	PHP 7/8	code_20_03.php
01	<?php	
02	\$i = 10;	
03		
04	do { //inicjacja pętli	
05	\$x = \$i * 2; //mnożenie	
06	print "\$i * 2 = \$x".PHP_EOL;	
07		
08	\$i++;	
09	} while (\$i < 15);	
\$ php code_20_03.php ↵		KONSOLA
10	* 2 = 20	
11	* 2 = 22	
12	* 2 = 24	
13	* 2 = 26	
14	* 2 = 28	

Ćwiczenia dodatkowe

1. Otwórz stronę <http://php.net> i wyszukaj opis pętli for, while i do-while. Spróbuj zrozumieć, jak działają. Przepisz i uruchom przykładowe listingi.



Pętla w akcji

W tej lekcji nauczysz się, jak wykorzystać pętle w praktyce. Poznasz również instrukcje `break` i `continue`, za pomocą których będziesz mógł przerwać wykonanie pętli lub cyklu. Ciekawą odmianą pętli jest funkcja rekurencyjna, czyli wykonująca samą siebie określoną liczbę razy. Przykładem funkcji rekurencyjnej jest funkcja licząca silnię.

Ćwiczenie 1. Sterowanie pętlą

W tym ćwiczeniu, tak jak w poprzedniej lekcji, zastosujesz pętlę `for` do wydrukowania wyników potęgowania kolejnych liczb z zakresu od 1 do 10. Tym razem użyjesz jednak instrukcji `continue`, aby wydrukowane zostały wyniki tylko dla liczb parzystych. Zauważ, że na listingu 21.1 w linii 04 pojawiła się instrukcja warunkowa `if`, która sprawdza, czy zmienna `$i` jest nieparzysta. Jeśli warunek jest spełniony, wywoływana jest instrukcja `continue`, która przerywa bieżący cykl, a pętla rozpoczyna wykonywanie następnego cyklu.

LISTING 21.1

PHP 7/8

code_21_01.php

```
01 <?php
02 for ($i = 1; $i <= 10; $i++) // inicjacja pętli
03 {
04     if ($i % 2 != 0) { // jeśli nieparzyste
05         continue; // to przerwij cykl i wykonaj następny
06     }
07
08     $pow = $i ** 2; // obliczenie kwadratu
09     print "$i do potęgi 2 = $pow".PHP_EOL;
10 }
```

\$ php code_21_01.php ↵

KONSOLA

```
2 do potęgi 2 = 4
4 do potęgi 2 = 16
6 do potęgi 2 = 36
8 do potęgi 2 = 64
10 do potęgi 2 = 100
```

Ćwiczenie 2. Przerwanie pętli

W tym ćwiczeniu, tak jak w poprzedniej lekcji, wykorzystasz pętlę `while` do wydrukowania wyników pierwiastkowania. Na listingu 21.2 w linii 05 zdefiniowałem pętlę w taki sposób, aby wykonywała swoje cykle w nieskończoność. Pętla `while` często przez programistów jest właśnie tak konstruowana, z zastrzeżeniem, że wewnątrz pętli muszą być zdefiniowane warunki, po których spełnieniu pętla zostanie przerwana. Taki warunek został określony w wierszu 10, a po jego spełnieniu pętla zostaje w linii 11 przerwana instrukcją `break`.

LISTING 21.2	PHP 7/8	code_21_02.php
01	<?php	
02	<code>\$i = 10;</code>	
03	<code>\$sqrt = 0;</code>	
04		
05	<code>while (true) // inicjacja pętli nieskończonej</code>	
06	<code>{</code>	
07	<code> \$sqrt = sqrt(\$i); // obliczenie pierwiastka</code>	
08	<code> printf("pierwiastek z %d = %.2f".PHP_EOL, \$i, \$sqrt);</code>	
09		
10	<code> if (\$sqrt > 8) { // czy pierwiastek większy od 8?</code>	
11	<code> break; // przerwij cykl i pętlę</code>	
12	<code> }</code>	
13		
14	<code> \$i += 25;</code>	
15	<code>}</code>	
\$ php code_21_02.php ↵		KONSOLA
pierwiastek z 10 = 3.16		
pierwiastek z 35 = 5.92		
pierwiastek z 60 = 7.75		
pierwiastek z 85 = 9.22		

Ćwiczenie 3. Rekurencja

W tym ćwiczeniu poznasz funkcję rekurencyjną, czyli taką, która odwołuje się do samej siebie. Klasycznym przykładem jest funkcja licząca silnię. Zauważ, że w linii 07 funkcja wywołuje samą siebie, aż wartość `$i` osiągnie 1.

Postaraj się zrozumieć, jak działa kod z listingu 21.3.

LISTING 21.3	PHP 7/8	code_21_03.php
01	<?php	
02	function silnia(\$i)	
03	{	
04	if (\$i < 2) {	
05	return 1;	
06	} else {	
07	return \$i * silnia(\$i - 1); <i>//funkcja wywołuje samą siebie</i>	
08	}	
09	}	
10		
11	for (\$x = 1; \$x <= 10; \$x++) <i>//inicjacja pętli</i>	
12	{	
13	print "\$x! = ".silnia(\$x).PHP_EOL;	
14	}	
\$ php code_21_03.php ↵		KONSOLA
1! = 1		
2! = 2		
3! = 6		
4! = 24		
5! = 120		
6! = 720		
7! = 5040		
8! = 40320		
9! = 362880		
10! = 3628800		

Ćwiczenia dodatkowe

1. Przy wykorzystaniu pętli for napisz program drukujący 10 liczb w taki sposób, że każda następna liczba jest większa od poprzedniej o 4.
2. Przy wykorzystaniu pętli while napisz program drukujący 10 liczb z zakresu od 20 do 29.
3. Napisz program liczący silnię z użyciem pętli for, tak aby uniknąć rekurencji.

Wskazówki

W praktyce pętlę `for` będziesz stosował w sytuacjach, kiedy z góry będziesz wiedział, co i ile razy ma być wykonane. Natomiast z pętli `while` skorzystasz, kiedy zakończenie jej działania jest uzależnione od wystąpienia jakiegoś zdarzenia czy spełnienia jakiegoś warunku. W swojej praktyce najczęściej używam pętli `for` oraz `foreach`, którą poznasz w następnych lekcjach przy omawianiu tablic.

Jest pewien kłopot z funkcjami rekurencyjnymi. Choć są proste i w wielu przypadkach wykorzystanie ich jest zasadne, wymagają do działania więcej pamięci niż zwykłe funkcje czy pętle. Pisząc kod, staraj się unikać rekurencji dla skomplikowanych funkcji, ponieważ może to negatywnie wpłynąć na wydajność programu.

Pytania

Jak napisać program liczący silnię przy wykorzystaniu pętli `for`?

Na listingu 21.4 przedstawiam kod programu liczący silnię z liczby 10.

LISTING 21.4	PHP 7/8	code_21_04.php
01	<?php	
02	\$x = 10;	
03	\$silnia = 1;	
04	for (\$i = 1; \$i <= \$x; \$i++)	
05	{	
06	\$silnia *= \$i;	
07	}	
08	print "\$x! = \$silnia";	
\$ php code_21_04.php ↵		KONSOLA
10! = 3628800		



Tablice i klucze

A teraz poznasz tablice, czyli struktury do przechowywania danych, bez których trudno wyobrazić sobie programowanie. W tablicach będziesz przechowywał dane w sposób uporządkowany, a dostęp do nich uzyskasz za pomocą klucza. W tej lekcji poznasz proste tablice i sposób ich wykorzystania w praktyce.

Deklaracja (utworzenie, inicjacja) tablicy w języku PHP wygląda tak:

```
$array = [  
    "klucz_1" => "wartość_1",  
    "klucz_2" => "wartość_2",  
    "klucz_3" => "wartość_3"  
];
```

Można również powyższą deklarację zapisać tak:

```
$array = [];  
$array["klucz_1"] = "wartość_1";  
$array["klucz_2"] = "wartość_2";  
$array["klucz_3"] = "wartość_3";
```

Do konkretnego elementu w takiej tablicy możesz się odwołać za pomocą klucza, przykładowo odwołanie `$array["klucz_1"]` zwróci wartość `wartość_1`. Uproszczona wersja powyższych deklaracji, w której klucze nadawane są automatycznie, począwszy od 0 (zera), wygląda tak:

```
$array = ["wartość_1", "wartość_2", "wartość_3"];
```

Oznacza to, że kluczem dla `wartość_1` będzie 0, dla `wartość_2` — 1 itd. W związku z tym, że kluczami w takiej tablicy są kolejne liczby, począwszy od 0 (zera), odwołanie np. `$array[1]` zwróci wartość `wartość_2`.

Tablice, w których zamiast kluczy numerycznych (0, 1, 2, 3 itd.) wykorzystane są klucze znakowe (`klucz_1`, `klucz_2` itd.), nazywamy tablicami asocjacyjnymi.

W tej lekcji poznasz nowe funkcje, takie jak `array_push`, `array_pop`, `in_array`, `sort`, `ksort`, `count`, `print_r`.

Ćwiczenie 1. Elementy tablicy

Przeanalizuj kod z listingu 22.1 linia po linii i postaraj się zrozumieć, co się dzieje z danymi umieszczonymi w tabeli \$owoce.

Przyjrzyj się dokładnie wartościom drukowanym funkcją `print_r` w wierszach 07, 17 i 24.

LISTING 22.1	PHP 7/8	code_22_01.php
01	<?php	
02	print "Tworzenie tablicy.".PHP_EOL;	
03		
04	<i>// deklaracja tablicy z 2 elementami</i>	
05	\$owoce = ["jabłko", "banan"];	
06		
07	print_r (\$owoce); <i>// drukowanie tablicy</i>	
08	<i>// funkcja count zwraca liczbę elementów w tablicy</i>	
09	print "Tablica ma ". count (\$owoce)." elementy.".PHP_EOL;	
10		
11	print str_repeat('-', 30).PHP_EOL;	
12	print "Dodanie 2 elementów.".PHP_EOL;	
13		
14	array_push (\$owoce, "arbuz"); <i>// dodanie elementu do tablicy</i>	
15	array_push (\$owoce, "gruszka"); <i>// dodanie elementu do tablicy</i>	
16		
17	print_r (\$owoce);	
18	print "Tablica ma ". count (\$owoce)." elementy.".PHP_EOL;	
19		
20	print str_repeat('-', 30).PHP_EOL;	
21	print "Pobranie 1 elementu z tablicy.".PHP_EOL;	
22		
23	\$owoc = array_pop (\$owoce); <i>// pobranie (i usunięcie) ostatniego elementu z tablicy</i>	
24	print_r (\$owoce);	
25		
26	print "Tablica ma ". count (\$owoce)." elementy.".PHP_EOL;	
27	print "Zmienna owoc ma wartość \$owoc.";	
\$ php code_22_01.php ↵		KONSOLA
<pre>Tworzenie tablicy. Array ([0] => jabłko [1] => banan) Tablica ma 2 elementy. ----- Dodanie 2 elementów. Array (</pre>		

```

    [0] => jabłko
    [1] => banan
    [2] => arbuz
    [3] => gruszka
)
Tablica ma 4 elementy.
-----
Pobranie 1 elementu z tablicy.
Array
(
    [0] => jabłko
    [1] => banan
    [2] => arbuz
)
Tablica ma 3 elementy.
Zmienna owoc ma wartość gruszka.

```

Jak działa powyższy kod?

1. W linii 05 utworzyłem zmienną tablicową \$owoce z dwoma elementami. W wierszu 07 drukowana jest zawartość tej tablicy funkcją print_r. Zapamiętaj tę funkcję, ponieważ będziesz z niej często korzystać. Pojawiła się też w linii 09 nowa funkcja count, która zwraca liczbę elementów w tablicy.
2. W wierszach 14 i 15 za pomocą funkcji array_push dodałem do tablicy \$owoce dwa nowe elementy. Zostały one wstawione na koniec tabeli.
3. W linii 23 do zmiennej \$owoc przypisałem za pomocą funkcji array_pop wartość ostatniego elementu tablicy \$owoce. Pamiętaj, że funkcja array_pop usuwa element z tablicy po jego pobraniu.

Ćwiczenie 2. Klucze elementów tablicy

W tym ćwiczeniu utworzysz tabelę z kolejnymi liczbami parzystymi. Zauważ, że zastosowałem inną formę dodawania kolejnych elementów do tablicy (linia 07), która jest alternatywą dla array_push. Przypomnij sobie lekcje na temat pętli i przeanalizuj kod z listingu 22.2.

LISTING 22.2

PHP 7/8

code_22_02.php

```

01 <?php
02 $liczby = []; // deklaracja pustej tablicy
03 $ile = 10;
04
05 for ($i = 0; $i <= $ile; $i++) {
06     if ($i % 2 == 0) { // parzyste?
07         $liczby[] = $i; // dodajemy do tablicy kolejny element
08         // array_push($liczby, $i); // alternatywa dla $liczby[] = $i

```

```

09     }
10 }
11
12 $e = count($liczby); // ile elementów ma tablica?
13
14 // dostęp do elementów tablicy
15 for ($i = 0; $i < $e; $i++) {
16     print "klucz = $i, wartość = $liczby[$i]".PHP_EOL;
17 }

```

```
$ php code_22_02.php ↵
```

KONSOLA

```

klucz = 0, wartość = 0
klucz = 1, wartość = 2
klucz = 2, wartość = 4
klucz = 3, wartość = 6
klucz = 4, wartość = 8
klucz = 5, wartość = 10

```

Jak działa powyższy kod?

1. W linii 02 utworzyłem pustą zmienną tablicową \$liczby, a w wierszu 03 zmienną pomocniczą \$ile.
2. W liniach od 05 do 10 zdefiniowałem pętlę for, w której kolejne liczby parzyste są dodawane do tablicy \$liczby. To, czy liczba jest parzysta, jest sprawdzane w wierszu 06. Zwróć uwagę na skróconą metodę dodawania kolejnych elementów do tablicy w linii 07. Możesz też wykorzystać metodę poznaną w poprzednim ćwiczeniu i użyć funkcji array_push (wiersz 08).
3. Za wydrukowanie wszystkich elementów tablicy odpowiedzialna jest pętla for zdefiniowana w liniach od 15 do 17. Zwróć uwagę na metodę odwołania się do kolejnego elementu tablicy \$liczby w wierszu 16. Do pierwszego elementu w tej tablicy odwołujemy się kluczem o wartości 0 (zero).

Ćwiczenie 3. Sortowanie tablicy

W tym ćwiczeniu posortujesz rosnąco tablicę \$owoce za pomocą funkcji sort. Jeśli chciałbyś posortować tablicę w odwrotnej kolejności, wykorzystaj w wierszu 08 funkcję rsort zamiast sort.

Pamiętaj, że funkcja sort sortuje tablicę bez zachowania oryginalnych kluczy. Musisz o tym zawsze pamiętać. Aby zachować po sortowaniu oryginalne klucze tabeli, zastosuj funkcję asort.

Na listingu 22.3 przeanalizuj dokładnie wartości drukowane funkcją `print_r` w liniach 06 i 11.

LISTING 22.3	PHP 7/8	code_22_03.php
01	<?php	
02	<i>// deklaracja tablicy z 4 elementami</i>	
03	\$owoce = ["jabłko", "banan", "arbuz", "gruszka"];	
04		
05	print "Przed sortowaniem:".PHP_EOL;	
06	print_r(\$owoce);	
07		
08	sort(\$owoce); <i>// sortowanie tablicy</i>	
09		
10	print "Po sortowaniu:".PHP_EOL;	
11	print_r(\$owoce);	
\$ php code_22_03.php ↵		KONSOLA
Przed sortowaniem:		
Array		
(		
[0] => jabłko		
[1] => banan		
[2] => arbuz		
[3] => gruszka		
)		
Po sortowaniu:		
Array		
(		
[0] => arbuz		
[1] => banan		
[2] => gruszka		
[3] => jabłko		
)		

Ćwiczenie 4. Sortowanie według kluczy

W tym ćwiczeniu (listing 22.4) posortujesz rosnąco tablicę `$owoce` za pomocą funkcji `ksort`. W odróżnieniu od poprzedniego ćwiczenia, sortowane będą klucze elementów tablicy. Jeśli chciałbyś posortować tablicę w odwrotnej kolejności, wykorzystaj w wierszu 13 funkcję `krsort` zamiast `ksort`.

Przeanalizuj dokładnie wartości drukowane funkcją `print_r` w liniach 11 i 16.

LISTING 22.4

PHP 7/8

code_22_04.php

```

01 <?php
02 // deklaracja tablicy z 4 elementami
03 $owoce = [
04     "b" => "jabłko",
05     "c" => "banan",
06     "d" => "arbuz",
07     "a" => "gruszka"
08 ];
09
10 print "Przed sortowaniem:".PHP_EOL;
11 print_r($owoce);
12
13 ksort($owoce); // sortowanie tablicy według kluczy
14
15 print "Po sortowaniu:".PHP_EOL;
16 print_r($owoce);

```

\$ php code_22_04.php ↵

KONSOLA

```

Przed sortowaniem:
Array
(
    [b] => jabłko
    [c] => banan
    [d] => arbuz
    [a] => gruszka
)
Po sortowaniu:
Array
(
    [a] => gruszka
    [b] => jabłko
    [c] => banan
    [d] => arbuz
)

```

Ćwiczenia dodatkowe

1. Otwórz stronę <http://php.net/manual/en/array.sorting.php> i przeanalizuj funkcje sortujące. Spróbuj zrozumieć, jak działają. Przepisz i uruchom przykładowe listingi.
2. Napisz program, który wydrukuje tabelę z kolejnymi liczbami nieparzystymi.
3. W ćwiczeniu 3 zastosuj funkcję `asort` zamiast `sort` i porównaj wyniki. Zwróć uwagę na wartości kluczy.

Wskazówki

Z tablic będziesz korzystał niemal za każdym razem, kiedy będziesz gromadził i przetwarzał dane. Prosty zbiór elementów to już tablica. Zaletą języka PHP jest to, że nie musisz z góry określać rozmiaru tablicy; zawsze możesz dodawać lub usuwać z niej elementy. W tej lekcji poznałeś bardzo proste tabele, w których wszystko jest jasne. W praktyce będziesz miał do czynienia z tabelami o wielu wymiarach i wielu elementach, ale nie martw się za bardzo, ich obsługa też jest prosta.

W deklaracji tablicy zamiast nawiasów klamrowych `[]` możesz spotkać się z taką konstrukcją:

```
$owoce = array("jabłko", "banan", "arbuz", "gruszka");
```

Powyższa deklaracja tablicy jest jak najbardziej prawidłowa, jednak ja preferuję taki zapis:

```
$owoce = ["jabłko", "banan", "arbuz", "gruszka"];
```

Pytania

Klucz czy indeks?

Możesz się spotkać z różnymi nazwami, jednak oznaczają dokładnie to samo. Część dostępnych w języku PHP funkcji obsługujących tablice już w swojej nazwie ma składnik *key* (czyli klucz), np. `array_keys`, `key_exists` czy `key`, zatem również ja proponuję posługiwać się słowem „klucz”.

Czy funkcję `print_r` wykorzystuje się tylko do drukowania tablic?

Oczywiście, że nie tylko. Zadaniem funkcji `print_r` jest drukowanie struktury zmiennych w sposób przyjazny dla programisty, co ma znaczenie przy analizie zmiennych tabelarycznych. Możesz wydrukować za pomocą tej funkcji także zwykłe zmienne, niebędące tabelami. Do drukowania bardziej szczegółowych informacji o zmiennych możesz wykorzystać również funkcje `var_dump` lub `var_export`. Więcej informacji o możliwościach tych funkcji znajdziesz na stronie <http://php.net>.

Wymiary tablic

Cieszę się, że poznałeś już tablice. Co zrobisz, kiedy będziesz chciał stworzyć bardziej złożoną strukturę danych? Możesz oczywiście zbudować kilka odrębnych i niezależnych tablic i tak organizować swoje dane, może to jednak w dużym stopniu skomplikować Twój program. Znacznie lepszym rozwiązaniem jest połączenie wszystkich tablic w jedną tablicę wielowymiarową. Klasycznym przykładem takiej tablicy jest książka telefoniczna w Twoim telefonie lub lista uczniów w dzienniku elektronicznym.

Tablicę wielowymiarową często nazywa się tablicą tablic.

W tej lekcji poznasz nowe funkcje operujące na tablicach, takie jak: `array_column`, `array_merge`, `array_sum`, `array_unique`, a także konstrukcję `foreach` umożliwiającą dostęp do kolejnych kluczy i elementów w tablicy.

Pętla foreach

Iterowanie (czyli przechodzenie) po kolejnych elementach tablicy umożliwia konstrukcja `foreach`. Jest to specjalna konstrukcja pętli przeznaczona dla tablic.

Poniżej znajdziesz prosty przykład wykorzystania pętli `foreach` na tablicy. Do wnętrza pętli `foreach` z każdą iteracją trafia kolejny element tablicy, który następnie może być w dowolny sposób przetworzony. Tak jak w przypadku pozostałych pętli, tutaj także można korzystać z instrukcji `continue` i `break`.

```
$tablica = [  
    'imie' => 'Tadek'  
];  
foreach ($tablica as $klucz=>$wartosc)  
{  
    // instrukcje do wykonania na każdym elemencie tablicy  
    print $klucz." ".$wartosc; // informacje o kolejnym elemencie tablicy  
}  
// wynik: imie:Tadek
```

Tablice dwuwymiarowe

Deklaracja (utworzenie, inicjacja) tablicy dwuwymiarowej w PHP wygląda tak:

```
$array = [ //tablica 2-wymiarowa
    "klucz_1" => [ //tablica 1-wymiarowa
        "klucz_1_a" => "wartość_1_a",
        "klucz_1_b" => "wartość_1_b",
        "klucz_1_c" => "wartość_1_c"
    ],
    "klucz_2" => [ //tablica 1-wymiarowa
        "klucz_2_a" => "wartość_2_a",
        "klucz_2_b" => "wartość_2_b",
        "klucz_2_c" => "wartość_2_c"
    ],
    "klucz_3" => [ //tablica 1-wymiarowa
        "klucz_3_a" => "wartość_3_a",
        "klucz_3_b" => "wartość_3_b",
        "klucz_3_c" => "wartość_3_c"
    ],
];
```

Do konkretnego elementu w takiej tablicy możesz się odwołać za pomocą ciągu kluczy, przykładowo odwołanie `$array["klucz_2"]["klucz_2_b"]` zwróci wartość `wartość_2_b`.

Uproszczona wersja powyższych deklaracji, w której klucze nadawane są automatycznie, począwszy od 0 (zera), wygląda tak:

```
$array = [ //tablica 2-wymiarowa
    ["wartość_1_a", "wartość_1_b", "wartość_1_c"], //tablica 1-wymiarowa
    ["wartość_2_a", "wartość_2_b", "wartość_2_c"], //tablica 1-wymiarowa
    ["wartość_3_a", "wartość_3_b", "wartość_3_c"] //tablica 1-wymiarowa
];
```

Oznacza to, że kluczem dla pierwszej tablicy będzie 0, dla drugiej tablicy — 1 itd. Podobnie będzie w przypadku wszystkich elementów zawartych w tablicach. Przykładowo odwołanie `$array[0][1]` zwróci wartość `wartość_1_b`.

Ćwiczenie 1. Tablica dwuwymiarowa

W tym ćwiczeniu stworzysz prostą tabelę dwuwymiarową z kontaktami telefonicznymi (podobną masz w swoim smartfonie). Przeanalizuj kod z listingu 23.1 linia po linii i postaraj się zrozumieć, co się dzieje z danymi umieszczonymi w tabelach `$kontakt` i `$kontakty`.

LISTING 23.1

PHP 7/8

code_23_01.php

```

01 <?php
02 $kontakt = []; // deklaracja tablicy
03
04 // pierwszy kontakt
05 $kontakt['imie'] = 'Adam';
06 $kontakt['nazwisko'] = 'Kowalski';
07 $kontakt['telefon'] = '123456789';
08
09 $kontakty = []; // deklaracja tablicy
10
11 // w tym momencie tablica $kontakty staje się już tablicą 2-wymiarową,
12 // ponieważ wstawiamy do niej tablicę $kontakt, tworzymy tablicę tablic
13 array_push($kontakty, $kontakt);
14
15 // drugi kontakt, wypełniamy ponownie tablicę $kontakt
16 $kontakt['imie'] = 'Ania';
17 $kontakt['nazwisko'] = 'Nowakówna';
18 $kontakt['telefon'] = '987654321';
19
20 // zamiast array_push możesz użyć formy skróconej []
21 $kontakty[] = $kontakt;
22
23 print_r($kontakty); // jak wygląda tablica w programie?
24
25 print PHP_EOL;
26 printf("Imię pierwszego kontaktu to: %s".PHP_EOL, $kontakty[0]['imie']);
27
28 print PHP_EOL;
29 print "Lista kontaktów:".PHP_EOL;
30
31 $e = count($kontakty); // ile elementów ma tablica?
32
33 // dostęp do elementów tablicy
34 for ($i = 0; $i < $e; $i++) {
35     printf(
36         "%s\t%s\ttel. %s".PHP_EOL,
37         $kontakty[$i]['imie'],
38         $kontakty[$i]['nazwisko'],
39         $kontakty[$i]['telefon']
40     );
41 }

```

\$ php code_23_01.php ↵

KONSOLA

```

Array
(
    [0] => Array

```

```

    (
        [imie] => Adam
        [nazwisko] => Kowalski
        [telefon] => 123456789
    )

[1] => Array
(
    [imie] => Ania
    [nazwisko] => Nowakówna
    [telefon] => 987654321
)

)

Imię pierwszego kontaktu to: Adam

Lista kontaktów:
Adam      Kowalski      tel. 123456789
Ania      Nowakówna      tel. 987654321

```

Jak działa powyższy kod?

1. W linii 02 utworzyłem zmienną tablicową \$kontakt, do której wstawiłem w wierszach od 05 do 07 przykładowe dane kontaktowe. Zauważ, że do powiązania danych wykorzystałem unikatowe i zrozumiałe klucze. Założyłem również, że tablica \$kontakt będzie tablicą jednowymiarową.
2. W linii 09 utworzyłem zmienną tablicową \$kontakty, w której będą przechowywane wszystkie dane (podtablice \$kontakt), zatem będzie to tablica dwuwymiarowa. Aby dobrze to zrozumieć, wyobraź sobie, że lista wszystkich znajomych w Twoim smartfonie będzie przechowywana w tablicy \$kontakty, a pojedynczy kontakt w tablicy \$kontakt.
3. W wierszu 13 wstawiam do tablicy \$kontakty tablicę \$kontakt, której automatycznie nadawany jest klucz 0 (zero). Od teraz tablica \$kontakty jest tak zwaną tablicą tablic.
4. W wierszach od 16 do 18 wstawiam nowe wartości w tablicy \$kontakt, aby w linii 21 wstawić ją jako kolejny element tablicy \$kontakty. Zwróć uwagę, że zamiast funkcji array_push użyłem formy skróconej [] (nawiasy kwadratowe).
5. W linii 23 za pomocą funkcji print_r drukowana jest zawartość tablicy \$kontakty. Zauważ, że tablica ta składa się z dwóch elementów, o kluczach 0 (zero) i 1. Pozostałe wiersze kodu powinny być już dla Ciebie zrozumiałe.

Ćwiczenie 2. Obliczenia na tablicach

W tym ćwiczeniu (listing 23.2) stworzysz listę uczniów klas pierwszych wraz z uzyskanymi przez nich ocenami. Dodatkowo obliczysz średnią wszystkich ocen dla każdej klasy i sporządzisz listę imion dzieci w porządku alfabetycznym.

LISTING 23.2	PHP 7/8	code_23_02.php
01	<?php	
02	<i>// tablice 2-wymiarowe z imionami i ocenami uczniów klas 1a i 1b</i>	
03	<code>\$klasa_1a = [</code>	
04	<code> ['imie' => 'Adam', 'ocena' => 4.5],</code>	
05	<code> ['imie' => 'Marek', 'ocena' => 5],</code>	
06	<code> ['imie' => 'Jola', 'ocena' => 5],</code>	
07	<code> ['imie' => 'Robert', 'ocena' => 4],</code>	
08	<code> ['imie' => 'Kasia', 'ocena' => 4]</code>	
09	<code>];</code>	
10	<code>\$klasa_1b = [</code>	
11	<code> ['imie' => 'Robert', 'ocena' => 5],</code>	
12	<code> ['imie' => 'Kasia', 'ocena' => 4],</code>	
13	<code> ['imie' => 'Adam', 'ocena' => 3.5],</code>	
14	<code> ['imie' => 'Magda', 'ocena' => 4],</code>	
15	<code> ['imie' => 'Rafał', 'ocena' => 5]</code>	
16	<code>];</code>	
17		
18	<i>// tablica 3-wymiarowa ze wszystkimi uczniami i ocenami klas 1</i>	
19	<code>\$klasy = [</code>	
20	<code> '1a' => \$klasa_1a,</code>	
21	<code> '1b' => \$klasa_1b</code>	
22	<code>];</code>	
23		
24	<code>#print_r(\$klasy); exit;</code>	
25		
26	<i>// drukowanie imion uczniów i uzyskanych ocen</i>	
27	<code>foreach (\$klasy as \$e => \$klasa) {</code>	
28		
29	<code> print "Klasa: ".\$e."\n".</code>	
30	<code> "-----\n".</code>	
31	<code> "Imię\tOcena\n".</code>	
32	<code> "-----\n";</code>	
33		
34	<code> foreach (\$klasa as \$k) {</code>	
35	<code> printf("%s\t%s\n", \$k['imie'], \$k['ocena']);</code>	
36	<code> }</code>	
37		
38	<code> print "-----\n";</code>	
39		

```

40      // sumujemy wszystkie oceny i dzielimy przez liczbę ocen
41      $oceny = array_column($klasa, 'ocena'); // wyodrębniamy kolumnę z ocenami
42      $srednia = array_sum($oceny) / count($oceny);
43
44      printf ("Średnia\t%.2f\n\n", $srednia);
45  }
46
47  $imiona = []; // tablica z imionami uczniów
48
49  $imiona = array_merge($klasa_1a, $klasa_1b); // łączymy tabele
50  #print_r($imiona);
51
52  $imiona = array_column($imiona, 'imie'); // wyodrębniamy kolumnę z imionami
53  #print_r($imiona);
54
55  $imiona = array_unique($imiona); // usuwamy powtarzające się imiona
56  #print_r($imiona);
57
58  sort ($imiona); // sortujemy alfabetycznie (rosnąco)
59  #print_r($imiona);
60
61  print "Dzieci w klasach mają następujące imiona:\n";
62
63  foreach ($imiona as $k => $imie) {
64      print $imie.", ";
65  }

```

\$ php code_23_02.php ↵

KONSOLA

```

Klasa: 1a
-----
Imię      Ocena
-----
Adam      4.5
Marek     5
Jola      5
Robert    4
Kasia     4
-----
Średnia 4.50

Klasa: 1b
-----
Imię      Ocena
-----
Robert    5
Kasia     4
Adam      3.5
Magda     4

```

```
Rafał    5
-----
Średnia 4.30
```

Dzieci w klasach mają następujące imiona:
Adam, Jola, Kasia, Magda, Marek, Rafał, Robert,

Jak działa powyższy kod?

1. W wierszach 03 i 10 utworzyłem tablice dwuwymiarowe (\$klasa_1a i \$klasa_1b) dla dwóch klas pierwszych.
2. W linii 19 utworzyłem tablicę trójwymiarową \$klasy, w której wstawiłem wcześniej utworzone klasy dwuwymiarowe. Proponuję, abys usunął teraz znak hasha (#) z początku linii 24 i uruchomił program. Zostanie wyświetlona struktura tabeli trójwymiarowej ze wszystkimi danymi. Postaraj się zrozumieć, dlaczego tak wygląda, a następnie przywróć znak hasha (#), który przed chwilą usunąłeś.
3. Począwszy od wiersza 47, funkcją array_merge połączyłem tablice (linia 49), następnie funkcją array_column wyodrębniłem z tablic wszystkie imiona (wiersz 52) i usunąłem duplikaty przy użyciu funkcji array_unique (linia 55), aby na końcu posortować wszystko alfabetycznie (wiersz 58).
4. Dla lepszego zrozumienia, jak zmienia się zawartość tablicy \$imiona, usuń znaki hasha (#) z linii 50, 53, 56 i 59, a następnie uruchom program.

Ćwiczenia dodatkowe

1. Otwórz stronę <https://www.php.net/manual/en/ref.array.php> i zapoznaj się z pozostałymi funkcjami operującymi na tablicach. Spróbuj zrozumieć, jak działają. Przepisz i uruchom przykładowe listingi.
2. Dodaj w ćwiczeniu 2 dodatkową tablicę z imionami uczniów i ocenami (np. 1c). Zmodyfikuj odpowiednio program, aby wydrukowana została również informacja o nowej klasie.
3. Otwórz stronę <http://php.net> i wyszukaj opis funkcji json_encode, json_decode i explode.

Wskazówki

W praktyce zawartości tabel nie będziesz umieszczał w kodzie programu, ponieważ jest to nieefektywne. Dane często się zmieniają, dlatego będziesz je zwykle pobierał ze źródeł zewnętrznych, takich jak bazy danych czy pliki tekstowe. Ważne, abyś zapamiętał, jak wygląda struktura tabeli jedno- lub wielowymiarowej i jak są w nich organizowane dane.

W następnej lekcji, dotyczącej operacji na plikach, pokażę Ci, jak pobierać dane z plików tekstowych, a następnie przetwarzać je i umieszczać w tabelach.

Pytania

Czy dane w tablicach można zapisać prościej?

Oczywiście, że można! Liczba rozwiązań jest bardzo duża. W pierwszej kolejności musisz się zastanowić, co dla Ciebie oznacza „prościej”. Możesz wykorzystać standard JSON i dane zapisać tak:

```
$klasa_1a_txt = '[
{"imie":"Adam", "ocena":"4.5"},
{"imie":"Marek", "ocena":"5"},
{"imie":"Jola", "ocena":"5"},
{"imie":"Robert", "ocena":"4"},
{"imie":"Kasia", "ocena":"4"}
]';
```

a następnie zdekodować je funkcją `json_decode`.

Możesz, rzecz jasna, opracować (co bardzo zalecam) własny format zapisu danych, który będzie dla Ciebie najwygodniejszy. Pamiętaj, że ma to sens wtedy, kiedy nie wykorzystujesz bazy danych do składowania informacji i zależy Ci na optymalizacji pamięci zajmowanej przez dane. Poniżej moja propozycja takiego rozwiązania, która sprowadza się do wykorzystania znaków specjalnych do odseparowania danych.

```
$klasa_1a_txt = "Adam,4.5|Marek,5|Jola,5|Robert,4|Kasia,4";
```

Znak `|` (pionowa kreska) oddziela uczniów, a znak `,` (przecinek) oddziela imię i ocenę konkretnego ucznia. Zapis ten jest znacznie krótszy i śmiało można powiedzieć, że bardzo czytelny (oczywiście, w tym konkretnym przypadku). Problem w tym, że w swoim programie będziesz musiał napisać dodatkowy kod obsługujący ten format.

Najlepiej napisać dodatkową funkcję, która zdekoduje format tekstowy i zwróci tablicę. Na listingu 23.3 zamieściłem 27 linii kodu. Zamień je w ćwiczeniu 2 i uruchom program.

LISTING 23.3	PHP 7/8	code_23_03.php
01	<?php	
02	<i>// proste zakodowanie informacji o uczniach i ocenach</i>	
03	<code>\$klasa_1a_txt = "Adam,4.5 Marek,5 Jola,5 Robert,4 Kasia,4";</code>	
04	<code>\$klasa_1b_txt = "Robert,5 Kasia,4 Adam,3.5 Magda,4 Rafał,5";</code>	
05		
06	function zdekoduj(\$dane) { <i>//zdekodowanie informacji</i>	
07	<code>\$r = [];</code>	
08	<code>\$e = explode(" ", \$dane);</code> <i>// znak podziału </i>	
09		
10	foreach (\$e as \$k) {	
11	<code>\$u = explode(",", \$k);</code> <i>//znak podziału ,</i>	
12	<code>\$r[] = ['imie' => \$u[0], 'ocena' => \$u[1]];</code>	
13	}	
14	return \$r;	
15	}	
16		
17	<i>// tablice 2-wymiarowe z imionami i ocenami uczniów klas 1a i 1b</i>	
18	<code>\$klasa_1a = zdekoduj(\$klasa_1a_txt);</code>	
19	<code>\$klasa_1b = zdekoduj(\$klasa_1b_txt);</code>	
20		
21	<i>// tablica 3-wymiarowa ze wszystkimi uczniami i ocenami klas 1</i>	
22	<code>\$klasy = [</code>	
23	<code>'1a' => \$klasa_1a,</code>	
24	<code>'1b' => \$klasa_1b</code>	
25	<code>];</code>	
26		
27	<i>/* dalsza część kodu z ćwiczenia 2 */</i>	



Odczyt i zapis plików

W trakcie przygody z programowaniem z pewnością dotrzesz do miejsca, w którym będziesz chciał wykorzystać dane z zewnętrznego źródła. Najprostszą formą przechowywania informacji są pliki tekstowe, które możesz dowolnie przetwarzać za pomocą wielu funkcji dostępnych w języku PHP. W tej lekcji nauczysz się stosować podstawowe funkcje do odczytywania i zapisywania danych z plików znajdujących się na dysku Twojego komputera, a także pobierzesz informacje dostępne w internecie.

Poniższa konstrukcja (listing 24.1) umożliwia odczytanie pliku (jeśli istnieje) i wstawienie jego zawartości do zmiennej:

LISTING 24.1	PHP 7/8	code_24_01.php
01	<?php	
02	\$filename = "jakisplik.txt"; <i>// nazwa pliku do odczytania</i>	
03	if (is_file(\$filename)) <i>// czy plik istnieje?</i>	
04	{	
05	<i>// otwarcie pliku w trybie tylko do odczytu</i>	
06	\$file = fopen(\$filename, "r");	
07		
08	<i>// pobranie zawartości pliku do zmiennej</i>	
09	\$contents = fread(\$file, filesize(\$filename));	
10		
11	fclose(\$file); <i>// zamknięcie pliku</i>	
12	}	

Właśnie poznałeś nowe funkcje do obsługi plików: `is_file`, `fopen`, `fread`, `filesize`, `fclose`. Myślę, że po analizie kodu nie będziesz miał większych problemów ze zrozumieniem, jak działają. W tym miejscu należy zwrócić uwagę na parametry funkcji `fopen` i `fread`.

Funkcja `fopen` otwiera plik wskazany w pierwszym parametrze, w konkretnym trybie dostępu, określonym w parametrze drugim. Funkcja `fread` pobiera zaś zawartość

pliku, którego rozmiar jest określony drugim parametrem. W powyższym przykładzie pobierana zawartość jest równa rozmiarowi pliku.

Tryby otwarcia plików w języku PHP, z których będziesz najczęściej korzystać:

Tryb otwarcia	Działanie	Wskaźnik pliku
r	Otwiera plik tylko do odczytu.	na początku
r+	Otwiera plik do odczytu i zapisu.	na początku
w	Otwiera plik tylko do zapisu, usuwa jego zawartość. Jeśli plik nie istnieje, próbuje go utworzyć.	na początku
w+	Otwiera plik do zapisu i odczytu, usuwa jego zawartość. Jeśli plik nie istnieje, próbuje go utworzyć.	na początku
a	Otwiera plik tylko do zapisu. Jeśli plik nie istnieje, próbuje go utworzyć.	na końcu
a+	Otwiera plik do odczytu i zapisu. Jeśli plik nie istnieje, próbuje go utworzyć.	na końcu

Język PHP zapewnia dodatkowe funkcje, za pomocą których będziesz mógł odczytać informacje z pliku linia po linii (`fgets`) czy znak po znaku (`fgetc`).

Zapis informacji do pliku umożliwia funkcja `fwrite`, która może wyglądać w programie tak jak na listingu 24.2.

LISTING 24.2	PHP 7/8	code_24_02.php
01	<?php	
02	\$file = fopen ("jakisplik.txt", "w"); // otwarcie pliku w trybie tylko do zapisu	
03	fwrite (\$file, "To będzie dobry dzień!"); // zapisanie tekstu w pliku	
04	fclose (\$file); // zamknięcie pliku	

Przy zapisywaniu informacji należy zawsze zwrócić uwagę na tryb otwarcia, gdyż to od niego zależy, czy dane będą zapisane na początku, czy na końcu pliku (zobacz wskaźnik pliku w tabeli).

Ćwiczenie 1. Zapisanie tekstu do pliku

W tym ćwiczeniu stworzysz plik, w którym zapiszesz przykładowy tekst (być może Twój pierwszy wiersz). Przeanalizuj program z listingu 24.3 linia po linii i postaraj się zrozumieć.

LISTING 24.3	PHP 7/8	code_24_03.php
01	<?php	
02	\$wiersz = <<<EOT	
03	To jest mój piękny wiersz	
04	o wszystkim, co mi się podoba!	
05	Kolejne linie są dla Ciebie!	
06	EOT;	
07		
08	\$filename = "wiersz.txt";	
09		
10	if (!\$file = fopen(\$filename, "w")) { <i>// otwarcie pliku w trybie do zapisu</i>	
11	die("Nie mogę utworzyć pliku \$filename");	
12	}	
13		
14	if (fwrite(\$file, \$wiersz) === false) { <i>// zapis tekstu w pliku</i>	
15	die("Nie mogę zapisać w pliku \$filename");	
16	}	
17		
18	fclose(\$file); <i>// zamknięcie pliku</i>	
19	print "Wiersz został zapisany w pliku \$filename";	
\$ php code_24_03.php ↵		KONSOLA
Wiersz został zapisany w pliku wiersz.txt		

Jak działa powyższy kod?

1. W liniach od 02 do 06 znajduje się definicja zmiennej `$wiersz` w formacie *heredoc*. Zwróć uwagę na definicję zmiennej pomiędzy `<<<EOT` a `EOT;`.
2. W wierszu 10 otwieram plik określony w zmiennej `$filename` w trybie do zapisu. Jeśli pliku nie ma, program go utworzy.
3. W linii 14 zapisuję tekst do pliku.
4. W wierszu 18 plik jest zamykany. Zwróć uwagę na konstrukcje warunkowe pomiędzy liniami 10 a 12 i 14 a 16 — mają one na celu zabezpieczenie programu przed ewentualnymi problemami.

Ćwiczenie 2. Odczytanie tekstu z pliku

W tym ćwiczeniu utworzysz plik utworzony w ćwiczeniu 1 i wydrukujesz jego wartość. Na listingu 24.4 w linii 04 sprawdzam, czy plik istnieje, a w wierszu 07 otwieram plik w trybie do odczytu. W linii 10 pobieram zawartość całego pliku i wstawiam do zmiennej `$contents`, a w wierszu 15 drukuję. Proste!

LISTING 24.4	PHP 7/8	code_24_04.php
01	<?php	
02	\$filename = "wiersz.txt";	
03		
04	if (is_file(\$filename)) // czy plik istnieje?	
05	{	
06	// otwarcie pliku w trybie tylko do odczytu	
07	\$file = fopen(\$filename, "r");	
08		
09	// pobranie zawartości do zmiennej	
10	\$contents = fread(\$file, filesize(\$filename));	
11		
12	fclose(\$file); // zamknięcie pliku	
13		
14	print "Mój wiersz z pliku \$filename".PHP_EOL;	
15	print \$contents;	
16	}	
\$ php code_24_04.php ↵		KONSOLA
Mój wiersz z pliku wiersz.txt To jest mój piękny wiersz o wszystkim, co mi się podoba! Kolejne linie są dla Ciebie!		

Ćwiczenie 3. Pobranie pliku z internetu

Teraz zajmiesz się czymś trudniejszym. W tym ćwiczeniu (listing 24.5) pobierzesz z internetu archiwalne wyniki losowań Dużego Lotka i zapiszesz je na dysku w pliku tekstowym. Następnym krokiem będzie wyszukanie w tym pliku wyników losowań dla konkretnego roku i miesiąca. Wszystkie wyniki wydrukujesz, zdekodujesz do formatu tablicowego i umieścisz w zmiennej.

Do realizacji zadania wykorzystasz funkcje `file_get_contents` i `file_put_contents`. Często ich używam. Więcej informacji o nich umieściłem we wskazówkach do tej lekcji. A zatem do pracy!

LISTING 24.5	PHP 7/8	code_24_05.php
01	<?php	
02	// Archiwalne wyniki losowań Dużego Lotka	
03	// Źródło: www.mbnet.com.pl	
04	// format: nr_losowania data (w formacie dd.mm.rrrr) 6_wylosowanych_liczb	
05	// przykład: 871. 04.03.1973 2,11,12,14,33,37	
06	\$url = "http://www.mbnet.com.pl/dl.txt";	
07		

```

08 // nazwa pliku, w którym zapiszemy wszystkie losowania
09 $filename = "losowanie.txt";
10
11 // jaki okres nas interesuje?
12 // pierwsze losowanie Dużego Lotka — 27 stycznia 1957 roku
13 $year = "1977";
14 $month = "09";
15
16 $table = []; // definicja zmiennej tablicowej
17
18 // pobierz dane archiwalne, jeśli wcześniej nie zostały pobrane
19 if (!is_file($filename)) { // czy plik istnieje?
20     $data = file_get_contents($url); // pobierz dane z internetu
21     file_put_contents($filename, $data); // zapisz dane w pliku na dysku
22 }
23
24 if (is_file($filename)) // czy plik istnieje?
25 {
26     // otwarcie pliku w trybie tylko do odczytu
27     $file = fopen($filename, "r");
28
29     while(!feof($file)) // czy koniec pliku?
30     {
31         $data = fgets($file); // pobierz linię tekstu z pliku
32         if ($data) { // czy zmienna coś zawiera?
33             $pos = mb_strpos($data, $month.".".$year);
34             if ($pos !== false) {
35                 echo $data; // drukowanie odnalezionego losowania
36
37                 // zdekoduj z formatu tekstowego do tablicy
38                 $t1 = explode(" ", $data); // znak podziału — spacja
39                 $t2 = explode(".", $t1[1]); // znak podziału — kropka
40
41                 // wstawienie danych do tablicy 3-wymiarowej
42                 // ([rok][miesiąc][dzień])
43                 $table[$t2[2]][$t2[1]][$t2[0]] = $t1[2];
44             }
45         }
46     }
47
48     fclose($file); // zamknięcie pliku
49 }
50
51 // zmienna tablicowa $table zawiera informacje o wyszukanych losowaniach
52 print_r($table);

```

\$ php code_24_05.php ↵

KONSOLA

```

1294. 04.09.1977 5,26,27,31,48,49
1295. 04.09.1977 2,13,16,19,25,33
1296. 11.09.1977 13,23,32,33,43,49
1297. 11.09.1977 3,8,9,10,29,46
1298. 18.09.1977 3,4,26,28,33,38
1299. 18.09.1977 1,4,15,23,28,36
1300. 25.09.1977 1,3,7,11,21,40
1301. 25.09.1977 17,21,24,25,39,41
Array
(
    [1977] => Array
        (
            [09] => Array
                (
                    [04] => 2,13,16,19,25,33
                    [11] => 3,8,9,10,29,46
                    [18] => 1,4,15,23,28,36
                    [25] => 17,21,24,25,39,41
                )
            )
        )
)

```

Jak działa powyższy kod?

1. W linii 06 umieściłem adres URL w zmiennej `$url`, prowadzący do archiwalnych wyników losowania Dużego Lotka (możesz wejść pod ten adres w przeglądarce i dowiedzieć się, od którego roku ludzie marzą o wielkiej wygranej).
2. W wierszu 09 w zmiennej `$filename` zdefiniowałem nazwę pliku, do którego zostaną zapisane archiwalne wyniki losowań Dużego Lotka.
3. W liniach 13 i 14 wskazałem okres, który ma być wyszukany w wynikach.
4. W wierszu 16 zdefiniowana jest zmienna tablicowa `$table`, w której będą przechowywane wyszukane wyniki losowań.
5. W liniach od 19 do 22 pobierane są archiwalne wyniki z internetu, które są zapisywane na dysk. Operacja ta jest wykonywana tylko raz, a przynajmniej do momentu, kiedy pobrany plik będzie dostępny na dysku.
6. W wierszu 27 otwierany jest plik w trybie tylko do odczytu.
7. W liniach od 29 do 46 wykonywana jest pętla `while`, w której odczytywane są kolejne linie z pliku (wiersz 31). Konstrukcja warunkowa, począwszy od linii 34, powinna być dla Ciebie doskonale zrozumiała, gdyż poznałeś ją we wcześniejszych lekcjach.

8. Do zdekodowania informacji (wiersze 38 i 39) wykorzystywana jest funkcja `explode`, za pomocą której tekst będzie rozbijany na mniejsze elementy.
9. W linii 43 zdekodowane dane wstawiane są do tablicy `$table`. Zwróć uwagę na tę część kodu i postaraj się dokładnie ją przeanalizować.
10. Na wyjściu zostaną wydrukowane wyniki losowań z okresu, który nas interesuje.

Ćwiczenie 4. Licznik odwiedzin

Przy wykorzystaniu plików tekstowych możesz napisać prosty licznik odwiedzin swojej strony internetowej. Na listingu 24.6 znajduje się przykładowy program takiego licznika. W zasadzie cały kod zawarty jest w funkcji `licznik`, której możesz w dowolny sposób użyć w innych swoich programach. Jestem przekonany, że analiza i zrozumienie tego programu nie będą dla Ciebie problemem.

Zapisz poniższą funkcję (kod bez linii 32) do nowego pliku o nazwie `licznik.php` w katalogu, gdzie znajdują się pozostałe pliki z przerabianymi lekcjami. Wykorzystasz go jeszcze w następnych ćwiczeniach.

LISTING 24.6	PHP 8	code_24_06.php
01	<?php	
02	function <code>licznik</code> (?string \$filename = 'licznik.txt') : int	
03	{	
04	<i>// czy plik istnieje?</i>	
05	if (! <code>is_file</code> (\$filename))	
06	{	
07	<i>// jeśli nie, utwórz plik z początkową wartością</i>	
08	<code>file_put_contents</code> (\$filename, "1");	
09	return (int)1; <i>// zwróć wartość funkcji</i>	
10	}	
11	<i>// OK, plik już istnieje</i>	
12	else	
13	{	
14	\$file = <code>fopen</code> (\$filename, "r+"); <i>// otwórz plik</i>	
15		
16	if (<code>flock</code> (\$file, <code>LOCK_EX</code>)) <i>// zablokuj plik</i>	
17	{	
18	<i>// odczytaj wartość licznika</i>	
19	\$counter = (int) <code>fread</code> (\$file, <code>filesize</code> (\$filename));	
20	\$counter++; <i>// zwiększ o 1</i>	
21	<code>fseek</code> (\$file, 0); <i>// ustaw wskaźnik pliku na początek</i>	
22	<code>fwrite</code> (\$file, \$counter); <i>// zapisz nową wartość licznika do pliku</i>	

```

23         flock($file, LOCK_UN); //zwolnij blokadę
24     }
25
26     fclose($file); //zamknij plik
27
28     return (int)$counter;
29 }
30 }
31 //zapisz funkcję w pliku pod nazwą licznik.php
32 print "Jesteś ".licznik(filename: 'licznik.txt')." gościem na mojej stronie.";
$ php code_24_06.php ↵ KONSOLA
Jesteś 1 gościem na mojej stronie.

```

Jak działa powyższy kod?

1. W linii 02 definiuję funkcję `licznik` z opcjonalnym argumentem `$filename`.
2. W wierszu 05 sprawdzam, czy plik z licznikiem istnieje; jeśli nie, tworzę nowy i wstawiam do niego wartość początkową równą 1 (linia 08).
3. Jeśli plik już istnieje, funkcją `flock` blokuję do niego dostęp (linia 16), a następnie odczytuję aktualną wartość licznika (linia 19), zwiększam o 1 (linia 20), ustawiam funkcją `fseek` wskaźnik pliku na początek (linia 21), zapisuję nową wartość do pliku (linia 22) i zwalnim blokadę (linia 23).
4. Funkcja zwraca wartość licznika w wierszu 09, kiedy plik z licznikiem jest tworzony, lub w linii 28, kiedy zwiększona wartość zostaje zapisana do pliku.

Ćwiczenia dodatkowe

1. Otwórz stronę <https://www.php.net/manual/en/ref.filesystem.php> i zapoznaj się z pozostałymi funkcjami operującymi na plikach. Spróbuj zrozumieć, jak działają. Przepisz i uruchom przykładowe listingi.
2. Zamień w ćwiczeniu 3 linię 43 na:

```
$table[$t2[2]][$t2[1]][$t2[0]] = explode(",", $t1[2]);
```

Jak zmieniła się struktura zmiennej tablicowej `$table`? Jak to wytłumaczysz?

3. Napisz program, który wyszuka w pliku z archiwalnymi wynikami Dużego Lotka dni, kiedy zostały wylosowane jednocześnie liczby 10, 20 i 30, a następnie te losowania wydrukuje.

Wskazówki

Pisząc program, który gromadzi i przetwarza dane w plikach, musisz pamiętać, że istnieje coś takiego jak prawdopodobieństwo zakleszczenia (wzajemnej blokady), czyli sytuacji, kiedy na pliku wykonywane są jednocześnie co najmniej dwie różne operacje.

Każda taka operacja przez cały czas czeka na swoją kolej, co w efekcie blokuje wykonanie programu i może spowodować uszkodzenie pliku. Zdarza się to często w przypadku operacji na dużych plikach, jednak Twój kod powinien taką sytuację „przewidzieć” dla każdego pliku. Możesz to osiągnąć przez zablokowanie dostępu do pliku na czas wykonywania na nim operacji, by następnie tę blokadę zdjąć i umożliwić dostęp innym procesom do pliku.

W języku PHP dostępna jest funkcja `flock`, którą możesz wykorzystać do tego celu. Na listingu 24.7 znajduje się konstrukcja obsługująca blokadę pliku. Zauważ, że przed próbą zapisu program zablokuje dostęp do pliku innym procesom (linia 03). Blokada jest automatycznie zdejmowana, kiedy plik jest zamykany funkcją `fclose` lub program kończy swoje działanie.

LISTING 24.7	PHP 7/8	code_24_07.php
01	<?php	
02	<code>\$file = fopen("dane.txt", "a+");</code>	<i>// otwórz plik</i>
03	<code>if (flock(\$file, LOCK_EX))</code>	<i>// zablokuj plik</i>
04	<code>{</code>	
05	<code> fwrite(\$file, "Jakiś tekst");</code>	<i>// zapisz do pliku</i>
06	<code> flock(\$file, LOCK_UN);</code>	<i>// zwolnij blokadę</i>
07	<code>}</code>	
08	<code>else</code>	<i>// plik jest już zablokowany</i>
09	<code>{</code>	
10	<code> print "Nie mogę zapisać, bo plik jest zablokowany!";</code>	
11	<code>}</code>	
12	<code>fclose(\$file);</code>	<i>// zamknij plik</i>

Jak zauważyłeś, powyższy program składa się z kilkunastu linii kodu. Program możesz znacznie skrócić przy wykorzystaniu funkcji `file_get_contents` i `file_put_contents`, za pomocą których szybko odczytasz i zapiszesz dane w pliku.

Powyższy przykład można zapisać o wiele prościej, czyli tak jak na listingu 24.8.

LISTING 24.8	PHP 7/8	code_24_08.php
01	<?php	
02	<code>file_put_contents("dane.txt", "Jakiś tekst", FILE_APPEND LOCK_EX);</code>	

Podobnie jest w przypadku odczytu danych z pliku lub z internetu (listing 24.9):

LISTING 24.9	PHP 7/8	code_24_09.php
01	<?php	
02	\$dane = file_get_contents ("dane.txt"); <i>// odczytanie pliku</i>	
03	\$html = file_get_contents ("https://helion.pl"); <i>// pobranie kodu strony WWW</i>	

Jeszcze kilka słów o wskaźniku pliku. Otóż miejsce, od którego po otwarciu pliku następuje zapis lub odczyt, nazywamy wskaźnikiem. Wskaźnik może być domyślnie ustawiony na początku lub na końcu. Plik to nic innego jak zbiór danych, po którym możesz się dowolnie poruszać. Znajdź opis funkcji `fseek` i spróbuj ją wykorzystać na dowolnym pliku, np. z pobranymi losowaniami.

Pytania

Jak utworzyć i usunąć katalog?

Katalog utworzysz funkcją `mkdir`, a usuniesz funkcją `rmdir` (listing 24.10):

LISTING 24.10	PHP 7/8	code_24_10.php
01	<?php	
02	\$dirname = "dane";	
03	if (! is_dir (\$dirname)) { <i>// jeśli katalog nie istnieje</i>	
04	mkdir ("dane"); <i>// to go utwórz</i>	
05	}	
06	if (is_dir (\$dirname)) { <i>// czy katalog istnieje?</i>	
07	rmdir ("dane"); <i>// jeśli tak, to go usuń</i>	
08	}	

Jak usunąć plik?

Plik możesz usunąć za pomocą funkcji `unlink`, (listing 24.11):

LISTING 24.11	PHP 7/8	code_24_11.php
01	<?php	
02	\$filename = "losowanie.txt";	
03	if (is_file (\$filename)) { <i>// czy plik istnieje?</i>	
04	unlink ("losowanie.txt"); <i>// jeśli tak, to go usuń</i>	
05	}	

To ile było takich losowań? Jak rozwiązać trzecie ćwiczenie dodatkowe?

Wstaw kod programu z listingu 24.12 w ćwiczeniu 3 zamiast linii od 32 do 45. Takich losowań w historii było jedenaście.

LISTING 24.12	PHP 7/8	code_24_12.php
01	if (\$data) {	
02	\$t1 = explode(" ", \$data); <i>//znak podziału — spacja</i>	
03	\$t2 = explode(".", \$t1[1]); <i>//znak podziału — kropka</i>	
04	\$pos1 = mb_strpos(\$t1[2], "10"); <i>//pierwsza szukana liczba</i>	
05	\$pos2 = mb_strpos(\$t1[2], "20"); <i>//druga szukana liczba</i>	
06	\$pos3 = mb_strpos(\$t1[2], "30"); <i>//trzecia szukana liczba</i>	
07		
08	if (\$pos1 !== false && \$pos2 !== false && \$pos3 !== false) {	
09	\$table[\$t2[2]][\$t2[1]][\$t2[0]] = \$t1[2];	
10	print \$data;	
11	}	
12	}	



Sytuacje wyjątkowe

Język PHP umożliwia programiście obsługę sytuacji wyjątkowych, czyli tak zwanych wyjątków (ang. *exceptions*), które pozwalają na reagowanie na nieprzewidziane sytuacje i błędy w kodzie. Sytuacje wyjątkowe to takie jak np. problem z dostępem do pliku, nieoczekiwane wartości argumentów funkcji, zbyt długi czas oczekiwania na odpowiedź z gniazda sieciowego i wiele innych, które negatywnie wpływają na działanie programu. W tej lekcji nauczysz się stosować technikę zgłaszania i obsługi wyjątków, wykorzystując instrukcję `throw` oraz konstrukcję `try-catch-finally`.

Nowością w wersji PHP 8 jest to, że przechwycony wyjątek w instrukcji `catch` nie musi być przypisany do zmiennej, co było dotychczas wymogiem.

PHP udostępnia programistom wyspecjalizowane klasy wyjątków, umożliwiające bardziej szczegółową obsługę konkretnych przypadków. Poniższa tabela prezentuje listę klas wyjątków wbudowanych w interpreter języka PHP.

Wyjątek	Czego dotyczy?	Wersja PHP
<code>Exception</code>	podstawowa klasa obsługi wyjątków	5 – 8
<code>ErrorException</code>	rozszerzona wersja klasy <code>Exception</code>	5.1 – 8
<code>Error</code>	wewnętrzny błąd interpretera PHP	7 – 8
<code>ArgumentCountError</code>	nieoczekiwana liczba argumentów	7.1 – 8
<code>ArithmeticError</code>	błąd w obliczeniu arytmetycznym	7 – 8
<code>AssertionError</code>	błąd wykonania funkcji <code>assert</code>	7 – 8
<code>DivisionByZeroError</code>	błąd dzielenia przez zero	7 – 8
<code>CompileError</code>	błąd podczas kompilacji kodu	7.3 – 8
<code>ParseError</code>	błąd w konstrukcji składni	7 – 8
<code>TypeError</code>	nieprawidłowy typ argumentu	7 – 8
<code>ValueError</code>	nieprawidłowa wartość argumentu	8
<code>UnhandledMatchError</code>	brak dopasowania w konstrukcji warunkowej	8

Przechwycony wyjątek może zawierać dodatkowe informacje, takie jak komunikat, numer, w której linii został wywołany, a także w którym pliku. Najważniejsze metody klasy zgłaszającej wyjątek przedstawia poniższa tabela.

Metoda	Co zwraca?	Typ
<code>getMessage()</code>	komunikat wyjątku	string
<code>getCode()</code>	numer komunikatu wyjątku	mixed
<code>getFile()</code>	nazwa pliku, w którym wystąpił wyjątek	string
<code>getLine()</code>	numer linii w pliku, w którym wystąpił wyjątek	int

Listing 25.1 prezentuje technikę obsługi wyjątków w języku PHP.

LISTING 25.1	PHP 8	code_25_01.php
01	<?php	
02	try {	
03	/*	
04	<i>wykonaj dowolny kod</i>	
05	*/	
06		
07	<i>// jeśli coś pójdzie nie tak, zgłoś wyjątek</i>	
08	throw new Exception('Coś poszło nie tak :(');	
09	} catch (\Exception) {	
10	<i>// obsługa wyjątku, np. kiedy wystąpił jakiś problem</i>	
11	}	

Jak działa powyższy kod?

1. W liniach od 02 do 11 znajduje się definicja konstrukcji try-catch.
2. W obrębie sekcji try wykonywany jest kod programu do momentu zgłoszenia wyjątku przez instrukcję throw w wierszu 08. Po zgłoszeniu wyjątku wykonywanie kodu w tej sekcji zostaje przerwane.
3. Zgłoszony wyjątek zostaje przechwycony w linii 09, po czym wykonywany jest kod programu w sekcji catch. Zauważ, że przechwycony wyjątek nie został przypisany do zmiennej, co jest nowością w ostatniej wersji języka PHP.

Przy rozbudowanych konstrukcjach try-catch przypisanie wyjątku do zmiennej umożliwia uzyskanie szczegółowych informacji o przyczynach i miejscu jego wystąpienia.

Obsługa wyjątków wiąże się z programowaniem zorientowanym obiektowo, które poznasz w następnych lekcjach. Jestem przekonany, że analiza kodu z ćwiczeń w niniejszej lekcji nie sprawi Ci problemu.

Ćwiczenie 1. Przechwycenie wyjątku

W tym ćwiczeniu wykorzystasz konstrukcję try-catch-finally do obsługi wyjątków w swoich programach. Przeanalizuj wszystkie komentarze, które umieściłem w kodzie z listingu 25.2. Jestem przekonany, że z ich pomocą bez problemu zrozumiesz jego działanie. Wszystkie drukowane komunikaty są ponumerowane, co powinno pomóc Ci prześledzić i przeanalizować kod linia po linii.

LISTING 25.2	PHP 8	code_25_02.php
01	<?php	
02	function mojaFunkcja(int \$x, int \$y) : float	
03	{	
04	<i>// jeśli coś pójdzie nie tak, zgłoś wyjątek</i>	
05	if (\$y == 0) {	
06	throw new Exception ('4. Nie wolno dzielić przez zero!');	
07	}	
08		
09	print "2. Funkcja zwraca wynik dzielenia.".PHP_EOL;	
10	return (float)\$x / \$y;	
11	}	
12		
13	try {	
14	<i>// monitorowana część kodu</i>	
15	print "1. Wywołuję funkcje z parametrami.".PHP_EOL;	
16	mojaFunkcja(x: 5, y: 20);	
17	mojaFunkcja(x: 5, y: 0); <i>// spróbujemy podzielić przez zero</i>	
18	} catch (\Exception \$e) {	
19	<i>// obsługa wyjątku, np. kiedy wystąpił jakiś problem</i>	
20	print "3. Funkcja zgłosiła wyjątek i przerwała swoje działanie.".PHP_EOL;	
21	print \$e->getMessage().PHP_EOL;	
22	} finally {	
23	<i>// wykonaj kod po bloku try-catch</i>	
24	print "5. Ten kod wykona się zawsze po try-catch.".PHP_EOL;	
25	}	
\$ php code_25_02.php ↵		KONSOLA
1. Wywołuję funkcje z parametrami. 2. Funkcja zwraca wynik dzielenia. 3. Funkcja zgłosiła wyjątek i przerwała swoje działanie. 4. Nie wolno dzielić przez zero! 5. Ten kod wykona się zawsze po try-catch.		

Jak działa powyższy kod?

1. W liniach od 02 do 11 zdefiniowana jest funkcja `mojaFunkcja` zwracająca wynik dzielenia dwóch liczb przekazanych jako argumenty.
2. W wierszach od 05 do 07 zdefiniowana jest konstrukcja warunkowa `if`, która w przypadku wykrycia argumentu `$y` równego zero zgłasza wyjątek `Exception`.
3. W liniach od 13 do 25 zdefiniowana jest konstrukcja obsługująca wyjątki `try-catch-finally`. Zwróć uwagę na nową sekcję `finally`; kod zawarty w tej sekcji zostanie zawsze wykonany, bez względu na to, czy wyjątek zostanie zgłoszony, czy nie.
4. W wierszu 16 wywoływana jest funkcja `mojaFunkcja`, która zwraca wynik dzielenia dwóch liczb przekazanych w argumentach. Funkcja nie zgłasza wyjątku, ponieważ argument `$y` jest różny od zera.
5. W linii 17 wywoływana jest funkcja `mojaFunkcja`, w której argument `$y` jest równy zero. W tej sytuacji konstrukcja warunkowa `if` w wierszu 05 powoduje zgłoszenie wyjątku przez instrukcję `throw` w linii 06.
6. Zgłoszony w wierszu 06 wyjątek zostaje przechwycony przez sekcję `catch` w linii 18 i przypisany do zmiennej `$e`. Komunikat wyjątku jest drukowany w wierszu 21.
7. Po wykonaniu kodu w sekcji `try` lub `catch` zostanie wykonany kod zdefiniowany w sekcji `finally`.

Ćwiczenie 2. Własna obsługa wyjątku

Pisząc program, możesz zdecydować, że rozbudujesz klasę obsługującą wyjątki o nowe funkcjonalności. Jeśli kod z listingu 25.3 nie będzie dla Ciebie jasny, proponuję zajrzeć tutaj po zapoznaniu się z lekcją poświęconą klasom i obiektom.

Poniższy listing prezentuje metodę utworzenia własnej klasy obsługującej wyjątki poprzez odziedziczenie własności po klasie `Exception`. Kod przedstawia najprostszą implementację takiej klasy. Przepisz kod i postaraj się go zrozumieć.

LISTING 25.3	PHP 8	code_25_03.php
01	<?php	
02	declare (strict_types = 1); <i>// dbamy o prawidłowe typy danych</i>	
03		
04	define ('ERROR_CODE', 4); <i>// definicja stałej</i>	
05		

```

06 // definicja niestandardowej klasy do obsługi wyjątków
07 class MyException extends Exception {
08     public function errorMessage() {
09         return sprintf("Wystąpił błąd '%s' o numerze %d w linii %d.".PHP_EOL,
10             $this->getMessage(), $this->getCode(), $this->getLine());
11     }
12 }
13
14 // funkcja anonimowa przypisana do zmiennej
15 $kalkulator = function(string $operacja, float $x, float $y) : float
16 {
17     if ($operacja == '/' && $y == 0) {
18         // zgłoś wyjątek do wcześniej zdefiniowanej klasy MyException
19         throw new MyException('Nie dziel przez zero!', ERROR_CODE);
20     }
21
22     return match($operacja) {
23         '+' => $x + $y,
24         '-' => $x - $y,
25         '*' => $x * $y,
26         '/' => fdiv($x, $y),
27     };
28 };
29
30 try {
31     print $kalkulator(operacja: '/', x: 5, y: 2); // dzielimy przez 2
32     print PHP_EOL;
33     print $kalkulator(operacja: '/', x: 5, y: 0); // próbujemy dzielić przez 0
34 } catch (\Exception $e) {
35     print $e->errorMessage();
36 }

```

```
$ php code_25_03.php ↵
```

KONSOLA

2.5

Wystąpił błąd 'Nie dziel przez zero!' o numerze 4 w linii 19.

Jak działa powyższy kod?

1. W liniach od 07 do 12 zdefiniowana jest klasa `MyException`, która za pomocą instrukcji `extends` odziedziczyła własności po klasie `Exception`. W klasie tej zdefiniowana jest metoda `errorMessage`, która zwraca odpowiedni komunikat.
2. W wierszach od 15 do 28 zdefiniowana jest funkcja anonimowa, która jest przypisana do zmiennej `$kalkulator`. W funkcji tej zdefiniowana jest konstrukcja warunkowa `if`, która w linii 19 zgłasza wyjątek, jeśli nastąpi próba dzielenia przez zero. Zwróć uwagę, że wyjątek zgłoszony jest do nowej klasy `MyException`.

3. W liniach od 30 do 36 zdefiniowana jest konstrukcja try-catch.
4. W wierszu 31 wywoływana jest funkcja anonimowa, która zwraca wynik dzielenia dwóch liczb przekazanych w argumentach. Funkcja nie zgłasza wyjątku, ponieważ argument \$y jest różny od zera.
5. W linii 33 wywoływana jest funkcja anonimowa, w której argument \$y jest równy zero. W tej sytuacji konstrukcja warunkowa if w wierszu 18 powoduje zgłoszenie wyjątku przez instrukcję throw w linii 19.
6. Zgłoszony w wierszu 19 wyjątek trafi do nowej klasy MyException, a następnie zostanie przechwycony przez sekcję catch w linii 34 i przypisany do zmiennej \$e.
7. Wywołanie metody errorMessage, zdefiniowanej w nowej klasie MyException, następuje w wierszu 35.

Ćwiczenia dodatkowe

1. Otwórz stronę <https://www.php.net/manual/en/language.exceptions.php> i zapoznaj się z informacjami na temat wyjątków. Spróbuj zrozumieć, jak działają. Przepisz i uruchom przykładowe listingi.

Pytania

Kiedy stosować finally w konstrukcji przechwytyjącej wyjątki?

Najczęściej spotkasz się z konstrukcją try-catch. Możesz też napotkać rozbudowane konstrukcje z dodatkową sekcją finally, w której kod jest wykonywany po opuszczeniu sekcji try lub catch. W jakich więc przypadkach stosować konstrukcję ze wszystkimi sekcjami?

Listing 25.4 przedstawia prostą konstrukcję obsługi zapytania do bazy danych. W sekcji try wykonywana jest monitorowana część kodu. Jeśli pojawi się problem, przez klasę \$db powinien zostać zgłoszony wyjątek, który sekcja catch przechwyci i obsłuży. Bez względu na to, czy kod zostanie wykonany poprawnie w sekcji try, czy zostanie zgłoszony wyjątek, kod zawarty w sekcji finally zostanie wykonany, czyli w poniższym przypadku zostanie zamknięte połączenie z bazą danych.

LISTING 25.4

PHP 7/8

code_25_04.php

```
01 try {  
02     $db->open(); // otwórz połączenie z bazą danych  
03     $db->query($sql); // wyślij zapytanie do bazy danych  
04 } catch (\Exception $e) {  
05     // wydrukuj komunikat przechwyconego wyjątku  
06     print $e->getMessage();  
07 } finally {  
08     $db->close(); // zamknij połączenie z bazą danych  
09 }
```



Dołącz kod źródłowy z pliku

Nauczyłeś się już tworzyć funkcje, które możesz wielokrotnie wykorzystywać w swoich programach. Jak wiesz, funkcje mają ułatwić programowanie, gdyż często powtarzający się kod jest umieszczany w jednej konstrukcji. Pytanie, jak zastosować raz napisany kod źródłowy w innych programach bez konieczności jego przepisywania. Umiejętność ponownego użycia kodu zapewnia większą niezawodność oraz spójność projektu i, co ważne, ułatwia pracę programistom. Dobrą praktyką jest umieszczanie części kodu (np. funkcji) w osobnych plikach, które później mogą być dołączone do głównego programu, w zależności od potrzeb.

W języku PHP dostępne są specjalne funkcje: `require` i `include`, za pomocą których dołączysz kod źródłowy z innych plików do swojego programu. Funkcje te działają w zasadzie tak samo, z jedną tylko różnicą: w przypadku gdy próba dołączenia pliku się nie powiedzie, `require` zwróci błąd krytyczny (i przerwie wykonywanie programu), natomiast `include` wygeneruje jedynie ostrzeżenie. Występują jeszcze dodatkowe (uzupełniające) funkcje: `include_once` i `require_once`, które w odróżnieniu od `include` i `require` wczytują kod źródłowy z pliku tylko raz, nawet jeśli zostaną wywołane w programie wielokrotnie.

Ćwiczenie 1. Plik z funkcjami

W tym ćwiczeniu dołączysz do swojego programu kod źródłowy z innego pliku. Przygotowałem dla Ciebie prostą funkcję. Dzięki niej dowiesz się, z którego województwa pochodzi konkretny numer rejestracyjny pojazdu. Na potrzeby niniejszej lekcji zapisz kod z listingu 26.1.1 w pliku o nazwie `funkcje.php` w katalogu, gdzie znajdują się pozostałe pliki z przerabianymi lekcjami.

LISTING 26.1.1	PHP 8	funkcje.php
01	<?php	
02	declare (strict_types = 1); <i>// dbamy o prawidłowe typy danych</i>	
03		
04	function rejestracja_pojazdu(?string \$numer = '') : string bool	

```

05  {
06      // deklaracja tabeli z prefiksami i nazwami województw
07      $rej = [
08          "B" => "podlaskie",
09          "C" => "kujawsko-pomorskie",
10          "D" => "dolnośląskie",
11          "E" => "łódzkie",
12          "F" => "lubuskie",
13          "G" => "pomorskie",
14          "K" => "małopolskie",
15          "L" => "lubelskie",
16          "N" => "warmińsko-mazurskie",
17          "O" => "opolskie",
18          "P" => "wielkopolskie",
19          "R" => "podkarpackie",
20          "S" => "śląskie",
21          "T" => "świętokrzyskie",
22          "W" => "mazowieckie",
23          "Z" => "zachodniopomorskie"
24      ];
25
26      // pobranie pierwszego znaku z numeru i zamiana na wielką literę
27      $prefiks = mb_strtoupper(mb_substr($numer, 0, 1));
28
29      // zwróć nazwę województwa, jeśli prefiks istnieje w tabeli
30      return isset($rej[$prefiks]) ? (string)$rej[$prefiks] : (bool>false;
31  }

```

Jak działa powyższy kod?

1. W liniach od 07 do 24 zadeklarowałem tablicę \$rej, w której umieściłem nazwy województw z przypisanymi prefiksami (kluczami).
2. W wierszu 27 wykorzystałem od razu dwie funkcje, za pomocą których wyodrębniam pierwszy znak, zamieniam na wielką literę i przypisuję do zmiennej \$prefiks.
3. W linii 30 zwracana jest nazwa województwa, jeśli istnieje odpowiedni prefiks w tabeli \$rej. Zwróć uwagę na użytą funkcję `isset`, która zwraca `true`, jeśli dana zmienna istnieje, lub `false` w przeciwnym razie.

A teraz przepisuj poniższy kod źródłowy (listing 26.1.2) i uruchom. Zwróć uwagę na linię 02, w której dołączasz do programu dodatkowy kod źródłowy zawarty w pliku `funkcje.php`. W wierszu 05 wykorzystujesz funkcję, którą właśnie dołączyłeś! Jestem przekonany, że analiza kodu nie będzie dla Ciebie problemem.

LISTING 26.1.2	PHP 8	code_26_01.php
01	<?php	
02	require_once "funkcje.php"; <i>// dołącz dodatkowy plik z kodem</i>	
03		
04	\$numer = 'SY 1234'; <i>// przykładowy numer rejestracyjny pojazdu</i>	
05	\$woj = rejestracja_pojazdu (numer: \$numer); <i>// wywołaj funkcję z dołączonego pliku</i>	
06		
07	if (\$woj !== false) {	
08	print "Rejestracja pojazdu \$numer to województwo \$woj.";	
09	} else {	
10	print "Wpisz poprawnie numer rejestracyjny pojazdu.";	
11	}	
\$ php code_26_01.php ↵		KONSOLA
Rejestracja pojazdu SY 1234 to województwo śląskie.		

Ćwiczenie 2. Dołączanie funkcji licznika

W tym ćwiczeniu wykorzystasz już napisany kod licznika odwiedzin. Mam nadzieję, że w lekcji o zapisywaniu danych do pliku poprawnie wykonałeś ćwiczenie 4 i zapisałeś kod licznika w pliku `licznik.php`, w przeciwnym razie kod z listingu 26.2 nie będzie działał poprawnie.

LISTING 26.2	PHP 8	code_26_02.php
01	<?php	
02	require_once "licznik.php"; <i>// dołączamy plik z kodem źródłowym</i>	
03	print "Jesteś ". licznik (filename: 'licznik.txt')." gościem na mojej stronie.";	
\$ php code_26_02.php ↵		KONSOLA
Jesteś 2 gościem na mojej stronie.		

Ćwiczenie 3. Sprawdzenie dołączonego kodu

W tym ćwiczeniu nauczysz się testować dostępność konkretnych funkcji. Język PHP zapewnia setki funkcji, które z kolei zawarte są w wielu modułach. Dodatkowo funkcje sam będziesz tworzył i dołączał do swoich programów. Problem może się pojawić, kiedy program będzie uruchamiany na różnych komputerach (serwerach). Aby sprawdzić, czy konkretna funkcja jest zdefiniowana i dostępna do użycia w Twoim programie, wykorzystasz specjalną funkcję `function_exists`.

Na listingu 26.3 w linii 04 sprawdzam, czy dostępna jest funkcja `triangle`. Jeśli tak, to zostanie wykonany kod od wiersza 05 do 12, a w przeciwnym razie kod od 14 do 16 linii. Taka konstrukcja nie spowoduje wygenerowania błędu i zakończenia działania programu. Spróbuj sam napisać funkcję `triangle` i wstaw ją do pliku `funkcje.php`, a następnie uruchom program. Jeśli będziesz miał z tym kłopot, odśledź definicję funkcji `triangle` we wcześniejszych lekcjach.

LISTING 26.3	PHP 8	code_26_03.php
01	<?php	
02	<code>require_once "funkcje.php";</code>	<i>// dołącz dodatkowy plik z kodem</i>
03		
04	<code>if (function_exists("triangle"))</code>	<i>// sprawdź, czy funkcja jest zdefiniowana</i>
05	<code>{</code>	
06	<code> \$a = 21;</code>	<i>// podstawa trójkąta</i>
07	<code> \$h = 43;</code>	<i>// wysokość trójkąta</i>
08		
09	<code> \$t = triangle(a: \$a, h: \$h);</code>	<i>// wywołaj funkcję i wynik przypisz do zmiennej</i>
10		
11	<code> printf("Pole trójkąta o podstawie %d i wysokości %d wynosi %d.", \$a, \$h, \$t);</code>	
12	<code>}</code>	
13	<code>else</code>	
14	<code>{</code>	
15	<code> print "Dołącz kod funkcji 'triangle' do swojego programu.";</code>	
16	<code>}</code>	
\$ php code_26_03.php ↵		KONSOLA
Pole trójkąta o podstawie 21 i wysokości 43 wynosi 451.		

Ćwiczenia dodatkowe

1. Napisz program dołączający jednocześnie pliki `funkcje.php` i `licznik.php`, a następnie wykorzystaj zawarte w nich funkcje.
2. Napisz funkcję pełniącą rolę kalkulatora i zapisz ją do pliku `funkcje-mat.php`, a następnie napisz program, który dołączy ten plik i wykorzysta zawartą w nim funkcję.
3. Zastanów się, jak byś rozszerzył funkcję `rejestracja_pojazdu` z ćwiczenia 1, aby zwracała również nazwę powiatu.

Wskazówki

Pisząc bardziej złożone programy, staraj się podzielić kod na tematyczne części, np. osobny plik z funkcjami do analizy tekstu, osobny plik z funkcjami obsługującymi grafikę itd. Z biegiem czasu stworzysz własne rozbudowane zbiory funkcji, które w zależności od potrzeb będziesz dołączał do swoich kolejnych programów.

Jak już wspomniałem, funkcje `require` i `include` działają podobnie. W swoich projektach przeważnie wykorzystuję funkcję `require_once`, ponieważ dołącza plik tylko raz. Jednak to od Ciebie zależy, czy powodzenie próby dołączenia konkretnego pliku jest niezbędne do prawidłowego działania programu.

Pytania

Czy mogę wstawić wszystkie funkcje do jednego pliku z programem?

Tak. Teoretycznie możesz mieć jeden plik z setką funkcji, jednak w praktyce programiści grupują kod w osobnych plikach, aby móc w komfortowy sposób nim zarządzać. To tak, jakbyś zamiast jednego, konkretnego elementu musiał za każdym razem wysyłać do serwisu cały komputer lub samochód.

Jak napisać funkcję `triangle` i wstawić ją do pliku `funkcje.php`?

Zajrzyj do ćwiczenia 2 w lekcji 18 — znajdziesz tam kod funkcji `triangle`. Skopiuj kod od linii 04 do 08 i wstaw do pliku `funkcje.php`, bezpośrednio po definicji funkcji `licznik`, a następnie zapisz plik.

Data i czas

Pojęcie czasu i daty w programowaniu jest niezwykle istotne. Trudno wyobrazić sobie program pełniący funkcję budzika czy prostego kalendarza bez odniesienia właśnie do czasu. Podstawową reprezentacją czasu w języku PHP jest czas uniksowy, mierzący liczbę sekund, począwszy od roku 1970, czyli od momentu, który został uznany za narodziny systemu Unix. Oczywiście nie musisz tego wiedzieć, ale kilka ciekawostek na ten temat umieściłem we wskazówkach.

W tej lekcji poznasz podstawowe funkcje związane z datą i czasem, tj. `date`, `time`, `mktime`, `strtotime`, `checkdate`. Poznasz również funkcję `rand` do generowania liczb losowych, które mają duże zastosowanie w programowaniu.

W języku PHP podstawową funkcją do formatowania czasu i daty jest `date`. W poniższej tabeli umieściłem najważniejsze znaczniki, które wykorzystasz razem z tą funkcją do formatowania czasu.

Znacznik czasu	Co funkcja <code>date</code> zwróci na wyjściu?	Przykład
G	godzina w formacie 24-godzinnym (bez zera)	od 0 do 23
H	godzina w formacie 24-godzinnym (z zerem)	od 00 do 23
i	minuty	od 00 do 59
s	sekundy	od 00 do 59
d	dzień miesiąca (z zerem)	od 01 do 31
D	nazwa dnia tygodnia (3 litery)	od Mon do Sun
l (małe L)	nazwa dnia tygodnia	od Sunday do Saturday
j	dzień miesiąca (bez zera)	od 1 do 31
w	dzień tygodnia (0 — niedziela ... 6 — sobota)	od 0 do 6
z	dzień roku	od 1 do 365
F	nazwa miesiąca	od January do December

Znacznik czasu	Co funkcja date zwróci na wyjściu?	Przykład
m	miesiąc roku (z zerem)	od 01 do 12
N	miesiąc roku (bez zera)	od 1 do 12
t	liczba dni w miesiącu	od 28 do 31
Y	rok (4 cyfry)	2021
y	rok (2 cyfry)	21

W praktyce funkcję `date` można wykorzystać do formatowania daty i czasu w sposób przedstawiony na listingu 27.1:

LISTING 27.1	PHP 7/8	code_27_01.php
01	<?php	
02	<code>date_default_timezone_set("Europe/Warsaw");</code>	<i>// definicja strefy czasowej</i>
03	<code>print "Aktualna data i czas: ".date("Y.m.d H:i:s");</code>	<i>// drukowanie daty i czasu</i>
\$	<code>php code_27_01.php ↵</code>	KONSOLA
Aktualna data i czas: 2021.03.19 09:24:11		

Zwróć uwagę na funkcję `date_default_timezone_set`, która definiuje strefę czasową dla wszystkich funkcji związanych z datą i czasem (staraj się zawsze ją definiować w swoich programach). Pamiętaj też, że standardowo nazwy dni i miesięcy zwracane są w języku angielskim.

Ćwiczenie 1. Upływający czas

W tym ćwiczeniu poznasz funkcje `time` i `mktime`, operujące czasem uniksowym do obliczenia czasu (wyrażonego w sekundach), który upłynął od początku 2000 roku. Funkcje te zwracają znacznik jako liczbę typu `integer`, co bardzo ułatwia jej przetwarzanie i zapisywanie w bazach danych.

Kod z listingu 27.2 jest bardzo prosty i nie powinien sprawić Ci większego problemu. W linii 05 za pomocą funkcji `time` do zmiennej `$unix_time` przypisywany jest aktualny znacznik czasu uniksowego, wskazujący, ile sekund upłynęło od początku 1970 roku do teraz. W wierszu 08 wykorzystałem zaś funkcję `mktime` do obliczenia i przypisania do zmiennej `$unix_time_2000` znacznika czasu dla konkretnej daty.

Postaraj się dobrze zrozumieć, jak ta funkcja działa, i spróbuj przećwiczyć program z innymi datami (parametry funkcji `mktime` znajdziesz w linii 10).

LISTING 27.2	PHP 7/8	code_27_02.php
01	<?php	
02	<code>date_default_timezone_set("Europe/Warsaw");</code>	<i>// definicja strefy czasowej</i>
03		
04	<i>// ile sekund minęło od początku 1970 roku do dzisiaj?</i>	
05	<code>\$unix_time = time();</code>	
06		
07	<i>// a ile sekund od początku 2000 roku?</i>	
08	<code>\$unix_time_2000 = mktime(0, 0, 0, 1, 1, 2000);</code>	
09		
10	<i>// mktime(\$godzina, \$minuta, \$sekunda, \$miesiac, \$dzien, \$rok)</i>	
11		
12	<code>print "Od początku 2000 roku upłynęło " . (\$unix_time - \$unix_time_2000) . " sekund.";</code>	
\$ php code_27_02.php ↵		KONSOLA
Od początku 2000 roku upłynęło 669461539 sekund.		

Ćwiczenie 2. Weryfikacja daty

W tym ćwiczeniu (listing 27.3) wydrukujesz nazwę dnia tygodnia w języku polskim dla dowolnej daty (może to być data Twoich urodzin). Wykorzystasz również funkcję `checkdate` do sprawdzenia poprawności daty.

LISTING 27.3	PHP 7/8	code_27_03.php
01	<?php	
02	<code>\$week = [</code>	
03	<code>'niedziela', 'poniedziałek', 'wtorek', 'środa',</code>	
04	<code>'czwartek', 'piątek', 'sobota'</code>	
05	<code>];</code>	
06		
07	<code>\$month = 12;</code>	
08	<code>\$day = 24;</code>	
09	<code>\$year = 2021;</code>	
10		
11	<i>// sprawdź poprawność daty</i>	
12	<code>if (checkdate(\$month, \$day, \$year))</code>	<i>// true</i>
13	<code>{</code>	
14	<code> \$d = date("w", mktime(0, 0, 0, \$month, \$day, \$year));</code>	
15	<code> print "\$year.\$month.\$day to ".\$week[\$d];</code>	
16	<code>}</code>	
17	<code>else</code>	<i>// false</i>

```
18 {
19     print "Data jest błędna.";
20 }
```

```
$ php code_27_03.php ↵
```

KONSOLA

```
2021.12.24 to piątek
```

Jak działa powyższy kod?

1. W liniach od 02 do 05 zadeklarowana jest tablica `$week` z nazwami dni tygodnia, która wykorzystywana jest w wierszu 15 do wydrukowania wyniku.
2. W liniach od 07 do 09 zdefiniowane są zmienne reprezentujące datę (możesz ją dowolnie zmienić).
3. W wierszu 12 sprawdzana jest poprawność daty przy użyciu funkcji `checkdate`.
4. W linii 14 funkcja `date` zwraca numer dnia tygodnia (od 0 do 6), który następnie jest wykorzystywany jako klucz do odczytania wartości z tablicy `$week`. Zwróć uwagę na użycie funkcji `date`, gdzie pierwszy parametr oznacza format („w” to numer dnia w tygodniu), a drugi parametr oznacza moment (znacznik czasu), dla którego funkcja ma zwrócić wynik.

Ćwiczenie 3. Początek weekendu

W tym ćwiczeniu wykorzystasz funkcję `strtotime` do wyznaczenia znacznika czasu. Parametrem funkcji jest tekst w języku angielskim, za pomocą którego w prosty sposób określasz, dla jakiej daty ma zostać zwrócony znacznik czasu: `now`, `yesterday`, `+1 week` itd.

Poniższy program oblicza czas do najbliższego weekendu, czyli do piątku do godziny 16. Zwróć uwagę na parametry funkcji `strtotime` w liniach 04 i 05 — mam nadzieję, że bez problemu je zrozumiesz. W wierszu 07 obliczam całkowitą liczbę sekund i przypisuję do zmiennej `$count`, a w liniach od 10 do 13 dzielę `$count` na liczbę dni, godzin, minut i sekund.

Przeanalizuj kod z listingu 27.4 wiersz po wierszu i postaraj się go zrozumieć.

LISTING 27.4

PHP 7/8

code_27_04.php

```
01 <?php
02 date_default_timezone_set("Europe/Warsaw"); // definicja strefy czasowej
03
```

```

04 $start = strtotime("now"); // bieżący znacznik czasu
05 $end = strtotime("next Friday 4pm"); // najbliższy piątek, godzina 16:00
06
07 $count = $end - $start; // ile sekund do weekendu?
08
09 // funkcja floor zaokrągla w dół, np. floor(4.8) = 4
10 $days = floor($count / (60 * 60 * 24)); // 86 400 sekund to 1 dzień
11 $hours = floor(($count / (60 * 60)) % 24); // 3600 sekund to 1 godzina
12 $minutes = floor(($count / 60) % 60); // 60 sekund to 1 minuta
13 $seconds = $count % 60;
14
15 printf(
16     "Weekend zaczyna się za %d dni, %d godzin, %d minut, %d sekund.",
17     $days, $hours, $minutes, $seconds
18 );

```

```
$ php code_27_04.php ↵
```

KONSOLA

```
Weekend zaczyna się za 2 dni, 6 godzin, 5 minut, 46 sekund.
```

Ćwiczenie 4. Losowanie daty

Teraz zapraszam Cię do małej zabawy. Program z listingu 27.5 generuje losowe daty (100 lat naprzód), kiedy prawdopodobnie odwiedzą nas przybysze z dalekich światów. Oczywiście, potraktuj to jako rozrywkę na koniec niniejszej lekcji. Pamiętaj, że każde uruchomienie wydrukiuje zupełnie inny zestaw dat. Jak zatem działa program?

LISTING 27.5

PHP 7/8

code_27_05.php

```

01 <?php
02 print "Kiedy odwiedzą nas kosmici?".PHP_EOL;
03
04 $table = []; // tablica na wylosowane daty
05
06 $i = 0;
07 do {
08     // wylosuj liczbę z danego zakresu
09     $day = rand(1, 31);
10     $month = rand(1, 12);
11     $year = rand(2021, 2121);
12     $hour = rand(0, 23);
13     $min = rand(0, 59);
14     $sec = rand(0, 59);
15
16     // sprawdź, czy data jest poprawna
17     if (checkdate($month, $day, $year)) {

```

```

18      // oblicz znacznik czasu
19      $table[] = strtotime("$year-$month-$day $hour:$min:$sec");
20      $i++;
21  }
22  } while ($i < 10);
23
24  sort($table); //sortuj rosnąco
25
26  foreach ($table as $t) {
27      print date("Y.m.d H:i:s (D)", $t).PHP_EOL;
28  }

```

```
$ php code_27_05.php ↵
```

KONSOLA

```
Kiedy odwiedzą nas kosmici?
```

```

2023.05.23 14:32:43 (Tue)
2055.07.16 11:23:00 (Fri)
2056.01.19 20:05:54 (Wed)
2059.04.19 02:30:29 (Sat)
2061.09.03 09:17:39 (Sat)
2075.04.25 02:39:11 (Thu)
2077.12.02 10:40:17 (Thu)
2089.09.08 02:11:20 (Thu)
2102.03.09 16:33:43 (Thu)
2110.07.12 02:06:24 (Sat)

```

Jak działa powyższy kod?

1. Zadeklarowałem tablicę `$table`, w której będą przechowywane znaczniki czasu (linia 04). Następnie wykonywana jest pętla, w której losowane są daty (wiersz od 07 do 22).
2. Do wylosowania składowych dat (rok, miesiąc, dzień, godzina, minuta i sekunda) wykorzystałem funkcję `rand` (linie od 09 do 14). Funkcja przyjmuje dwa parametry: początek i koniec zbioru, z którego ma być wylosowana liczba.
3. Prawidłowość wylosowanych składowych daty sprawdzamy funkcją `checkdate` (wiersz 17), aby uniknąć sytuacji, w której np. wylosowany został miesiąc 2 (luty) i dzień 31.
4. Wszystkie wylosowane daty zamieniane są na znacznik czasu uniksowego i wstawiane do tablicy `$table` (linia 19).
5. Po sortowaniu (wiersz 24) wszystkie wylosowane daty są drukowane (linie od 26 do 28).

Ćwiczenia dodatkowe

1. Otwórz stronę <https://www.php.net/manual/en/ref.datetime.php> i zapoznaj się z pozostałymi funkcjami związanymi z obsługą daty i czasu. Spróbuj zrozumieć, jak działają. Przepisz i uruchom przykładowe listingi.
2. Napisz program, który wydrukuje dla bieżącej daty (znacznika czasu) nazwę miesiąca w języku polskim. Wykorzystaj pomysł z tabelą z ćwiczenia 2.
3. Osiem lat to ile sekund? Jak szybko to obliczyć?
4. Napisz program drukujący 6 losowych liczb z zakresu od 1 do 49. Wylosowane liczby nie mogą się powtarzać.

Wskazówki

Począwszy od roku 1970, czas uniksowy był zapisywany w formacie 32-bitowym, który może reprezentować maksymalnie 2 147 483 647 sekund, czyli 68 lat. Czy to dużo? Niestety nie — i jest to bardzo poważny problem, ponieważ nie wiadomo, co się dokładnie stanie z 32-bitowymi systemami informatycznymi 19 stycznia 2038 roku. W tym dniu „przekręci się licznik” 32-bitowego czasu i trudno przewidzieć tego konsekwencje, np. w bankowości, medycynie czy komunikacji. Problemu można uniknąć przez wykorzystanie zapisu daty w formie 64-bitowej — wówczas liczba możliwości zwiększa nam się do 9 223 372 036 854 775 807 sekund, czyli do 292 miliardów lat. Dzisiaj większość nowych systemów informatycznych jest już 64-bitowa, dlatego nie musisz się tym problemem przejmować. Dobra wiadomość dla programisty jest taka, że w języku PHP data zapisywana jest w formacie 64-bitowym, oczywiście jeśli system również jest 64-bitowy.

Uruchom kod z listingu 27.6 i dowiedz się, czy możesz kodować 64-bitowo w języku PHP.

LISTING 27.6

PHP 7/8

code_27_06.php

```
01 <?php
02 if (PHP_INT_SIZE === 8) {
03     print "64-bity to moja specjalność!";
04 }
```


Pytania

Co oznacza skrót Y2K38?

Skrót ten oznacza „problem roku 2038”, podobnie zresztą jak skrót Y2K („problem roku 2000”). Wyszukaj w internecie więcej informacji na ten temat.

Jak napisać program losujący liczby od 1 do 49?

Poniżej znajduje się kod programu pełniący funkcję generatora liczb do popularnej gry Lotto. Jeśli przerobiłeś ćwiczenie 4, analiza kodu z listingu 27.7 nie powinna być dla Ciebie trudna.

LISTING 27.7	PHP 7/8	code_27_07.php
01	<?php	
02	\$liczby = []; <i>// tablica na wylosowane liczby</i>	
03		
04	\$i = 0;	
05	do {	
06	\$liczba = rand(1, 49); <i>// losuj liczbę</i>	
07		
08	<i>// sprawdź, czy liczba została już wcześniej wylosowana</i>	
09	if (!in_array(\$liczba, \$liczby)) {	
10	\$liczby[] = \$liczba;	
11	\$i++;	
12	}	
13	} while (\$i < 6);	
14		
15	sort(\$liczby); <i>// sortuj rosnąco</i>	
16		
17	foreach (\$liczby as \$liczba) {	
18	print \$liczba." ";	
19	}	
\$ php code_27_07.php ↵		KONSOLA
8 9 13 24 34 42		



Zrób coś z klasą

Podczas ćwiczeń przerabianych we wcześniejszych lekcjach pisałeś kod składający się m.in. z różnych funkcji, czyli z niezależnych fragmentów programu wykonujących ściśle określone zadania. Takie programowanie nazywa się programowaniem proceduralnym. W tej lekcji nauczę Cię czegoś nowego, a mianowicie programowania zorientowanego obiektowo (ang. *object-oriented programming*, OOP). Takie programowanie polega na grupowaniu wszystkich funkcji i danych wewnątrz jednego elementu, który nazywany jest obiektem. Przeważnie obiekty reprezentują coś konkretnego, np. konkretną osobę, markę samochodu czy produkt w koszyku zakupów. Zdefiniowana struktura opisująca taki obiekt nazywa się zaś klasą. Niniejsza książka przeznaczona jest dla początkujących programistów, dlatego omówione zostaną jedynie podstawy programowania obiektowego.

Klasę definiuje się za pomocą słowa kluczowego `class`. Umieszcza się w niej jej właściwości i metody. Dostęp do właściwości i metod wewnątrz klasy uzyskuje się za pomocą specjalnej zmiennej o nazwie `$this` i symbolu strzałki (`->`) — postaraj się to zapamiętać. Tworzenie instancji, czyli obiektu, na podstawie zdefiniowanej klasy odbywa się z użyciem słowa kluczowego `new`. We wskazówkach do niniejszej lekcji umieściłem więcej informacji na temat definiowania dostępu do właściwości klasy.

Przeanalizuj kod z listingu 28.1 linia po linii i postaraj się go zrozumieć.

LISTING 28.1	PHP 8	code_28_01.php
01	<?php	
02	class Person // definicja klasy	
03	{	
04	private \$name; // właściwość klasy	
05		
06	public function setName(string \$name) // metoda klasy	
07	{	
08	\$this-> name = \$name; // przypisanie wartości do właściwości klasy	
09	}	
10		

```

11     public function getName() : string //metoda klasy
12     {
13         return $this->name; //pobranie wartości właściwości klasy
14     }
15 }
16 $contact = new Person(); //tworzenie egzemplarza klasy
17 $contact->setName(name: "Robert"); //wywołanie metody klasy
18 print $contact->getName(); //wydrukowanie wartości właściwości klasy

```

```
$ php code_28_01.php ↵
```

KONSOLA

Robert

Na podstawie powyższej klasy można zbudować bardziej złożoną konstrukcję, reprezentującą konkretną grupę ludzi, np. klasę szkolną. W takiej grupie zarówno Ty, jak i Twoje koleżanki oraz koledzy będziecie obiektami klasy Person (człowiek). Możecie wykonywać różne czynności (metody), takie jak chodzenie, jedzenie i spanie. Każdy obiekt może mieć indywidualne właściwości (niezależnie od instancji), takie jak imię, nazwisko, wiek czy data urodzenia.

W języku PHP 8 pojawiła się możliwość automatycznego definiowania właściwości przez konstruktor. Dotychczas należało przekazać argumenty do konstruktora, a następnie przypisać wartości do wcześniej zdefiniowanych właściwości klasy.

Poniższa tabela prezentuje dwie metody przypisywania wartości do właściwości klasy. Jak można zauważyć, w PHP 8 jest to znacznie uproszczone.

PHP 7	PHP 8
<pre> class Produkt //definicja klasy { private \$nazwa; private \$brutto; //konstruktor public function __construct(string \$nazwa, float \$brutto) { \$this->nazwa = \$nazwa; \$this->brutto = \$brutto; } } </pre>	<pre> class Produkt //definicja klasy { //konstruktor public function __construct(private string \$nazwa, private float \$brutto) {} } </pre>

Programowanie zorientowane obiektowo jest bardzo złożonym procesem, z którym wiążą się takie pojęcia jak hermetyzacja (enkapsulacja), dziedziczenie, polimorfizm i kompozycja. Wiem, trudne wyrazy, jednak warto je poznać i postarać się je z czasem

zrozumieć. Nie martw się, podstawy poznasz podczas lektury niniejszej książki. Jeśli chciałbyś poszerzyć swoją wiedzę na ten temat, możesz skorzystać z bogatej literatury, dostępnej również w języku polskim.

Ćwiczenie 1. Tworzenie obiektów

Na początek (listing 28.2) napiszesz prostą klasę `Auto` i utworzysz na jej podstawie dwa obiekty (instancje). W związku z tym, że kolejne instancje działają niezależnie od siebie, zmiana właściwości pierwszego obiektu nie powinna wpłynąć na właściwość obiektu drugiego. Sprawdź, czy faktycznie tak jest.

LISTING 28.2	PHP 8	code_28_02.php
01	<?php	
02	class <code>Auto</code> <i>// definicja klasy</i>	
03	{	
04	private <code>\$marka</code> ; <i>// właściwość klasy</i>	
05		
06	public function <code>setMarka(string \$marka)</code> <i>// metoda klasy</i>	
07	{	
08	<code>\$this->marka = \$marka</code> ; <i>// przypisanie wartości do właściwości klasy</i>	
09	}	
10		
11	public function <code>getMarka()</code> : <code>string</code> <i>// metoda klasy</i>	
12	{	
13	return <code>\$this->marka</code> ; <i>// pobranie wartości właściwości klasy</i>	
14	}	
15	}	
16		
17	<code>\$fiat = new Auto()</code> ; <i>// tworzenie pierwszego egzemplarza klasy</i>	
18	<code>\$fiat->setMarka(marka: "Fiat 125p")</code> ; <i>// i wywołanie metody</i>	
19		
20	<code>\$jeep = new Auto()</code> ; <i>// tworzenie drugiego egzemplarza klasy</i>	
21	<code>\$jeep->setMarka(marka: "Jeep Grand Cherokee")</code> ; <i>// i wywołanie metody</i>	
22		
23	<code>print "Samochody występujące jako obiekty:".PHP_EOL</code> ;	
24	<code>print \$fiat->getMarka()</code> ;	
25	<code>print PHP_EOL</code> ;	
26	<code>print \$jeep->getMarka()</code> ;	
\$ php code_28_02.php ↵		KONSOLA
Samochody występujące jako obiekty:		
Fiat 125p		
Jeep Grand Cherokee		

Jak działa powyższy kod?

1. W liniach od 02 do 15 zdefiniowałem prostą klasę `Auto`, w której znajduje się jedna właściwość `$marka` (wiersz 04) i dwie metody `setMarka` i `getMarka` (wiersze 06 i 11). Pierwsza na podstawie parametru ustawia wartość właściwości `$marka`, druga zwraca jej wartość.
2. W linii 17 utworzyłem pierwszy egzemplarz klasy `Auto` o nazwie `$fiat`.
3. W wierszu 18 metodą `setMarka` przypisałem nową wartość do właściwości `$marka` wewnątrz egzemplarza `$fiat`.
4. Następnie podobne czynności wykonałem dla następnej instancji klasy `Auto` o nazwie `$jeep` (w liniach 20 i 21). W tym momencie oba egzemplarze, `$fiat` i `$jeep`, są od siebie niezależne.
5. Aby się o tym przekonać, w wierszach od 23 do 26 drukowane są wartości właściwości utworzonych obiektów.

Ćwiczenie 2. Konstruktor klasy

W tym ćwiczeniu poznasz specjalną metodę `__construct`, która nazywana jest konstruktorem klasy i wywoływana jest automatycznie w momencie utworzenia egzemplarza obiektu. Metoda ta jest jedną z tak zwanych metod magicznych, które są wywoływane niejawnie. Konstruktor, podobnie jak funkcja, może być wywołany z parametrami. Jak zatem działa kod z listingu 28.3?

LISTING 28.3	PHP 8	code_28_03.php
01	<?php	
02	class Kwadrat // definicja klasy	
03	{	
04	private \$pole;	
05	private \$obwod;	
06		
07	public function __construct(int float \$bok) // konstruktor	
08	{	
09	\$this->pole = \$bok * \$bok;	
10	\$this->obwod = 4 * \$bok;	
11	}	
12		
13	public function pole() : int float // metoda zwracająca pole kwadratu	
14	{	
15	return \$this->pole;	
16	}	

```

17
18     public function obwod(): int|float //metoda zwracająca obwód kwadratu
19     {
20         return $this->obwod;
21     }
22 }
23
24 $bok = 12.5; //długość boku kwadratu
25
26 $kwadrat = new Kwadrat(bok: $bok); //tworzenie egzemplarza klasy
27
28 $pole = $kwadrat->pole(); //wywołanie metody zwracającej pole
29 $obwod = $kwadrat->obwod(); //i obwód kwadratu
30
31 printf("Pole kwadratu o boku %.2f wynosi %.2f.".PHP_EOL, $bok, $pole);
32 printf("Obwód kwadratu o boku %.2f wynosi %.2f.".PHP_EOL, $bok, $obwod);

```

```
$ php code_28_03.php ↵
```

KONSOLA

```

Pole kwadratu o boku 12.50 wynosi 156.25.
Obwód kwadratu o boku 12.50 wynosi 50.00.

```

Jak działa powyższy kod?

1. W liniach od 02 do 22 zdefiniowałem klasę Kwadrat.
2. Wewnątrz klasy, w wierszach 04 i 05, zadeklarowałem dwie właściwości: \$pole i \$obwod.
3. W liniach od 07 do 11 zadeklarowałem metodę magiczną __construct z jednym parametrem — \$bok.
4. Konstruktor w wierszach 09 i 10 na podstawie parametru oblicza pole i obwód kwadratu oraz przypisuje odpowiednie wyniki do zadeklarowanych właściwości klasy.
5. Definicja metod pole i obwod (w liniach 13 i 18) ma na celu zwrócenie wartości konkretnej właściwości klasy. Zwróć uwagę, że dostęp do właściwości uzyskuje się za pomocą specjalnej zmiennej \$this i symbolu strzałki (->).
6. Po definicji klasy (wiersze od 02 do 22) przypisałem w linii 24 do zmiennej \$bok przykładową wartość.
7. W wierszu 26 do zmiennej \$kwadrat przypisałem (instancję) egzemplarz klasy Kwadrat z parametrem \$bok.

8. W momencie utworzenia obiektu (instancji klasy) została wykonana magiczna metoda `__construct`, co skutkuje przypisaniem do właściwości klas `$pole` i `$obwod` konkretnych wartości, które w liniach 28 i 29 zostają odczytane oraz przypisane do zmiennych. Choć nazwy właściwości klasy i zmiennych poza nią są zbieżne, nie mają ze sobą nic wspólnego.

Ćwiczenie 3. Dostęp do właściwości i metod klasy

W tym ćwiczeniu zapoznam Cię z pojęciem enkapsulacji danych, czyli powiem o publicznym i prywatnym dostępie do właściwości i metod. Przeanalizuj dokładnie kod z listingu 28.4, najlepiej linia po linii, i postaraj się go zrozumieć.

LISTING 28.4	PHP 8	code_28_04.php
01	<?php	
02	class Produkt <i>// definicja klasy</i>	
03	{	
04	private \$netto; <i>// właściwość dostępna lokalnie</i>	
05	public \$vat = 23; <i>// właściwość dostępna publicznie</i>	
06		
07	<i>// konstruktor</i>	
08	public function __construct (private string \$nazwa, private int float \$brutto)	
09	{	
10	/*	
11	<i>PHP 8 automatycznie utworzy i zdefiniuje właściwości:</i>	
12	\$this->nazwa = \$nazwa;	
13	\$this->brutto = \$brutto;	
14	*/	
15	\$this->netto = \$this->setNetto(); <i>// wywołaj metodę lokalną</i>	
16	}	
17		
18	public function drukuj() <i>// metoda dostępna publicznie</i>	
19	{	
20	printf(	
21	"Produkt: %s".PHP_EOL."Cena netto: %.2f PLN".PHP_EOL.	
22	"Podatek: %.2f%".PHP_EOL."Cena brutto: %.2f PLN",	
23	\$this->nazwa,	
24	\$this->netto,	
25	\$this->vat,	
26	\$this->brutto	
27	);	
28	}	
29		
30	private function setNetto() : int float <i>// metoda dostępna lokalnie</i>	
31	{	

```

32         return $this->brutto - ($this->brutto / (1 + $this->vat));
33     }
34 }
35
36 // tworzenie egzemplarza klasy
37 $owoc = new Produkt(nazwa: "Gruszka", brutto: 4.85);
38 print $owoc->drukuj(); // wywołanie metody publicznej
$ php code_28_04.php ↵

```

KONSOLA

```

Produkt: Gruszka
Cena netto: 4.65 PLN
Podatek: 23.00%
Cena brutto: 4.85 PLN

```

Jak działa powyższy kod?

1. W liniach od 02 do 34 zdefiniowałem klasę Produkt.
2. Wewnątrz klasy zadeklarowałem właściwość prywatną \$netto (linia 04), która dostępna jest tylko wewnątrz klasy.
3. W wierszu 05 zadeklarowałem właściwość \$vat, która dostępna jest publicznie. Właściwość można odczytać, np. poprzez konstrukcję `print $owoc->vat`, lub zmienić jej wartość, np. `$owoc->vat = 7`, jednak byłoby to złamaniem zasad przyjętej konwencji. Znacznie lepszym wyjściem jest napisanie i wykorzystanie dodatkowych metod do zmiany tej właściwości, np. `setVAT` lub `getVAT`.
4. Podobnie jak w poprzednim ćwiczeniu, klasa zawiera konstruktor, tym razem z dwoma parametrami. W wersji PHP 8 konstruktor na podstawie argumentów automatycznie utworzy właściwości klasy. Zwróć uwagę, że w linii 15 obliczana jest wartość właściwości \$netto za pomocą lokalnej metody `setNetto`, zdefiniowanej w wierszach od 30 do 33.
5. W linii 37 tworzymy obiekt \$owoc o wskazanych parametrach, a w wierszu 38 wywoływana jest publiczna metoda `drukuj`.

Ćwiczenia dodatkowe

1. Wyszukaj w internecie więcej informacji na temat programowania obiektowego (OOP).
2. Otwórz stronę <https://www.php.net/manual/en/language.oop5.php> i zapoznaj się z dodatkowymi informacjami na temat klas i obiektów w języku PHP. Przepisz i uruchom przykładowe listingi. Postaraj się je zrozumieć.

3. Napisz klasę reprezentującą zwierzę o co najmniej dwóch właściwościach i metodach.
4. W ćwiczeniu 3 zmień w linii 18 charakter metody drukuj z publicznej na prywatną, a następnie uruchom program. Wyjaśnij, dlaczego program wyświetlił błąd.

Wskazówki

Język PHP zapewnia bardzo dobre możliwości programowania obiektowego, o czym świadczy to, że niemal każdy nowy projekt oparty na tym języku jest obiektowy. Bardzo zachęcam Cię do eksperymentowania i przyswojenia podstaw obiektowości, które z czasem będziesz rozwijał.

Tworząc definicje klas, zawsze kieruj się zasadą, aby publicznie udostępniać tylko te właściwości i metody, które są niezbędne do prawidłowego wykorzystania danej klasy, szczególnie kiedy będziesz je udostępniał innym programistom. Inaczej mówiąc, jeśli tworzysz klasę obliczającą np. średni poziom opadów na danym terenie, wystarczy, że udostępnisz publicznie jedną metodę, zwracającą wynik dla konkretnych parametrów wejściowych.

Pytania

Zmienna czy właściwość? Metoda czy funkcja?

Dla uproszczenia możesz traktować właściwość klasy tak jak zmienną, a metodę jak funkcję. Pamiętaj o tym, że właściwości i metody mogą mieć różny charakter (np. prywatny lub publiczny).

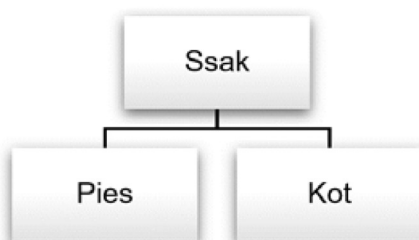


Jest czy ma? Oto jest pytanie

W programowaniu zorientowanym obiektowo (OOP) między obiektami może występować relacja typu „jest” i związek typu „ma”. Ma to ścisły związek z najważniejszymi cechami obiektowości, czyli możliwością wielokrotnego wykorzystania kodu (dziedziczenie) oraz możliwością budowania obiektów z pewnej liczby innych obiektów (kompozycja).

Związek typu „jest”

Projektując system, możesz zdefiniować powiązania między klasami, które umożliwiają wielokrotne wykorzystanie kodu i ułatwiają pogrupowanie klas według wspólnych cech. Przykład takiego związku przedstawia rysunek 29.1.



Rysunek 29.1. Związek typu „jest”

Pies z pewnością jest ssakiem, tak samo jak kot, mają zatem wiele cech wspólnych dla ssaków. Jeśli przykładowo zdefiniujemy klasę `Ssak` i nadamy jej właściwość związaną z kolorem oczu, to klasy `Pies` i `Kot` mogą ją odziedziczyć. Inaczej mówiąc, klasy `Pies` i `Kot` są podklasami klasy `Ssak`, która jest klasą nadrzędną, i przejmują wcześniej zdefiniowane właściwości i metody. Nazywa się to dziedziczeniem. W języku PHP możesz zdefiniować nową klasę na podstawie innej przy wykorzystaniu słowa kluczowego `extends`.

Dziedziczenie znacznie ułatwia pracę programiście, ponieważ nie musi on za każdym razem pisać kodu dla każdej klasy z osobna, natomiast może umiejętnie wykorzystać

już wcześniej zdefiniowane elementy klasy nadrzędnej, dzięki czemu tworzony kod jest o wiele przejrzystszy. Tak więc przykładowo w klasie `Ssak` można zdefiniować właściwość `$kolorOczu`, która zostanie przekazana klasom potomnym (podklasom), dzięki czemu nie trzeba jej już w nich definiować.

Ćwiczenie 1. Związek typu „jest”

Teraz przećwiczysz wariant z ssakami, czyli utworzysz klasę nadrzędną `Ssak` i klasę potomną (podklasę) `Pies`. Ćwiczenie to jest bardzo proste, postaraj się jednak dokładnie przepisać kod z listingu 29.1, a następnie go przeanalizować i zrozumieć.

LISTING 29.1

PHP 8

code_29_01.php

```

01  <?php
02  class Ssak // definicja klasy
03  {
04      private $kolorOczu; // wspólna właściwość dla ssaków
05
06      public function setKolorOczu(string $kolor)
07      {
08          $this->kolorOczu = $kolor;
09      }
10
11      public function getKolorOczu() : string
12      {
13          return $this->kolorOczu;
14      }
15  }
16
17  class Pies extends Ssak // definicja klasy z dziedziczeniem
18  {
19      private $rasa;
20
21      public function setRasa(string $rasa)
22      {
23          $this->rasa = $rasa;
24      }
25
26      public function getRasa() : string
27      {
28          return $this->rasa;
29      }
30  }
31
32  $pies = new Pies();

```

```

33 $pies->setKolorOczu(kolor: "brązowe"); // metoda odziedziczona po klasie Ssak
34 $pies->setRasa(rasa: "szpic pomorski"); // metoda zdefiniowana w klasie Pies
35
36 print "Mój pies:".PHP_EOL;
37 print "rasa: ".$pies->getRasa().PHP_EOL;
38 print "kolor oczu: ".$pies->getKolorOczu();
$ php code_29_01.php ↵

```

KONSOLA

```

Mój pies:
rasa: szpic pomorski
kolor oczu: brązowe

```

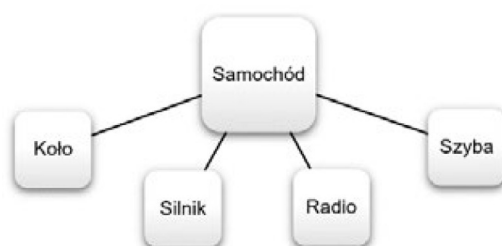
Jak działa powyższy kod?

1. W wierszach od 02 do 15 zdefiniowałem prostą klasę `Ssak` z jedną właściwością `$kolorOczu` w linii 04 oraz dwiema metodami `setKolorOczu` (wiersz 06) i `getKolorOczu` (wiersz 11).
2. W linii 17 zdefiniowałem klasę `Pies` i rozszerzyłem ją (`extends`) o klasę `Ssak`. W klasie `Pies` zdefiniowałem właściwość `$rasa` (wiersz 19) oraz dwie metody: `setRasa` (linia 21) i `getRasa` (linia 26). Teraz przyszedł moment wykorzystania tych klas w kodzie.
3. W wierszu 32 do zmiennej `$pies` przypisałem egzemplarz (instancję) klasy `Pies`.
4. W linii 33 przez wywołanie metody `setKolorOczu` zmieniłem wartość właściwości `$kolorOczu`, a w wierszu 34 wywołałem metodę `setRasa`, aby zmienić wartość właściwości `$rasa`. Zwróć uwagę, że w klasie `Pies` dostępne są również metody zdefiniowane w klasie nadrzędnej `Ssak` — oznacza to, że metody te zostały oddziedziczone.
5. W wierszach od 36 do 38 drukowane są wartości właściwości klas. Zauważ, że w linii 38 wywoływana jest metoda zdefiniowana w klasie `Ssak`, a nie w klasie `Pies`.

Związek typu „ma”

Tworzenie większych konstrukcji z mniejszych elementów jest naturalnym tokiem postępowania zarówno przy budowie mostów, jak i samochodów czy komputerów. Gotowa konstrukcja jest kompozycją zbudowaną z wielu niezależnych obiektów.

W wielu książkach na temat programowania obiektowego przykładem takiej kompozycji jest samochód, dlatego ja również z niego skorzystam (rysunek 29.2).



Rysunek 29.2. Związek typu „ma”

Ćwiczenie 2. Związek typu „ma”

W tym ćwiczeniu zdefiniujesz prostą klasę reprezentującą samochód i wkomponujesz w nią obiekt reprezentujący silnik.

Postaraj się dokładnie przepisać kod z listingu 29.2. Zwróć uwagę, że nie występuje tutaj dziedziczenie, lecz kompozycja.

LISTING 29.2	PHP 8	code_29_02.php
01	<?php	
02	class Silnik <i>// definicja klasy</i>	
03	{	
04	private \$moc;	
05	private \$moment;	
06	private \$paliwo;	
07		
08	public function setMoc(int float \$moc)	
09	{	
10	\$this->moc = \$moc;	
11	}	
12		
13	public function getMoc() : int float	
14	{	
15	return \$this->moc;	
16	}	
17	}	
18		
19	class Auto <i>// definicja klasy</i>	
20	{	
21	private \$silnik;	
22		
23	public function __construct(<i>// konstruktor</i>	
24	private string \$marka,	
25	private string \$model,	
26	int float \$moc)	
27	{	

```

28     $this->silnik = new Silnik; // tworzenie egzemplarza klasy
29     $this->silnik->setMoc($moc); // wywołanie metody egzemplarza klasy
30 }
31
32 public function print()
33 {
34     print "Parametry samochodu:".PHP_EOL;
35     print "Marka: ".$this->marka.PHP_EOL;
36     print "Model: ".$this->model.PHP_EOL;
37     print "Moc silnika: ".$this->silnik->getMoc()." KM";
38 }
39 }
40
41 $auto = new Auto(marka: "Syrena", model: "S-31", moc: 45);
42 $auto->print();

```

\$ php code_29_02.php ↵

KONSOLA

```

Parametry samochodu:
Marka: Syrena
Model: S-31
Moc silnika: 45 KM

```

Jak działa powyższy kod?

1. W liniach od 02 do 17 zdefiniowałem klasę `Silnik`, reprezentującą silnik samochodu. Wewnątrz klasy znajdują się właściwości związane z mocą silnika, momentem obrotowym i zużyciem paliwa (wiersze od 04 do 06). Dodatkowo zdefiniowałem dwie metody obsługujące właściwość `$moc`.
2. W wierszach od 19 do 39 zdefiniowałem klasę `Auto`, która w swoim konstruktorze dla właściwości `$silnik` tworzy egzemplarz klasy `Silnik` (linia 28). W tym momencie wkomponowałem klasę `Silnik` w klasę `Auto`. Zwróć uwagę na konstruktor i jego trzy parametry. Konstruktor na podstawie pierwszych dwóch argumentów utworzy właściwości wewnątrz klasy `Auto`, a trzeci zostanie wykorzystany do wywołania metody `setMoc` zdefiniowanej w klasie `Silnik`.
3. Począwszy od linii 32, zdefiniowana jest metoda `print` drukująca informacje o samochodzie.

Ćwiczenia dodatkowe

1. Wyszukaj w internecie informacje na temat języka UML (ang. *Unified Modelling Language*). Możesz go w przyszłości wykorzystać do projektowania systemów informatycznych.
2. Rozbuduj dowolnie kod programu z ćwiczenia 2 o nowe elementy składowe samochodu, właściwości i metody.
3. Zaprojektuj klasy w związku typu „jest” i „ma” z dowolnej dziedziny, np. człowiek i komputer.

Pytania

Czy można dziedziczyć więcej niż jedną klasę?

W języku PHP klasa (podklasa) może dziedziczyć tylko po jednej klasie (nadklasie). Przykładowo poniższa konstrukcja nie jest możliwa do wykonania:

```
class Radio extends Stereo, Samochod {}
```

Pewnym rozwiązaniem powyższego problemu jest konstrukcja przedstawiona na listingu 29.3:

LISTING 29.3	PHP 7/8	code_29_03.php
01	<?php	
02	class Auto	
03	{	
04	// właściwości i metody klasy Auto	
05	}	
06	class Stereo extends Auto	
07	{	
08	// właściwości i metody klasy Stereo	
09	}	
10	class Radio extends Stereo	
11	{	
12	// właściwości i metody klasy Radio	
13	}	

Więcej na ten temat dowiesz się w następnej lekcji.

Dlaczego klasy tak dziwnie się nazywają, np. klasa dziedzicząca?

W literaturze możesz spotkać różne nazwy tych samych typów klas. Klasa dziedzicząca po prostu dziedziczy od swojego przodka pewne cechy. Inna nazwa to klasa rozszerzająca, ponieważ wykorzystane jest słowo `extends`. Możesz też spotkać takie nazwy jak potomek i podklasa.



Trochę więcej o klasach

W tej lekcji rozszerzysz swoją wiedzę związaną z konstrukcją klas, obiektów, dziedziczeniem i kompozycją. Choć tematyka związana z programowaniem zorientowanym obiektowo jest bardzo szeroka i napisano na jej temat wiele książek, postaram się nauczyć Cię kilku dodatkowych elementów. Jak już wiesz, możesz wykorzystać modyfikatory `public` lub `private` do zdefiniowania dostępu do właściwości, metod, a także stałych (od wersji PHP 7.1) w klasach. Jest jeszcze dodatkowy modyfikator o nazwie `protected`, który określa zasady widoczności w klasach dziedziczących.

W zależności od deklaracji właściwości metody lub stałe mogą być:

- `public` — są widoczne wewnątrz klasy, w klasach dziedziczących, jak i na zewnątrz klasy,
- `private` — są widoczne tylko wewnątrz klasy i nie mogą być dziedziczone,
- `protected` — są widoczne tylko wewnątrz klasy i mogą być dziedziczone.

Poprawne zdefiniowanie modyfikatora dostępu ma istotne znaczenie podczas konstruowania systemu. Kontrola udostępnianych danych wpływa również na jego bezpieczeństwo.

Interfejsy

W programowaniu obiektowym występują jeszcze takie konstrukcje jak interfejsy i klasy abstrakcyjne. Interfejsy wymuszają na klasach implementację konkretnego, z góry zdefiniowanego zestawu metod, które są niezbędne do prawidłowej interakcji z innymi elementami systemu. Przykładowo aplikacja medyczna obsługująca pacjentów powinna zawierać odpowiedni zestaw informacji o każdym z nich. Definicja interfejsu jest podobna do definicji klasy (listing 30.1):

LISTING 30.1

PHP 7/8

code_30_01.php

```

01 <?php
02 interface Information
03 {
04     public function getID();
05     public function getName();
06     public function getCode();
07 }
08
09 class Patient implements Information
10 {
11     public function getID()
12     {
13         /* kod metody */
14     }
15     public function getName()
16     {
17         /* kod metody */
18     }
19     public function getCode()
20     {
21         /* kod metody */
22     }
23 }

```

Przy definiowaniu interfejsu używa się słowa kluczowego `interface`. Wewnątrz interfejsu definiuje się tylko prototypy metod, a w klasie implementującej interfejs ich implementacje. W definicji takiej klasy należy użyć słowa kluczowego `implements`. Trzeba pamiętać, że klasa powinna zawierać implementacje wszystkich metod ujętych w interfejsie, gdyż w przeciwnym razie zostanie wygenerowany błąd.

Klasy abstrakcyjne

Klasy abstrakcyjne to klasy, na podstawie których nie możesz utworzyć obiektu. Klasę taką oznacza się słowem kluczowym `abstract` przed słowem kluczowym `class`. Wewnątrz takiej klasy musi się znajdować przynajmniej jedna metoda abstrakcyjna, która musi być oznaczona słowem kluczowym `abstract`. Metody abstrakcyjne, podobnie jak w interfejsach, nie są implementowane, a jedynie definiowane. Dopiero w klasach dziedziczących, rozszerzających abstrakcyjnego przodka, metody abstrakcyjne są implementowane.

Definicja klasy abstrakcyjnej jest podobna do definicji klasy (listing 30.2):

LISTING 30.2	PHP 7/8	code_30_02.php
01	<?php	
02	abstract class Information	
03	{	
04	abstract public function getID();	
05	abstract public function getName();	
06	abstract public function getCode();	
07	}	
08		
09	class Patient extends Information	
10	{	
11	public function getID()	
12	{	
13	<i>/* kod metody */</i>	
14	}	
15	public function getName()	
16	{	
17	<i>/* kod metody */</i>	
18	}	
19	public function getCode()	
20	{	
21	<i>/* kod metody */</i>	
22	}	
23	}	

Jeśli klasa dziedzicząca nie będzie zawierać implementacji wszystkich metod abstrakcyjnych przodka, wówczas sama stanie się klasą abstrakcyjną. Metody abstrakcyjne nie mogą być zdefiniowane jako `private`, ponieważ muszą być dziedziczone, a także nie mogą być zdefiniowane jako `final`, gdyż muszą się pokrywać.

Kiedy klasa jest dziedziczona, potomek może zmienić metodę przodka własną implementacją. Możesz temu zapobiec przez wykorzystanie słowa kluczowego `final`, kiedy uznasz, że napisałeś idealną metodę i już nie ma sensu jej poprawiać. Praktyczny przykład może wyglądać tak jak na listingu 30.3.

LISTING 30.3	PHP 7/8	code_30_03.php
01	<?php	
02	class Ssak	
03	{	
04	private \$kolorOczu;	
05		
06	final public function setKolorOczu(\$kolor)	

```

07     {
08         $this->kolorOczu = $kolor;
09     }
10 }

```

Powyższa konstrukcja zapewnia, że w klasie dziedziczącej (potomnej) nie zostanie utworzona nowa metoda `setKolorOczu`. Można również zapobiec rozszerzaniu całej klasy bez deklarowania wszystkich metod jako `final`. Wystarczy oznaczyć całą klasę jako finalną, czyli przed słowem kluczowym `class` dodać słowo kluczowe `final`, np. `final class Ssak { ... }`.

Ćwiczenie 1. Dziedziczenie

Jak już wiesz z poprzedniej lekcji, w języku PHP nie ma możliwości jednoczesnego dziedziczenia więcej niż jednej klasy. W tym ćwiczeniu pokażę Ci, jak rozwiązać ten problem przy wykorzystaniu słowa kluczowego `protected`.

Przepisz kod z listingu 30.4, przeanalizuj linia po linii i postaraj się go zrozumieć.

LISTING 30.4	PHP 7/8	code_30_04.php
01	<?php	
02	class Auto	
03	{	
04	protected function print1()	
05	{	
06	print "Klasa Auto".PHP_EOL;	
07	}	
08	}	
09		
10	class Stereo extends Auto	
11	{	
12	protected function print2()	
13	{	
14	print "Klasa Stereo".PHP_EOL;	
15	}	
16	}	
17		
18	class Radio extends Stereo	
19	{	
20	public function print3()	
21	{	
22	\$this->print1(); <i>// wywołanie metody odziedziczonej po klasie Auto</i>	
23	\$this->print2(); <i>// wywołanie metody odziedziczonej po klasie Stereo</i>	

```

24         print "Klasa Radio";
25     }
26 }
27
28 $radio = new Radio();
29 $radio->print3();

```

```
$ php code_30_04.php ↵
```

KONSOLA

```

Klasa Auto
Klasa Stereo
Klasa Radio

```

Jak działa powyższy kod?

1. W liniach od 02 do 08 zdefiniowałem klasę Auto z jedną metodą — print1. Metoda ta została oznaczona jako protected, czyli jest dostępna wewnątrz klasy Auto oraz w klasach potomnych.
2. W wierszach od 10 do 16 zdefiniowałem następną klasę, Stereo, i rozszerzyłem ją o klasę Auto. Wewnątrz klasy Stereo również znajduje się metoda oznaczona jako protected.
3. Ostatnia klasa, Radio, jest rozszerzeniem klasy Stereo i jest zdefiniowana w liniach od 18 do 26. W związku z tym, że metody w klasach Auto i Stereo zostały oznaczone jako protected, są dostępne w klasie Radio. Zwróć uwagę na konstrukcję metody print3 w klasie Radio.
4. W wierszu 28 do zmiennej \$radio przypisałem egzemplarz klasy Radio, aby w linii 29 wywołać metodę print3, która z kolei wywołuje odziedziczone metody print1 i print2.

Ćwiczenie 2. Implementacja interfejsu

Teraz przećwiczysz implementację prostego interfejsu. Jak już się dowiedziałeś, klasa implementująca interfejs musi zawierać implementację wszystkich metod zawartych w interfejsie. Wiem, jak to brzmi, ale tak to się nazywa. Przepisz kod z listingu 30.5 i postaraj się go zrozumieć.

LISTING 30.5

PHP 7/8

code_30_05.php

```

01 <?php
02 interface Data // definicja interfejsu
03 {
04     public function getID(); // definicja metody

```

```

05     public function getName(); //definicja metody
06     public function getCode(); //definicja metody
07 }
08
09 class Patient implements Data //implementacja interfejsu
10 {
11     public function getID() //implementacja metody
12     {
13         print "Obsługa metody getID()".PHP_EOL;
14     }
15     public function getName() //implementacja metody
16     {
17         print "Obsługa metody getName()".PHP_EOL;
18     }
19     public function getCode() //implementacja metody
20     {
21         print "Obsługa metody getCode()".PHP_EOL;
22     }
23 }
24
25 $chory = new Patient;
26 $chory->getID();
27 $chory->getName();
28 $chory->getCode();

```

```
$ php code_30_05.php ↵
```

KONSOLA

```

Obsługa metody getID()
Obsługa metody getName()
Obsługa metody getCode()

```

Jak działa powyższy kod?

1. W liniach od 02 do 07 zdefiniowałem interfejs Data z trzema metodami. Zwróć uwagę, że w interfejsie metody są tylko definiowane, a implementowane są w klasie Patient (wiersze od 09 do 23).
2. O tym, że klasa implementuje interfejs, świadczy wykorzystane słowo kluczowe implements. Klasa Patient, implementując interfejs Data, musi zatem obsłużyć trzy metody: getID, getName i getCode.
3. Na koniec (w linii 25) do zmiennej \$chory przypisałem egzemplarz klasy Patient, a następnie wywołałem po kolei wszystkie zaimplementowane metody.

Ćwiczenie 3. Implementacja klasy abstrakcyjnej

A teraz przećwiczysz implementację prostej klasy abstrakcyjnej. W odróżnieniu od interfejsu klasa, aby była abstrakcyjna, musi mieć przynajmniej jedną metodę abstrakcyjną. Wiem, to też brzmi dziwnie, ale tak to się po prostu nazywa. Przepisz kod z listingu 30.6 i postaraj się go zrozumieć.

LISTING 30.6	PHP 7/8	code_30_06.php
01	<?php	
02	abstract class Info <i>// definicja klasy abstrakcyjnej</i>	
03	{	
04	abstract public function getID(); <i>// definicja metody abstrakcyjnej</i>	
05		
06	public function getName() <i>// implementacja metody</i>	
07	{	
08	print "Obsługa metody getName()".PHP_EOL;	
09	}	
10		
11	final public function getCode() <i>// implementacja metody</i>	
12	{	
13	print "Obsługa metody getCode()".PHP_EOL;	
14	}	
15	}	
16		
17	class Patient extends Info <i>// definicja klasy z dziedziczeniem</i>	
18	{	
19	public function getID() <i>// implementacja metody abstrakcyjnej</i>	
20	{	
21	print "Obsługa metody getID()".PHP_EOL;	
22	}	
23	}	
24		
25	\$chory = new Patient;	
26	\$chory->getID();	
27	\$chory->getName();	
28	\$chory->getCode();	
\$ php code_30_06.php ↵		KONSOLA
Obsługa metody getID()		
Obsługa metody getName()		
Obsługa metody getCode()		

Jak działa powyższy kod?

1. W tym przypadku zdefiniowana w liniach od 02 do 15 klasa abstrakcyjna `Info` ma jedną metodę abstrakcyjną, `getID`, oraz dwie metody dodatkowe, `getName` i `getCode`. Oznacza to, że klasa dziedzicząca powinna zaimplementować metodę `getID` we własnym zakresie. Zauważ, że metodę `getCode` zdefiniowałem jako finalną za pomocą słowa kluczowego `final` (wiersz 11), co oznacza, że w klasach potomnych nie będzie jej można na nowo zaimplementować.
2. W wierszach od 17 do 23 zdefiniowałem klasę `Patient`, która dziedziczy metody po klasie abstrakcyjnej `Info`. W związku z tym, że metoda `getID` jest zdefiniowana jako abstrakcyjna, w klasie `Patient` zaimplementowałem jej obsługę.
3. Podobnie jak w poprzednim ćwiczeniu, w linii 25 do zmiennej `$chory` przypisałem egzemplarz klasy `Patient`, a następnie wywołałem po kolei wszystkie zaimplementowane metody.

Ćwiczenia dodatkowe

1. W ćwiczeniu 3 spróbuj w klasie `Patient` zaimplementować metodę `getCode`. Co należy zrobić, aby było to możliwe?
2. Jeśli uważasz, że chciałbyś poszerzyć zdobytą już wiedzę, to poszukaj w internecie więcej informacji na temat abstrakcji w programowaniu.
3. Zrób sobie małe powtórzenie materiału i przejrzyj wszystkie wcześniejsze lekcje. Poświęć na to więcej czasu. Przeanalizuj ponownie kod, najlepiej linia po linii.

Wskazówki

Klasa może dziedziczyć tylko jedną klasę abstrakcyjną, ale może implementować wiele interfejsów. W ćwiczeniu 1 pokazałem Ci pewne obejście tego ograniczenia, ale przy interfejsach sprawa jest znacznie prostsza. Implementacja wielu interfejsów może wyglądać np. tak:

```
interface silnik { ... }  
interface stereo { ... }  
class Auto implements silnik, stereo { ... }
```

Poza tym w klasie abstrakcyjnej masz możliwość implementacji części metod, podczas gdy w interfejsach tylko je definiujesz.

PHP zapewnia jeszcze metody statyczne, które można wywołać bez inicjowania egzemplarza klasy, definiowane słowem kluczowym `static`. Inaczej mówiąc, nie musisz tworzyć obiektu, aby wykorzystać potrzebną metodę. Przykładowa definicja klasy z metodą statyczną może mieć postać jak na listingu 30.7.

LISTING 30.7	PHP 8	code_30_07.php
01	<?php	
02	class Kwadrat	
03	{	
04	public static function pole(int float \$bok) : int float	
05	{	
06	return \$bok * \$bok;	
07	}	
08	}	
09	print "Pole kwadratu o boku 8 wynosi ".Kwadrat::pole(bok: 8);	
\$ php code_30_07.php ↵		KONSOLA
Pole kwadratu o boku 8 wynosi 64		

W powyższej klasie Kwadrat zdefiniowałem statyczną metodę `pole`, którą następnie wywołałem za pomocą specjalnej konstrukcji `Kwadrat::pole(8)`. Pomiedzy nazwą klasy a nazwą metody statycznej znajduje się podwójny znak dwukropka. Przepisz powyższy kod i uruchom.

Pytania

Co to znaczy „implementować”?

W informatyce implementacja oznacza wdrożenie, zastosowanie, integrowanie, programowanie. Jest to proces tworzenia kodu źródłowego, który w efekcie prowadzi do napisania programu.



Pierwsza baza danych

Dotychczas do przechowywania niewielkich informacji używałeś plików tekstowych, jednak kiedy danych będzie znacznie więcej, najlepszym rozwiązaniem jest skorzystanie z bazy danych. Taka baza to nic innego jak zbiór rekordów, które można umieszczać w wielu tabelach w sposób zorganizowany. Do obsługi bazy danych stosuje się język SQL (ang. *Structured Query Language*), za pomocą którego informacje mogą być w bazie umieszczane, modyfikowane, usuwane i wyszukiwane. Trudno sobie wyobrazić działanie nawet bardzo prostej strony WWW z koszykiem zakupów bez jej wykorzystania. Kiedy szukasz informacji w wyszukiwarce Google, wyniki otrzymujesz właśnie z takiej bazy. Język PHP obsługuje wiele silników baz danych²⁰, takich jak MySQL, PostgreSQL, MSSQL, Oracle, a także ma wbudowaną obsługę SQLite. Ta ostatnia różni się od większości baz danych tym, że nie jest samodzielną aplikacją, lecz rozszerzeniem wchodzącym w skład języka PHP, zapisującym dane w zwykłych plikach na dysku twardym. Inaczej mówiąc, możesz wykorzystać bazę danych SQLite do nauki języka SQL bez instalowania i konfigurowania dodatkowego oprogramowania.

W niniejszej książce zamieściłem kody źródłowe programów zarówno dla bazy danych SQLite, jak i MySQL, gdyż za ich pomocą początkującemu programiście najłatwiej nauczyć się podstaw języka SQL. Mam nadzieję, że zainstalowałeś pakiet XAMPP i skonfigurowałeś dostęp do bazy danych MySQL.

Kody źródłowe celowo umieściłem oddzielnie dla bazy SQLite i MySQL, abyś mógł w prosty sposób szybko z nich skorzystać. Jak pewnie zauważysz, kody źródłowe dla obydwu typów baz danych są podobne, a jedyna różnica to część definiująca metodę połączenia się z bazą; z tego też powodu nie powielam opisów działania programów przy każdym listingu. Opanowanie tego materiału ułatwi Ci napisanie kodu obsługującego inne bazy danych, jak np. PostgreSQL czy Firebird.

²⁰ PHP, <https://www.php.net/manual/en/refs.database.php> [dostęp: 26.05.2021].

W swoich pierwszych projektach webowych korzystałem z bazy PostgreSQL, dzisiaj wydaje się, że standardem jest MySQL. Warto też wspomnieć o bazie Oracle, którą również możesz wykorzystać w swoich projektach. Możliwości bazy SQL są ogromne!

Podstawy języka SQL

Język SQL został zaprojektowany do komunikacji z bazami danych i w odróżnieniu od innych języków składa się tylko z kilku słów. Zadaniem języka SQL jest proste i wydajne odczytywanie i zapisywanie informacji w bazie. Mimo jego prostoty za jego pomocą można budować bardzo złożone zapytania.

Typy danych w języku SQL

Informacje przechowywane w bazie danych mogą mieć różny charakter. Mogą to być dane tekstowe, numeryczne, binarne oraz związane z datą i czasem.

SQLite zapewnia następujące typy danych²¹:

Typ danych	Zakres danych
INTEGER	liczby całkowite ze znakiem, zapisane na 1, 2, 3, 4, 6 lub 8 bajtach, w zależności od wartości
REAL	liczby zmiennoprzecinkowe, zapisane na 8 bajtach
TEXT	łańcuchy znaków zakodowane w UTF-8, UTF-16B lub UTF-16LE
BLOB	dane binarne
NULL	brak wartości

MySQL zapewnia następujące typy danych (najważniejsze)²²:

Typ danych	Zakres danych
TINYINT	1-bajtowa liczba całkowita ze znakiem od -128 do 127 lub bez znaku od 0 do 255
SMALLINT	2-bajtowa liczba całkowita ze znakiem od -32 768 do 32 767 lub bez znaku od 0 do 65 535
MEDIUMINT	3-bajtowa liczba całkowita ze znakiem od -8 388 608 do 8 388 607 lub bez znaku od 0 do 16 777 215

²¹ SQLite, <https://www.sqlite.org/datatype3.html> [dostęp: 26.05.2021].

²² MySQL, <https://dev.mysql.com/doc/refman/8.0/en/data-types.html> [dostęp: 26.05.2021].

Typ danych	Zakres danych
INT	4-bajtowa liczba całkowita ze znakiem od -2 147 483 648 do 2 147 483 647 lub bez znaku od 0 do 4 294 967 295
BIGINT	8-bajtowa liczba całkowita ze znakiem od -9 223 372 036 854 775 808 do 9 223 372 036 854 775 807 lub bez znaku od 0 do 18 446 744 073 709 551 615
FLOAT	4-bajtowa liczba zmiennoprzecinkowa ze znakiem od -3.402823466E+38 do -1.175494351E-38 lub bez znaku od 1.175494351E-38 do 3.402823466E+38
DOUBLE	8-bajtowa liczba zmiennoprzecinkowa ze znakiem od -1.7976931348623157E+308 do -2.2250738585072014E-308 lub bez znaku od 2.2250738585072014E-308 do 1.7976931348623157E+308
DECIMAL	8-bajtowa liczba zmiennoprzecinkowa w formacie znakowym
DATE	data w formacie RRRR-MM-DD
DATETIME	data i czas w formacie RRRR-MM-DD GG:MM:SS
TIMESTAMP	data i czas liczony od początku epoki systemu Unix, czyli od 1970-01-01 00:00:00
TIME	czas w formacie GG:MM:SS
CHAR	pole znakowe o stałej długości znaków od 1 do 255 bajtów
VARCHAR	pole znakowe o zmiennej długości znaków od 1 do 255 bajtów
TEXT	pole tekstowe o rozmiarze nieprzekraczającym 65 535 bajtów
BLOB	pole binarne o rozmiarze nieprzekraczającym 65 535 bajtów
MEDIUMTEXT	pole tekstowe o rozmiarze nieprzekraczającym 16 777 215 bajtów
MEDIUMBLOB	pole binarne o rozmiarze nieprzekraczającym 16 777 215 bajtów
LONGTEXT	pole tekstowe o rozmiarze nieprzekraczającym 4 294 967 295 bajtów
LONGBLOB	pole binarne o rozmiarze nieprzekraczającym 4 294 967 295 bajtów
ENUM(e1, e2)	lista zdefiniowanych wartości

Tworzenie tabeli

W bazie danych informacje przechowywane są w tabelach, które z kolei mają ściśle zdefiniowane kolumny. Poniższa przykładowa tabela przedstawia listę uczestników balu dziecięcego, podzieloną na cztery kolumny. Każda kolumna reprezentuje konkretną informację o uczestniku, taką jak numer porządkowy, imię, odgrywana postać i wiek. Oczywiście tabela może zawierać bardziej szczegółowe informacje w wielu kolumnach. Jak zatem za pomocą języka SQL utworzyć taką tabelę w bazie danych?

ID	Imię	Postać	Wiek
1	Ariel	Szeryf	8
2	Róża	Księżniczka	6

W języku SQL do utworzenia tabeli wykorzystujemy instrukcję `CREATE TABLE`.

Poniżej przedstawiam definicję tabeli w języku SQL dla bazy SQLite i MySQL, reprezentującą przykładową listę uczestników balu dziecięcego:

SQLite

```
CREATE TABLE UczestnicyBalu
(
  id INTEGER,
  imie TEXT,
  postac TEXT,
  wiek INTEGER
);
```

MySQL

```
CREATE TABLE `UczestnicyBalu` (
  `id` int DEFAULT NULL,
  `imie` varchar(50) DEFAULT '',
  `postac` varchar(50) DEFAULT '',
  `wiek` tinyint UNSIGNED DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

W swojej praktyce kolumny `id`, reprezentujące numer porządkowy w tabeli, definiuję jako klucz główny (ang. *primary key*). Dodatkowo wykorzystuję *autoinkrementację*, czyli za każdym razem, gdy dodawany jest nowy rekord, numer `id` jest automatycznie zwiększany. Jest to bardzo powszechna praktyka wśród projektantów baz danych. Ostateczna definicja tabeli może być zatem następująca:

SQLite

```
CREATE TABLE IF NOT EXISTS UczestnicyBalu
(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  imie TEXT DEFAULT '',
  postac TEXT DEFAULT '',
  wiek INTEGER DEFAULT 0
);
```

MySQL

```
CREATE TABLE IF NOT EXISTS `UczestnicyBalu` (
  `id` int PRIMARY KEY AUTO_INCREMENT,
```

```
`imie` varchar(50) DEFAULT '',
`postac` varchar(50) DEFAULT '',
`wiek` tinyint UNSIGNED DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Dodanie instrukcji `IF NOT EXISTS` instruuje silnik bazy danych, aby przed próbą utworzenia tabeli sprawdził, czy tabela nie została wcześniej utworzona. Zachęcam też do zdefiniowania domyślnej wartości za pomocą słowa kluczowego `DEFAULT`, ponieważ wówczas będziemy mieli pewność, że podczas dodawania rekordu do bazy w przypadku braku definicji konkretnej kolumny otrzyma on zdefiniowaną przez nas wartość domyślną.

Wstawianie danych

Do wprowadzania informacji do bazy danych służy instrukcja `INSERT`. Za pomocą tej instrukcji możemy wstawić wiersz z danymi do wszystkich lub wybranych kolumn w tabeli.

Poniższa konstrukcja dodaje do tabeli nowy rekord i wstawia do kolejnych kolumn otrzymane wartości. W kolumnie `id` wstawiana jest wartość 1, w kolumnie `imie` — wartość `Ariel` itd.

```
INSERT INTO UczestnicyBalu
VALUES
(1, 'Ariel', 'Szeryf', 8);
```

Inną metodą jest jawne wskazanie kolumn, które mają być uwzględnione podczas dodawania danych. Jeśli pominiesz np. kolumnę `wiek`, to automatycznie zostanie wstawiona wartość 0, ponieważ w definicji tabeli dla tej kolumny wartość ta była zdefiniowana jako domyślna.

```
INSERT INTO UczestnicyBalu
(imie, postac, wiek)
VALUES
('Ariel', 'Szeryf', 8);
```

Odczytywanie danych

Najczęściej wykorzystywaną instrukcją w języku SQL jest `SELECT`, która służy do odczytywania danych z jednej lub wielu tabel. Informacje mogą być pobrane dla wszystkich kolumn lub tylko dla wybranych. Język SQL zapewnia również wiele instrukcji umożliwiających sortowanie, filtrowanie i grupowanie danych.

Najprostsza konstrukcja pobierająca wszystkie dane z tabeli wygląda tak:

```
SELECT * FROM UczestnicyBalu;
```

W praktyce przy pobieraniu wielu danych z tabeli, aby zoptymalizować wykorzystanie pamięci serwera, wskazuje się, które kolumny mają być zwrócone. W poniższym przykładzie zwrócone zostaną rekordy zawierające tylko imię i wiek.

```
SELECT imie, wiek FROM UczestnicyBalu;
```

Oczywiście język SQL możemy również wykorzystać do posortowania wyników i wskazać konkretną liczbę zwróconych rekordów. Poniższa konstrukcja jest tego przykładem:

```
SELECT imie, wiek FROM UczestnicyBalu ORDER BY wiek ASC LIMIT 10;
```

Ponieważ w bazach danych zwykle znajduje się bardzo dużo informacji, język SQL umożliwia ich filtrowanie. Pobranie z bazy danych rekordów spełniających konkretny warunek może wyglądać tak:

```
SELECT imie, wiek FROM UczestnicyBalu WHERE wiek = 8;
```

Powyższa konstrukcja zwróci wyniki tylko tych rekordów, w których kolumna wiek jest równa 8.

Aktualizacja i usuwanie danych

Do aktualizacji rekordów w tabeli wykorzystywana jest instrukcja UPDATE. Możemy jej użyć do aktualizacji konkretnego wiersza w tabeli, grupy wierszy lub wszystkich wierszy. Podczas aktualizacji danych należy być bardzo ostrożnym i umiejętnie stosować klauzulę WHERE, ponieważ może to doprowadzić do nieprzewidzianych sytuacji (np. modyfikacji przypadkowych rekordów).

```
UPDATE UczestnicyBalu SET imie = 'Pako' WHERE id = 1;
```

Powyższa konstrukcja powoduje zmianę kolumny imie w wierszu z kolumną id równą 1. Jednoczesna aktualizacja wszystkich rekordów może być wykonana w następujący sposób:

```
UPDATE UczestnicyBalu SET wiek = 9;
```

Do usuwania rekordów z tabeli służy instrukcja DELETE. Tutaj również należy pamiętać o prawidłowym wykorzystaniu klauzuli WHERE, gdyż przez przypadek mogą zostać usunięte przypadkowe rekordy.

```
DELETE FROM UczestnicyBalu WHERE id = 1;
```

Powyższa konstrukcja usuwa z tabeli rekord o kolumnie id równej 1.

Należy pamiętać, że przy wykorzystaniu instrukcji UPDATE i DELETE powinno się zawsze poprawnie stosować klauzulę WHERE. Jeśli nie ma się pewności, które dane zostaną zmodyfikowane lub usunięte, należy testowo zastosować instrukcję SELECT, aby przekonać się o poprawności użycia klauzuli WHERE.

Ćwiczenie 1. Wstawianie danych do bazy

W tym ćwiczeniu nauczysz się wykorzystywać rozszerzenie PDO (ang. *PHP Data Objects*), które jest dostępne w języku PHP. Rozszerzenie to obsługuje wiele silników baz danych, w tym SQLite i MySQL. Utworzysz przykładową tabelę i dodasz kilka rekordów. Polecam powtórkę z programowania obiektowego z poprzednich lekcji.

Poniższy kod zawiera znaną Ci konstrukcję try-catch, aby móc obsłużyć ewentualne zgłoszone wyjątki, czyli po prostu jakieś problemy, które mogą się pojawić podczas połączenia się z bazą danych.

SQLite

Na listingu 31.1 w linii 06 tworzony jest egzemplarz klasy PDO, który jest przypisany do zmiennej \$db_pdo. Zwróć uwagę na argument przekazany do konstruktora klasy PDO, w którym jasno informujemy, jaki typ (driver) bazy danych zamierzamy wykorzystać. W naszym przypadku argumentem będzie nazwa pliku zdefiniowana w zmiennej \$db_file w linii 03. Przy pierwszym uruchomieniu programu lub w przypadku niedostępności pliku na dysku zostanie utworzona nowa, pusta baza danych. Następnie w wierszu 09 włączamy obsługę wyjątków dla interfejsu PDO, które będą przechwytywane w sekcji catch w linii 34. W wierszu 12 następuje próba utworzenia tabeli zdefiniowanej w liniach od 13 do 19. W wierszach od 23 do 26 za pomocą funkcji exec dodawane są kolejne rekordy do tabeli. W linii 28 połączenie z bazą zostaje zamknięte i jeśli wszystko przebiegło prawidłowo, na dysku powinien się pojawić plik o nazwie UczestnicyBalu.db, a w konsoli wyświetlić się odpowiedni komunikat (linia 32).

LISTING 31.1	PHP 7/8	code_31_01.php
01	<?php	
02	try {	
03	\$db_file = 'UczestnicyBalu.db';	
04		
05	// otwórz połączenie z bazą SQLite lub utwórz nowy plik dla bazy	
06	\$db_pdo = new PDO('sqlite:'.__DIR__.'/'.\$db_file);	
07		


```

08      //włącz obsługę wyjątków dla obsługi interfejsu PDO
09      $db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
10
11      //utwórz tabelę, jeśli nie została wcześniej utworzona
12      $db_pdo->exec(
13          "CREATE TABLE IF NOT EXISTS UczestnicyBalu
14          (
15              id INTEGER PRIMARY KEY AUTOINCREMENT,
16              imie TEXT DEFAULT '',
17              postac TEXT DEFAULT '',
18              wiek INTEGER DEFAULT 0
19          );"
20      );
21
22      /*dodawanie rekordów do tabeli */
23      $db_pdo->exec("INSERT INTO UczestnicyBalu (imie, postac, wiek)
24      VALUES ('Ariel', 'Szeryf', 8);");
25      $db_pdo->exec("INSERT INTO UczestnicyBalu (imie, postac, wiek)
26      VALUES ('Róża', 'Księżniczka', 6);");
27
28      $db_pdo = NULL; //zamknij połączenie z bazą
29
30      //sprawdź, czy plik z bazą został utworzony
31      if (file_exists($db_file)) {
32          print "Plik z bazą danych SQLite został utworzony!";
33      }
34  } catch (PDOException $e) {
35      print "Błąd!: " . $e->getMessage();
36      die();
37  }

```

```
$ php code_31_01.php ↵
```

KONSOLA

```
Plik z bazą danych SQLite został utworzony!
```

MySQL

Na listingu 31.2 w liniach od 04 do 07 zdefiniowane są parametry, takie jak nazwa serwera, nazwa bazy danych, nazwa i hasło użytkownika, które są niezbędne do połączenia się z bazą danych MySQL. W wierszu 10 tworzony jest egzemplarz klasy PDO, który jest przypisany do zmiennej \$db_pdo. Zwróć uwagę na argument przekazany do konstruktora klasy PDO, w którym jasno informujemy, jaki typ (driver) bazy danych zamierzamy wykorzystać. Dodatkowy parametr w linii 14 informuje, że połączenie z bazą danych będzie kodowane w formacie UTF-8. Następnie w wierszu 18 włączamy obsługę wyjątków dla interfejsu PDO, które będą

przechwytywane w sekcji catch w linii 40. W wierszu 21 następuje próba utworzenia tabeli zdefiniowanej w liniach od 22 do 27. W wierszach od 31 do 34 za pomocą funkcji exec dodawane są kolejne rekordy do tabeli. W linii 36 połączenie z bazą zostaje zamknięte i jeśli wszystko przebiegło prawidłowo, w tabeli powinny się pojawić nowe dane, a w konsoli wyświetlić odpowiedni komunikat (wiersz 38). Wykorzystaj narzędzie phpMyAdmin do sprawdzenia, czy została utworzona tabela z danymi.

LISTING 31.2	PHP 7/8	code_31_02.php
01	<?php	
02	try {	
03	<i>/* Konfiguracja serwera MySQL */</i>	
04	\$db_server = 'localhost';	
05	\$db_name = 'db_php8';	
06	\$db_user = 'db_php8';	
07	\$db_passwd = 'WETFD8toJyKLWtXC'; <i>//u Ciebie będzie z pewnością inne :)</i>	
08		
09	<i>// otwórz połączenie z bazą danych MySQL</i>	
10	\$db_pdo = new PDO(	
11	"mysql:host=\$db_server;dbname=\$db_name",	
12	\$db_user,	
13	\$db_passwd,	
14	[PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8"]	
15	);	
16		
17	<i>//włącz obsługę wyjątków dla obsługi interfejsu PDO</i>	
18	\$db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);	
19		
20	<i>// utwórz tabelę, jeśli nie została wcześniej utworzona</i>	
21	\$db_pdo->exec(	
22	"CREATE TABLE IF NOT EXISTS `UczestnicyBalu` (	
23	`id` int PRIMARY KEY AUTO_INCREMENT,	
24	`imie` varchar(50) DEFAULT '',	
25	`postac` varchar(50) DEFAULT '',	
26	`wiek` tinyint UNSIGNED DEFAULT 0	
27	) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;"	
28	);	
29		
30	<i>/* dodawanie rekordów do tabeli */</i>	
31	\$db_pdo->exec("INSERT INTO UczestnicyBalu (imie, postac, wiek)	
32	VALUES ('Ariel', 'Szeryf', 8);");	
33	\$db_pdo->exec("INSERT INTO UczestnicyBalu (imie, postac, wiek)	
34	VALUES ('Róża', 'Księżniczka', 6);");	
35		
36	\$db_pdo = NULL; <i>//zamknij połączenie z bazą</i>	
37		

```

38     print "Tabela w bazie danych MySQL została utworzona!";
39
40 } catch (PDOException $e) {
41     print "Błąd!: " . $e->getMessage();
42     die();
43 }

```

```
$ php code_31_02.php ↵
```

KONSOLA

```
Tabela w bazie danych MySQL została utworzona!
```

Ćwiczenie 2. Odczytanie danych z bazy

A teraz (listingi 31.3 i 31.4) odczytasz informacje dodane do bazy danych w poprzednim ćwiczeniu. W celu pobrania danych z bazy użyjesz funkcji query, która zwraca wynik w postaci obiektu. A zatem do dzieła!

SQLite

LISTING 31.3	PHP 7/8	code_31_03.php
01	<?php	
02	try {	
03	\$db_file = 'UczestnicyBalu.db';	
04		
05	<i>// otwórz połączenie z bazą SQLite lub utwórz nowy plik dla bazy</i>	
06	\$db_pdo = new PDO('sqlite:'.__DIR__.'/'.\$db_file);	
07		
08	<i>// włącz obsługę wyjątków dla obsługi interfejsu PDO</i>	
09	\$db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);	
10		
11	<i>// wyślij zapytanie do bazy i pobierz wyniki</i>	
12	\$wynik = \$db_pdo->query(	
13	"SELECT imie, wiek FROM UczestnicyBalu ORDER BY wiek ASC;"	
14	);	
15		
16	\$db_pdo = NULL; <i>// zamknij połączenie z bazą</i>	
17		
18	<i>// przetwórz pobrane rekordy i wydrukuj</i>	
19	print "Uczestnicy balu:".PHP_EOL;	
20	while (\$wiersz = \$wynik->fetch()) {	
21	printf("Imię: %s, wiek: %d".PHP_EOL, \$wiersz['imie'], \$wiersz['wiek']);	
22	}	
23	} catch (PDOException \$e) {	
24	print "Błąd!: " . \$e->getMessage();	
25	die();	
26	}	

```
$ php code_31_03.php ↵
```

KONSOLA

```
Uczestnicy balu:
Imię: Róża, wiek: 6
Imię: Ariel, wiek: 8
```

Jak działa powyższy kod?

1. W linii 06 otwierane jest połączenie z bazą danych UczestnicyBalu.db, która powinna już się znajdować na Twoim dysku.
2. W wierszu 12 wywoływane jest zapytanie do bazy, które ma zwrócić wyniki posortowane rosnąco według wieku. Odczytane dane wstawiane są do zmiennej \$wynik.
3. W linii 16 połączenie z bazą jest zamykane.
4. W wierszach od 20 do 22 informacje zawarte w zmiennej \$wynik są kolejno odczytywane za pomocą funkcji fetch i wstawiane do zmiennej \$wiersz.
5. W związku z tym, że zmienna \$wiersz jest zmienną tabelaryczną, odczytanie z niej danych jest już bardzo proste (linia 21).

MySQL

LISTING 31.4

PHP 7/8

code_31_04.php

```
01 <?php
02 try {
03     /* Konfiguracja serwera MySQL */
04     $db_server = 'localhost';
05     $db_name   = 'db_php8';
06     $db_user   = 'db_php8';
07     $db_passwd = 'WETFD8toJyKLWtXC'; //u Ciebie będzie z pewnością inne :)
08
09     // otwórz połączenie z bazą danych MYSQL
10     $db_pdo = new PDO(
11         "mysql:host=$db_server;dbname=$db_name",
12         $db_user,
13         $db_passwd,
14         [PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8"]
15     );
16
17     // włącz obsługę wyjątków dla obsługi interfejsu PDO
18     $db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
19
20     // wyślij zapytanie do bazy i pobierz wyniki
```

```

21     $wynik = $db_pdo->query(
22         "SELECT imie, wiek FROM UczestnicyBalu ORDER BY wiek ASC;"
23     );
24
25     $db_pdo = NULL; //zamknij połączenie z bazą
26
27     //przetwórz pobrane rekordy i wydrukuj
28     print "Uczestnicy balu:".PHP_EOL;
29     while ($wiersz = $wynik->fetch()) {
30         printf("Imię: %s, wiek: %d".PHP_EOL, $wiersz['imie'], $wiersz['wiek']);
31     }
32
33 } catch (PDOException $e) {
34     print "Błąd!: " . $e->getMessage();
35     die();
36 }

```

```
$ php code_31_04.php ↵
```

KONSOLA

```

Uczestnicy balu:
Imię: Róża, wiek: 6
Imię: Ariel, wiek: 8

```

Jak działa powyższy kod?

1. W liniach od 04 do 07 zdefiniowane są parametry, takie jak nazwa serwera, nazwa bazy danych, nazwa i hasło użytkownika, które są niezbędne do połączenia się z bazą danych MySQL.
2. W wierszu 10 otwierane jest połączenie z bazą danych MySQL.
3. W linii 21 wywoływane jest zapytanie do bazy, które ma zwrócić wyniki posortowane rosnąco według wieku. Odczytane dane wstawiane są do zmiennej \$wynik.
4. W linii 25 połączenie z bazą jest zamykane.
5. W wierszach od 29 do 31 informacje zawarte w zmiennej \$wynik są kolejno odczytywane za pomocą funkcji fetch i wstawiane do zmiennej \$wiersz.
6. W związku z tym, że zmienna \$wiersz jest zmienną tabelaryczną, odczytanie z niej danych jest już bardzo proste (linia 30).

Ćwiczenia dodatkowe

1. Wejdź na stronę <https://www.sqlite.org> i dowiedz się więcej na temat bazy SQLite. Poszukaj w internecie więcej informacji na temat języka SQL. Poszukaj też przykładów dla SQLite.
2. Odwiedź również stronę <https://www.mysql.com>; znajdziesz tam dokumentację najpopularniejszej bazy danych wykorzystywanej w projektach webowych.
3. Otwórz stronę <https://www.php.net/manual/en/book.sqlite3.php> i zapoznaj się z dodatkowymi funkcjami obsługującymi SQLite. Przepisz i uruchom przykładowe listingi. Postaraj się je zrozumieć.
4. Otwórz stronę <https://www.php.net/manual/en/set.mysqlinfo.php> i zapoznaj się z dodatkowymi funkcjami obsługującymi MySQL. Przepisz i uruchom przykładowe listingi. Postaraj się je zrozumieć.
5. Usuń plik UczestnicyBaLu.db i uruchom kod z ćwiczenia 2. Zostanie wyświetlony komunikat o błędzie. Jak myślisz, dlaczego? Co można poprawić, aby kod działał poprawnie?
6. Napisz program tworzący bazę danych i tabelę, w której umieścisz informacje o swoich ulubionych filmach, aktorach lub postaciach z gier.

Wskazówki

Aby zachęcić Cię do nauki podstaw języka SQL, wybrałem bazę danych SQLite, ponieważ jest bardzo prosta w obsłudze i nie wymaga instalacji dodatkowego oprogramowania. Większość systemów bazodanowych implementowana jest jako odrębne procesy serwera, z którymi program kliencki się komunikuje. Pojawia się wówczas problem związany z administrowaniem takim systemem, prawami użytkowników, tworzeniem baz danych itp. W SQLite nie ma tego problemu, ponieważ informacje są zapisywane bezpośrednio w plikach bazy danych na dysku. Oczywiście, rozwiązanie to ma wiele zalet i wad, ale na tym etapie Twojej nauki jest optymalne. Sam bardzo często wykorzystuję SQLite w swojej pracy zawodowej, do czego zachęcam również Ciebie.

Obsługa języka SQL może się delikatnie różnić w różnych bazach danych, jednak jeśli poznasz podstawy, nie powinno być to dla Ciebie problemem. Przykładowo struktura SQL przygotowana dla SQLite powinna bez problemu zadziałać w bazie

danych MySQL czy PostgreSQL i odwrotnie (w podstawowym zakresie). W celu zapewnienia kompatybilności z innymi bazami danych SQLite dokonuje konwersji nazw typów występujących w innych bazach.

Konwersja nazw przedstawiona jest w poniższej tabeli:

MySQL, PostgreSQL	SQLite
INT	INTEGER
FLOAT i DOUBLE	REAL
CHAR i VARCHAR	TEXT

W niniejszej książce poznasz podstawy języka SQL, aby móc zacząć budować i wypełniać informacjami swoje pierwsze bazy danych. Z tego właśnie powodu oszczędzę Ci omówienia roli wyzwalaczy, widoków i transakcji, w nadziei, że wiedzę tę uzupełnisz samodzielnie.

Pytania

Jaka jest wydajność SQLite?

W niewielkich projektach, w których baza danych będzie używana głównie do odczytu informacji, wykorzystanie SQLite jest optymalnym rozwiązaniem. Wiele testów wydajnościowych wskazuje, że instrukcja `SELECT` wykonywana jest szybciej w SQLite niż w pozostałych systemach bazodanowych.

Czy mogę wstawić do tabeli zawartość plików?

Oczywiście, że tak. Wystarczy, że zdefiniujesz kolumnę dla danych binarnych (BLOB), do której wstawiś zawartość dowolnego pliku. W zasadzie nie ma przeszkód, abyś do tabeli wstawił zawartość dowolnego pliku muzycznego, graficznego, tekstowego czy spakowanego w formacie `.zip`.



Proste tabele z danymi

W poprzedniej lekcji dowiedziałeś się, czym jest baza danych i do czego możesz ją wykorzystać. Dodatkowo poznałeś podstawy języka SQL, za pomocą którego będziesz zarządzał informacjami zgromadzonymi w bazach danych. W tej lekcji zastosujesz zdobytą już wiedzę do utworzenia prostych tabel. Zaprojektujesz strukturę bazy danych do obsługi rodzinnej wypożyczalni książek. Myślę, że będzie to dobra wprawka przed opanowaniem języka SQL. Do dzieła!

Więcej o języku SQL

SQL umożliwia zaawansowane filtrowanie i sortowanie danych zawartych w tabelach, a także wydobywanie informacji z wielu tabel za jednym razem przy wykorzystaniu tak zwanego łączenia tabel.

Filtrowanie danych

W praktyce tabele zawierają bardzo dużo informacji, czasem nawet wiele milionów rekordów! Jak zatem wyodrębnić tylko te, które nas najbardziej interesują? Otóż w języku SQL możesz dane filtrować za pomocą klauzuli `WHERE`, które wpisujesz po klauzuli `FROM`, np.:

```
SELECT imie FROM osoby WHERE id = 1;
```

Powyższe polecenie zwróci imię z wiersza, w którym kolumna `id` jest równa 1.

Operator	Co oznacza?
=	równy (jeden znak równości), ale pamiętaj, że w PHP równość to == (dwa znaki równości)
<> i !=	różny
<	mniejszy od
<=	mniejszy lub równy
!<	nie mniejszy niż

Operator	Co oznacza?
>	wiekszy od
>=	wiekszy lub równy
!>	nie większy od
BETWEEN	między dwiema wartościami, np. SELECT imie FROM osoby WHERE id BETWEEN 5 AND 7
IS NULL	jest to wartość NULL

W celu konstruowania bardziej złożonych zapytań do bazy operatory możesz łączyć przy wykorzystaniu operatorów logicznych AND, OR i NOT, a także specjalnego operatora IN, określającego zakres warunków, z których spełniony może być dowolny.

Przeanalizuj poniższe zapytanie SQL:

```
SELECT imie FROM osoby WHERE (id = 1 OR id = 4) AND nazwisko != "Kowalski";
```

Powyższe polecenie zwróci pole imie z wierszy, w których kolumna id jest równa 1 lub 4, a kolumna nazwisko nie jest równa Kowalski. Wykorzystanie operatora IN może wyglądać tak:

```
SELECT imie FROM osoby WHERE id IN (1,3,5);
```

Powyższe polecenie zwróci pole imie z wierszy, w których kolumna id jest równa 1, 3 i 5.

Język SQL umożliwia również zaawansowane filtrowanie danych, których nie można odnaleźć za pomocą klasycznych operatorów. Wyobraźmy sobie, że chcemy przeszukać rekordy w tabeli pod kątem występowania w tytule książki wyrażenia „dla początkujących”. Żeby to zrobić, musimy zastosować wyszukiwanie ze znakami wieloznacznymi z użyciem klauzuli LIKE. Najczęściej wykorzystywanym znakiem wieloznacznym jest znak % (procentu), który oznacza dowolną liczbę wystąpień dowolnego znaku.

Oto przykład takiego zapytania:

```
SELECT autor, tytuł FROM ksiazki WHERE tytuł LIKE "%dla początkujących%";
```

W związku z tym, że znak % został wstawiony przed i za szukaną frazą, wyniki będą zawierały wszystkie tytuły, w których szukana fraza występuje w dowolnym miejscu, np.: „Podręcznik dla początkujących programistów”, „Dla początkujących”, „Tylko dla początkujących” itd.

Istnieje również możliwość przefiltrowania danych według zakresu z wykorzystaniem klauzuli `BETWEEN`. Klauzula ta umożliwia np. odszukanie wszystkich książek wydanych w konkretnym przedziale lat. Wykonanie tego polecenia zwróci listę książek wydanych między 2012 a 2021 rokiem:

```
SELECT autor, tytuł FROM książki WHERE rok BETWEEN 2012 AND 2021;
```

Sortowanie danych

Wykorzystanie klauzuli `ORDER` wiąże się z sortowaniem wyników otrzymanych w wyniku użycia instrukcji `SELECT`. Możesz dane sortować rosnąco `ASC` lub malejąco `DESC`, np.:

```
SELECT id, nazwisko FROM osoby WHERE imie = "Natalia" ORDER BY id DESC;
```

Powyższe polecenie zwróci pola `id` i `nazwisko` posortowane malejąco względem pola `id`, czyli od największych wartości do najmniejszych. Możesz również sortować alfabetycznie pola tekstowe, np.:

```
SELECT id, imie, nazwisko FROM osoby ORDER BY nazwisko ASC;
```

Łączenie tablic

Należy pamiętać, że złączenia tabel w trakcie pobierania danych instrukcją `SELECT` są jedną z najważniejszych cech języka SQL. Dla zapewnienia większej efektywności przechowywania informacji w bazach danych dane rozbija się na tabele. Przykładowo baza danych wypożyczalni książek może się składać z kilku tabel, w których osobno będą przechowywane informacje o książkach, wypożyczających je osobach oraz dokładnej dacie ich wypożyczenia. Aby zatem za jednym razem wydobyć z bazy informację o dacie wypożyczenia, tytule książki i osobie wypożyczającej, najlepiej wykorzystać możliwość złączenia tablic.

Jeśli nie użyjemy złączenia tablic, to dla poniższego wyniku będziemy zmuszeni wykonać dodatkowe dwie instrukcje `SELECT`, aby poznać tytuł wypożyczonej książki oraz imię i nazwisko osoby wypożyczającej.

```
SELECT * FROM wypozyczenia ORDER BY id ASC LIMIT 1;
```

WYPOŻYCZENIA

ID	DATA	ID KSIĄŻKI	ID OSOBY
1	2021-03-20	1	1

Wykorzystując złączenia tablic, możemy od razu uzyskać wszystkie potrzebne informacje. W poniższym przykładzie połączyłem trzy tabele w jednym zapytaniu.

Dla wygody w zapytaniu użyłem polecenia AS, którym przypisałem tablicom aliasy, aby np. zamiast wyrażenia osoba.id móc użyć wersji skróconej o.id. Warunki postawione za klauzulą WHERE (w.ksiazka = k.id AND w.osoba = o.id) gwarantują, że informacje zwrócone w polach (w.id, w.data, k.tytuł, o.imie, o.nazwisko) zostaną odpowiednio dopasowane.

```
SELECT w.id, w.data, k.tytuł, o.imie, o.nazwisko
FROM wypozyczenia AS w, ksiazki AS k, osoby AS o
WHERE
w.ksiazka = k.id AND w.osoba = o.id
ORDER BY w.id ASC LIMIT 1;
```

ID	DATA	TYTUŁ KSIĄŻKI	WYPOŻYCZAJĄCY
1	2021-03-20	Władca Pierścieni	Zofia Nowakówna

Ćwiczenie 1. Tabele i struktura danych

Przepisz dokładnie kod z listingu 32.1 — pamiętaj, że każdy znak ma znaczenie. Zadaniem programu jest utworzenie bazy danych i struktury tabel, a następnie wypełnienie ich przykładowymi danymi o książkach i osobach. Utworzoną bazę danych będziesz wykorzystywał w dalszych ćwiczeniach, dlatego postaraj się zrobić to jak najdokładniej. Jak to działa?

Poniższy kod zawiera znaną Ci konstrukcję try-catch, aby obsłużyć zgłoszone wyjątki, czyli po prostu jakieś problemy, które mogą się pojawić podczas połączenia z bazą danych.

SQLite

LISTING 32.1	PHP 7/8	code_32_01.php
01	<?php	
02	try {	
03	\$db_file = 'kartoteka.db';	
04		
05	// otwórz połączenie z bazą SQLite lub utwórz nowy plik dla bazy	
06	\$db_pdo = new PDO('sqlite:'.__DIR__.'/'.\$db_file);	
07		
08	// włącz obsługę wyjątków dla obsługi interfejsu PDO	
09	\$db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);	
10		
11	// utwórz tabelę, jeśli nie została wcześniej utworzona	
12	\$db_pdo->exec("	
13	CREATE TABLE IF NOT EXISTS ksiazki	
14	(	

```

15      id INTEGER PRIMARY KEY AUTOINCREMENT,
16      autor TEXT DEFAULT '',
17      tytuł TEXT DEFAULT '',
18      rok INTEGER DEFAULT 0,
19      stron INTEGER DEFAULT 0
20  );
21
22  CREATE TABLE IF NOT EXISTS osoby
23  (
24      id INTEGER PRIMARY KEY AUTOINCREMENT,
25      imie TEXT DEFAULT '',
26      nazwisko TEXT DEFAULT ''
27  );
28
29  CREATE TABLE IF NOT EXISTS wypożyczenia
30  (
31      id INTEGER PRIMARY KEY AUTOINCREMENT,
32      książka INTEGER DEFAULT 0,
33      osoba INTEGER DEFAULT 0,
34      data TEXT DEFAULT ''
35  );
36
37  INSERT INTO książki (autor, tytuł, rok, stron)
38  VALUES ('J.R.R. Tolkien', 'Władca Pierścieni', 2012, 1280);
39  INSERT INTO książki (autor, tytuł, rok, stron)
40  VALUES ('Dan Brown', 'Kod Leonarda Da Vinci', 2016, 456);
41  INSERT INTO książki (autor, tytuł, rok, stron)
42  VALUES ('J.D. Salinger', 'Buszujący w zbożu', 2017, 304);
43  INSERT INTO książki (autor, tytuł, rok, stron)
44  VALUES ('Stephen King', 'Mroczna Wieża VII', 2021, 768);
45
46  INSERT INTO osoby (imie, nazwisko) VALUES ('Zofia', 'Nowakówna');
47  INSERT INTO osoby (imie, nazwisko) VALUES ('Oskar', 'Milewski');
48  INSERT INTO osoby (imie, nazwisko) VALUES ('Karina', 'Promyczek');
49  INSERT INTO osoby (imie, nazwisko) VALUES ('Antoni', 'Iksiński');
50  INSERT INTO osoby (imie, nazwisko) VALUES ('Natalia', 'Kornatowska');
51
52  INSERT INTO wypożyczenia (książka, osoba, data)
53      VALUES (1, 1, '2021-03-20');
54  INSERT INTO wypożyczenia (książka, osoba, data)
55      VALUES (1, 2, '2021-03-22');
56  INSERT INTO wypożyczenia (książka, osoba, data)
57      VALUES (2, 3, '2021-03-24');
58  INSERT INTO wypożyczenia (książka, osoba, data)
59      VALUES (3, 4, '2021-03-27');
60  INSERT INTO wypożyczenia (książka, osoba, data)

```

```

61         VALUES (4, 5, '2021-03-31');
62     ");
63
64     print "Baza danych SQLite została utworzona!".PHP_EOL;
65     print "Zawartość bazy danych:".PHP_EOL;
66
67     print PHP_EOL;
68     print "KSIAŻKI".PHP_EOL;
69     print "ID\tAUTOR\tTYTUŁ\tROK\tSTRON".PHP_EOL;
70
71     $wynik = $db_pdo->query("SELECT * FROM ksiazki ORDER BY id ASC;");
72     while ($wiersz = $wynik->fetch()) {
73         printf(
74             "%d\t%s\t%s\t%d\t%d".PHP_EOL,
75             $wiersz['id'], $wiersz['autor'], $wiersz['tytul'],
76             $wiersz['rok'], $wiersz['stron']
77         );
78     }
79
80     print PHP_EOL;
81     print "OSOBY".PHP_EOL;
82     print "ID\tIMIĘ\tNAZWISKO".PHP_EOL;
83
84     $wynik = $db_pdo->query("SELECT * FROM osoby ORDER BY id ASC;");
85     while ($wiersz = $wynik->fetch()) {
86         printf(
87             "%d\t%s\t%s".PHP_EOL,
88             $wiersz['id'], $wiersz['imie'], $wiersz['nazwisko']
89         );
90     }
91
92     print PHP_EOL;
93     print "WYPOŻYCZENIA".PHP_EOL;
94     print "ID\tDATA\tID KSIĄŻKI\tID OSOBY".PHP_EOL;
95
96     $wynik = $db_pdo->query("SELECT * FROM wypozyczenia ORDER BY id ASC;");
97     while ($wiersz = $wynik->fetch()) {
98         printf(
99             "%d\t%s\t%s\t%s".PHP_EOL,
100             $wiersz['id'], $wiersz['data'],
101             $wiersz['ksiazka'], $wiersz['osoba']
102         );
103     }
104
105     $db_pdo = NULL; //zamknij połączenie z bazą
106

```

```

107 } catch (PDOException $e) {
108     print "Błąd!: " . $e->getMessage();
109     die();
110 }

```

```
$ php code_32_01.php ↵
```

KONSOLA

Baza danych SQLite została utworzona!
Zawartość bazy danych:

KSIAŹKI

ID	AUTOR	TYTUŁ	ROK	STRON
1	J.R.R. Tolkien	Władca Pierścieni	2012	1280
2	Dan Brown	Kod Leonarda Da Vinci	2016	456
3	J.D. Salinger	Buszujący w zbożu	2017	304
4	Stephen King	Mroczna Wieża VII	2021	768

OSOBY

ID	IMIĘ	NAZWISKO
1	Zofia	Nowakówna
2	Oskar	Milewski
3	Karina	Promyczek
4	Antoni	Iksiński
5	Natalia	Kornatowska

WYPOŻYCZENIA

ID	DATA	ID KSIAŹKI	ID OSOBY
1	2021-03-20	1	1
2	2021-03-22	1	2
3	2021-03-24	2	3
4	2021-03-27	3	4
5	2021-03-31	4	5

Jak działa powyższy kod?

1. W linii 06 otwierane jest połączenie z bazą danych kartoteka.db, która powinna się już znajdować na Twoim dysku.
2. W wierszu 12 następuje próba utworzenia trzech tabel zdefiniowanych w języku SQL w liniach od 13 do 35 oraz umieszczenia przykładowych danych w tabelach (wiersze od 37 do 61).
3. Jeśli wszystko zostanie wykonane prawidłowo, informacje zawarte w tabelach zostaną wydrukowane w konsoli (linie od 64 do 103).
4. W wierszu 105 połączenie z bazą zostaje zamknięte.

MySQL

W pierwszej kolejności wykorzystaj narzędzie phpMyAdmin do utworzenia struktury danych, czyli tabel i zawartych w nich informacji. Wybierz nazwę bazy, potem kliknij w zakładkę *SQL* i w oknie edycyjnym wpisz poniższy kod w języku SQL.

```
CREATE TABLE IF NOT EXISTS `ksiazki` (
  `id` int PRIMARY KEY AUTO_INCREMENT,
  `autor` varchar(50) DEFAULT '',
  `tytul` varchar(50) DEFAULT '',
  `rok` mediumint UNSIGNED DEFAULT 0,
  `stron` mediumint UNSIGNED DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS `osoby` (
  `id` int PRIMARY KEY AUTO_INCREMENT,
  `imie` varchar(50) DEFAULT '',
  `nazwisko` varchar(50) DEFAULT ''
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS `wypozyczenia` (
  `id` int PRIMARY KEY AUTO_INCREMENT,
  `ksiazka` int UNSIGNED DEFAULT 0,
  `osoba` int UNSIGNED DEFAULT 0,
  `data` date
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO ksiazki (autor, tytul, rok, stron)
VALUES ('J.R.R. Tolkien', 'Władca Pierścieni', 2012, 1280);
INSERT INTO ksiazki (autor, tytul, rok, stron)
VALUES ('Dan Brown', 'Kod Leonarda Da Vinci', 2016, 456);
INSERT INTO ksiazki (autor, tytul, rok, stron)
VALUES ('J.D. Salinger', 'Buszujący w zbożu', 2017, 304);
INSERT INTO ksiazki (autor, tytul, rok, stron)
VALUES ('Stephen King', 'Mroczna Wieża VII', 2021, 768);

INSERT INTO osoby (imie, nazwisko) VALUES ('Zofia', 'Nowakówna');
INSERT INTO osoby (imie, nazwisko) VALUES ('Oskar', 'Milewski');
INSERT INTO osoby (imie, nazwisko) VALUES ('Karina', 'Promyczek');
INSERT INTO osoby (imie, nazwisko) VALUES ('Antoni', 'Iksiński');
INSERT INTO osoby (imie, nazwisko) VALUES ('Natalia', 'Kornatowska');

INSERT INTO wypozyczenia (ksiazka, osoba, data) VALUES (1, 1, '2021-03-20');
INSERT INTO wypozyczenia (ksiazka, osoba, data) VALUES (2, 2, '2021-03-22');
INSERT INTO wypozyczenia (ksiazka, osoba, data) VALUES (2, 3, '2021-03-24');
INSERT INTO wypozyczenia (ksiazka, osoba, data) VALUES (3, 4, '2021-03-27');
INSERT INTO wypozyczenia (ksiazka, osoba, data) VALUES (4, 5, '2021-03-31');
```

Po utworzeniu tabel w narzędziu phpMyAdmin uruchom kod źródłowy z listingu 32.2.

LISTING 32.2	PHP 7/8	code_32_02.php
01	<?php	
02	try {	
03	<i>/* Konfiguracja serwera MySQL */</i>	
04	\$db_server = 'localhost';	
05	\$db_name = 'db_php8';	
06	\$db_user = 'db_php8';	
07	\$db_passwd = 'WETFD8toJyKLWtXC'; <i>// u Ciebie będzie z pewnością inne :)</i>	
08		
09	<i>// otwórz połączenie z bazą danych MySQL</i>	
10	\$db_pdo = new PDO(	
11	"mysql:host=\$db_server;dbname=\$db_name",	
12	\$db_user,	
13	\$db_passwd,	
14	[PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8"]	
15	);	
16		
17	<i>// włącz obsługę wyjątków dla obsługi interfejsu PDO</i>	
18	\$db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);	
19		
20	print "Zawartość bazy danych MySQL:".PHP_EOL;	
21		
22	print PHP_EOL;	
23	print "KSIAŻKI".PHP_EOL;	
24	print "ID\tAUTOR\tTYTUŁ\tROK\tSTRON".PHP_EOL;	
25		
26	\$wynik = \$db_pdo->query("SELECT * FROM ksiazki ORDER BY id ASC;");	
27	while (\$wiersz = \$wynik->fetch()) {	
28	printf(	
29	"%d\t%s\t%s\t%d\t%d".PHP_EOL,	
30	\$wiersz['id'], \$wiersz['autor'], \$wiersz['tytul'],	
31	\$wiersz['rok'], \$wiersz['stron']	
32	);	
33	}	
34		
35	print PHP_EOL;	
36	print "OSOBY".PHP_EOL;	
37	print "ID\tIMIE\tNAZWISKO".PHP_EOL;	
38		
39	\$wynik = \$db_pdo->query("SELECT * FROM osoby ORDER BY id ASC;");	
40	while (\$wiersz = \$wynik->fetch()) {	
41	printf(	
42	"%d\t%s\t%s".PHP_EOL,	
43	\$wiersz['id'], \$wiersz['imie'], \$wiersz['nazwisko']	
44	);	


```

45     }
46
47     print PHP_EOL;
48     print "WYPOŻYCZENIA".PHP_EOL;
49     print "ID\tDATA\t\tID KSIĄŻKI\tID OSOBY".PHP_EOL;
50
51     $wynik = $db_pdo->query("SELECT * FROM wypozyczenia ORDER BY id ASC;");
52     while ($wiersz = $wynik->fetch()) {
53         printf(
54             "%d\t%s\t%s\t\t%s".PHP_EOL,
55             $wiersz['id'], $wiersz['data'],
56             $wiersz['ksiazka'], $wiersz['osoba']
57         );
58     }
59
60     $db_pdo = NULL; //zamknij połączenie z bazą
61
62 } catch (PDOException $e) {
63     print "Błąd!: " . $e->getMessage();
64     die();
65 }

```

\$ php code_32_02.php ↵

KONSOLA

Zawartość bazy danych MySQL:

KSIĄŻKI

ID	AUTOR	TYTUŁ	ROK	STRON
1	J.R.R. Tolkien	Władca Pierścieni	2012	1280
2	Dan Brown	Kod Leonarda Da Vinci	2016	456
3	J.D. Salinger	Buszujący w zbożu	2017	304
4	Stephen King	Mroczna Wieża VII	2021	768

OSOBY

ID	IMIĘ	NAZWISKO
1	Zofia	Nowakówna
2	Oskar	Milewski
3	Karina	Promycek
4	Antoni	Iksiński
5	Natalia	Kornatowska

WYPOŻYCZENIA

ID	DATA	ID KSIĄŻKI	ID OSOBY
1	2021-03-20	1	1
2	2021-03-22	1	2
3	2021-03-24	2	3
4	2021-03-27	3	4
5	2021-03-31	4	5

Jak działa powyższy kod?

1. W liniach od 04 do 07 zdefiniowane są parametry, takie jak nazwa serwera, nazwa bazy danych, nazwa i hasło użytkownika, które są niezbędne do połączenia się z bazą danych MySQL.
2. W wierszu 10 otwierane jest połączenie z bazą danych MySQL.
3. Informacje zawarte w tabelach są pobierane poleceniem query w liniach 26, 39 i 51 i drukowane w konsoli.
4. W wierszu 60 połączenie z bazą zostaje zamknięte.

Ćwiczenie 2. Złożone zapytania do bazy

W tym ćwiczeniu (listingi 32.3 i 32.4) wykorzystasz funkcję złączenia tablic, jednej z najważniejszych cech języka SQL, do odczytania szczegółowych informacji o wypożyczeniach.

SQLite

Struktura kodu jest Ci już znana i jego analiza nie powinna sprawić Ci problemu. Zwróć uwagę na zapytanie SQL skierowane do bazy danych w linii 12. Użyłem złączeń tablic do pobrania informacji za jednym razem ze wszystkich trzech tabel.

LISTING 32.3	PHP 7/8	code_32_03.php
01	<?php	
02	try {	
03	\$db_file = 'kartoteka.db';	
04		
05	<i>// otwórz połączenie z bazą SQLite lub utwórz nowy plik dla bazy</i>	
06	\$db_pdo = new PDO('sqlite:'.__DIR__.'/'.\$db_file);	
07		
08	<i>// włącz obsługę wyjątków dla obsługi interfejsu PDO</i>	
09	\$db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);	
10		
11	<i>/* pobierz informacje o wypożyczeniach */</i>	
12	\$result = \$db_pdo->query("	
13	SELECT w.id, w.data, k.tytuł, o.imię, o.nazwisko	
14	FROM wypozyczenia AS w, ksiazki AS k, osoby AS o	
15	WHERE	
16	w.ksiazka = k.id AND w.osoba = o.id	
17	ORDER BY w.id ASC;	
18	");	
19		

```

20     print "ID\tDATA\t\tTYTUŁ KSIĄŻKI\t\tWYPOŻYCZAJĄCY".PHP_EOL;
21
22     foreach ($result as $r) {
23         printf(
24             "%s\t%s\t%s\t%s %s".PHP_EOL,
25             $r['id'], $r['data'], $r['tytuł'], $r['imie'], $r['nazwisko']
26         );
27     }
28
29     $db_pdo = NULL; //zamknij połączenie z bazą
30
31 } catch (PDOException $e) {
32     print "Błąd!: " . $e->getMessage();
33     die();
34 }

```

```
$ php code_32_03.php ↵
```

KONSOLA

ID	DATA	TYTUŁ KSIĄŻKI	WYPOŻYCZAJĄCY
1	2021-03-20	Władca Pierścieni	Zofia Nowakówna
2	2021-03-22	Władca Pierścieni	Oskar Milewski
3	2021-03-24	Kod Leonarda Da Vinci	Karina Promyczek
4	2021-03-27	Buszujący w zbożu	Antoni Iksiński
5	2021-03-31	Mroczna Wieża VII	Natalia Kornatowska

MySQL

LISTING 32.4	PHP 7/8	code_32_04.php
01	<?php	
02	try {	
03	/* Konfiguracja serwera MySQL */	
04	\$db_server = 'localhost';	
05	\$db_name = 'db_php8';	
06	\$db_user = 'db_php8';	
07	\$db_passwd = 'WETFD8toJyKLWtXC'; //u Ciebie będzie z pewnością inne :)	
08		
09	// otwórz połączenie z bazą danych MYSQL	
10	\$db_pdo = new PDO(	
11	"mysql:host=\$db_server;dbname=\$db_name",	
12	\$db_user,	
13	\$db_passwd,	
14	[PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8"]	
15	);	
16		
17	// włącz obsługę wyjątków dla obsługi interfejsu PDO	
18	\$db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);	
19		

```

20      /* pobierz informacje o wypożyczeniach */
21      $result = $db_pdo->query("
22          SELECT w.id, w.data, k.tytuł, o.imie, o.nazwisko
23          FROM wypozyczenia AS w, ksiazki AS k, osoby AS o
24          WHERE
25          w.ksiazka = k.id AND w.osoba = o.id
26          ORDER BY w.id ASC;
27      ");
28
29      print "ID\tDATA\tTYTUŁ KSIĄŻKI\tWYPOŻYCZAJĄCY".PHP_EOL;
30
31      foreach ($result as $r) {
32          printf(
33              "%s\t%s\t%s\t%s %s".PHP_EOL,
34              $r['id'], $r['data'], $r['tytuł'], $r['imie'], $r['nazwisko']
35          );
36      }
37
38      $db_pdo = NULL; //zamknij połączenie z bazą
39
40  } catch (PDOException $e) {
41      print "Błąd!: " . $e->getMessage();
42      die();
43  }

```

```
$ php code_32_04.php ↵
```

KONSOLA

ID	DATA	TYTUŁ KSIĄŻKI	WYPOŻYCZAJĄCY
1	2021-03-20	Władca Pierścieni	Zofia Nowakówna
2	2021-03-22	Władca Pierścieni	Oskar Milewski
3	2021-03-24	Kod Leonarda Da Vinci	Karina Promyczek
4	2021-03-27	Buszujący w zbożu	Antoni Iksiński
5	2021-03-31	Mroczna Wieża VII	Natalia Kornatowska

Ćwiczenia dodatkowe

1. Poszukaj więcej informacji na temat sortowania, filtrowania i łączenia tablic w języku SQL.
2. Poszukaj informacji na temat ataków SQL Injection. Postaraj się zrozumieć, dlaczego są tak groźne dla systemu bazodanowego.
3. Napisz zapytanie SQL pobierające z bazy danych informacje o wypożyczonych książkach, które zostały wydane po 2016 roku.

Pytania

Czy mogę sortować dane względem dwóch kolumn?

Oczywiście, że tak! Przykładem może być poniższe zapytanie SQL:

```
SELECT imie, nazwisko FROM osoby ORDER BY nazwisko ASC, imie ASC;
```

Zostaną zwrócone wyniki posortowane alfabetycznie (rosnąco, czyli od A do Z), w pierwszej kolejności względem kolumny `nazwisko`, a w drugiej względem kolumny `imie`. Ma to znaczenie, kiedy w tak zwanych osobowych bazach danych istnieją rekordy z osobami o identycznych nazwiskach. Dopiero kolejne sortowanie względem imion powoduje, że zwracane wyniki są poprawnie posortowane.



Bezpieczeństwo w SQL-u

Kiedy będziesz projektował system do obsługi baz danych SQL, z pewnością wykorzystasz takie instrukcje jak SELECT, UPDATE i DELETE. Niemal zawsze bazy danych przeszukiwane są pod kątem konkretnych informacji, np. wyszukiwarka Google. Zastanów się przez chwilę nad tym, że w oknie takiej wyszukiwarki możesz wpisać cokolwiek, nawet specjalnie przygotowany złośliwy kod SQL, który w niezabezpieczonych systemach może wyrządzić wiele szkód. Jednym z najbardziej złośliwych ataków na bazę danych jest SQL Injection, który umożliwia bezprawny dostęp do najbardziej wrażliwych i tajnych danych systemu.

Twój system musi być odporny na ataki SQL, przynajmniej w podstawowym zakresie, aby informacje uzyskane np. od użytkownika Twojej strony internetowej można było bezpiecznie zapisać w bazie danych. Jedną z podstawowych metod bezpieczeństwa SQL jest walidacja danych, czyli dokładne sprawdzenie danych pod kątem typu i zakresu.

O co dokładnie chodzi w atakach SQL i dlaczego są tak niebezpieczne? Załóżmy, że użytkownik strony internetowej przeszukuje bazę danych z książkami. W formularzu na stronie podaje tytuł książki i klika odpowiedni przycisk, a program po stronie serwera wykonuje odpowiednie zapytanie do bazy. Często szukana fraza umieszczana jest w zmiennej, która z kolei wstawiana jest do zapytania SQL. Oto przykładowy kod:

```
$db->query("SELECT * FROM ksiazki WHERE id = '$id'");
```

Jeśli użytkownik przekaże w zmiennej \$id wartość 758, to zapytanie SQL przyjmie postać:

```
SELECT * FROM ksiazki WHERE id = '758';
```

Problem pojawi się wówczas, kiedy użytkownik będzie chciał celowo wykorzystać lukę w bezpieczeństwie i uzyskać dostęp do wszystkich danych w tabeli albo, co najgorsze, będzie chciał wszystkie dane usunąć. Problemem będzie przekazanie w zmiennej \$id wartości 1' OR '1'='1, ponieważ spowoduje to tak zwane wstrzyknięcie złośliwego kodu, co w konsekwencji zmodyfikuje pierwotne zapytanie SQL do postaci:

```
SELECT * FROM ksiazki WHERE id = '1' OR '1'='1';
```

Powyższe zapytanie pobierze zawartość całej tabeli zamiast wybranego rekordu.

A teraz wyobraź sobie, co się stanie, gdy \$id przyjmie wartość 1'; DROP TABLE osoby; SELECT '1. Będzie to katastrofalne w skutkach dla danych zawartych w tabeli, ponieważ zapytanie SQL przyjmie następującą postać:

```
SELECT * FROM osoby WHERE id = '1'; DROP TABLE osoby; SELECT '1';
```

Powyższy kod SQL składa się tak naprawdę z trzech zapytań SQL oddzielonych znakiem średnika. W drugim zapytaniu instrukcja DROP TABLE usuwa tabelę ze wszystkimi danymi. Na szczęście zarówno bazy danych, jak i obsługujące je biblioteki (wbudowane w język PHP) mają zaimplementowane mechanizmy obronne i udostępniają programistom narzędzia do walidacji danych.

Pamiętaj! Nigdy nie ufaj danym pobieranym z nieznanych źródeł, zwłaszcza od użytkowników np. Twojej strony internetowej. Najbezpieczniej będzie, jeśli wszystkie zapytania do bazy będą walidowane — wówczas prawdopodobieństwo wystąpienia problemów będzie znacznie mniejsze.

Ćwiczenie 1. Przygotowanie zapytania do bazy

Podstawowych zasad bezpieczeństwa przy konstruowaniu zapytań w języku SQL nauczysz się w trakcie tego ćwiczenia (listingi 33.1 i 33.2). Poznasz funkcje z biblioteki PDO i wykorzystasz je do przygotowania zapytania SQL (prepare), sprawdzenia jego poprawności pod kątem wprowadzonych parametrów (bindValue), a także jego wykonania (execute). We wskazówkach do tej lekcji znajdziesz więcej informacji na temat funkcji bindValue, przeznaczonej do walidacji parametrów w zapytaniu SQL.

Poniższy kod zawiera znaną Ci konstrukcję try-catch, aby móc obsłużyć ewentualne zgłoszone wyjątki, czyli po prostu jakieś problemy, które mogą się pojawić podczas połączenia się z bazą danych.

SQLite

Wykorzystałem funkcję prepare do przygotowania zapytania SQL do dalszej obróbki (linia 12). W tym momencie biblioteka obsługująca bazę SQLite przyjmuje do wiadomości, że zamierzam wykonać konkretne zapytanie SQL. Zauważ, że w samym zapytaniu, za klauzulą WHERE, umieściłem specjalny zapis informujący, że do zapytania będzie wstawiany parametr (id=:id). Tak przygotowane (spreparowane) zapytanie

trafia w formie obiektu do zmiennej `$stmt`. W wierszu 15 wywoływana jest funkcja `bindValue`, która odpowiada za prawidłową walidację danych. Wskazałem, że wartość ma reprezentować liczby całkowite, w przeciwnym razie zapytanie SQL nie zostanie przygotowane (spreparowane). W linii 18 użyłem funkcji `execute` do wysłania zapytania SQL do bazy danych. Wynik zapytania trafia do zmiennej tabelarycznej `$wynik`, a następnie jest w wierszu 19 drukowany.

LISTING 33.1	PHP 7/8	code_33_01.php
01	<?php	
02	try {	
03	\$db_file = 'kartoteka.db';	
04		
05	<i>// otwórz połączenie z bazą SQLite lub utwórz nowy plik dla bazy</i>	
06	\$db_pdo = new PDO('sqlite:'.__DIR__.'/'.\$db_file);	
07		
08	<i>// włącz obsługę wyjątków dla obsługi interfejsu PDO</i>	
09	\$db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);	
10		
11	<i>/* przygotowanie zapytania SQL */</i>	
12	\$stmt = \$db_pdo->prepare("SELECT * FROM ksiazki WHERE id=:id");	
13		
14	<i>// sprawdź, czy przekazany parametr jest liczbą całkowitą</i>	
15	if (\$stmt->bindValue(':id', 1, PDO::PARAM_INT))	
16	{	
17	<i>/* wykonaj przygotowane zapytanie SQL */</i>	
18	\$stmt->execute();	
19	print_r(\$stmt->fetch(PDO::FETCH_ASSOC));	
20	}	
21		
22	\$db_pdo = NULL; <i>// zamknij połączenie z bazą</i>	
23		
24	} catch (PDOException \$e) {	
25	print "Błąd!: " . \$e->getMessage();	
26	die();	
27	}	
\$ php code_33_01.php ↵		
		KONSOLA
<pre>Array ([id] => 1 [autor] => J.R.R. Tolkien [tytul] => Władca Pierścieni [rok] => 2012 [stron] => 1280)</pre>		

MySQL

LISTING 33.2

PHP 7/8

code_33_02.php

```

01  <?php
02  try {
03      /* Konfiguracja serwera MySQL */
04      $db_server = 'localhost';
05      $db_name   = 'db_php8';
06      $db_user   = 'db_php8';
07      $db_passwd = 'WETFD8toJyKLWtXC'; // u Ciebie będzie z pewnością inne :)
08
09      // otwórz połączenie z bazą danych MySQL
10      $db_pdo = new PDO(
11          "mysql:host=$db_server;dbname=$db_name",
12          $db_user,
13          $db_passwd,
14          [PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8"]
15      );
16
17      // włącz obsługę wyjątków dla obsługi interfejsu PDO
18      $db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
19
20      /* przygotowanie zapytania SQL */
21      $stmt = $db_pdo->prepare("SELECT * FROM ksiazki WHERE id=:id");
22
23      // sprawdź, czy przekazany parametr jest liczbą całkowitą
24      if ($stmt->bindValue(':id', 1, PDO::PARAM_INT))
25      {
26          /* wykonaj przygotowane zapytanie SQL */
27          $stmt->execute();
28          print_r($stmt->fetch(PDO::FETCH_ASSOC));
29      }
30
31      $db_pdo = NULL; // zamknij połączenie z bazą
32
33  } catch (PDOException $e) {
34      print "Błąd!: " . $e->getMessage();
35      die();
36  }

```

\$ php code_33_02.php ↵

KONSOLA

```

Array
(
    [id] => 1
    [autor] => J.R.R. Tolkien
    [tytuł] => Władca Pierścieni
    [rok] => 2012
    [stron] => 1280
)

```

Ćwiczenie 2. Optymalizacja zapytania do bazy

W tym ćwiczeniu (listingi 33.3 i 33.4) nauczę Cię, jak możesz walidować dane, które trafią do Twojego programu np. z internetu. Odpowiednia obsługa takich danych wpływa bezpośrednio na bezpieczeństwo programu. Nigdy nie będziesz miał pewności, jakie informacje wpisze użytkownik na stronie WWW lub w jakim formacie otrzymasz dane z innego programu. Przepisz dokładnie poniższy kod, przeanalizuj go linia po linii i postaraj się zrozumieć.

Jak działa program? Informacje przesyłane np. z formularzy wstawiane są przez serwer WWW do specjalnych tablic superglobalnych `$_GET` lub `$_POST`, które z kolei mogą być odczytane przez program. Informacje w tych tablicach są surowe, to znaczy takie, jakie wpisał je użytkownik, bez modyfikacji. Twoim zadaniem jest obsługa tych danych pod kątem ich poprawności i zgodności z oczekiwanym formatem. Jeśli oczekujesz wartości związanej z numerem telefonu, to musisz dokonać walidacji właśnie pod tym kątem.

SQLite

W naszym przypadku informacja, którą wpisał użytkownik, znajduje się w zmiennej `$_GET['tytuł']` (linia 02). Jak możesz zauważyć, użytkownik popełnił błąd, gdyż wstawił na początku szukanego tekstu znak spacji, a na końcu znak kratki. Aby zobaczyć, jak wartość zmiennej `$szukaj` ulegała modyfikacji w trakcie działania programu, dodałem pomocniczą zmienną tabelaryczną `$proces`. W wierszu 07 tworzę zmienną `$szukaj` i wstawiam do niej zawartość zmiennej `$_GET['tytuł']`, a jednocześnie, dla pewności, narzucam zmiennej typ tekstowy `string`. W linii 11 usuwam zbędne spacje z obu stron (`trim`) oraz zamieniam tekst na małe litery (`mb_strtolower`). W wierszu 15 korzystam z funkcji `preg_replace` do usunięcia z tekstu wszystkich niepotrzebnych znaków, których tytuł książki z pewnością nie będzie zawierał.

W linii 34 dodałem do zmiennej `$szukaj` z obu stron znak `%` (procentu). Związane jest to z późniejszym zastosowaniem zaawansowanego filtrowania w języku SQL przy wykorzystaniu klauzuli `LIKE` (wiersz 32). Oprócz wyników wyszukiwania drukowane są (dla zobrazowania) kolejne fazy obróbki zmiennej `$szukaj`. Program kończy działanie w linii 52, w której zamyka połączenie z bazą danych.

LISTING 33.3

PHP 7/8

code_33_03.php

```
01 <?php
02 $_GET['tytuł'] = " Wieża#"; //szukana fraza
03
```

```

04 $proces = [];
05
06 /* pierwsza optymalizacja danych */
07 $szukaj = (string)$_GET['tytuł']; // rzutujemy na typ string
08 $proces[] = $szukaj;
09
10 // obcinamy boczne spacje i zamieniamy na małe litery
11 $szukaj = trim(mb_strtolower($szukaj));
12 $proces[] = $szukaj;
13
14 // wycinamy wszystko, co nie jest cyfrą, literą, podkreśleniem itp.
15 $szukaj = preg_replace('/[^\0-9 a-z-!\_\p{L}]/u', '', $szukaj);
16 $proces[] = $szukaj;
17
18 if ($szukaj == '') {
19     die("Nie ma co szukać ;");
20 }
21
22 try {
23     $db_file = 'kartoteka.db';
24
25     // otwórz połączenie z bazą SQLite lub utwórz nowy plik dla bazy
26     $db_pdo = new PDO('sqlite:'.__DIR__.'/'.$db_file);
27
28     // włącz obsługę wyjątków dla obsługi interfejsu PDO
29     $db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
30
31     /* przygotowanie zapytania SQL */
32     $stmt = $db_pdo->prepare("SELECT * FROM ksiazki WHERE tytuł LIKE :tytuł");
33
34     $szukaj = '%'.$szukaj.'%'; // znaki specjalne dla klauzuli LIKE
35
36     // sprawdź, czy przekazany parametr jest tekstem
37     if ($stmt->bindValue(':tytuł', $szukaj, PDO::PARAM_STR))
38     {
39         /* wykonaj przygotowane zapytanie SQL */
40         $stmt->execute();
41         print_r($stmt->fetch(PDO::FETCH_ASSOC));
42
43         print "Co się działo ze zmienną 'szukaj' w trakcie działania programu?";
44         print PHP_EOL.str_repeat('-', 63).PHP_EOL;
45         foreach ($proces as $p) {
46             print $p." => ";
47         }
48
49         print $szukaj;

```

```

50     }
51
52     $db_pdo = NULL; //zamknij połączenie z bazą
53
54 } catch (PDOException $e) {
55     print "Błąd!: " . $e->getMessage();
56     die();
57 }

```

```
$ php code_33_03.php ↵
```

KONSOLA

```

Array
(
    [id] => 4
    [autor] => Stephen King
    [tytul] => Mroczna Wieża VII
    [rok] => 2021
    [stron] => 768
)
Co się działo ze zmienną 'szukaj' w trakcie działania programu?
-----
Wieża# => wieża# => wieża => %wieża%

```

MySQL

LISTING 33.4

PHP 7/8

code_33_04.php

```

01 <?php
02 $_GET['tytul'] = " Wieża#"; //szukana fraza
03
04 $proces = [];
05
06 /* pierwsza optymalizacja danych */
07 $szukaj = (string)$_GET['tytul']; //rzutujemy na typ string
08 $proces[] = $szukaj;
09
10 //obcinamy boczne spacje i zamieniamy na małe litery
11 $szukaj = trim(mb_strtolower($szukaj));
12 $proces[] = $szukaj;
13
14 //wycinamy wszystko, co nie jest cyfrą, literą, podkreśleniem itp.
15 $szukaj = preg_replace('/[^0-9 a-z-!\_\\p{L}]/u', '', $szukaj);
16 $proces[] = $szukaj;
17
18 if ($szukaj == '') {
19     die("Nie ma co szukać ;");
20 }
21
22 try {

```

```

23      /* Konfiguracja serwera MySQL */
24      $db_server = 'localhost';
25      $db_name   = 'db_php8';
26      $db_user   = 'db_php8';
27      $db_passwd = 'WETFD8toJyKLWtXC'; // u Ciebie będzie z pewnością inne :)
28
29      // otwórz połączenie z bazą danych MySQL
30      $db_pdo = new PDO(
31          "mysql:host=$db_server;dbname=$db_name",
32          $db_user,
33          $db_passwd,
34          [PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8"]
35      );
36
37      // włącz obsługę wyjątków dla obsługi interfejsu PDO
38      $db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
39
40      /* przygotowanie zapytania SQL */
41      $stmt = $db_pdo->prepare("SELECT * FROM ksiazki WHERE tytuł LIKE :tytuł");
42
43      $szukaj = '%'.$szukaj.'%'; // znaki specjalne dla klauzuli LIKE
44
45      // sprawdź, czy przekazany parametr jest tekstem
46      if ($stmt->bindValue(':tytuł', $szukaj, PDO::PARAM_STR))
47      {
48          /* wykonaj przygotowane zapytanie SQL */
49          $stmt->execute();
50          print_r($stmt->fetch(PDO::FETCH_ASSOC));
51
52          print "Co się działo ze zmienną 'szukaj' w trakcie działania programu?";
53          print PHP_EOL.str_repeat('-', 63).PHP_EOL;
54          foreach ($proces as $p) {
55              print $p." => ";
56          }
57
58          print $szukaj;
59      }
60
61      $db_pdo = NULL; // zamknij połączenie z bazą
62
63  } catch (PDOException $e) {
64      print "Błąd!: " . $e->getMessage();
65      die();
66  }

```

```
$ php code_33_04.php ↵
```

KONSOLA

```
Array
(
    [id] => 4
    [autor] => Stephen King
    [tytul] => Mroczna Wieża VII
    [rok] => 2021
    [stron] => 768
)
Co się działo ze zmienną 'szukaj' w trakcie działania programu?
-----
Wieża# => wieża# => wieża => %wieża%
```

Ćwiczenia dodatkowe

1. Poszukaj w sieci informacji na temat ataków XSS (ang. *Cross Site Scripting*) i HTML Injection. Postaraj się zrozumieć, jak działają i jak się przed nimi skutecznie bronić przy wykorzystaniu składni języka PHP.

Wskazówki

Poprawne przygotowanie zapytania SQL do bazy jest bardzo istotne, biorąc pod uwagę bezpieczeństwo i integralność danych. Walidacja danych w zasadzie dotyczy wszystkich informacji przetwarzanych przez program, bez względu na to, czy są to zapytania w języku SQL, zmienne, stałe, czy nazwy plików.

Biblioteka PDO udostępnia funkcje ułatwiające poprawne przygotowanie zapytania SQL, a mianowicie `bindParam` i `bindValue`. Obie funkcje przyjmują trzy parametry. Pierwszy oznacza identyfikator w zapytaniu SQL, drugi — wartość, jaka ma być podstawiana pod identyfikator, a trzeci — typ podstawianej wartości.

Poniższy przykład demonstruje wykorzystanie funkcji `bindValue` do sprawdzenia, czy wartość podstawiana pod `:id` jest liczbą całkowitą, czyli czy jest typu `INTEGER`.

```
$stmt = $db->prepare("SELECT * FROM ksiazki WHERE id=:id");
if ($stmt->bindValue(':id', 1, PDO::PARAM_INT)) { ... };
```

Możesz również sprawdzić inne typy; najważniejsze umieściłem w poniższej tabeli.

Typ (trzeci parametr)	Co sprawdza?
PDO::PARAM_BOOL	wartość boolowska
PDO::PARAM_INT	liczby, odpowiednik INTEGER w języku SQL
PDO::PARAM_STR	łańcuchy znaków, odpowiednik CHAR, VARCHAR w języku SQL
PDO::PARAM_LOB	dane binarne, duże struktury danych
PDO::PARAM_NULL	wartość NULL

W swojej praktyce dane przekazywane do systemu z internetu waliduję co najmniej dwukrotnie, zanim wykorzystam je w zapytaniu SQL. Pierwszą walidację zlecam serwerowi WWW (odpowiednie reguły umieszczam w plikach `.htaccess`), a druga wykonywana jest już w kodzie przez wyspecjalizowane funkcje dostępne w języku PHP (np. `preg_match`, `is_integer`, `is_string`) lub funkcje specjalnie zaprojektowane przeze mnie na potrzeby konkretnego projektu. Jeśli wszystkie dane przeszły wszystkie testy pomyślnie, wykorzystuję je do zbudowania zapytania SQL. Możesz uznać to za bardzo rozbudowany proces, jednak takie postępowanie wymuszają praktyka i doświadczenie.

Pytania

Jakie mogą być konsekwencje ataków hakerskich?

Każdy taki atak skierowany na firmy i instytucje może doprowadzić do tak zwanego wycieku danych, czyli nieautoryzowanego dostępu do tajnych informacji, co w efekcie może skutkować ogromnymi stratami finansowymi. Poszukaj w sieci więcej informacji na temat udanych ataków hakerskich na popularne serwisy społecznościowe, sklepy internetowe i portale informacyjne.

Czym jest walidowanie danych?

Walidacja jest procesem polegającym na sprawdzeniu danych pod kątem ich zgodności z założeniami systemu, jego logiką i wytycznymi określonymi przez projektanta (programistę). Przykładem może być wartość liczbową (np. numer telefonu) wprowadzana przez użytkownika do systemu. Podczas walidacji sprawdzony zostanie np. typ wprowadzonych danych (jak `integer`) oraz jego ściśle określony i oczekiwany format.



Arytmetyka w SQL-u

Język SQL pozwala wykonywać operacje arytmetyczne, takie jak dodawanie, odejmowanie, mnożenie i dzielenie, jak również zapewnia szereg funkcji agregujących, umożliwiających obliczenie liczby wierszy w tabeli, pobranie zbiorczej sumy konkretnych wartości w kolumnach, a także odszukanie najmniejszej i największej wartości w kolumnie tabeli.

Operacje arytmetyczne można wykonać bezpośrednio w instrukcji SELECT, np. wywołanie poniższego polecenia zwróci wynik z prognozowanym czasem, liczonym w minutach, potrzebnym na przeczytanie konkretnej książki (oczywiście jest to tylko przykład).

```
SELECT (stron * 3) AS czas FROM ksiazki;
```

Czasami niezbędne są informacje podsumowujące dane bez konieczności ich pobierania. Inaczej mówiąc, nie musisz pobierać zawartości całej tabeli do programu, aby dokonać obliczeń, możesz je po prostu wykonać bezpośrednio w bazie danych.

Funkcja	Co zwraca?
COUNT (*)	liczba wierszy w kolumnie
SUM()	suma wartości w kolumnie
MIN()	najmniejsza wartość w kolumnie
MAX()	największa wartość w kolumnie
AVG()	średnia wartość kolumny

Przykładem wykorzystania funkcji agregującej może być polecenie zwracające sumę stron wszystkich książek w tabeli:

```
SELECT SUM(stron) AS strony FROM ksiazki;
```

Możesz oczywiście łączyć funkcje agregujące, aby wydobyć z bazy bardziej szczegółowe informacje. Wywołanie polecenia:

```
SELECT MAX(rok) AS najnowsza, MIN(rok) AS najstarsza FROM ksiazki;
```


zwróci na podstawie kolumny reprezentującej rok wydania jednocześnie informację o najstarszym i najnowszym wydaniu książki w tabeli.

Ćwiczenie 1. Operacje arytmetyczne

W tym prostym ćwiczeniu (listingi 34.1 i 34.2) wykorzystasz operacje arytmetyczne do obliczenia czasu, który powinienś przeznaczyć na przeczytanie konkretnej książki umieszczonej w tabeli. Założyłem, że czas potrzebny na przeczytanie jednej strony wynosi 3 minuty. Oczywiście, ćwiczenie to potraktuj z lekkim przymrużeniem oka, gdyż dla każdego z nas czas ten będzie inny.

Poniższy kod zawiera znaną Ci konstrukcję try-catch, aby obsłużyć zgłoszone wyjątki, czyli po prostu jakieś problemy, które mogą się pojawić podczas połączenia z bazą danych.

SQLite

LISTING 34.1	PHP 7/8	code_34_01.php
01	<?php	
02	try {	
03	\$db_file = 'kartoteka.db';	
04		
05	<i>// otwórz połączenie z bazą SQLite lub utwórz nowy plik dla bazy</i>	
06	\$db_pdo = new PDO('sqlite:'.__DIR__.'/'.\$db_file);	
07		
08	<i>// włącz obsługę wyjątków dla obsługi interfejsu PDO</i>	
09	\$db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);	
10		
11	print "Ile potrzebuję czasu na przeczytanie książki?".PHP_EOL;	
12	print "AUTOR\t\tTYTUŁ\t\t\tCZAS".PHP_EOL;	
13		
14	<i>// wyślij zapytanie do bazy i pobierz wyniki</i>	
15	\$wynik = \$db_pdo->query("	
16	SELECT (((stron * 3) / 60) + 1) AS czas, autor, tytuł	
17	FROM ksiazki	
18	ORDER BY stron DESC;	
19	");	
20		
21	<i>// przetwórz pobrane rekordy i wydrukuj</i>	
22	while (\$wiersz = \$wynik->fetch()) {	
23	printf(	
24	"%s\t%s\t%d godz.".PHP_EOL,	
25	\$wiersz['autor'], \$wiersz['tytuł'], \$wiersz['czas']	

```

26         );
27     }
28
29     $db_pdo = NULL; //zamknij połączenie z bazą
30
31 } catch (PDOException $e) {
32     print "Błąd!: " . $e->getMessage();
33     die();
34 }

```

```
$ php code_34_01.php ↵
```

KONSOLA

```

Ile potrzebuję czasu na przeczytanie książki?
AUTOR      TYTUŁ      CZAS
J.R.R. Tolkien Władca Pierścieni 65 godz.
Stephen King  Mroczna Wieża VII 39 godz.
Dan Brown    Kod Leonarda Da Vinci 23 godz.
J.D. Salinger Buszujący w zbożu   16 godz.

```

Jak działa powyższy kod?

1. W linii 06 otwierane jest połączenie z bazą danych. Następnie wywoływane jest polecenie SELECT, zdefiniowane w wierszach od 16 do 18.
2. Zwróć uwagę na operację arytmetyczną obliczającą czas potrzebny na przeczytanie książki, wyrażony w godzinach (linia 16). Po prostu liczba stron mnożona jest przez 3, a następnie dzielona przez 60, na samym końcu zaś dodajemy 1, aby wynik reprezentował pełne godziny po zaokrągleniu.
3. Wyniki drukowane są w konsoli (wiersze od 22 do 27). Połączenie z bazą danych zamykane jest w linii 29 i program kończy swoje działanie.

MySQL

LISTING 34.2

PHP 7/8

code_34_02.php

```

01 <?php
02 try {
03     /* Konfiguracja serwera MySQL */
04     $db_server = 'localhost';
05     $db_name   = 'db_php8';
06     $db_user   = 'db_php8';
07     $db_passwd = 'WETFD8toJyKLWtXC'; //u Ciebie będzie z pewnością inne :)
08
09     //otwórz połączenie z bazą danych MYSQL
10     $db_pdo = new PDO(
11         "mysql:host=$db_server;dbname=$db_name",
12         $db_user,

```

```

13      $db_passwd,
14      [PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8"]
15  );
16
17  // włącz obsługę wyjątków dla obsługi interfejsu PDO
18  $db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
19
20  print "Ile potrzebuję czasu na przeczytanie książki?".PHP_EOL;
21  print "AUTOR\t\tTYTUŁ\t\t\tCZAS".PHP_EOL;
22
23  // wyślij zapytanie do bazy i pobierz wyniki
24  $wynik = $db_pdo->query("
25  SELECT (((stron * 3) / 60) + 1) AS czas, autor, tytuł
26  FROM ksiazki
27  ORDER BY stron DESC;
28  ");
29
30  // przetwórz pobrane rekordy i wydrukuj
31  while ($wiersz = $wynik->fetch()) {
32      printf(
33          "%s\t%s\t%d godz.".PHP_EOL,
34          $wiersz['autor'], $wiersz['tytuł'], $wiersz['czas']
35      );
36  }
37
38  $db_pdo = NULL; // zamknij połączenie z bazą
39
40  } catch (PDOException $e) {
41      print "Błąd!: " . $e->getMessage();
42      die();
43  }

```

```
$ php code_34_02.php ↵
```

KONSOLA

```

Ile potrzebuję czasu na przeczytanie książki?
AUTOR      TYTUŁ      CZAS
J.R.R. Tolkien Władca Pierścieni 65 godz.
Stephen King  Mroczna Wieża VII 39 godz.
Dan Brown    Kod Leonarda Da Vinci 23 godz.
J.D. Salinger Buszujący w zbożu 16 godz.

```

Ćwiczenie 2. Agregacja danych

A teraz proste ćwiczenie, w którym zastosujesz funkcje agregujące na tabeli reprezentującej książki (listingi 34.3 i 34.4). Pobierzesz liczbę rekordów w tabeli, oznaczającą w naszym przypadku liczbę książek, a także sumę wszystkich stron, średnią wartość

stron przypadających na jedną książkę, jak również informacje o roku najstarszego i najnowszego wydania. Przy większych zbiorach danych informacje statystyczne mogą mieć bardzo duże znaczenie zarówno dla biblioteki, jak i czytelnika.

SQLite

LISTING 34.3

PHP 7/8

code_34_03.php

```

01 <?php
02 try {
03     $db_file = 'kartoteka.db';
04
05     // otwórz połączenie z bazą SQLite lub utwórz nowy plik dla bazy
06     $db_pdo = new PDO('sqlite:'.__DIR__.'/'.$db_file);
07
08     // włącz obsługę wyjątków dla obsługi interfejsu PDO
09     $db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
10
11     print "STATYSTYKA TABELI Z KSIĄŻKAMI".PHP_EOL;
12
13     // wyślij zapytanie do bazy i pobierz wyniki
14     $wynik = $db_pdo->query("
15     SELECT
16     count(*) AS liczba,
17     SUM(stron) AS suma,
18     AVG(stron) AS srednia,
19     MIN(rok) AS min,
20     MAX(rok) AS max
21     FROM ksiazki;
22     ")->fetch();
23
24     $db_pdo = NULL; // zamknij połączenie z bazą
25
26     // wydrukuj wyniki
27     printf(
28         "Liczba książek w tabeli: %d".PHP_EOL.
29         "Suma wszystkich stron: %d".PHP_EOL.
30         "Średnia liczba stron w książce: %d".PHP_EOL.
31         "Najstarsze wydanie: %d rok".PHP_EOL.
32         "Najnowsze wydanie: %d rok".PHP_EOL,
33         $wynik['liczba'], $wynik['suma'], $wynik['srednia'],
34         $wynik['min'], $wynik['max']
35     );
36 } catch (PDOException $e) {
37     print "Błąd!: " . $e->getMessage();
38     die();
39 }

```

```
$ php code_34_03.php ↵
```

KONSOLA

```
STATYSTYKA TABELI Z KSIĄŻKAMI
Liczba książek w tabeli: 3
Suma wszystkich stron: 305
Średnia liczba stron w książce: 101
Najstarsze wydanie: 2015 rok
Najnowsze wydanie: 2021 rok
```

Jak działa powyższy kod?

1. W linii 06 otwierane jest połączenie z bazą danych.
2. Następnie wywoływane jest polecenie SELECT, zdefiniowane w wierszach od 15 do 21, zawierające funkcje agregujące. Zwracane informacje wstawiane są do zmiennej tabelarycznej \$wynik (linia 14). Przeanalizuj dokładnie to zapytanie i postaraj się je zrozumieć.
3. Połączenie z bazą danych zamykane jest w wierszu 24.
4. Wyniki drukowane są w konsoli (linie od 27 do 35) i program kończy swoje działanie.

MySQL

LISTING 34.4

PHP 7/8

code_34_04.php

```
01 <?php
02 try {
03     /* Konfiguracja serwera MySQL */
04     $db_server = 'localhost';
05     $db_name   = 'db_php8';
06     $db_user   = 'db_php8';
07     $db_passwd = 'WETFD8toJyKLWtXC'; // u Ciebie będzie z pewnością inne :)
08
09     // otwórz połączenie z bazą danych MYSQL
10     $db_pdo = new PDO(
11         "mysql:host=$db_server;dbname=$db_name",
12         $db_user,
13         $db_passwd,
14         [PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8"]
15     );
16
17     // włącz obsługę wyjątków dla obsługi interfejsu PDO
18     $db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
19
20     print "STATYSTYKA TABELI Z KSIĄŻKAMI".PHP_EOL;
```

```

21
22     // wyślij zapytanie do bazy i pobierz wyniki
23     $wynik = $db_pdo->query("
24     SELECT
25     count(*) AS liczba,
26     SUM(stron) AS suma,
27     AVG(stron) AS srednia,
28     MIN(rok) AS min,
29     MAX(rok) AS max
30     FROM ksiazki;
31     ")->fetch();
32
33     $db_pdo = NULL; // zamknij połączenie z bazą
34
35     // wydrukuj wyniki
36     printf(
37         "Liczba książek w tabeli: %d".PHP_EOL.
38         "Suma wszystkich stron: %d".PHP_EOL.
39         "Średnia liczba stron w książce: %d".PHP_EOL.
40         "Najstarsze wydanie: %d rok".PHP_EOL.
41         "Najnowsze wydanie: %d rok".PHP_EOL,
42         $wynik['liczba'], $wynik['suma'], $wynik['srednia'],
43         $wynik['min'], $wynik['max']
44     );
45 } catch (PDOException $e) {
46     print "Błąd!: " . $e->getMessage();
47     die();
48 }

```

```
$ php code_34_04.php ↵
```

KONSOLA

```

STATYSTYKA TABELI Z KSIĄŻKAMI
Liczba książek w tabeli: 4
Suma wszystkich stron: 2808
Średnia liczba stron w książce: 702
Najstarsze wydanie: 2012 rok
Najnowsze wydanie: 2021 rok

```

Jak działa powyższy kod?

1. W linii 10 otwierane jest połączenie z bazą danych.
2. Następnie wywoływane jest polecenie SELECT, zdefiniowane w wierszach od 24 do 30, zawierające funkcje agregujące. Zwracane informacje wstawiane są do zmiennej tabelarycznej \$wynik (linia 23). Przeanalizuj dokładnie to zapytanie i postaraj się je zrozumieć.

3. Połączenie z bazą danych zamykane jest w wierszu 33.
4. Wyniki drukowane są w konsoli (linie od 36 do 44) i program kończy swoje działanie.

Ćwiczenia dodatkowe

1. Poszukaj więcej informacji na temat funkcji agregujących w języku SQL, a w szczególności o klauzulach GROUP BY oraz HAVING.
2. Poszukaj informacji na temat wyzwalaczy (ang. *triggers*) w języku SQL.



Rozmówki klienta z serwerem

Pojęcie „serwer” z pewnością jest Ci doskonale znane. Wyróżniamy choćby serwer pocztowy, serwer WWW, serwer plików (popularna chmura), serwer aplikacji czy komunikacji online. Choć nazwa „serwer” może się kojarzyć bezpośrednio z fizycznym sprzętem komputerowym, to w rzeczywistości są to bardzo skomplikowane i rozbudowane programy działające właśnie na komputerach. Popularne serwery WWW, takie jak Apache, Microsoft i nginx, świetnie współpracują z programami napisanymi w języku PHP i popularnymi bazami danych (MySQL, PostgreSQL czy SQLite).

W tej lekcji zaprojektujesz i uruchomisz program pełniący funkcję serwera i program pełniący funkcję klienta, który będzie się łączył z tym serwerem i pobierał z niego dane. Pisząc program, wykorzystasz specjalistyczne funkcje sieciowe obsługujące strumienie (ang. *streams*) w języku PHP. Z pojęciami serwera i klienta wiążą się pojęcia hosta (adres IP serwera, komputera) i portu, na którym serwer będzie „nasłuchiwał” w oczekiwaniu na połączenia klientów. Klasycznym przykładem ułatwiającym zrozumienie, czym są host i port, jest sytuacja, kiedy w przeglądarce wpisujesz adres, np. `https://helion.pl:443/`, co dokładnie oznacza, że przeglądarka otwiera połączenie z serwerem WWW o adresie `helion.pl` na porcie 443. Poniższa tabela przedstawia numery portów najpopularniejszych usług sieciowych, takich jak poczta, WWW, FTP.

Port	Protokół
21	FTP (serwer FTP)
22	SSH (szyfrowane połączenie ze zdalnym komputerem)
25	SMTP (serwer poczty wychodzącej, port niezalecany)
80	HTTP (serwer WWW)
110	POP3 (serwer poczty przychodzącej)
143	IMAP (serwer pocztowy)
443	HTTPS (serwer WWW, połączenie szyfrowane)
465	SMTP (serwer poczty wychodzącej, połączenie szyfrowane)
587	SMTP (serwer poczty wychodzącej, port zalecany)

Port	Protokół
993	IMAP (serwer poczty, połączenie szyfrowane)
995	POP3 (serwer poczty przychodzącej, połączenie szyfrowane)
3306	MySQL (serwer baz danych)
5432	PostgreSQL (serwer baz danych)

W języku PHP możesz bez problemu użyć funkcji sieciowych do budowy programu klienckiego dla popularnych usług sieciowych, np. możesz napisać program odbierający pocztę z serwera za pomocą protokołu POP3.

W tym ćwiczeniu wykorzystasz dwie konsole systemowe jednocześnie, jedną dla serwera, a drugą dla klienta. W pierwszej kolejności przepisuj dokładnie kody źródłowe (serwera i klienta) i zapisz na dysku, odpowiednio w plikach `code_35_01.php` i `code_35_02.php`. Pamiętaj, że program kliencki nie zadziała, jeśli program serwera nie będzie wcześniej uruchomiony.

Po zapisaniu na dysku kodu programu z listingów 35.1 i 35.2 naciśnij klawisze **⌘ + R** i wpisz `cmd`. Po uruchomieniu się konsoli sprawdź, czy znajdujesz się w katalogu, w którym zapisane są pliki klienta i serwera. Jeśli pliki z programami z niniejszej książki zapisujesz w katalogu `c:\xampp\apps\php8\htdocs`, wpisz w konsoli polecenie:

```
cd /d c:\xampp\apps\php8\htdocs ↵
```

Wykonaj ponownie powyższe czynności dla drugiego okna konsoli. Pamiętaj, abyś w obu oknach znajdował się w tym samym miejscu na dysku.

W pierwszym oknie uruchom program serwera:

```
php code_35_01.php ↵
```

Po uruchomieniu się serwera w konsoli pojawi się napis:

```
<> Uruchamiam serwer...
Gotowe!
Limit połączeń: 2
```

W drugim oknie uruchom program klienta:

```
php code_35_02.php wersja-php ↵
```

Klient połączy się z serwerem, pobierze informację o aktualnej wersji języka PHP i wyświetli ją w konsoli:

```
Wersja PHP: 8.0.3
```

W konsoli z uruchomionym serwerem zostanie wyświetlona dodatkowa informacja:

```
[1] Zaakceptowane połączenie z klientem.
<= Odebrałem wiadomość...
<?> Przygotowuję informację o wersji PHP...
=> Wysłałem wiadomość...
```

```

c:\xampp\apps\php8\htdocs>php code_35_01.php
<> Uruchamiam serwer...
Gotowe!
Limit połączeń: 2
[1] Zaakceptowane połączenie z klientem.
<= Odebrałem wiadomość...
<?> Przygotowuję informację o wersji PHP...
=> Wysłałem wiadomość...
[2] Zaakceptowane połączenie z klientem.
<= Odebrałem wiadomość...
<?> Przygotowuję informację o aktualnym czasie...
=> Wysłałem wiadomość...
[*] Przekroczono limit połączeń.
<> Wyłączam serwer...
Gotowe!
c:\xampp\apps\php8\htdocs>

c:\xampp\apps\php8\htdocs>php code_35_02.php wersja php
<> Nawiązuję połączenie z serwerem...
Wersja PHP: 8.0.3
c:\xampp\apps\php8\htdocs>php code_35_02.php czas
<> Nawiązuję połączenie z serwerem...
Czas lokalny: 2021-07-21 09:58:09
c:\xampp\apps\php8\htdocs>

```

Rysunek 35.1. Praktyczne wykorzystanie programu klienta i serwera w konsoli systemowej

Czyli działa! Bez problemu możesz uzyskać z serwera informację o aktualnym czasie. W tym celu wywołaj polecenie w oknie z klientem:

```
php code_35_02.php czas ↵
```

W ten oto sposób uruchomiłeś oprogramowanie działające po stronie serwera i po stronie klienta jako niezależne procesy komunikujące się za pomocą protokołu sieciowego. Nawet tak prosty kod możesz uruchomić na dwóch bardzo oddalonych od siebie komputerach (np. w różnych krajach) — i też będzie działać. Wystarczy wprowadzić do programu odpowiedni adres hosta i numer portu.

Teraz kilka słów o kodzie serwera i klienta. Do obsługi połączeń sieciowych wykorzystałem bibliotekę Streams, wbudowaną domyślnie w język PHP. Zanim objaśnię, jak to działa, proszę, abyś w pierwszej kolejności samodzielnie przeanalizował kod źródłowy linia po linii.

Serwer

W programie serwera (listing 35.1) w liniach od 03 do 05 umieściłem funkcje definiujące czas działania programu i czas oczekiwania na połączenie klienckie. Program pełniący funkcję serwera czeka w pętli while (wiersze od 26 do 63), dlatego niezbędne jest zdefiniowanie czasu jego działania. W liniach 08 i 09 zdefiniowałem adres hosta (domyślnie localhost) i numer portu, pod którym serwer będzie

„nasłuchiwał”. Dla bezpieczeństwa wprowadziłem limit obsługiwanych połączeń (wiersz 11), który możesz dowolnie zmienić.

Po przekroczeniu limitu serwer kończy działanie. W linii 17 następuje wywołanie funkcji `stream_socket_server`, odpowiedzialnej za uruchomienie serwera i przejście w stan „nasłuchu”. W wierszu 26, po wykryciu połączenia przychodzącego, funkcja `stream_socket_accept` przypisuje do zmiennej `$socket` obiekt będący odwołaniem do klienta. W linii 30 za pomocą funkcji `stream_socket_recvfrom` odczytywana jest wiadomość nadana przez klienta, która następnie jest przetwarzana (wiersze od 33 do 48). Zwróć uwagę, że klient może wysłać dowolny tekst, który zostanie mu zwrócony na zasadzie echa, lecz dla specyficznych wyrażeń (ciągów znaków), takich jak `czas` i `wersja-php`, serwer przygotowuje inną odpowiedź (przeanalizuj dokładnie ten fragment kodu). Po przygotowaniu odpowiedzi serwer wysyła ją za pomocą funkcji `stream_socket_sendto` (linia 50) i zamyka połączenie z klientem (wiersz 54). W liniach od 57 do 62 sprawdzana jest liczba obsługiwanych połączeń klientów i w przypadku przekroczenia limitu serwer kończy swoje działanie (wiersz 68).

LISTING 35.1	PHP 7/8	code_35_01.php
01	<?php	
02	<i>/* optymalizacja działania serwera */</i>	
03	set_time_limit (600); <i>// program będzie działał co najmniej 10 minut</i>	
04	ini_set ("default_socket_timeout", 300); <i>// oczekuj najwyżej 5 minut na połączenie</i>	
05	ob_implicit_flush (); <i>// wyświetl informacje trafiające do bufora w konsoli</i>	
06		
07	<i>/* parametry połączenia */</i>	
08	<code>\$host = 'localhost';</code>	
09	<code>\$port = 12345;</code>	
10		
11	<code>\$limit = 2;</code> <i>// limit przyjętych połączeń</i>	
12	<code>\$cnt = 1;</code> <i>// licznik połączeń</i>	
13		
14	<code>print "<> Uruchamiam serwer...".PHP_EOL;</code>	
15		
16	<i>// utwórz serwer</i>	
17	<code>if ((\$connect = stream_socket_server('tcp://'.\$host.':'.\$port)) === false)</code>	
18	<code>{</code>	
19	<code> die('Nie mogę uruchomić serwera.');</code>	
20	<code>}</code>	
21		
22	<code>print "Gotowe!".PHP_EOL;</code>	
23	<code>print "Limit połączeń: \$limit".PHP_EOL;</code>	
24		

```

25 //zaakceptuj połączenie przychodzące
26 while ($socket = stream_socket_accept($connect))
27 {
28     print "[$cnt] Zaakceptowane połączenie z klientem.".PHP_EOL;
29
30     $read = stream_socket_recvfrom($socket, 1500, 0, $sender);
31     if (empty($read) === false)
32     {
33         print "<= Odebrałem wiadomość...".PHP_EOL;
34         $response = '';
35
36         switch ($read)
37         {
38             case 'czas':
39                 $response = 'Czas lokalny: ' . date('Y-m-d H:i:s');
40                 print "<> Przygotowuję informację o aktualnym czasie...".PHP_EOL;
41                 break;
42             case 'wersja-php':
43                 $response = 'Wersja PHP: ' . phpversion();
44                 print "<> Przygotowuję informację o wersji PHP...".PHP_EOL;
45                 break;
46             default:
47                 $response = 'Echo: '.$read;
48         }
49
50         stream_socket_sendto($socket, $response, 0, $sender);
51         print "=> Wysłałem wiadomość...".PHP_EOL;
52     }
53
54     fclose($socket);
55     sleep(1); //poczekaj chwilkę na kolejne połączenie
56
57     $cnt++;
58     if ($cnt > $limit) //sprawdź limit połączeń
59     {
60         print "[*] Przekroczono limit połączeń.".PHP_EOL;
61         break;
62     }
63 }
64
65 print "<> Wyłączam serwer...".PHP_EOL;
66
67 //zamknij połączenie (serwera i klienta)
68 stream_socket_shutdown($connect, \STREAM_SHUT_RDWR);
69
70 print "Gotowe!".PHP_EOL;

```

```
$ php code_35_01.php ↵
```

KONSOLA

```
<> Uruchamiam serwer...
Gotowe!
Limit połączeń: 2
[1] Zaakceptowane połączenie z klientem.
<= Odebrałem wiadomość...
<?> Przygotowuję informację o wersji PHP...
=> Wysłałem wiadomość...
[2] Zaakceptowane połączenie z klientem.
<= Odebrałem wiadomość...
<?> Przygotowuję informację o aktualnym czasie...
=> Wysłałem wiadomość...
[*] Przekroczono limit połączeń.
<> Wyłączam serwer...
Gotowe!
```

Klient

W programie klienta (listing 35.2) w liniach 03 i 04 zdefiniowałem adres hosta i numer portu, pod którym klient ma się łączyć z serwerem. Wykorzystałem specjalną zmienną tablicową \$argv (wiersz 06), która przechowuje argumenty wywołanego programu.

Wywołanie w konsoli polecenia:

```
php code_35_02.php wersja-php ↵
```

przypisze zmiennej tablicowej \$argv następujące wartości:

```
$argv[0] = code_35_02.php
$argv[1] = wersja-php
```

W linii 11 funkcja stream_socket_client otwiera połączenie ze zdalnym serwerem. Po poprawnym połączeniu wysyłana jest wiadomość za pomocą funkcji stream_socket_sendto z jednoczesnym oczekiwaniem na odpowiedź serwera (wiersze od 17 do 21). Odebrana wiadomość jest drukowana w konsoli (linie 19 i 20). Klient kończy połączenie przez wywołanie funkcji stream_socket_shutdown (wiersz 24).

LISTING 35.2

PHP 7/8

code_35_02.php

```
01 <?php
02 /* parametry połączenia */
03 $host = 'localhost';
04 $port = 12345;
05
```

```

06 $command = (isset($argv[1])) ? trim($argv[1]) : 'czas';
07
08 print "<> Nawiązuję połączenie z serwerem...".PHP_EOL;
09
10 //utwórz klienta i połącz się z serwerem
11 if (($socket = stream_socket_client('tcp://'.$host.':'.$port)) === false)
12 {
13     die('Nie mogę uruchomić klienta.');
```

```

14 }
15
16 //wyślij wiadomość do serwera
17 if (stream_socket_sendto($socket, $command) !== false)
18 {
19     $response = fread($socket, 4096); //odbierz wiadomość z serwera
20     print $response;
21 }
22
23 //zamknij połączenie
24 stream_socket_shutdown($socket, STREAM_SHUT_RDWR);
```

```
$ php code_35_02.php ↵
```

KONSOLA

Wersja PHP: 8.0.3

Czas lokalny: 2021-03-21 09:58:09

Ćwiczenia dodatkowe

1. Otwórz stronę <https://www.php.net/manual/en/book.stream.php> i zapoznaj się z pozostałymi funkcjami sieciowymi biblioteki Streams. Spróbuj zrozumieć, jak działają. Przepisz i uruchom przykładowe listingi.
2. Jeśli masz dostęp do kilku komputerów w sieci lokalnej, uruchom program serwera na jednym komputerze, a na pozostałych uruchom program klienta. Pamiętaj, aby w kodzie źródłowym wprowadzić adres IP serwera.
3. Zadanie z gwiazdką. Spróbuj zmodyfikować kod źródłowy serwera, aby pełnił funkcję prostego kalkulatora. Klient wysyła np. „2+2”, a serwer dekoduje wiadomość i zwraca wynik.
4. Kolejne zadanie z gwiazdką. Napisz kod prostego serwera czatu, tak aby kilku klientów mogło między sobą prowadzić rozmowy.

Wskazówki

W języku PHP oprócz biblioteki Streams dostępna jest również dodatkowa biblioteka obsługująca tak zwane gniazda (ang. *sockets*), którą możesz wykorzystać do programowania sieciowego. Gniazdo sieciowe oznacza punkt do komunikacji dwustronnej, czyli do wysyłania i odbierania danych. Aby móc korzystać z biblioteki Sockets, należy ją aktywować w pliku konfiguracyjnym `php.ini`. Zajrzyj do pierwszych lekcji, w których dokładnie opisałem, jak można aktywować dodatkowe biblioteki programistyczne.

Otwórz w edytorze tekstu plik `php.ini` i odszukaj następującą linię (u mnie jest to linia 952):

```
;extension=sockets
```

a następnie usuń początkowy znak średnika i zapisz plik na dysku.

Na stronie <https://www.php.net/manual/en/book.sockets.php> znajdziesz wiele ciekawych przykładów wykorzystania biblioteki Sockets. Postaraj się je przepisać i uruchomić.

Pytania

Czym jest protokół sieciowy?

Protokół to zestaw ściśle określonych i przestrzeganych reguł wykorzystywanych w komunikacji między co najmniej dwoma elementami sieci. Klasycznym przykładem protokołu sieciowego jest HTTP, który używany jest do komunikacji między klientem (przeglądarką internetową) a serwerem (oprogramowaniem pełniącym funkcję serwera WWW). W sieci znajdziesz wiele informacji na ten temat, dlatego zachęcam Cię do samodzielnego poszerzenia wiedzy w tym zakresie. Tak z przymrużeniem oka: odpowiednikiem protokołu sieciowego w życiu codziennym może być *savoir-vivre*, który z pewnością warto znać i stosować na co dzień.



Serwer webowy w PHP

Dotarłeś do miejsca, w którym warto podsumować zdobytą wiedzę. Przerobiłeś już 35 lekcji, w których poznałeś podstawy programowania w języku PHP i nauczyłeś się stosować język SQL do obsługi baz danych. Wykonałeś ogromną pracę, za co Ci bardzo dziękuję. Gratuluję!

Dotychczas do prezentowania wyników działania programów używałeś konsoli systemowej. Umiejętność tę na pewno wykorzystasz podczas dalszej nauki programowania zarówno w języku PHP, jak i C++ czy Java, a także kiedy będziesz łączył się ze zdalnym serwerem i uruchamiał własne programy w linii poleceń. Masz za sobą swój pierwszy program w języku PHP, poznałeś, czym są zmienne i jak z nich korzystać, przećwiczyłeś projektowanie własnych funkcji i użyłeś pętli. Mam nadzieję, że dobrze opanowałeś tablice, operacje na plikach i przyswoiłeś zasady projektowania obiektowego. Jeśli zrozumiałeś znaczenie baz danych, a język SQL nie jest Ci już obcy, jesteś gotowy do wykorzystania zdobytej wiedzy w projekcie webowym.

Począwszy od wersji PHP 5.4, programista ma dostęp do wbudowanego serwera WWW, który doskonale nadaje się do przetestowania projektów bezpośrednio w przeglądarce internetowej. Dodatkowo pakiet XAMPP zawiera serwer Apache, który również możesz wykorzystać do testowania swoich aplikacji webowych.

W tej lekcji nauczysz się uruchamiać i testować serwer WWW. Jeśli będziesz miał kłopoty z uruchomieniem serwera, zajrzyj do wskazówek do niniejszej lekcji.

Ćwiczenie 1. Informacja o wersji języka PHP

Celem ćwiczenia jest wyświetlenie w konsoli lub przeglądarce szczegółowych informacji technicznych na temat wykorzystywanej wersji języka PHP. Popularną praktyką jest sprawdzenie wersji języka PHP na serwerze dystrybucyjnym, zanim gotowy projekt zostanie na niego wgrany. W pierwszej kolejności zapisz kod z listingu 36.1 w pliku na dysku.

Serwer PHP

Interpreter języka PHP umożliwia uruchomienie serwera WWW z poziomu linii poleceń w konsoli systemowej. Uruchamiając serwer, wskazujesz nazwę serwera (zazwyczaj localhost) i numer portu. Tym sposobem możesz uruchomić kilka niezależnych serwerów WWW i w każdym z nich testować inny projekt.

Naciśnij klawisze **Win + R** i wpisz `cmd`. Po otwarciu się okna przejdź do katalogu roboczego — może to być dowolny katalog, w którym masz zapisane pliki z kodem źródłowym.

Zakładając, że katalogiem roboczym jest, tak jak w przypadku serwera wirtualnego `php8.localhost`, `c:\xampp\apps\php8\htdocs`, wykonaj polecenie:

```
cd /d c:\xampp\apps\php8\htdocs ↵
```

Teraz uruchom serwer WWW poleceniem:

```
php -S localhost:8008 ↵
```

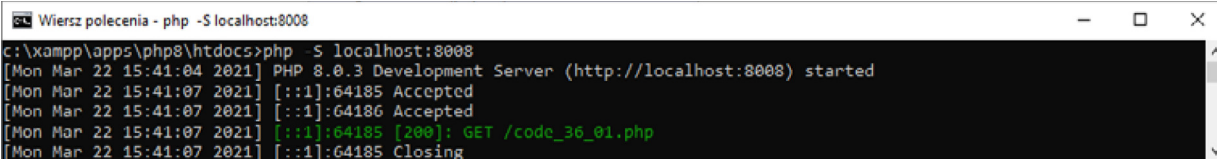
Po uruchomieniu się serwera w konsoli zostanie wyświetlony komunikat:

```
PHP 8.0.3 Development Server (http://localhost:8008) started
```

Brawo! Serwer działa!

Otwórz stronę `http://localhost:8008/code_36_01.php` w oknie swojej przeglądarki. Powinieneś od razu zauważyć aktualny numer wersji PHP oraz charakterystyczne logo (patrz rysunek 36.1).

Uruchomienie serwera z poziomu interpretera języka PHP przedstawia rysunek 36.2.



```
Wiersz polecenia - php -S localhost:8008
c:\xampp\apps\php8\htdocs>php -S localhost:8008
[Mon Mar 22 15:41:04 2021] PHP 8.0.3 Development Server (http://localhost:8008) started
[Mon Mar 22 15:41:07 2021] [::1]:64185 Accepted
[Mon Mar 22 15:41:07 2021] [::1]:64186 Accepted
[Mon Mar 22 15:41:07 2021] [::1]:64185 [200]: GET /code_36_01.php
[Mon Mar 22 15:41:07 2021] [::1]:64185 Closing
```

Rysunek 36.2. Uruchomienie serwera WWW z poziomu interpretera języka PHP

Ćwiczenie 2. Połączenie kodu PHP z HTML-em

W tym ćwiczeniu nauczysz się łączyć kod PHP i HTML w jednym pliku. Jak już wiesz, tekst, który nie znajduje się między znacznikiem otwierającym `<?php` a znacznikiem zamykającym `?>`, jest przez interpreter PHP ignorowany i przepisywany na wyjście,

czyli wyświetlany w konsoli lub przeglądarce. Właśnie tę funkcjonalność wykorzystasz do napisania swojej pierwszej strony internetowej. Rozpocznij ćwiczenie od zapisania poniższego kodu w pliku na dysku.

Listing 36.2 zawiera obsługę bazy SQLite, ale na podstawie wcześniejszych ćwiczeń bez problemu możesz poprawić kod, dodając obsługę bazy MySQL. Zwróć uwagę, że kod w pierwszej linii nie rozpoczyna się znacznikiem otwierającym `<?php`, znacznik ten pojawia się dopiero w wierszu 09. W linii 25 drukowane są znaczniki HTML, aby przeglądarka mogła poprawnie zaprezentować dane.

Przepisz poniższy listing i zapisz w katalogu roboczym. Uruchom pakiet XAMPP i otwórz stronę `http://php8.localhost/code_36_02.php` w oknie przeglądarki. Jeśli chcesz wykorzystać serwer wbudowany w interpreter języka PHP, naciśnij klawisze **⌘ + R** i wpisz `cmd`. Po otwarciu się okna uruchom serwer poleceniem:

```
php -S localhost:8008 -t c:\xampp\apps\php8\htdocs .\
```

Tym razem za pomocą parametru `-t` wskazałem serwerowi WWW, gdzie znajduje się katalog roboczy. Po uruchomieniu się serwera otwórz stronę `http://localhost:8008/code_36_02.php` w oknie przeglądarki.

LISTING 36.2	PHP 7/8	code_36_02.php
01	<!doctype html>	
02	<head>	
03	<meta charset="utf-8">	
04	<title>Domowa biblioteka</title>	
05	</head>	
06	<body>	
07	Lista książek	
08		
09	<?php	
10	try {	
11	\$db_file = 'kartoteka.db';	
12		
13	<i>// otwórz połączenie z bazą SQLite lub utwórz nowy plik dla bazy</i>	
14	\$db_pdo = new PDO('sqlite:'.__DIR__.'/'.\$db_file);	
15		
16	<i>// włącz obsługę wyjątków dla obsługi interfejsu PDO</i>	
17	\$db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);	
18		
19	<i>/* przygotowanie zapytania SQL */</i>	
20	\$wynik = \$db_pdo->query("SELECT autor, tytul FROM ksiazki ORDER BY id ASC;");	
21		

```

22 //przetwórz pobrane rekordy i wydrukuj
23 while ($wiersz = $wynik->fetch()) {
24     printf(
25         "<li>%s, \"%s\"</li>".PHP_EOL,
26         $wiersz['autor'], $wiersz['tytul']
27     );
28 }
29
30 $db_pdo = NULL; //zamknij połączenie z bazą
31
32 } catch (PDOException $e) {
33     print "Błąd!: " . $e->getMessage();
34     die();
35 }
36 ?>
37 </ol>
38 </body>
39 </html>

```

http://php8.localhost/code_36_02.php

PRZEGLĄDARKA

Program wygeneruje i prześle do przeglądarki następujący kod HTML:

```

<!doctype html>
<head>
<meta charset="utf-8">
<title>Domowa biblioteka</title>
</head>
<body>
<h2>Lista książek</h2>
<ol>
<li>J.R.R. Tolkien, "Władca Pierścieni"</li>
<li>Dan Brown, "Kod Leonarda Da Vinci"</li>
<li>J.D. Salinger, "Buszujący w zbożu"</li>
<li>Stephen King, "Mroczna Wieża VII"</li>
</ol>
</body>
</html>

```

Przesłany kod HTML zostanie zrenderowany w przeglądarce (patrz rysunek 36.3).



Rysunek 36.3. Prezentacja danych pochodzących z bazy SQLite w języku PHP

Zauważ, że karta w przeglądarce otrzymała nazwę *Domowa biblioteka*, a w oknie zostały wyświetlone rekordy z bazy danych *kartoteka.db*. Przeanalizuj dokładnie kod źródłowy; zwróć uwagę, jak połączony jest kod PHP i HTML.

Ćwiczenia dodatkowe

1. Wejdź na stronę <https://www.php.net/manual/en/features.commandline.webserver.php> i zapoznaj się ze szczegółowym opisem wbudowanego serwera WWW.
2. Na początku lat 90. ubiegłego wieku powstała pierwsza strona internetowa — poszukaj w sieci więcej informacji na ten temat. Kto był autorem i dlaczego powstała?

Wskazówki

Wbudowany w PHP serwer WWW jest przeznaczony dla programisty w celach testowych; możesz się spotkać z określeniem, że jest to wersja deweloperska. Oznacza to, że w praktyce wykorzystasz go głównie do szybkiego sprawdzenia poprawności działania kodu w przeglądarce internetowej. To tyle i aż tyle. Kompletny projekt, który będziesz chciał udostępnić publicznie, będziesz już instalował na serwerze docelowym, czyli produkcyjnym. Serwery tego typu zapewniają znacznie więcej możliwości dla programisty i potrafią poradzić sobie z bardzo dużym obciążeniem, to znaczy wtedy, kiedy duża liczba internautów odwiedzi Twoją stronę internetową w tym samym czasie.

Pamiętaj, aby dodać ścieżkę `c:\xampp\php` w zmiennej środowiskowej `PATH`. Ułatwi to dostęp do interpretera PHP w linii poleceń (konsoli) po wpisaniu tylko nazwy `php`. Jeśli z jakichś przyczyn nie jesteś w stanie tego zrobić, serwer WWW w wierszu poleceń powinien być uruchomiony w następujący sposób:

```
c:\xampp\php\php -S localhost:8008 -t c:\xampp\apps\php8\htdocs ↵
```

Jeśli powyższe polecenie nie uruchomi serwera, to najprawdopodobniej błędnie wskazałeś położenie pliku interpretera PHP lub katalogu roboczego. Spróbuj zatem sprawdzić, gdzie dokładnie znajduje się na dysku plik `php.exe`, a także czy prawidłowo wskazałeś katalog roboczy.

Poprawnie uruchomiony serwer WWW wydrukuje w konsoli następujące informacje:

```
PHP 8.0.3 Development Server (http://localhost:8008) started
```

Po uruchomieniu w przeglądarce adresu *http://localhost:8008* serwer domyślnie będzie szukał pliku o nazwie `index.php`, a jeśli go nie odnajdzie, wyświetli komunikat o błędzie *Not Found*. Plik `index.php` z reguły jest plikiem startowym każdego projektu, dlatego umieszcza się go w katalogu głównym (ang. *document root*). Dla zobrazowania sobie, jak to działa, zapisz kod z ćwiczenia 2 na dysku, w katalogu `c:\xampp\apps\php8\htdocs`, pod nazwą `index.php`, a następnie uruchom w przeglądarce adres *http://localhost:8008*.



HTML dla zielonych

Budując projekty webowe, z pewnością będziesz miał do czynienia z HTML-em, czyli z językiem znaczników, za pomocą których będziesz nadawał stronom WWW indywidualny wygląd. Choć rozwinięcie skrótu HTML (ang. *HyperText Markup Language*) po polsku oznacza hipertekstowy język znaczników, nie jest to język programowania (jak PHP, Python, C++ czy Java). HTML-em opisujesz dokument hipertekstowy, który z kolei jest zrozumiały dla przeglądarek internetowych. Obecnie standardem jest HTML5, który rozwiązuje szereg problemów technicznych i pozwala projektantowi osadzić w dokumencie nie tylko tekst, ale również materiały audio, wideo czy grafikę wektorową.

Dzisiejsze projekty wykorzystują wiele technologii jednocześnie; bardzo często strony internetowe budowane są z użyciem języków PHP, HTML5, CSS i JavaScript. Na temat każdej z wymienionych technologii wydano wiele ciekawych książek, do których lektury bardzo gorąco Cię zachęcam. W tej lekcji poznasz podstawowe znaczniki HTML, które zastosujesz do budowy pierwszych stron WWW.

Na listingu 37.1 przedstawiłem szkielet dokumentu hipertekstowego w języku HTML:

LISTING 37.1	HTML5	code_37_01.html
01	<!doctype html>	
02	<html>	
03	<head>	
04	<meta charset="utf-8">	
05	<meta name="description" content="Opis strony">	
06	<title>Tytuł strony</title>	
07	</head>	
08	<body>	
09	Treść strony	
10	</body>	
11	</html>	
http://php8.localhost/code_37_01.html ↵		PRZEGLĄDARKA

Co oznacza powyższy kod?

1. Pierwsza linia oznacza, że dokument jest zakodowany w HTML5, który dzisiaj jest już standardem. W kolejnych liniach znajdują się znaczniki, które występują w dwóch wariantach, otwierające i zamykające, np. `<html>` i `</html>`, oraz tylko otwierające, np. `<meta>`. Zwróć uwagę na hierarchiczny charakter znaczników, np. znacznik `<title>` jest w sekcji `<head>`, która z kolei jest w sekcji `<html>`.
2. Dokument hipertekstowy zawiera nagłówek (sekcję `head` — linie od 03 do 07), który jest przeznaczony do definiowania parametrów, takich jak typ kodowania (wiersz 04), tytuł dokumentu (wiersz 06), a także ważny z punktu widzenia wyszukiwarek opis strony (wiersz 05).
3. Najważniejszą częścią dokumentu HTML jest jego właściwa treść, która jest umieszczona wewnątrz znacznika `<body>` (linie od 08 do 10). To właśnie w tym miejscu dokumentu hipertekstowego zawarte są wszystkie znaczniki odpowiedzialne za prezentację tekstu, obrazów i plików multimedialnych. Zazwyczaj kod HTML zapisywany jest w plikach tekstowych z rozszerzeniem `.htm` lub `.html`.

W poniższej tabeli zawarłem najważniejsze znaczniki HTML²³, od których możesz zacząć budowanie swoich pierwszych dokumentów hipertekstowych.

Znacznik	Opis
<code><title></code>	tytuł dokumentu
<code><meta></code>	dodatkowe informacje o dokumencie
<code><div></code>	warstwa, sekcja, grupowanie elementów
<code><p></code>	akapit
<code>
</code>	łamanie wiersza
<code></code>	pogrubienie
<code></code>	kursywa
<code><hr></code>	linia pozioma
<code><h1> ... </h6></code>	nagłówki: najpierw najważniejszy, h1, następnie h2 – h6
<code></code>	obraz
<code><a></code>	hiperłącza (odnośniki do innych dokumentów)
<code></code> i <code></code>	elementy w formie listy, np. spis książek
<code><table></code>	tabela
<code><form></code>	formularz

²³ HTML, <https://html.spec.whatwg.org> [dostęp: 26.05.2021].

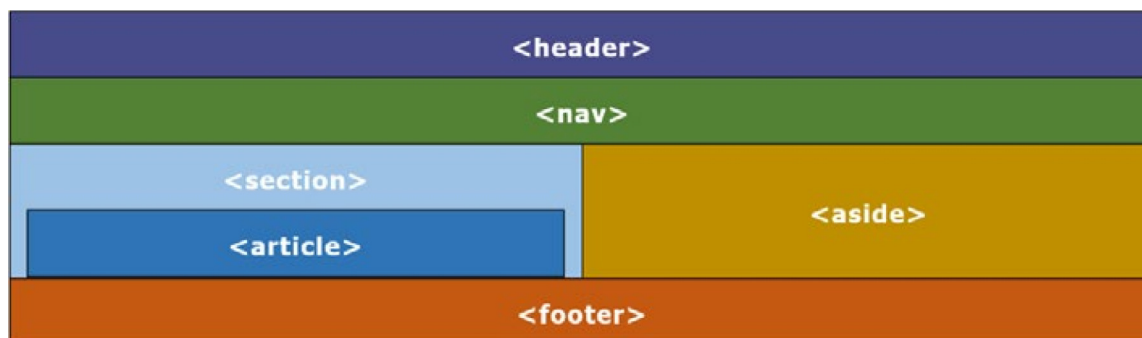
W następniej tabeli umieściłem kilka wybranych znaczników HTML5²⁴.

Znacznik	Opis
<header>	nagłówek dokumentu
<section>	sekcja dokumentu
<article>	treść uznana za artykuł, np. wpis na blogu
<figure>	obraz lub rysunek
<nav>	zbiór hiperłączy w jednym miejscu, np. menu
<main>	główna część dokumentu
<aside>	dodatkowa treść, odbiegająca od głównego tematu
<footer>	stopka dokumentu

Szkielet dokumentu hipertekstowego w języku HTML5:

```
<!doctype html>
<html>
  <head>...</head>
  <body>
    <header>...</header>
    <nav>...</nav>
    <section>
      <article>...</article>
    </section>
    <aside>...</aside>
    <footer>...</footer>
  </body>
</html>
```

Poniżej zamieszczam wizualizację struktury dokumentu z wykorzystaniem znaczników HTML5 (rysunek 37.1).



Rysunek 37.1. Struktura dokumentu z wykorzystaniem znaczników HTML5

²⁴ HTML 5.3, <https://www.w3.org/TR/html53/> [dostęp: 26.05.2021].

Jak korzystać ze znaczników HTML

Tytuł dokumentu — znacznik <title>:

```
<title>Może kilka słów o programistach</title>
```

Dodatkowe informacje o dokumencie — znacznik <meta>:

```
<meta name="description" content="Opis strony">
<meta name="author" content="Jan Kowalski">
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Warstwa, sekcja, grupowanie elementów — znacznik <div>.

Akapit — znacznik <p>:

```
<div style="text-align: right;">
<p>Ten tekst będzie wyświetlany z prawej strony ekranu.</p>
</div>
```

Łamanie wiersza, przejście do nowej linii — znacznik
:

```
<p>
Wiersz 1<br>
Wiersz 2<br>
Wiersz 3
</p>
```

Pogrubienie i kursywa — znaczniki i :

```
<p>
Ten tekst jest <strong>pogrubiony</strong>,<br>
a ten w nowej linii pisany jest <em>kursywą</em>.
</p>
```

Pozioma linia — znacznik <hr>:

```
<p>
Tekst rozdzielony poziomą linią
<hr>
wygląda właśnie tak!
</p>
```

Nagłówki — znaczniki od <h1> do <h6>:

```
<h1>Tytuł strony</h1>
<p>Blok tekstu</p>
<h2>Kolejny nagłówek, ale już mniej ważny.</h2>
<p>Kolejny blok tekstu</p>
```

Obrazy — znacznik :

```

```

Znacznik `<img>` ma następujące atrybuty:

- `src` — adres pliku graficznego,
- `width` — szerokość obrazu,
- `height` — wysokość obrazu,
- `alt` — informacja dodatkowa o obrazie.

Hiperłącza, odnośniki do innych dokumentów i stron — znacznik `<a>`:

- tekstowy:

```
<a href="https://twojastrona.pl/">
Zapraszam na moją stronę!
</a>
```

- graficzny:

```
<a href="https://twojastrona.pl/">

</a>
```

Listy uporządkowane (numerowane) — znacznik `<ol>`:

```
<ol> → znacznik listy
  <li>pierwszy</li> → znacznik elementu listy
  <li>drugi</li>
  <li>trzeci</li>
</ol>
```

Listy nieuporządkowane (elementy listy oznaczone znakiem, np. kropką) — znacznik `<ul>`:

```
<ul> → znacznik listy
  <li>pierwszy</li> → znacznik elementu listy
  <li>drugi</li>
  <li>trzeci</li>
</ul>
```

Tabele — znaczniki `<table>`, `<tr>`, `<th>`, `<td>`, `<tr>`:

```
<table> → znacznik tabeli
  <tr> → znacznik wiersza
    <th>Imię ucznia</th> → znacznik komórki
    <th>Ocena</th>
  </tr>
  <tr>
    <td>Janek</td> → znacznik komórki
    <td>5</td>
  </tr>
</table>
```

Formularze — znaczniki `<form>`, `<input>`, `<textarea>`:

```
<form method="post" action="kontakt.php"> → znacznik formularza
  <input type="text" value="Imię" name="name"> → znacznik pola tekstowego
  <input type="email" value="E-mail" name="email">
  <textarea rows="5" cols="30" name="message"> → znacznik obszaru tekstowego
    Napisz coś!
  </textarea>
  <input type="submit" value="Wyślij">
</form>
```

Ćwiczenie 1. Pierwszy dokument hipertekstowy

W tym prostym ćwiczeniu wykorzystasz podstawowe znaczniki HTML do konstruowania prostego dokumentu hipertekstowego. Przeanalizuj dokładnie kod z listingu 37.2, a następnie zapisz na dysku. Pamiętaj, że tym razem zapisujesz kod do pliku z rozszerzeniem *.html*, np. *code_37_02.html*.

Wykorzystaj pakiety XAMPP i Apache do otworzenia pliku na serwerze wirtualnym *php8.localhost* lub użyj serwera wbudowanego w interpreter języka PHP, czyli naciśnij klawisze **⌘ + R** i wpisz *cmd*, a po otwarciu się okna wpisz polecenie:

```
php -S localhost:8008 -t c:\xampp\apps\php8\htdocs ↵
```

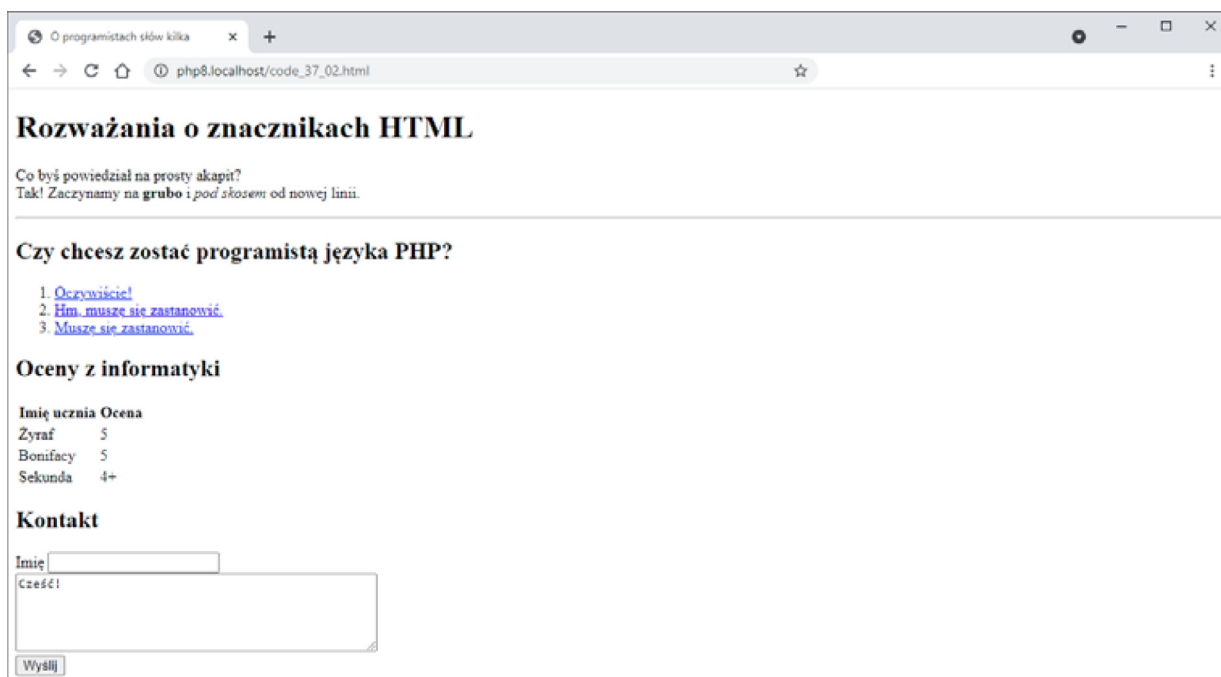
Po uruchomieniu się serwera otwórz w przeglądarce stronę *http://localhost:8008/code_37_02.html* (rysunek 37.2).

LISTING 37.2	HTML5	code_37_02.html
01	<code><!doctype html></code>	
02	<code><html lang="pl"></code>	
03	<code> <head></code>	
04	<code> <meta charset="utf-8"></code>	
05	<code> <meta name="viewport" content="width=device-width, initial-scale=1"></code>	
06	<code> <title>0 programistach słów kilka</title></code>	
07	<code> </head></code>	
08	<code> <body></code>	
09	<code> <h1>Rozważania o znacznikach HTML</h1></code>	
10	<code> <p></code>	
11	<code> Co byś powiedział na prosty akapit?
</code>	
12	<code> Tak! Zaczynamy na grubo</code>	
13	<code> i pod skosem od nowej linii.</code>	
14	<code> </p></code>	
15	<code> <hr></code>	
16	<code> <h2>Czy chcesz zostać programistą języka PHP?</h2></code>	
17	<code> </code>	

```

18         <li><a href="#">Oczywiście!</a></li>
19         <li><a href="#">Hm, muszę się zastanowić.</a></li>
20         <li><a href="#">Muszę się zastanowić.</a></li>
21     </ol>
22     <h2>Oceny z informatyki</h2>
23     <table>
24         <tr>
25             <th>Imię ucznia</th>
26             <th>Ocena</th>
27         </tr>
28         <tr>
29             <td>Żyraf</td>
30             <td>5</td>
31         </tr>
32         <tr>
33             <td>Bonifacy</td>
34             <td>5</td>
35         </tr>
36         <tr>
37             <td>Sekunda</td>
38             <td>4+</td>
39         </tr>
40     </table>
41     <h2>Kontakt</h2>
42     <form action="#" name="formularz">
43         Imię <input type="text" name="name" size="20" maxlength="30"><br>
44         <textarea cols="50" rows="5" name="message">Cześć!</textarea><br>
45         <input type="submit" name="send" value="Wyślij">
46     </form>
47 </body>
48 </html>

```



Rysunek 37.2. Wizualizacja kodu HTML z listingu 37.2 w przeglądarce internetowej

Co oznacza powyższy kod?

1. Najważniejszy kod jest w sekcji <body>, czyli w liniach od 08 do 47.
2. Na początku wykorzystałem nagłówek <h1> (wiersz 09).
3. W pierwszym akapicie <p> (linie od 10 do 14) wykorzystałem znacznik przejścia do nowego wiersza
, wyróżniłem tekst pogrubieniem i kursywą .
4. Następnie definiowana jest linia pozioma <hr> (wiersz 15), kolejny nagłówek, tym razem <h2> (linia 16), oraz lista z dalszymi pytaniami, i <a> (wiersze od 17 do 21).
5. Po nich definiowany jest kolejny nagłówek, też <h2> (linia 22), i tabela <table> (wiersze od 23 do 40). Poszczególne komórki tabeli są zdefiniowane znacznikami <tr>, <th> i <td>.
6. W linii 41 znajduje się jeszcze jeden nagłówek <h2>, a w wierszach od 42 do 46 zdefiniowany jest formularz kontaktowy, wykorzystujący znaczniki <input> i <textarea>.

Umieszczone w kodzie znaczniki nie wskazują na żaden adres (i nie działają); są wstawione tylko dla zobrazowania, jak można je wykorzystać.

Ćwiczenia dodatkowe

1. Zmodyfikuj dowolnie kod z ćwiczenia, np. dodaj dalsze wiersze w tabeli, nowy akapit itd. Wszystkie zmiany sprawdzaj w przeglądarce internetowej.
2. Poszukaj w sieci więcej informacji na temat znaczników HTML, arkuszy stylów CSS, języka programowania JavaScript, a także możliwości wykorzystania tych technologii w praktyce.
3. Kod strony internetowej możesz sprawdzić w przeglądarce po wciśnięciu klawiszy *Ctrl+U* (Chrome, Internet Explorer i Edge). Sprawdź z ciekawości, jak wygląda kod HTML w kilku dowolnie wybranych serwisach WWW.

Wskazówki

Dzisiejsze strony internetowe zawierają bardzo skomplikowany kod HTML, który często połączony jest z CSS (ang. *Cascading Style Sheets*), czyli kaskadowymi arkuszami stylów, a także z wieloma frameworkami (bibliotekami) napisanymi w języku JavaScript. To obecnie niezbędne technologie, aby móc tworzyć nowoczesne i funkcjonalne projekty webowe. Technologie webowe, takie jak HTML, CSS i JavaScript, są odpowiedzialne za wszystko, co dzieje się w przeglądarce internetowej, czyli za właściwą prezentację treści, obsługę multimediów, płynną nawigację, komunikację z serwerem itd. Technologie te nazywa się front-endem. Back-endem nazywamy zaś oprogramowanie uruchamiane po stronie serwera, np. skrypty PHP.

W praktyce podczas budowy nowego projektu webowego tworzone są grupy robocze odpowiedzialne za poszczególne jego elementy. Jedna grupa projektuje front-end, czyli wszystko to, co się będzie działo w przeglądarce, a inna grupa tworzy kod uruchamiany po stronie serwera, czyli back-end. Pamiętaj, że ten model dotyczy przeważnie średnich i dużych firm, w których zazwyczaj tworzone są duże projekty webowe. W mniejszych firmach w przypadku nieskomplikowanych projektów całość może być zaprojektowana i wykonana nawet przez jedną osobę!

Jesteś na etapie, na którym warto zainteresować się technologiami webowymi oraz możliwościami, jakie zapewniają projektantom. Znasz już na tyle dobrze język PHP, że śmiało możesz rozpocząć kolejną przygodę z programowaniem — nauczyć się wykorzystywać w swoich projektach HTML, JavaScript i CSS.

Pytania

Czym są kaskadowe arkusze stylów?

W arkuszach stylów CSS umieszczane są reguły odnoszące się do wyglądu dokumentu. W takim arkuszu można dokładnie określić, jak mają wyglądać np. nagłówki, akapity, odnośniki itd. Style umieszcza się w zewnętrznych plikach z rozszerzeniem .css lub bezpośrednio w kodzie HTML, z użyciem znacznika <style>. Wstaw poniższy kod pomiędzy linie 06 a 07 i sprawdź w przeglądarce, co się zmieni.

```
<style>
h1 {
  color: white;
  background-color: black;
}
</style>
```

Czym jest JavaScript?

Jest to skryptowy język programowania działający w przeglądarce, a także na serwerze, np. w środowisku Node.js. Stosowany jest do poprawy interaktywności z użytkownikiem, wykonywania skomplikowanych obliczeń, dynamicznego pobierania danych, efektów graficznych itd. Trudno sobie wyobrazić nowoczesną stronę WWW bez wykorzystania tego języka. Kod JavaScript umieszcza się w osobnych plikach z rozszerzeniem .js lub wstawia do kodu HTML za pomocą znacznika <script>. Wstaw poniższy kod po znaczniku otwierającym <body>.

```
<script>
  var imie = prompt("Cześć! Jak masz na imię?", "Bonifacy");
  alert("Witaj na mojej stronie " + imie + "!");
</script>
```




Czas na pierwszy projekt

Przyszedł czas na pierwszy projekt webowy, w którym wykorzystasz bardzo popularne i rozbudowane narzędzie do obsługi szablonów, a mianowicie Smarty. W swoich projektach, nawet tych bardzo skomplikowanych, często stosuję właśnie tę technologię, ponieważ umożliwia ona efektywną współpracę pomiędzy projektantami (front-endowcami) a programistami (back-endowcami). Istotą tej technologii jest oddzielenie warstwy logiki od warstwy widoku (prezentacji), co w praktyce sprowadza się do tego, że zadaniem projektanta jest opracowanie szablonu projektu (kod HTML), a zadaniem programisty jest opracowanie oprogramowania (kod PHP). Praca w takim systemie jest bardzo efektywna. Nikt nikomu nie przeszkadza i wszyscy są zadowoleni.

W pierwszej kolejności zainstaluj narzędzie Composer zgodnie z instrukcją na początku niniejszej książki, czyli otwórz stronę <https://getcomposer.org>, pobierz najnowszą wersję instalacyjną dla systemu Windows i zainstaluj.

W tej lekcji zajmiesz się konfiguracją pakietu Smarty. Na końcu przetestujesz, czy wszystko zostało wykonane zgodnie z planem.

Pamiętaj, wszystkie pliki z kodem źródłowym dostępne są na stronie wydawnictwa Helion. W każdej chwili możesz je pobrać i uruchomić na swoim komputerze.

Przygotowanie katalogu dla nowego projektu

Na początku niniejszej książki opisałem sposób instalacji popularnego wśród programistów języka PHP narzędzia Composer, które świetnie nadaje się do zarządzania pakietami programistycznymi. Jeśli zainstalowałeś narzędzie Composer z dodatkowymi pakietami, to powinieneś mieć już gotowy katalog na nowy projekt.

Na początku sprawdź, czy katalog na nowy projekt jest dostępny; jeśli nie, należy go utworzyć. Naciśnij klawisze **⌘ + R** i wpisz `cmd`, a po uruchomieniu się konsoli wpisz polecenia:

```
cd /d c:\xampp\apps\php8\htdocs ↵
mkdir projekt\libs ↵
cd projekt\libs ↵
```

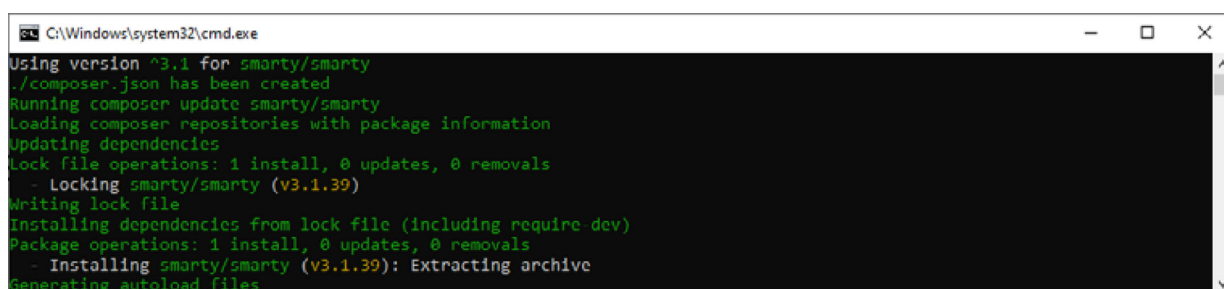
Jeśli podczas tworzenia katalogu roboczego polecenie `mkdir` wyświetli komunikat *A subdirectory or file projekt\libs already exists.*, oznacza to, że wcześniej katalog ten został przez Ciebie utworzony. Zakładam, że skonfigurowałeś serwer wirtualny w pakiecie XAMPP zgodnie z wytycznymi na początku książki. Nowy projekt będzie dostępny w przeglądarce pod adresem *http://php8.localhost/projekt*.

Instalacja Smarty

W podkatalogu `libs` w katalogu roboczym zainstaluj pakiet Smarty poleceniem:

```
cd /d c:\xampp\apps\php8\htdocs\projekt\libs ↵
composer require smarty/smarty ↵
```

Powinieneś zobaczyć w konsoli informację o przebiegu instalacji pakietu (rysunek 38.1).



Rysunek 38.1. Instalacja pakietu Smarty narzędziem Composer

Z informacji, które pojawiły się podczas instalacji (rysunek 38.1), możemy odczytać, że został utworzony specjalny plik `composer.json` w formacie JSON, w którym będą przechowywane informacje o zainstalowanych pakietach. Informacyjnie dodam, że Composer utworzy dodatkowy plik `composer.lock` ze swoją konfiguracją.

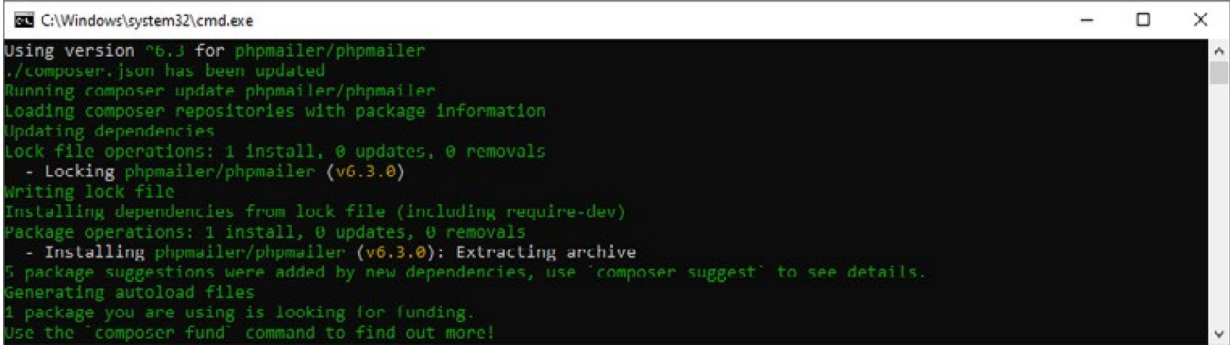
Instalacja PHPMailer

Zainstaluj pakiet PHPMailer poleceniem:

```
cd /d c:\xampp\apps\php8\htdocs\projekt\libs ↵
composer require phpmailer/phpmailer ↵
```

Z informacji, które pojawiły się podczas instalacji pakietu (rysunek 38.2), możemy odczytać, że plik `composer.json` został zaktualizowany. Uaktualniona zawartość pliku może być taka:

```
{
  "require": {
    "smarty/smarty": "^3.1",
    "phpmailer/phpmailer": "^6.3"
  }
}
```



```
C:\Windows\system32\cmd.exe
Using version ^6.3 for phpmailer/phpmailer
./composer.json has been updated
Running composer update phpmailer/phpmailer
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking phpmailer/phpmailer (v6.3.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
  - Installing phpmailer/phpmailer (v6.3.0): Extracting archive
5 package suggestions were added by new dependencies, use 'composer suggest' to see details.
Generating autoload files
1 package you are using is looking for funding.
Use the 'composer fund' command to find out more!
```

Rysunek 38.2. Instalacja pakietu PHPMailer narzędziem Composer

Zachęcam do odświeżenia informacji na temat pakietu Composer — znajdziesz je na początku niniejszej książki lub na stronie <https://getcomposer.org>.

Struktura katalogów

W tym momencie należy utworzyć katalogi, w których będą przechowywane pliki nowego projektu, czyli skrypty PHP, szablony Smarty, arkusze CSS i kod JavaScript.

Zwróć uwagę, że wszystkie poniższe czynności będziesz wykonywał w katalogu głównym projektu, a nie w podkatalogu `libs`.

Wykonaj zatem następujące polecenia w konsoli:

```
cd /d c:\xampp\apps\php8\htdocs\projekt &
mkdir inc &
mkdir smarty &
mkdir smarty\cache &
mkdir smarty\configs &
mkdir smarty\templates &
mkdir smarty\templates_c &
exit &
```

Po utworzeniu wszystkich katalogów struktura projektu powinna wyglądać tak:

```
c:\xampp\apps\php8\htdocs\projekt
|—inc
|—libs
|—libs\vendor
|—libs\vendor\composer
|—libs\vendor\phpmailer[...]
|—libs\vendor\smarty[...]
```

```

├──smarty
├──smarty\cache
├──smarty\configs
├──smarty\templates
└──smarty\templates_c

```

Test instalacji

W tym momencie należy sprawdzić, czy pakiety Smarty i PHPMailer są poprawnie zainstalowane, to znaczy czy wszystkie katalogi i pliki znajdują się tam, gdzie powinny. Pamiętaj, że od teraz katalogiem roboczym będzie katalog projekt, w którym będziesz umieszczał wszystkie pliki związane z nowym projektem.

W dalszej części książki będę używał wersji skróconych nazw katalogów, np.:

```

projekt = c:\xampp\apps\php8\htdocs\projekt
projekt\inc = c:\xampp\apps\php8\htdocs\projekt\inc
projekt\smarty\template = c:\xampp\apps\php8\htdocs\projekt\smarty\template

```

Zainstalowane pakiety należy za pomocą *autoloadera* dołączyć do nowego projektu. Na listingu 38.1 pokazałem, jak można to zrobić. Zapisz kod w pliku `test.php` w katalogu projektu i otwórz w przeglądarce. Jeśli nie pojawiły się błędy typu *PHP Fatal error*, to oznacza, że wszystko poprawnie zainstalowałeś i skonfigurowałeś.

LISTING 38.1	PHP 7/8	test.php
01	<?php	
02	use PHPMailer\PHPMailer\{PHPMailer, SMTP, Exception};	
03		
04	require_once __DIR__."/libs/vendor/autoload.php"; // autoloader	
05		
06	\$smarty = new Smarty;	
07	\$mail = new PHPMailer(true);	
08		
09	// definicja katalogów roboczych	
10	\$smarty->setTemplateDir('smarty/templates'); //szablony	
11	\$smarty->setCompileDir('smarty/templates_c');	
12	\$smarty->setCacheDir('smarty/cache');	
13	\$smarty->setConfigDir('smarty/configs');	
14		
15	\$smarty->testInstall(); // test instalacji Smarty	
http://php8.localhost/projekt/test.php ↵		PRZEGLĄDARKA

Aby wykorzystać technologię Smarty, należy utworzyć egzemplarz klasy (linia 06). Podstawowa konfiguracja wymaga zdefiniowania katalogów roboczych (wiersze od 10 do 13). Pakiet Smarty ma wbudowaną funkcję testującą poprawność instalacji (linia 15), dlatego warto z niej skorzystać.

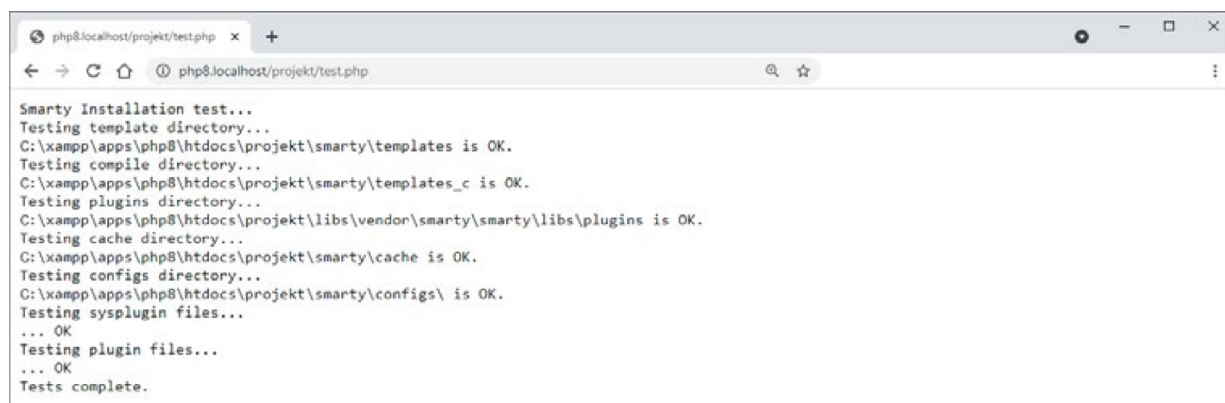
Pamiętaj, aby od teraz do testowania projektu wykorzystywać pakiet XAMPP z serwerem Apache lub serwer WWW wbudowany w interpreter języka PHP.

W przypadku pakietu XAMPP wystarczy go uruchomić i aktywować moduł Apache oraz pamiętać, aby wszystkie pliki z projektem wgrywać do katalogu roboczego, czyli do `c:\xampp\apps\php8\htdocs\projekt`. W przeglądarce projekt będzie dostępny pod adresem `http://php8.localhost/projekt`.

Jeśli będziesz chciał skorzystać z serwera WWW wbudowanego w interpreter PHP, naciśnij klawisze **Windows + R** i wpisz `cmd`, a po uruchomieniu się konsoli wpisz polecenie:

```
php -S localhost:8008 -t c:\xampp\apps\php8\htdocs
```

Następnie otwórz w przeglądarce stronę `http://localhost:8008/projekt/test.php`. Powinieneś zobaczyć raport z przeprowadzonego testu; porównaj go z raportem przedstawionym na rysunku 38.3. W przypadku pojawienia się błędów zostaniesz o tym poinformowany stosownym komunikatem.



Rysunek 38.3. Raport z przeprowadzonego testu instalacji pakietu Smarty

Ćwiczenia dodatkowe

1. Wejdź na stronę projektu Smarty — <https://www.smarty.net> — i zapoznaj się z dokumentacją. Niestety, dokumentacja nie jest dostępna w języku polskim, jednak bez problemu zrozumiesz zawarte w niej liczne przykłady wykorzystania Smarty. Na stronie projektu, w dziale *Quick Install*, znajduje się również szczegółowa instrukcja instalacji.



Dwa światy, logika i widok

Oddzielenie warstwy logiki (kod PHP po stronie serwera) od warstwy widoku (kod HTML, CSS i JavaScript po stronie klienta) znacznie ułatwia pracę projektantów i programistów. Projektant, tworząc szablon, nie zastanawia się nad tym, w jaki sposób programista dostarczy dane. Całkiem prawdopodobne, że projekt webowy się rozwinie i z biegiem czasu zmieni się format przechowywanych informacji w bazie danych, co wymusi aktualizację warstwy logiki, ale bez ingerencji w warstwę widoku (prezentacji). Tak samo będzie w przypadku aktualizacji kodu HTML czy dodania kolejnej wtyczki w JavaScriptcie — wówczas zmieni się tylko warstwa widoku, bez zmiany warstwy logiki. W dużym uproszczeniu taki model ułatwia sprawne zarządzanie projektem, gdyż angażuje do realizacji konkretnego zadania najbardziej kompetentne osoby.

Na potrzeby tej lekcji będziesz jednocześnie projektantem i programistą, co ma wiele zalet, ponieważ nauczysz się wykorzystywać technologię Smarty do budowy nowego projektu. Opracujesz szablon z użyciem języka HTML i napiszesz obsługujący go kod w języku PHP. Nie martw się, to nie jest trudne!

Technologia Smarty

Smarty to funkcjonalny język skryptowy, który warto poznać i opanować. Systemów do obsługi szablonów jest znacznie więcej, choćby Mustache, Twig czy Plates, a każdy z nich ma własną logikę i składnię. Jeśli dobrze poznasz technologię Smarty, bez większych problemów nauczysz się wykorzystywać w swoich projektach pozostałe systemy do obsługi szablonów. Przykładowy szablon łączący technologię Smarty z kodem HTML jest zawarty w poniższym fragmencie kodu:

```
<p>Cześć <strong>{$name}</strong>!</p>
```

System Smarty zmodyfikuje powyższy kod (warstwa widoku) w taki sposób, że w miejsce {\$name} wstawi zmienną, którą otrzyma za pomocą funkcji assign (warstwa logiki), np.:

```
$smarty->assign('name', 'Krzysiek');
```

Po przetworzeniu przeglądarka internetowa wyświetli tekst:

Cześć Krzysiek!

Istotą takiego rozwiązania jest umożliwienie projektantom i programistom niezależnej i bezpiecznej pracy nad projektem. Dzięki temu projektant zwraca się z prośbą do programisty, by ten przekazał konkretne dane do szablonu w zmiennej tekstowej lub tablicowej.

Oczywiście, technologia Smarty zapewnia znacznie więcej możliwości i funkcji, choćby bogatą bibliotekę wtyczek (ang. *plugins*), które umożliwiają wielokrotne wykorzystywanie kodu. Na potrzeby tej lekcji ograniczę się do podstaw związanych z szablonami w Smarty.

Oprogramowanie ionCube, za pomocą którego programiści na całym świecie zabezpieczają swój kod źródłowy, potrafi szyfrować również szablony pakietu Smarty. ionCube zainstalowane jest prawie na każdym serwerze hostingowym.

Szablony należy zapisywać w plikach z rozszerzeniem *.tpl* w katalogu zgodnie z deklaracją:

```
$smarty->setTemplateDir('smarty/templates');
```

Przepisz dokładnie kod z listingu 39.1 i zapisz w pliku *index.tpl* w katalogu z szablonami.

LISTING 39.1	SMARTY	smarty\templates\index.tpl
01	<!doctype html>	
02	<html>	
03	<head>	
04	<meta charset="utf-8">	
05	<title>{\$title}</title>	
06	</head>	
07	<body>	
08	Cześć {\$name}!	
09	Witaj na mojej stronie ;)	
10	Zmienna \$x = {\$x}, a \$y = {\$y}	
11	{\$x} * {\$y} = {math equation="x * y" x=\$x y=\$y}	
12	<p>	
13	{* to jest komentarz w szablonie Smarty *}	
14	{if \$x > 100}	
15	Wartość zmiennej \$x jest większa od 100!	
16	{/if}	
17	</p>	
18	<p>	


```

19      Początkowe liczby pierwsze to:
20      {foreach item=liczba from=$liczby}
21          {$liczba},
22      {/foreach}
23  </p>
24 </body>
25 </html>

```

Jak pewnie zauważyłeś, kod szablonu składa się ze znaczników HTML oraz dodatkowego kodu umieszczonego wewnątrz nawiasów klamrowych { i }. Zmienne przekazane do szablonu wyglądają podobnie jak w PHP, to znaczy zaczynają się od znaku \$ (dolar). Składnia języka skryptowego Smarty zawiera również obsługę konstrukcji warunkowych (linie od 14 do 16) oraz pętli (wiersze od 20 do 22), a zastosowanie wtyczki `math` (linia 11) pozwala wykonywać obliczenia matematyczne.

Teraz przygotujesz plik z kodem PHP, za pomocą którego przekażesz niezbędne dane do szablonu. Do przekazania danych użyłem funkcji `assign` (linie od 13 do 17), a do wyświetlenia szablonu funkcję `display` (wiersz 20). Możesz przekazać dane tekstowe (linie 13 i 14), liczbowe (wiersze 15 i 16) lub tablicowe (linia 17). Przeanalizuj, jak przekazywane są dane do szablonu i w jaki sposób są one w nim wykorzystywane. Zrozumienie tego jest podstawą do praktycznego zastosowania technologii Smarty we własnych projektach webowych.

Przepisz dokładnie kod z listingu 39.2 i zapisz w pliku `index.php` w katalogu projektu.

LISTING 39.2	PHP 7/8	index.php
01	<?php	
02	require_once __DIR__."/libs/vendor/autoload.php"; <i>// autoloader</i>	
03		
04	\$smarty = new Smarty;	
05		
06	<i>// definicja katalogów roboczych</i>	
07	\$smarty->setTemplateDir ('smarty/templates'); <i>// szablony</i>	
08	\$smarty->setCompileDir ('smarty/templates_c');	
09	\$smarty->setCacheDir ('smarty/cache');	
10	\$smarty->setConfigDir ('smarty/configs');	
11		
12	<i>// przekazanie danych do szablonu</i>	
13	\$smarty->assign ('title', 'Zmienne w Smarty'); <i>// przekaz tekst</i>	
14	\$smarty->assign ('name', 'Krzysiek'); <i>// przekaz tekst</i>	
15	\$smarty->assign ('x', 120); <i>// przekaz liczbę</i>	
16	\$smarty->assign ('y', 540); <i>// przekaz liczbę</i>	


```

17 $smarty->assign('liczby', [2,3,5,7,11,13]); //przekaz tablice
18
19 //wyświetlenie szablonu
20 $smarty->display('index.tpl');
http://php8.localhost/projekt ↵

```

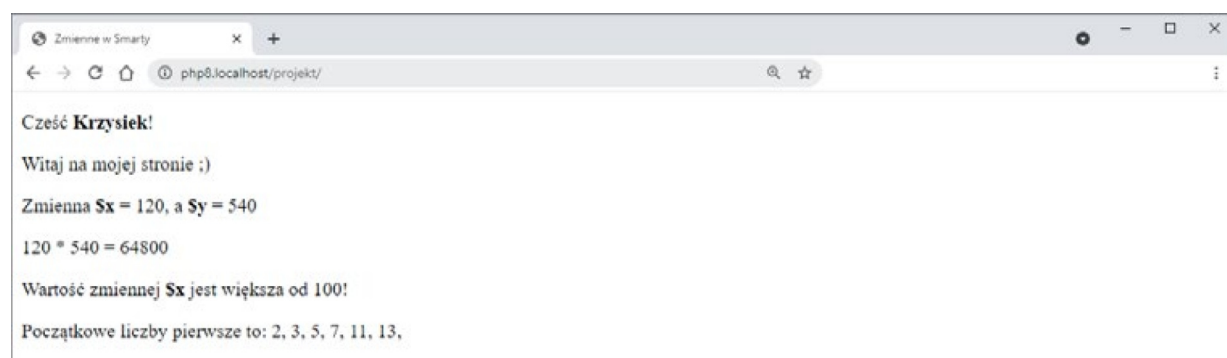
PRZEGLĄDARKA

Teraz uruchom pakiet XAMPP i serwer Apache. Następnie otwórz w przeglądarce stronę *http://php8.localhost/projekt* i sprawdź, czy wszystko działa poprawnie. Możesz też wykorzystać wbudowany w interpreter języka PHP serwer WWW, czyli naciśnij klawisze **Windows** + **R** i wpisz `cmd`, a po uruchomieniu się konsoli wpisz polecenie:

```
php -S localhost:8008 -t c:\xampp\apps\php8\htdocs ↵
```

Następnie otwórz w przeglądarce stronę *http://localhost:8008/projekt/*.

Powinieneś zobaczyć stronę WWW wygenerowaną przez system Smarty; porównaj ją z rysunkiem 39.1.



Rysunek 39.1. Strona wygenerowana przez system Smarty

Gotowe! Gratuluję!

Pierwszy projekt stworzony z wykorzystaniem technologii Smarty masz już za sobą.

Ćwiczenia dodatkowe

1. Przekaz do szablonu 10 kolejnych liczb pierwszych i wyświetl je w tabeli. Zaproponuj dowolny wariant rozwiązania tego zadania (zajrzyj do wskazówek).

Wskazówki

Na programiście spoczywa odpowiedzialność za prawidłowe przygotowanie i przetworzenie danych. Jest on odpowiedzialny za warstwę logiki, podobnie jak za warstwę widoku odpowiedzialny jest projektant. Poznanie obu światów (warstwy logiki i widoku) jest bardzo inspirującym doświadczeniem dla większości programistów.

Umiejętność poruszania się między logiką a widokiem znacząco ułatwia zarządzanie projektami, co może mieć decydujący wpływ na decyzje biznesowe.

Na zachętę zaproponowałem swoje rozwiązanie ćwiczenia dodatkowego (listing 39.3). Aby nie było tak łatwo, dodałem funkcję generującą liczby pierwsze. Możesz ją wykorzystać również w innych swoich projektach.

W pliku `index.php` wprowadź następujące zmiany:

LISTING 39.3	PHP 8	index.php
01	<?php	
02	require_once __DIR__."/libs/vendor/autoload.php"; <i>// autoloader</i>	
03	<i>//funkcja zwracająca liczby pierwsze</i>	
04	function liczbyPierwsze(int \$ile = 1, int \$start = 2) : array	
05	{	
06	\$wynik = []; <i>// tablica z liczbami pierwszymi</i>	
07	\$x = 1; <i>// licznik</i>	
08	\$y = \$start; <i>// od jakiej liczby zaczynamy?</i>	
09	while (\$x <= \$ile)	
10	{ \$z = 0;	
11	for (\$i=1; \$i<=\$y; \$i++)	
12	{	
13	if ((\$y % \$i) == 0) {	
14	\$z++;	
15	}	
16	}	
17	if (\$z < 3) {	
18	\$wynik[] = \$y;	
19	\$x++;	
20	}	
21	\$y++;	
22	}	
23	return \$wynik;	
24	}	
25		
26	\$smarty = new Smarty;	
27		

```

28 //definicja katalogów roboczych
29 $smarty->setTemplateDir('smarty/templates'); //szablony
30 $smarty->setCompileDir('smarty/templates_c');
31 $smarty->setCacheDir('smarty/cache');
32 $smarty->setConfigDir('smarty/configs');
33
34 //przekazanie danych do szablonu
35 $smarty->assign('title', 'Zmienne w Smarty'); //przekaż tekst
36 $smarty->assign('name', 'Krzysiek'); //przekaż tekst
37 $smarty->assign('x', 120); //przekaż liczbę
38 $smarty->assign('y', 540); //przekaż liczbę
39 $smarty->assign('liczby', liczbyPierwsze(ile: 10, start: 2)); //przekaż tablicę
40
41 //wyświetlenie szablonu
42 $smarty->display('index.tpl');

```

http://php8.localhost/projekt ↵ PRZEGLĄDARKA

Funkcja `liczbyPierwsze` ma dwa argumenty: pierwszy określa liczbę generowanych liczb pierwszych, drugi definiuje zakres początkowy. Do szablonu, za pomocą funkcji `assign`, przekazywany jest wynik działania funkcji generującej liczby pierwsze.

W szablonie Smarty, w pliku `index.tpl`, wprowadź następujące zmiany (patrz listing 39.4):

LISTING 39.4	SMARTY	smarty\templates\index.tpl
01	<!doctype html>	
02	<html>	
03	<head>	
04	<meta charset="utf-8">	
05	<title>{\$title}</title>	
06	</head>	
07	<body>	
08	Cześć {\$name}!	
09	Witaj na mojej stronie ;)	
10	Zmienna \$x = {\$x}, a \$y = {\$y}	
11	{\$x} * {\$y} = {math equation="x * y" x=\$x y=\$y}	
12	<p>	
13	{* to jest komentarz w szablonie Smarty *}	
14	{if \$x > 100}	
15	Wartość zmiennej \$x jest większa od 100!	
16	{/if}	
17	</p>	
18	{\$liczby count} początkowych liczb pierwszych to:	
19	<table border="1">	

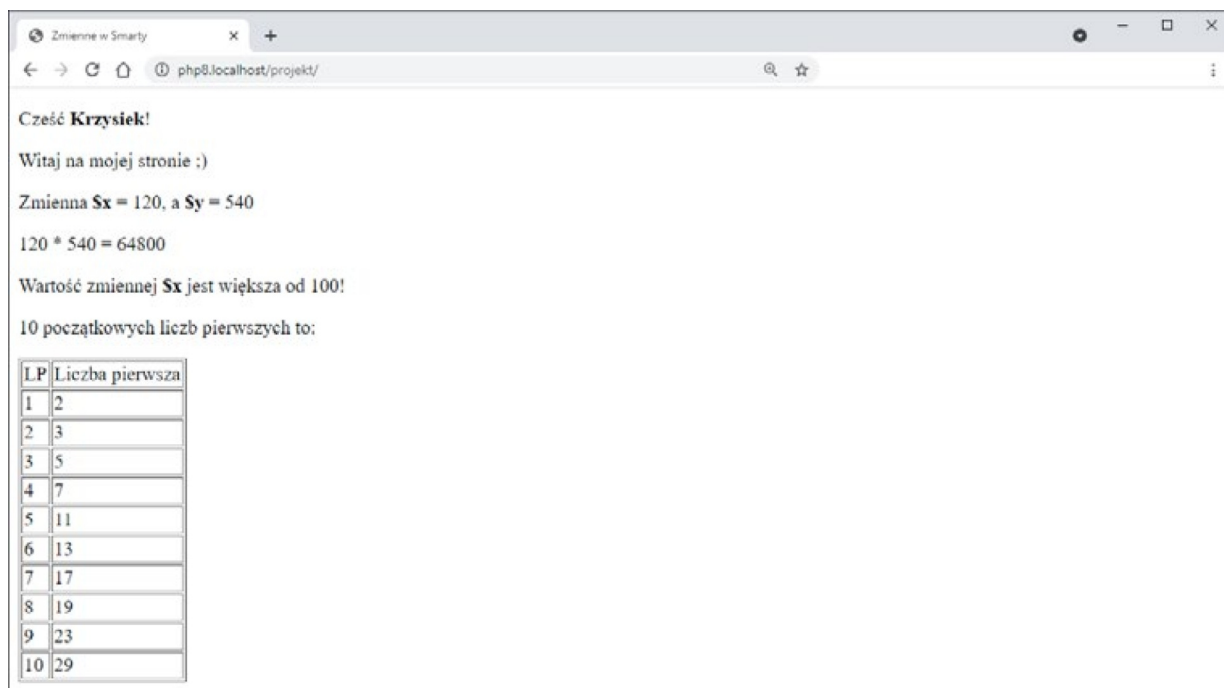
```

20      <tr>
21          <td>LP</td>
22          <td>Liczba pierwsza</td>
23      </tr>
24      {foreach item=liczba name=liczby from=$liczby}
25          <tr>
26              <td>{$smarty.foreach.liczby.iteration}</td>
27              <td>{$liczba}</td>
28          </tr>
29      {/foreach}
30  </table></body>
31  </html>

```

Zmodyfikowałem pętlę foreach i obudowałem ją znacznikami HTML definiującymi tabelę. Wykorzystałem również modyfikator count do obliczenia liczby elementów w zmiennej tablicowej \$liczby.

Rysunek 39.2 przedstawia wygląd strony po wprowadzonych zmianach.



Rysunek 39.2. Zmodyfikowana wersja strony z ćwiczenia dodatkowego

Pytania

Który system szablonów wybrać?

Trudne pytanie, ponieważ każdy projektant czy programista ma swoje zdanie na ten temat. Warto jednak poznać kilka różnych systemów i wybrać ten optymalny (Smarty, Mustache, Twig, Plates itd.).

Ja wybrałem Smarty, technologię zapewniającą bardzo zaawansowane narzędzia do budowy naprawdę dużych serwisów webowych. Wykorzystanie własnych wtyczek i buforowania, obsługa wielojęzyczności w serwisach WWW i wysoka wydajność to z pewnością istotne atuty Smarty.



Uruchomienie pierwszej strony

Czas już najwyższy wykorzystać zdobytą wiedzę do uruchomienia pierwszej strony internetowej, którą zbudujesz z użyciem najnowszej wersji frameworku (biblioteki) Bootstrap, rozwijanego przez programistów Twittera. Biblioteka ta zawiera wiele ciekawych narzędzi do projektowania interfejsu użytkownika, takich jak stylizacja tekstów, formularze, przyciski, nawigacja, elementy graficzne i multimedia, a także idealnie nadaje się do projektowania serwisów WWW obsługujących urządzenia mobilne. Biblioteka Bootstrap jest bardzo popularna wśród programistów na całym świecie, o czym świadczą miliony stron opartych właśnie na niej. Bootstrap świetnie nadaje się do projektowania stron, które działają optymalnie zarówno w zwykłych przeglądarkach desktopowych, jak i na tabletach i smartfonach.

Co masz do zrobienia?

W projekcie wykorzystasz wcześniej utworzoną bazę danych. Jeśli będziesz chciał wykorzystać bazę SQLite, skopiuj plik `kartoteka.db` do katalogu `projekt`. Możesz to zrobić w prosty sposób w konsoli systemowej, czyli naciśnij klawisze `⌘ + R` i wpisz `cmd`, a po otwarciu się okna wpisz następujące polecenie:

```
copy c:\xampp\apps\php8\htdocs\kartoteka.db c:\xampp\apps\php8\htdocs\projekt
```

W przypadku wykorzystania bazy MySQL pamiętaj, aby w pakiecie XAMPP włączyć moduł MySQL — zgodnie z opisem, który znajduje się na początku niniejszej książki.

Po pierwsze, najważniejszym zadaniem będzie dokładne przepisanie wszystkich listingów związanych z warstwą logiki i widoku. Ten etap jest bardzo ważny, dlatego zwróć szczególną uwagę na nazwy plików z kodem PHP i szablonami Smarty oraz na nazwy katalogów, w których będziesz zapisywał pliki.

Po drugie, kiedy listingi zostaną już zapisane na dysku, należy sprawdzić w przeglądarce internetowej, czy strona wyświetla się prawidłowo i czy nie pojawiają się komunikaty o błędach. Tak jak w poprzednich ćwiczeniach, wykorzystasz do tego

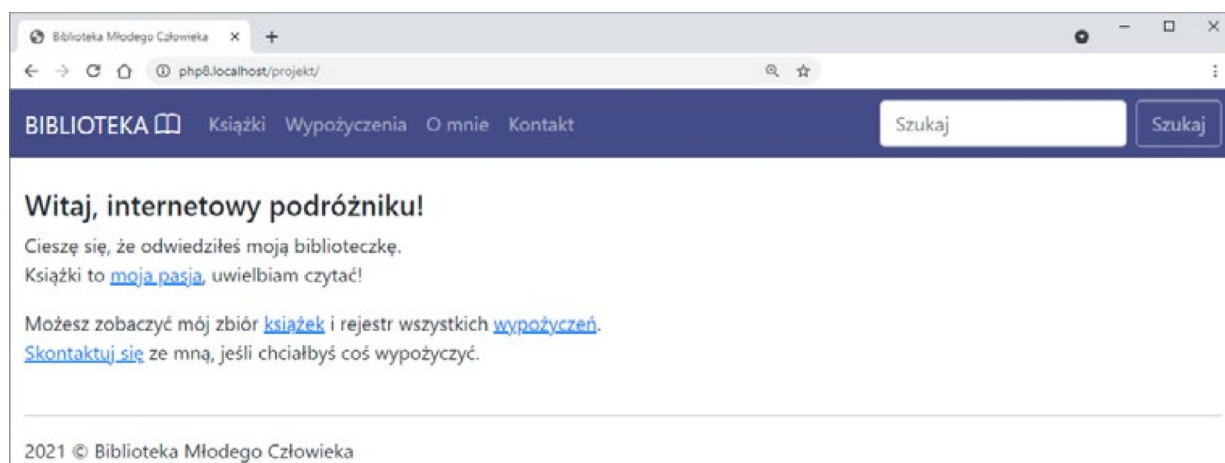
celu serwer WWW z pakietu XAMPP lub dostępny w interpreterze języka PHP w konsoli systemowej. Aby uruchomić serwer WWW w konsoli systemowej, naciśnij klawisze **⌘ + R** i wpisz `cmd`, a po otwarciu się okna wpisz polecenie:

```
php -S localhost:8008 -t c:\xampp\apps\php8\htdocs ↵
```

Otwórz w przeglądarce stronę `http://localhost:8008/projekt`.

Pamiętaj, że jeśli używasz serwera WWW z pakietu XAMPP, to w przeglądarce należy otworzyć stronę `http://php8.localhost/projekt`.

Gotowe! Rysunek 40.1 przedstawia stronę startową projektu. Porównaj, czy u Ciebie jest podobnie.



Rysunek 40.1. Strona startowa projektu

Na dalszych stronach znajdziesz wszystkie listingi niezbędne do uruchomienia witryny. Zarezerwuj trochę czasu, aby na spokojnie przepisać je do edytora i zapisać na dysku. Powodzenia!

Pamiętaj, wszystkie pliki z kodem źródłowym dostępne są na stronie wydawnictwa Helion. W każdej chwili możesz je pobrać i uruchomić na swoim komputerze.

Listingi

Warstwa logiki

Plik `index.php` (listing 40.1) zawiera obsługę biblioteki Smarty, baz danych SQLite i MySQL oraz wyświetlanych szablonów (nowa wersja pliku z poprzedniej lekcji). W zmiennej `$_REQUEST['page']` przekazywana jest nazwa sekcji, która została wybrana (kliknięta) w menu strony (linia 13). Instrukcja warunkowa `switch`

(wiersze od 26 do 63) jest odpowiedzialna za wyświetlanie konkretnych szablonów. W zmiennej `$_POST['search']` przekazywany jest szukany tekst, który trafia do funkcji `$project->books` (linia 33).

Wprowadziłem dwie stałe `E_MYSQL` i `E_SQLITE`, abyś mógł wybrać, z której bazy danych będziesz chciał skorzystać (linia 23). Pamiętaj, aby w pliku `db.class.php` wstawić poprawne dane konfiguracyjne do połączenia się z bazą MySQL (patrz listing 40.2).

LISTING 40.1	PHP 8	index.php
01	<?php	
02	require_once __DIR__."/libs/vendor/autoload.php";	<i>// autoloader</i>
03		
04	\$smarty = new Smarty;	
05		
06	<i>// definicja katalogów roboczych</i>	
07	\$smarty->setTemplateDir('smarty/templates');	<i>// szablony</i>
08	\$smarty->setCompileDir('smarty/templates_c');	
09	\$smarty->setCacheDir('smarty/cache');	
10	\$smarty->setConfigDir('smarty/configs');	
11		
12	<i>// którą stronę wyświetlić?</i>	
13	\$page = (isset(\$_REQUEST['page'])) ? trim(strip_tags(\$_REQUEST['page'])) : '';	
14	\$smarty->assign('page', \$page);	
15		
16	require_once 'inc/db.class.php';	<i>// obsługa bazy danych</i>
17	require_once 'inc/book.class.php';	<i>// główna klasa projektu</i>
18		
19	<i>/*</i>	
20	<i>E_MYSQL — baza danych MySQL</i>	
21	<i>E_SQLITE — baza danych SQLite</i>	
22	<i>*/</i>	
23	\$project = new MyBook(E_SQLITE);	<i>// tworzenie egzemplarza klasy</i>
24		
25	<i>// wyświetlenie szablonu</i>	
26	switch (\$page)	
27	{	
28	case 'books':	
29	<i>// może czegoś szukasz?</i>	
30	\$search = (isset(\$_POST['search'])) ? trim(strip_tags(\$_POST['search']))	
31	: '';	
32	<i>/* pobierz informacje o książkach */</i>	
33	\$books = \$project->books(search: \$search);	
34	\$smarty->assign('books', \$books);	


```

35     $smarty->assign('search', $search);
36
37     $smarty->assign('title', 'Lista książek');
38     $smarty->display('books.tpl');
39     break;
40
41     case 'register':
42         /* pobierz informacje o wypożyczeniach */
43         $register = $project->register();
44         $smarty->assign('register', $register);
45
46         $smarty->assign('title', 'Wypożyczenia');
47         $smarty->display('register.tpl');
48         break;
49
50     case 'about':
51         $smarty->assign('title', 'O mnie');
52         $smarty->display('about.tpl');
53         break;
54
55     case 'contact':
56         $smarty->assign('title', 'Kontakt');
57         $smarty->display('contact.tpl');
58         break;
59
60     default:
61         $smarty->assign('title', 'Biblioteka Młodego Człowieka');
62         $smarty->display('index.tpl');
63 }
64
65 unset($project);

```

Plik `db.class.php` zawiera definicję klasy obsługującej bazę danych (patrz listing 40.2). Klasa `MyDB` obsługuje jednocześnie bazy SQLite i MySQL, w zależności od argumentu podanego w konstruktorze. Pamiętaj o poprawnym wprowadzeniu konfiguracji MySQL w liniach od 10 do 13. Klasa ma zdefiniowane dwie metody, pierwsza `myQuery` zwraca wynik zapytania w języku SQL do bazy danych, druga `mySearch` zwraca wynik wyszukiwania autorów i tytułów książek w bazie. Zwróć uwagę na konstruktor klasy, w którym tworzona jest instancja klasy PDO. Nowością jest dodany destruktor klasy (wiersz 39), który powoduje zamknięcie połączenia z bazą danych. Destruktor to metoda wywoływana automatycznie, kiedy obiekt danej klasy jest usuwany.

LISTING 40.2

PHP 8

inc\db.class.php

```

01  <?php
02  class MyDB
03  {
04      private $db_pdo;
05
06      /* Plik z bazą danych w formacie SQLite */
07      private $db_file = 'kartoteka.db';
08
09      /* Konfiguracja serwera MySQL */
10      private $db_server = 'localhost';
11      private $db_name   = 'db_php8';
12      private $db_user   = 'db_php8';
13      private $db_passwd = 'WETFD8toJyKLWtXC'; // u Ciebie będzie inne :) 
14
15      public function __construct(private int $engine = E_SQLITE)
16      {
17          try {
18              $this->db_pdo = match($this->engine) {
19                  // SQLite
20                  E_SQLITE => new PDO('sqlite:'.__DIR__.'/../'.$this->db_file),
21                  // MySQL
22                  E_MYSQL => new PDO(
23                      "mysql:host=".$this->db_server.";dbname=".$this->db_name,
24                      $this->db_user,
25                      $this->db_passwd,
26                      [PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8"]
27                  )
28              };
29
30              // włącz obsługę wyjątków dla obsługi interfejsu PDO
31              $this->db_pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
32
33          } catch (PDOException $e) {
34              print "Błąd!: " . $e->getMessage();
35              die();
36          }
37      }
38
39      public function __destruct() {
40          $this->db_pdo = NULL;
41      }
42
43      public function myQuery(string $sql) : array
44      {
45          // wyślij zapytanie do bazy i pobierz wyniki

```

```

46         return $this->db_pdo->query($sql)->fetchAll(PDO::FETCH_ASSOC);
47     }
48
49     public function mySearch(string $sql, ?string $search = '') : array
50     {
51         // wyślij zapytanie do bazy i pobierz wyniki
52         $stmt = $this->db_pdo->prepare($sql);
53         $stmt->execute(["%$search%", "%$search%"]);
54         return $stmt->fetchAll(PDO::FETCH_ASSOC);
55     }
56 }

```

Plik `book.class.php` zawiera definicję klasy obsługującej nasz projekt (patrz listing 40.3). Definicja klasy `MyBook` (linia 04) wskazuje na to, że klasa dziedziczy metody po klasie `MyDB` — o czym świadczy instrukcja `extends`. W konstruktorze klasy wywoływany jest konstruktor klasy odziedziczonej, aby utworzyć połączenie z bazą danych (wiersz 08). Metoda `books` (linie od 11 do 30) zwraca listę wszystkich książek w bazie. Metoda ta może zwrócić również wynik wyszukiwania na podstawie zdefiniowanego argumentu `$search` (wiersz 11). Metoda `register` (linie od 32 do 49) zwraca rejestr wszystkich wypożyczeń. Dla ujednolicenia nazewnictwa zmiennych funkcje te zwracają pola w języku angielskim, np. pole `["tytuł"]` zamieniane jest na `["title"]` (wiersze od 24 do 27 i od 39 do 43).

LISTING 40.3	PHP 8	inc\book.class.php
01	<?php	
02	declare(strict_types = 1);	<i>// dbamy o prawidłowe typy danych</i>
03		
04	class MyBook extends MyDB	
05	{	
06	public function __construct(private int \$engine = E_SQLITE)	
07	{	
08	parent::__construct(engine: \$engine);	
09	}	
10		
11	public function books(string \$search = '') : array	
12	{	
13	if (\$search != '')	
14	{	
15	<i>// wycinamy wszystko, co nie jest cyfrą, literą, podkreśleniem itp.</i>	
16	\$search = preg_replace('/[^0-9 a-z-!_\\p{L}]/u', '', \$search);	
17	}	
18		
19	<i>// lista wszystkich książek</i>	
20	<i>// zmiana nazw pól na język angielski</i>	

```

21         return $this->mySearch(sql: "
22             SELECT
23             id,
24             autor as author,
25             tytuł as title,
26             rok as year,
27             stron as pages
28             FROM ksiazki WHERE autor LIKE ? OR tytuł LIKE ? ORDER BY id ASC;
29         ", search: $search);
30     }
31
32     public function register() : array
33     {
34         // lista wszystkich książek
35         // zmiana nazw pól na język angielski
36         return $this->myQuery(sql: "
37             SELECT
38             w.id,
39             w.data as date,
40             k.tytuł as title,
41             k.rok as year,
42             o.imie as name,
43             o.nazwisko as surname
44             FROM wypozyczenia AS w, ksiazki AS k, osoby AS o
45             WHERE
46             w.ksiazka = k.id AND w.osoba = o.id
47             ORDER BY w.id ASC;
48         ");
49     }
50 }

```

Warstwa widoku

Szablony w systemie Smarty można łączyć, aby utworzyć większą całość. Jest to bardzo ważna właściwość, ponieważ część kodu możemy wykorzystywać wielokrotnie. Jak można zauważyć na listingu 40.4, plik `index.tpl` składa się z mniejszych elementów, wstawionych do szablonu poleceniem `include` (linie 01, 02, 03 i 18). Całość tworzy dokument hipertekstowy wykorzystujący właściwości charakterystyczne dla biblioteki Bootstrap.

LISTING 40.4	SMARTY	smarty\templates\index.tpl
--------------	--------	----------------------------

01	{include file="header.tpl"}
02	{include file="js.tpl"}
03	{include file="nav.tpl"}
04	

```

05 <div class="container-fluid">
06 <br>
07 <h4>Witaj, internetowy podróżniku!</h4>
08 <p>
09 Cieszę się, że odwiedziłeś moją biblioteczkę.<br>
10 Książki to <a href="?page=about">moja pasja</a>, uwielbiam czytać!
11 </p>
12 <p>
13 Możesz zobaczyć mój zbiór <a href="?page=books">ksiązek</a> i rejestr wszystkich
14 <a href="?page=register">wypożyczeń</a>.<br>
15 <a href="?page=contact">Skontaktuj się</a> ze mną, jeśli chciałbyś coś wypożyczyć.
16 </p>
17 </div>
18 {include file="footer.tpl"}

```

Plik `header.tpl` zawiera definicję nagłówka dokumentu hipertekstowego, czyli deklarację zgodności z HTML5, znaczniki meta oraz odnośniki do arkuszy stylów CSS, które zostaną załadowane razem z dokumentem. Na listingu 40.5 zwróć uwagę na linię 10 — Smarty wstawi tutaj otrzymaną z pliku `index.php` zmienną `$title`.

LISTING 40.5	SMARTY	smarty\templates\header.tpl
01	<!doctype html>	
02	<html lang="pl">	
03	<head>	
04	<meta charset="utf-8">	
05	<meta name="viewport" content="width=device-width, initial-scale=1">	
06	<!-- Bootstrap CSS -->	
07	<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/twitter-bootstrap/5.0.0-beta2/css/bootstrap.min.css">	
08	<!-- Bootstrap Icons -->	
09	<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/bootstrap-icons/1.4.0/font/bootstrap-icons.min.css">	
10	<title>{\$title}</title>	
11	</head>	
12	<body>	

Plik `js.tpl` zawiera listę bibliotek JavaScript, które będą ładowane razem ze stroną. Bootstrap opiera się na JQuery i Popper, również popularnych bibliotekach. Na listingu 40.6 w liniach od 04 do 09 zawarłem kod JavaScript odpowiedzialny za aktywację przycisku *Szukaj*, jeśli w polu tekstowym pojawi się tekst.

LISTING 40.6	SMARTY	smarty\templates\js.tpl
01	<script src="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.6.0/	
02	jquery.min.js"></script>	
03	<script src="https://cdnjs.cloudflare.com/ajax/libs/twitter-bootstrap/	
04	5.0.0-beta2/js/bootstrap.bundle.min.js"></script>	
05	<script>	
06	\$(function() {	
07	\$('input:first', '#form').on('input', function() {	
08	var msg = \$(this).val();	
09	\$('button:first', '#form').attr("disabled", (msg.length > 0) ? false : true);	
10	});	
11	});	
12	</script>	

Plik `nav.tpl` zawiera kod z nawigacją (menu) strony. Nie przejmuj się, jeśli kod HTML jest dla Ciebie niezrozumiały. Biblioteka Bootstrap wykorzystuje arkusze stylów kaskadowych, które deklarowane są w znacznikach HTML za pomocą atrybutów `class`. Po prostu dokładnie przepisz kod z listingu 40.7.

LISTING 40.7	SMARTY	smarty\templates\nav.tpl
01	<nav class="navbar navbar-expand-lg navbar-dark navbar-light" style="background-color: #474A8A;">	
02	<div class="container-fluid">	
03	BIBLIOTEKA <i class="bi bi-book"></i>	
04	<button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle navigation">	
05		
06	</button>	
07	<div class="collapse navbar-collapse" id="navbarSupportedContent">	
08	<ul class="navbar-nav me-auto mb-2 mb-lg-0">	
09	<li class="nav-item">	
10	Książki	
11		
12	<li class="nav-item">	
13	Wypożyczenia	
14		
15	<li class="nav-item">	
16	O mnie	
17		
18	<li class="nav-item">	
19	Kontakt	

```

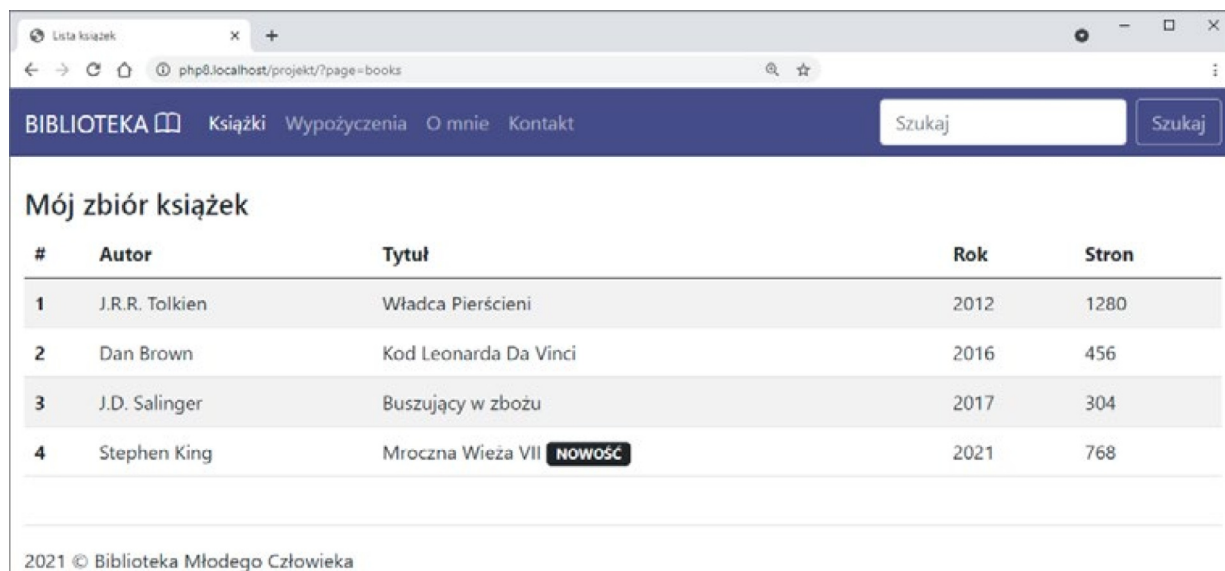
20         </li>
21     </ul>
22     <form id="form" method="post" class="d-flex">
23         <input class="form-control me-2" type="search" name="search"
24             placeholder="Szukaj" aria-label="Szukaj">
25         <button class="btn btn-outline-light" type="submit" disa-
26             bled>Szukaj</button>
27         <input type="hidden" name="page" value="books">
28     </form>
29 </div>

```

Plik footer.tpl zawiera kod ze stopką strony (patrz listing 40.8).

LISTING 40.8	SMARTY	smarty\templates\footer.tpl
01	<div class="container-fluid" style="margin-top: 40px;">	
02	<hr>	
03	{'Y' date} © Biblioteka Młodego Człowieka	
04	</div>	
05	</body>	
06	</html>	

LISTA KSIĄŻEK



Rysunek 40.2. Widok strony przy aktywnej zakładce Książki

Plik `books.tpl` wyświetla listę rekordów (książek) przekazanych w zmiennej `$books` (patrz rysunek 40.2 i listing 40.9). Do wyświetlenia listy wykorzystałem pętlę `foreach` (linie od 23 do 31). Jeśli lista jest wynikiem wyszukiwania, szukana fraza również jest drukowana (wiersze od 08 do 10).

LISTING 40.9	SMARTY	smarty\templates\books.tpl
01	<code>{include file="header.tpl"}</code>	
02	<code>{include file="js.tpl"}</code>	
03	<code>{include file="nav.tpl"}</code>	
04		
05	<code><div class="container-fluid"></code>	
06	<code>
</code>	
07	<code> <h4>Mój zbiór książek</h4></code>	
08	<code> {if \$search != ''}</code>	
09	<code> <h6>Szukana fraza: {\$search}</h6></code>	
10	<code> {/if}</code>	
11	<code> {* tabela z książkami *}</code>	
12	<code> <table class="table table-striped table-hover"></code>	
13	<code> <thead></code>	
14	<code> <tr></code>	
15	<code> <th scope="col">#</th></code>	
16	<code> <th scope="col">Autor</th></code>	
17	<code> <th scope="col">Tytuł</th></code>	
18	<code> <th scope="col">Rok</th></code>	
19	<code> <th scope="col">Stron</th></code>	
20	<code> </tr></code>	
21	<code> </thead></code>	
22	<code> <tbody></code>	
23	<code> {foreach item=book name=books from=\$books}</code>	
24	<code> <tr></code>	
25	<code> <th scope="row">{\$smarty.foreach.books.iteration}</th></code>	
26	<code> <td>{\$book.author}</td></code>	
27	<code> <td>{\$book.title} {if \$book.year == 'Y' date}<span class="badge</code>	
	<code>bg-dark">NOWOŚĆ{/if}</td></code>	
28	<code> <td>{\$book.year}</td></code>	
29	<code> <td>{\$book.pages}</td></code>	
30	<code> </tr></code>	
31	<code> {/foreach}</code>	
32	<code> </tbody></code>	
33	<code> </table></code>	
34	<code></div></code>	
35		
36	<code>{include file="footer.tpl"}</code>	

REJESTR WYPOŻYCZEŃ

#	Data	Tytuł książki	Wypożyczający
1	2021-03-20	Władca Pierścieni	Zofia Nowakówna
2	2021-03-22	Kod Leonarda Da Vinci	Oskar Milewski
3	2021-03-24	Kod Leonarda Da Vinci	Karina Promycek
4	2021-03-27	Buszujący w zbożu	Antoni Iksiński
5	2021-03-31	Mroczna Wieża VII NOWOŚĆ	Natalia Kornatowska

2021 © Biblioteka Młodego Człowieka

Rysunek 40.3. Widok strony przy aktywnej zakładce Wypożyczenia

Plik `register.tpl` wyświetla listę rekordów (wypożyczeń) przekazanych w zmiennej `$register` (patrz rysunek 40.3 i listing 40.10). Podobnie jak w przypadku listy książek, wykorzystałem pętlę `foreach` do wyświetlenia rejestru wypożyczeń (linie od 19 do 26).

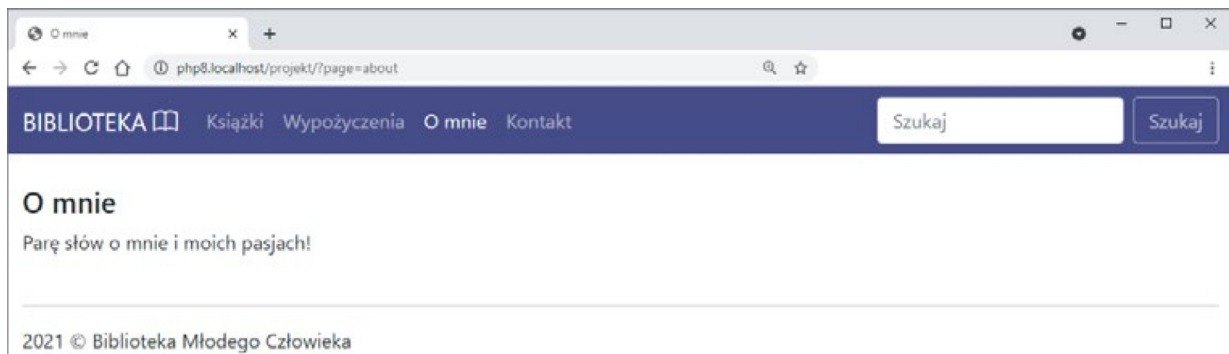
LISTING 40.10	SMARTY	smarty\templates\register.tpl
01	<code>{include file="header.tpl"}</code>	
02	<code>{include file="js.tpl"}</code>	
03	<code>{include file="nav.tpl"}</code>	
04		
05	<code><div class="container-fluid"></code>	
06	<code>
</code>	
07	<code><h4>Rejestr wszystkich wypożyczeń</h4></code>	
08	<code>{* tabela z wypożyczeniami *}</code>	
09	<code><table class="table table-striped table-hover"></code>	
10	<code><thead></code>	
11	<code><tr></code>	
12	<code><th scope="col">#</th></code>	
13	<code><th scope="col">Data</th></code>	
14	<code><th scope="col">Tytuł książki</th></code>	
15	<code><th scope="col">Wypożyczający</th></code>	
16	<code></tr></code>	
17	<code></thead></code>	
18	<code><tbody></code>	
19	<code>{foreach item=record name=register from=\$register}</code>	

```

20         <tr>
21             <th scope="row">{$smarty.foreach.register.iteration}</th>
22             <td>{$record.date}</td>
23             <td>{$record.title} {if $record.year == 'Y'|date}<span class="badge
                bg-dark">NOWOŚĆ</span>{/if}</td>
24             <td>{$record.name} {$record.surname}</td>
25         </tr>
26     </foreach>
27 </tbody>
28 </table>
29 </div>
30
31 {include file="footer.tpl"}

```

O MNIE

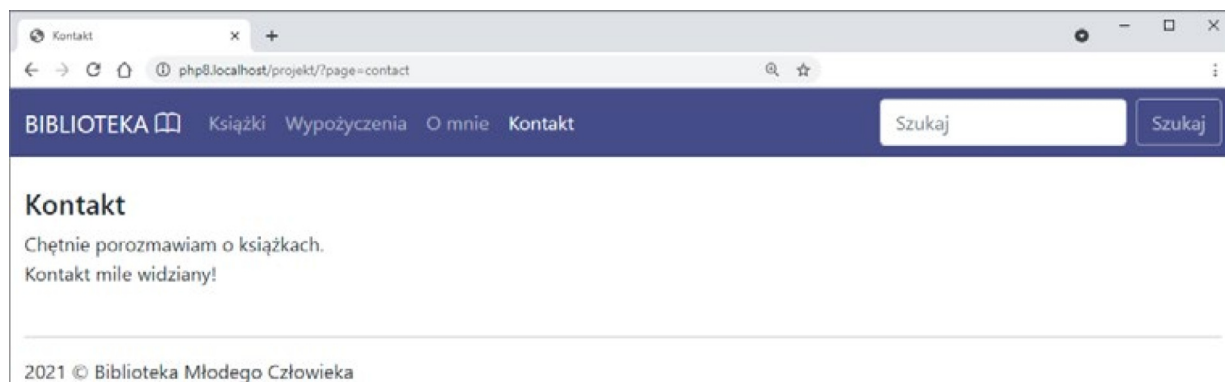


Rysunek 40.4. Widok strony przy aktywnej zakładce O mnie

Plik `about.tpl` zawiera informacje o autorze, które możesz śmiało uzupełnić (patrz rysunek 40.4 i listing 40.11).

LISTING 40.11	SMARTY	smarty\templates\about.tpl
01	{include file="header.tpl"}	
02	{include file="js.tpl"}	
03	{include file="nav.tpl"}	
04		
05	<div class="container-fluid">	
06	 	
07	O mnie	
08	Parę słów o mnie i moich pasjach!	
09	</div>	
10		
11	{include file="footer.tpl"}	

KONTAKT

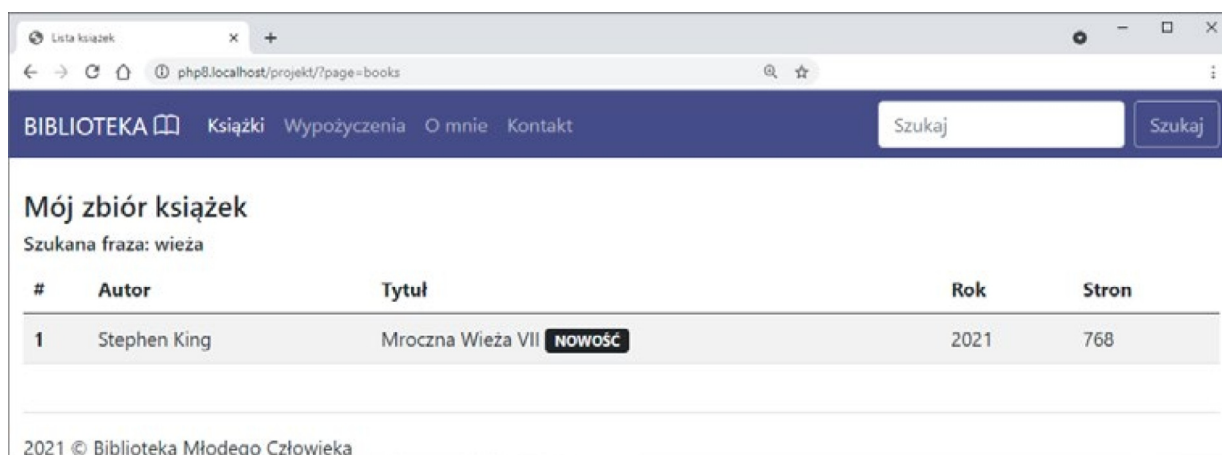


Rysunek 40.5. Widok strony przy aktywnej zakładce Kontakt

Plik `contact.tpl` zawiera informacje umożliwiające kontakt z autorem strony, czyli z Tobą (patrz rysunek 40.5 i listing 40.12). W praktyce powinien zawierać również formularz kontaktowy, którym zajmiesz się w następnej lekcji.

LISTING 40.12	SMARTY	smarty\templates\contact.tpl
01	<code>{include file="header.tpl"}</code>	
02	<code>{include file="js.tpl"}</code>	
03	<code>{include file="nav.tpl"}</code>	
04		
05	<code><div class="container-fluid"></code>	
06	<code>
</code>	
07	<code> <h4>Kontakt</h4></code>	
08	<code> <p></code>	
09	<code> Chętnie porozmawiam o książkach.
</code>	
10	<code> Kontakt mile widziany!</code>	
11	<code> </p></code>	
12	<code></div></code>	
13		
14	<code>{include file="footer.tpl"}</code>	

WYSZUKIWARKA



Rysunek 40.6. Widok strony z wynikiem wyszukiwania

Szukana fraza wpisana w pole tekstowe trafia do pliku `index.php` w zmiennej `$_POST['search']`. Przycisk *Szukaj* uaktywni się w momencie wpisania co najmniej jednego znaku (rysunek 40.6).

Komputer, tablet i smartfon

Strona internetowa powinna być responsywna (ang. *Responsive Web Design*, RWD), czyli dobrze wyglądać zarówno w przeglądarce na komputerze stacjonarnym, jak i na tablecie czy smartfonie. Biorąc pod uwagę różnorodność urządzeń mobilnych dostępnych na rynku i liczbę możliwych wariantów związanych z rozdzielczością ekranu, projektowanie stron responsywnych stało się już w branży IT standardem. Zwróć uwagę na to, że strony internetowe coraz częściej otwierane są właśnie na takich urządzeniach jak tablety i smartfony, co oznacza, że projektanci, budując nowoczesne projekty webowe, muszą to uwzględnić. Jeśli strona internetowa jest responsywna, jest bardzo duża szansa na to, że znajdzie się na wyższych pozycjach w wynikach wyszukiwarki Google, co ma istotne znaczenie w promowaniu serwisu WWW.

Jak udostępnić swoją stronę w sieci domowej?

Ciekawym pomysłem może być pochwalenie się swoim projektem w domu lub na lekcji informatyki. Możesz to zrobić przez udostępnienie serwera WWW bezpośrednio ze swojego komputera. Zakładam, że jesteś podłączony do sieci razem z innymi użytkownikami poprzez wspólny punkt dostępowy (np. router wi-fi) i nie masz włączonego oprogramowania blokującego ruch przychodzący, np. firewalla.

W pierwszej kolejności musisz się dowiedzieć, jaki adres IP został przydzielony Twojemu komputerowi w sieci lokalnej. Najprawdopodobniej będzie to adres rozpoczynający się od 192.168.x.x lub 172.16.x.x, np. 192.168.1.100 lub 172.16.1.80. Naciśnij klawisze **Win** + **R** i wpisz `cmd`, a po otwarciu się okna wpisz następujące polecenie:

```
ipconfig ↵
```

Zobaczysz listę wszystkich interfejsów sieciowych dostępnych na Twoim komputerze. Odszukaj ten, za pomocą którego łączysz się z siecią, np. karta ethernet (kabel podłączony do karty sieciowej, czyli tak zwana skrętka) lub karta wi-fi (sieć bezprzewodowa). Jeśli np. łączysz się z siecią przez wi-fi, konfiguracja może wyglądać tak:

```
Wireless LAN adapter Wi-Fi:
Connection-specific DNS Suffix . :
Link-local IPv6 Address . . . . . : fe80::a4ad:614a:16e3:d8fa%14
IPv4 Address. . . . . : 192.168.1.100
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.1.1
```

Powyższy zapis oznacza, że Twój komputer w sieci lokalnej ma adres IP 192.168.1.100 i pod tym adresem jest widoczny dla innych użytkowników (tej samej sieci).

Dotychczas uruchamiałeś serwer WWW w konsoli systemowej dla adresu lokalnego, czyli `localhost`; tym razem uruchom serwer, podając adres IP swojego komputera, np.:

```
php -S 192.168.1.100:8008 -t c:\xampp\apps\php8\htdocs ↵
```

Otwórz w przeglądarce stronę `http://192.168.1.100:8008/projekt`. Jeśli strona WWW uruchomi się poprawnie, będzie to oznaczać, że wszystko działa prawidłowo.

Teraz możesz się pochwalić swoim projektem przed domownikami i przyjaciółmi. Przekaż im powyższy adres z prośbą o wpisanie go w przeglądarce smartfona, tabletu czy komputera.

Ćwiczenia dodatkowe

1. Uzupełnij szablon `about.tp1` informacjami o książkach lub czasopismach, które przeczytałeś w ciągu ostatnich 12 miesięcy.
2. Wejdź na stronę <https://getbootstrap.com>, gdzie w zakładce *Documentation* znajdziesz liczne przykłady wykorzystania biblioteki Bootstrap. Przejrzyj je i postaraj się zrozumieć.
3. Poszukaj w sieci więcej informacji na temat RWD, czyli responsywności stron internetowych.

Pytania

Strona mobilna czy responsywna?

Choć potocznie terminy te używane są zamiennie, strona mobilna i strona responsywna znacznie się od siebie różnią. Przedstawiając to w dużym skrócie, strona responsywna zawiera w sobie cały kod dla wszystkich urządzeń (desktop, tablet, smartfon) i elastycznie dostosowuje swój wygląd oraz układ, co może wpływać na jej wydajność (np. powodować wolne ładowanie się na komórkach). Natomiast strona mobilna jest prostsza w nawigacji i dostarcza tylko najważniejszych informacji, jednak dostosowana jest wyłącznie do urządzeń mobilnych i wymaga dodatkowego adresu WWW (np. `mobile.adres.pl`).

Pamiętam czasy (początek lat dwutysięcznych), gdy oprócz strony klasycznej wykonywało się dodatkową wersję na urządzenia mobilne (stare komórki) z użyciem protokołu WAP (ang. *Wireless Application Protocol*). Z biegiem czasu, kiedy przybywało coraz to nowocześniejszych urządzeń wyposażonych w przeglądarkę internetową, technologia WAP odeszła w niepamięć. Potem pojawił się następny standard, XHTML Mobile Profile, który nie ma już wielkiego znaczenia. Dzisiejsze smartfony mają dużą moc obliczeniową i świetnie sobie radzą z większością responsywnych stron WWW, wykorzystujących najnowsze technologie webowe, takie jak HTML5, JavaScript i CSS.



Kontakt mile widziany

Istotnym elementem każdej strony internetowej jest bez wątpienia zakładka kontaktu, czyli miejsce, gdzie odwiedzający Twój serwis WWW dowie się, jak może się z Tobą skontaktować. Najlepiej będzie, jeśli umożliwisz kontakt przez wiele kanałów komunikacyjnych, czyli telefonicznie, mailowo, osobiście czy przez profile w witrynach społecznościowych. Z pewnością dodatkowym atutem będzie formularz kontaktowy, za pomocą którego będą wysyłane do Ciebie wiadomości. W tej lekcji zajmiesz się uruchomieniem takiego formularza na swojej stronie.

Co masz do zrobienia?

W pierwszej kolejności zmodyfikuj pliki `index.php` i `contact.tpl` — dodaj do nich obsługę formularza kontaktowego. Po aktualizacji plików uruchom stronę i wejdź do zakładki z kontaktem. Pojawi się dodatkowy przycisk *Formularz kontaktowy*, którego kliknięcie spowoduje wyświetlenie się formularza. Śmiało możesz go wypełnić i wysłać do siebie wiadomość. Rysunek 41.1 przedstawia nową stronę z kontaktem; porównaj, czy u Ciebie jest podobnie.

Wszystkie potrzebne listingi znajdziesz na następnych stronach. Przepisz je bardzo dokładnie.

Listingi

Warstwa logiki

Poniższy listing zawiera nowy kod źródłowy pliku `index.php`, uaktualniony o obsługę formularza kontaktowego. Dane z formularza przekazywane są w zmiennej globalnej `$_POST`, a następnie testowane (linie od 62 do 65) i filtrowane (linie od 68 do 70). Wszystkie informacje wstawiane są do zmiennej `$write` (wiersz 74) i zapisywane do pliku `messages.txt` na dysku komputera (wiersz 82).

Wiadomość z formularza kontaktowego może być również wysłana na adres e-mail za pomocą pakietu PHPMailer. Pakiet ten wykorzystuje uwierzytelnianie SMTP do wysyłania wiadomości, dlatego w liniach od 93 do 98 wprowadź konfigurację konta, z którego będą wysyłane wiadomości z formularza. Nie zapomnij podać swojego adresu e-mail i imienia w wierszach 88 i 89. Jeśli wszystkie dane konfiguracyjne będą wprowadzone, zmień w linii 20 wartość stałej E_SEND na true.

Rysunek 41.1. Widok strony z aktywnym formularzem kontaktowym

Na listingu 41.1 stylem pogrubionym oznaczyłem nowe linie kodu względem listingu 40.1. Uaktualnij plik `index.php` w katalogu projektu.

LISTING 41.1	PHP 8	index.php
01	<code><?php</code>	
02	<code>use PHPMailer\PHPMailer\{PHPMailer, SMTP, Exception};</code>	
03	<code>require_once __DIR__."/libs/vendor/autoload.php"; // autoloader</code>	
04		
05	<code>\$smarty = new Smarty;</code>	
06		
07	<code>// definicja katalogów roboczych</code>	
08	<code>\$smarty->setTemplateDir('smarty/templates'); // szablony</code>	


```

09 $smarty->setCompileDir('smarty/templates_c');
10 $smarty->setCacheDir('smarty/cache');
11 $smarty->setConfigDir('smarty/configs');
12
13 // którą stronę wyświetlić?
14 $page = (isset($_REQUEST['page'])) ? trim(strip_tags($_REQUEST['page'])) : '';
15 $smarty->assign('page', $page);
16
17 require_once 'inc/db.class.php'; // obsługa bazy danych
18 require_once 'inc/book.class.php'; // główna klasa projektu
19
20 define('E_SEND', false); // wysłać wiadomość z formularza na adres e-mail?
21
22 /*
23  E_MYSQL — baza danych MySQL
24  E_SQLITE — baza danych SQLite
25  */
26 $project = new MyBook(E_MYSQL); // tworzenie egzemplarza klasy
27
28 // wyświetlenie szablonu
29 switch ($page)
30 {
31     case 'books':
32         // może czegoś szukasz?
33         $search =
34             (isset($_POST['search'])) ? trim(strip_tags($_POST['search'])) : '';
35
36         /* pobierz informacje o książkach */
37         $books = $project->books(search: $search);
38         $smarty->assign('books', $books);
39         $smarty->assign('search', $search);
40
41         $smarty->assign('title', 'Lista książek');
42         $smarty->display('books.tpl');
43         break;
44
45     case 'register':
46         /* pobierz informacje o wypożyczeniach */
47         $register = $project->register();
48         $smarty->assign('register', $register);
49
50         $smarty->assign('title', 'Wypożyczenia');
51         $smarty->display('register.tpl');
52         break;
53
54     case 'about':
55         $smarty->assign('title', 'O mnie');

```

```

56     $smarty->display('about.tpl');
57     break;
58
59     case 'contact':
60         //formularz kontaktowy
61         if (
62             isset($_POST['contactForm']) &&
63             isset($_POST['name']) && $_POST['name'] != '' &&
64             isset($_POST['email']) && $_POST['email'] != '' &&
65             isset($_POST['message']) && $_POST['message'] != ''
66         ) {
67             // proste filtrowanie danych z formularza
68             $name = trim(strip_tags($_POST['name']));
69             $email = trim(strip_tags($_POST['email']));
70             $message = trim(strip_tags($_POST['message']));
71
72             $smarty->assign('contactForm', $name);
73
74             $write =
75             "===> Data: ".date("Y-m-d H:i:s").PHP_EOL.
76             "Imię: ".$name.PHP_EOL.
77             "E-mail: ".$email.PHP_EOL.
78             "Wiadomość:\r\n".$message.PHP_EOL.
79             "<===" . PHP_EOL . PHP_EOL;
80
81             // zapisz do pliku
82             file_put_contents ("messages.txt", $write, FILE_APPEND);
83
84             // wyślij na adres e-mail
85             if (E_SEND) {
86                 $mail = new PHPMailer(true);
87                 try {
88                     $myEmail = 'moj@email.pl';
89                     $myName = 'Ja';
90
91                     /* konfiguracja z uwierzytelnianiem SMTP */
92                     $mail->isSMTP();
93                     $mail->Host      = 'smtp.serwer.pl';
94                     $mail->SMTPAuth  = true;
95                     $mail->Username  = 'moj@email.pl';
96                     $mail->Password  = 'hasło';
97                     $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
98                     $mail->Port      = 587;
99
100                    $mail->CharSet = 'UTF-8';
101                    $mail->setFrom($myEmail, $myName);
102                    $mail->addAddress($myEmail, $myName);

```

```

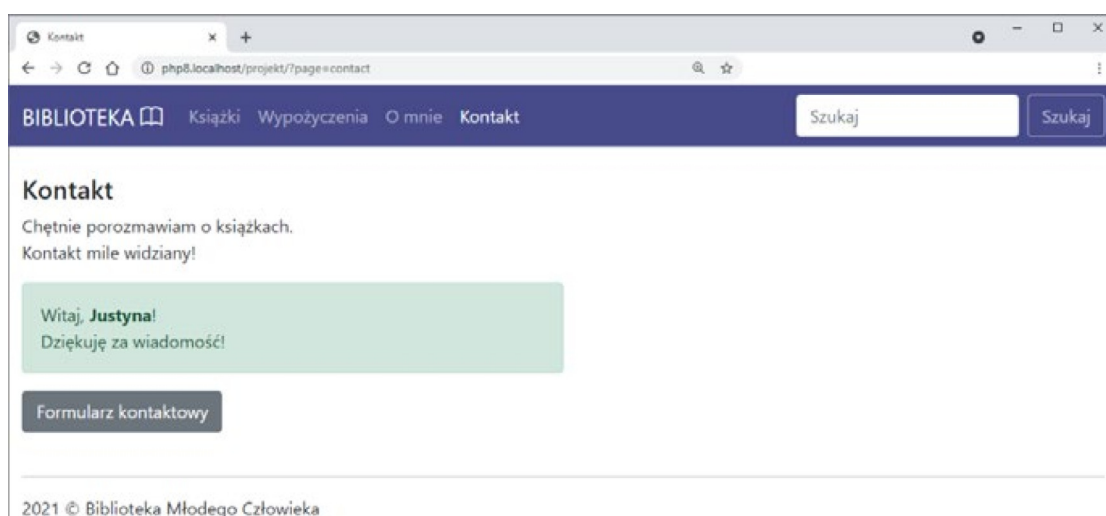
103         $mail->Subject = 'Wiadomość ze strony';
104         $mail->Body = $write;
105         $mail->send();
106     } catch (Exception $e) {
107         $smarty->assign('contactError', "Wiadomość nie została
wysłana.<br>Komunikat błędu: {$mail->ErrorInfo}");
108     }
109 }
110 }
111 $smarty->assign('title', 'Kontakt');
112 $smarty->display('contact.tpl');
113 break;
114
115 default:
116     $smarty->assign('title', 'Biblioteka Młodego Człowieka');
117     $smarty->display('index.tpl');
118 }
119
120 unset($project);

```

Warstwa widoku

Poniżej znajduje się aktualna zawartość szablonu kontaktowego, którą zapiszesz ponownie w pliku `contact.tpl` na dysku. Jak pewnie zauważyłeś, nowy kod jest umieszczony pomiędzy liniami 13 a 44. Po wysłaniu formularza zostanie wyświetlony odpowiedni komunikat (wiersze od 13 do 17). W przypadku wystąpienia problemów z wysłaniem wiadomości na podany adres e-mail zostanie wyświetlony komunikat o błędzie (linie od 18 do 22).

Informację o poprawnym wysłaniu wiadomości przedstawia rysunek 41.2.



Rysunek 41.2. Informacja o poprawnym wysłaniu formularza kontaktowego

Przepisz listing 41.2 i sprawdź w przeglądarce działanie formularza kontaktowego. Pamiętaj, że aby wykorzystać pakiet PHPMailer do wysyłania wiadomości na adres e-mail, należy poprawnie wpisać konfigurację konta SMTP.

LISTING 41.2	SMARTY	smarty\templates\contact.tpl
01	<code>{include file="header.tpl"}</code>	
02	<code>{include file="js.tpl"}</code>	
03	<code>{include file="nav.tpl"}</code>	
04		
05	<code><div class="container-fluid"></code>	
06	<code>
</code>	
07	<code> <h4>Kontakt</h4></code>	
08	<code> <p></code>	
09	<code> Chętnie porozmawiam o książkach.
</code>	
10	<code> Kontakt mile widziany!</code>	
11	<code> </p></code>	
12		
13	<code> {if isset(\$contactForm)}</code>	
14	<code> <div class="alert alert-success col-6" role="alert"></code>	
15	<code> Witaj, {\$contactForm}!
Dziękuję za wiadomość!</code>	
16	<code> </div></code>	
17	<code> {/if}</code>	
18	<code> {if isset(\$contactError)}</code>	
19	<code> <div class="alert alert-danger col-6" role="alert"></code>	
20	<code> {\$contactError}</code>	
21	<code> </div></code>	
22	<code> {/if}</code>	
23	<code> <p></code>	
24	<code> <a class="btn btn-secondary" data-bs-toggle="collapse" href="#contact-</code>	
	<code>Form" role="button" aria-expanded="false" aria-controls="contactForm">Formularz</code>	
	<code>kontaktowy</code>	
25	<code> </p></code>	
26	<code> <form method="post" id="contactForm" class="collapse multi-collapse col-6"></code>	
27	<code> <div class="mb-3"></code>	
28	<code> <label class="form-label">Twoje imię</label></code>	
29	<code> <input type="text" class="form-control" id="name" name="name" place-</code>	
	<code>holder="Twoje imię" required></code>	
30	<code> </div></code>	
31	<code> <div class="mb-3"></code>	
32	<code> <label class="form-label">Twój adres e-mail</label></code>	
33	<code> <input type="email" class="form-control" id="email" name="email"</code>	
	<code>placeholder="Twój adres e-mail" required></code>	
34	<code> </div></code>	
35	<code> <div class="mb-3"></code>	
36	<code> <label class="form-label">Wiadomość</label></code>	
37	<code> <textarea class="form-control" id="message" name="message" rows="4"</code>	
	<code>placeholder="Napisz coś do mnie ;)" required></textarea></code>	
38	<code> </div></code>	

```

39         <div class="mb-3">
40             <button type="submit" name="contactForm" class="btn btn-second-
               ary">Wyślij</button>
41         </div>
42         <input type="hidden" name="page" value="contact">
43     </form>
44 </div>
45
46 {include file="footer.tpl"}

```

Ćwiczenia dodatkowe

1. Spróbuj zapisać dane pochodzące z formularza do bazy danych. Wykorzystaj istniejącą bazę SQLite — `kartoteka.db` lub bazę MySQL. Będziesz w tym celu musiał utworzyć dodatkową tabelę, np.:

SQLite

```

CREATE TABLE IF NOT EXISTS wiadomosci
(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    imie TEXT DEFAULT '',
    email TEXT DEFAULT '',
    tekst TEXT DEFAULT '',
    data TEXT DEFAULT ''
);

```

MySQL

```

CREATE TABLE IF NOT EXISTS `wiadomosci` (
  `id` int PRIMARY KEY AUTO_INCREMENT,
  `imie` tinytext NOT NULL,
  `email` tinytext NOT NULL,
  `tekst` text NOT NULL,
  `data` date
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

```

2. Jeśli ćwiczenie 1 nie będzie dla Ciebie dużym problemem, napisz kod wyświetlający wszystkie wiadomości zapisane w bazie danych na stronie z kontaktem. Sposób realizacji zadania i prezentacji wyników może być dowolny.
3. Jeśli ćwiczenia 1 i 2 jednak sprawiają Ci trudności, spróbuj wyświetlić na stronie z kontaktem wiadomości zapisane w pliku tekstowym `messages.txt`. Wykorzystaj w tym celu funkcję `file_get_contents`.
4. Poszukaj w sieci informacji na temat metod zabezpieczania formularzy przed spamem.

Pytania

Co to jest CAPTCHA?

Jest to rozwiązanie informatyczne umożliwiające rozpoznanie, czy formularz umieszczony na stronie jest wypełniany przez człowieka, czy przez maszynę (robota internetowego). Technika ta chroni portale przed sztucznym zakładaniem kont, blogi przed automatycznym dodawaniem komentarzy, a formularze przed spamem. Z CAPTCHA najprawdopodobniej spotkałeś się przy wypełnianiu formularzy na stronach WWW, kiedy to należało odczytać tekst umieszczony na obrazku. Człowiekowi odczytanie tekstu z obrazka nie sprawia dużego kłopotu, jednak dla komputera jest to bardzo trudne (przynajmniej w założeniu).



A może trochę rozrywki?

Przyszedł czas, aby trochę odetchnąć i się zrelaksować. W zasadzie przerobiłeś już wszystkie lekcje zawarte w niniejszej książce, z czego powinieneś być bardzo dumny. Wykonałeś ogromną pracę, która z pewnością zaowocuje w przyszłości ciekawymi i innowacyjnymi projektami webowymi. Pamiętaj, że tylko praktyka czyni mistrza, dlatego czytaj, ćwicz i programuj, kiedy tylko masz na to ochotę.

W tej lekcji napiszesz kod popularnej gry w wisielca (ang. *hangman*), którą umieścisz na swojej stronie. Jestem przekonany, że zasady gry są Ci doskonale znane; po prostu wprowadzając kolejne litery, musisz odgadnąć hasło. Wygrywasz, jeśli hasło zostanie na czas odgadnięte.

Co masz do zrobienia?

W pierwszej kolejności dodaj nowe pliki `game.class.php` i `game.tpl`, zawierające kod gry, oraz uaktualnij pliki `index.php` i `nav.tpl`. Kiedy już to zrobisz, wejdź na stronę w przeglądarce i sprawdź, czy nie pojawiają się komunikaty o błędach. Rysunek 42.1 przedstawia nową stronę startową z grą — porównaj, czy u Ciebie jest podobnie. Zwróć uwagę, że w menu pojawiła się nowa zakładka o nazwie *Gra*.

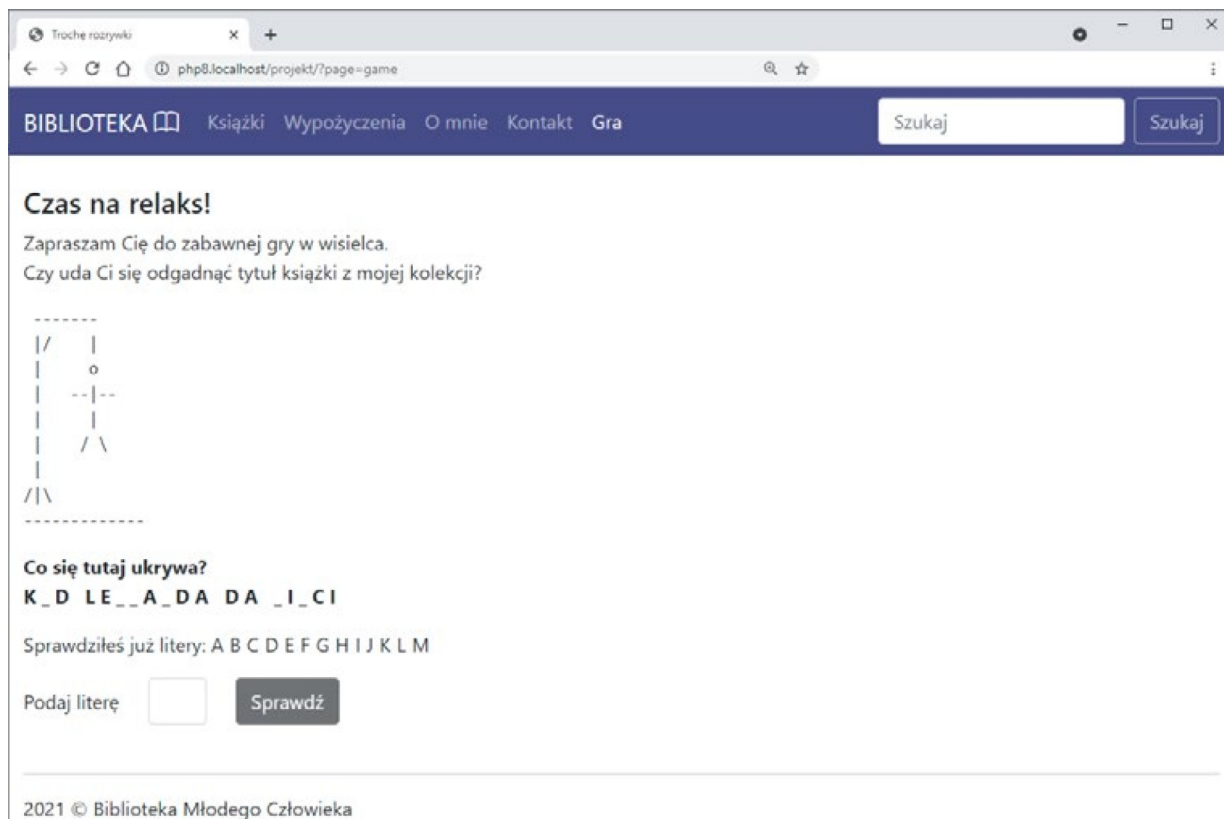
Wszystkie potrzebne listingi znajdziesz na następnych stronach. Przepisz je bardzo dokładnie.

Listingi

Warstwa logiki

Plik `game.class.php` zawiera definicję klasy gry w wisielca (listing 42.1). Projektując tę klasę, wykorzystałem popularne w sieci tekstowe wizerunki (znaki ASCII) kolejnych scen w grze. Konstruktor klasy definiuje wszystkie parametry gry (linie od 100 do 106). Metoda `check` (wiersz 110) sprawdza, czy wprowadzona w formularzu litera znajduje się w hasle. Metoda `start` (linia 125) uruchamiana jest tylko podczas inicjacji gry,

a metoda `play` (linia 139) na kolejnych etapach. Kod starałem się dokładnie opisać w komentarzach, dlatego jestem przekonany, że świetnie poradzisz sobie z jego zrozumieniem.



Rysunek 42.1. Widok strony przy aktywnej zakładce *Gra*

LISTING 42.1	PHP 8	inc\game.class.php
01	<?php	
02	declare(strict_types = 1); // dbamy o prawidłowe typy danych	
03		
04	mb_internal_encoding("UTF-8");	
05		
06	// etapy gry	
07	define('GAME_DURING', 1);	
08	define('GAME_WIN', 2);	
09	define('GAME_OVER', 3);	
10		
11	// definicja naszego wisielca	
12	define('HANGMAN', [	
13	// scena 1	
14	' -----	
15	/	
16		
17		


```

18  |
19  |
20  |
21  /|\
22  -----',
23  //scena 2
24  ' -----
25  |/   |
26  |    o
27  |
28  |
29  |
30  |
31  /|\
32  -----',
33  //scena 3
34  ' -----
35  |/   |
36  |    o
37  |    |
38  |    |
39  |
40  |
41  /|\
42  -----',
43  //scena 4
44  ' -----
45  |/   |
46  |    o
47  |    |
48  |    |
49  |    /
50  |
51  /|\
52  -----',
53  //scena 5
54  ' -----
55  |/   |
56  |    o
57  |    |
58  |    |
59  |    / \
60  |
61  /|\
62  -----',

```

```

63 //scena 6
64 ' -----
65  |/   |
66  |   o
67  |  --|
68  |   |
69  |   / \
70  |
71  /|\
72 -----',
73 //scena 7
74 ' -----
75  |/   |
76  |   o
77  |  --|--
78  |   |
79  |   / \
80  |
81  /|\
82 -----'
83 ]);
84
85 class Hangman
86 {
87     private $count = 0; //liczba możliwych haseł do odgadnięcia
88     private $current = 0; //indeks aktualnego hasła
89     private $texts = []; //tablica z hasłami
90     private $text = ''; //aktualne hasło
91     private $letters = 0; //liczba liter w hasle
92     private $template = ''; //szablon z ukrytymi literami
93     private $errors = 0; //liczba błędnych typowań
94
95     public function __construct(array $texts, int $current = -1, int $errors = 0)
96     {
97         foreach ($texts as $k=>$v) {
98             $texts[$k] = str_replace(' ', '^', $v); //zamień spacje na znak ^
99         }
100         $this->texts = $texts;
101         $this->count = count($this->texts);
102         $this->current = ($current == -1) ? rand(0, $this->count - 1) : $current;
103         $this->text = mb_strtoupper($this->texts[$this->current]);
104         $this->letters = mb_strlen($this->text);
105         $this->template = str_repeat('_ ', $this->letters);
106         $this->errors = $errors;
107     }
108

```

```

109 // sprawdź, czy wprowadzone litery znajdują się w hasle
110 private function check(string $entered) : string
111 {
112     for ($x = 0; $x < $this->letters; $x++)
113     {
114         $char = $this->text[$x];
115         if (mb_strstr($entered, $char))
116         {
117             $pos = $x * 2;
118             $this->template[$pos] = $char;
119         }
120     }
121
122     return $this->template;
123 }
124
125 public function start() : array
126 {
127     $this->check(entered: '^'); // oddziel wyrazy
128
129     return [
130         'status' => GAME_DURING,
131         'stage' => HANGMAN[0],
132         'template' => $this->template,
133         'current' => $this->current,
134         'entered' => '',
135         'errors' => 0,
136     ];
137 }
138
139 public function play(string $letter, string $entered) : array
140 {
141     $this->check(entered: '^'); // oddziel wyrazy
142
143     $letter = mb_strtoupper(trim($letter));
144     $entered = mb_strtoupper(trim($entered));
145
146     // jeśli nie ma litery w hasle, to zwiększ licznik błędów
147     if (!mb_strstr($this->text, $letter))
148     {
149         $this->errors++;
150     }
151
152     // zakończ grę po 7 scenie
153     if ($this->errors > 6)
154     {

```

```

155         return [
156             'status' => GAME_OVER,
157             'stage' => HANGMAN[6],
158             'text' => $this->text
159         ];
160     }
161
162     $entered .= $letter;
163
164     //usuń duplikaty i posortuj typowane litery
165     $tmp = array_unique(mb_str_split($entered), SORT_STRING);
166     sort($tmp);
167     $entered = implode('', $tmp);
168
169     $this->check($entered); //sprawdź hasło
170
171     if (!mb_strstr($this->template, "_"))
172     {
173         return [
174             'status' => GAME_WIN,
175             'stage' => HANGMAN[$this->errors],
176             'text' => $this->text
177         ];
178     }
179
180     return [
181         'status' => GAME_DURING,
182         'stage' => HANGMAN[$this->errors],
183         'template' => $this->template,
184         'current' => $this->current,
185         'entered' => $entered,
186         'errors' => $this->errors,
187     ];
188 }
189 }

```

Odszukaj w pliku `index.php` sekcję `switch($page){...}` i wstaw poniższy kod. Możesz go wstawić po sekcji obsługującej zakładkę *Kontakt*, np.:

```

switch ($page) {
    case 'contact':
        ...
        break;

    case 'game': ← tutaj wstaw poniższy kod z listingu 42.2
        ...
        break;
}

```

LISTING 42.2

PHP 8

index.php

```

01 case 'game':
02     // tymczasowa tablica z informacjami o aktualnym stanie gry
03     $progres = [];
04     $progres['letter'] = (isset($_POST['letter']) && $_POST['letter'] != '')
    ? $_POST['letter'] : '_';
05     $progres['entered'] = (isset($_POST['entered']) && $_POST['entered'] !=
    '') ? $_POST['entered'] : '';
06     $progres['current'] = (isset($_POST['current']) && $_POST['current'] !=
    '') ? (int)$_POST['current'] : -1;
07     $progres['errors'] = (isset($_POST['errors']) && $_POST['errors'] != '')
    ? (int)$_POST['errors'] : 0;
08
09     // tytuły książek z bazy danych będą hasłami do odgadnięcia w grze
10     $books = $project->books();
11     foreach ($books as $book) {
12         $progres['books'][] = $book['title'];
13     }
14
15     require_once 'inc/game.class.php';
16     $hangman = new Hangman(
17         texts: $progres['books'],
18         current: $progres['current'],
19         errors: $progres['errors']
20     );
21
22     // nowa gra czy kontynuacja?
23     $game = (isset($_POST['game'])) ? $hangman->play($progres['letter'],
    $progres['entered']) : $hangman->start();
24
25     $smarty->assign('title', 'Trochę rozrywki');
26     $smarty->assign('game', $game);
27     $smarty->display('game.tpl');
28     break;

```

Warstwa widoku

Plik `game.tpl` zawiera obsługę kolejnych etapów gry (listing 42.3). Zwróć uwagę na konstrukcję formularza (linie od 14 do 28), w którym umieściłem ukryte pola (wiersze od 24 do 27). Podczas wysyłania formularza z kolejną literą (linia 19) dołączane są informacje dodatkowe związane z aktualnym stanem gry. Dodałem trochę kodu w JavaScriptcie (wiersze od 30 do 35), aby zabezpieczyć formularz. Przycisk *Wyślij* uaktywni się więc w momencie, gdy w polu tekstowym zostanie wpisany jeden znak. Wykorzystałem charakterystyczną dla szablonów Smarty instrukcję warunkową `{if}{elseif}{/if}` (linie 11, 37 i 43) do obsługi kolejnych stanów gry.

LISTING 42.3

SMARTY

smarty\templates\game.tpl

```

01 {include file="header.tpl"}
02 {include file="js.tpl"}
03 {include file="nav.tpl"}
04
05 <div class="container-fluid">
06     <br>
07     <h4>Czas na relaks!</h4>
08     <p>Zapraszam Cię do zabawnej gry w wisielca.<br>
09     Czy uda Ci się odgadnąć tytuł książki z mojej kolekcji?</p>
10     <pre>{$game.stage}</pre>
11     {if $game.status == 1}
12         <p><strong>Co się tutaj ukrywa?<br>{$game.template|re-
13         place:'^': '&nbsp;'}</strong></p>
14         {if $game.entered != ''}<p>Sprawdziłeś już litery: <span style="letter-spa-
15         cing: 5px;">{$game.entered}</span></p>{/if}
16         <form method="post" id="game" class="row">
17             <div class="col-auto">
18                 <label for="letter" class="col-form-label">Podaj literę</label>
19             </div>
20             <div class="col-auto">
21                 <input type="text" class="form-control" name="letter" id="letter"
22                 minlength="1" maxlength="1" placeholder="" style="width: 3em;" required>
23             </div>
24             <div class="col-auto">
25                 <button type="submit" name="game" class="btn btn-dark" disa-
26                 bled>Sprawdź</button>
27             </div>
28             <input type="hidden" name="page" value="game">
29             <input type="hidden" name="current" value="{ $game.current }">
30             <input type="hidden" name="entered" value="{ $game.entered }">
31             <input type="hidden" name="errors" value="{ $game.errors }">
32         </form>
33         <script>
34             $(function() {
35                 $('#letter', '#game').on('input', function() {
36                     var letter = $(this).val();
37                     $('button[name="game"]', '#game').attr("disabled", (letter.length ==
38                     1) ? false : true);
39                 });
40             });
41         </script>
42     {elseif $game.status == 2}
43         <div class="alert alert-success" role="alert">
44             <p>Brawo! Odgadłeś prawidłowo!<br>
45             Oczywiście, że chodziło o "<strong>{$game.text|replace:'^': '
46             '}</strong>"</p>

```

```

41         <a href="?page=game" class="btn btn-primary btn-sm">Spróbuj jeszcze
    raz!</a>
42     </div>
43     {elseif $game.status == 3}
44         <div class="alert alert-danger" role="alert">
45             <p>Przykro mi, ale nie odgadłeś tytułu mojej książki ;(<br>
46             Chodziło o "<strong>{$game.text|replace:'^: '}</strong>"</p>
47             <a href="?page=game" class="btn btn-primary btn-sm">Spróbuj jeszcze
    raz!</a>
48         </div>
49     {/if}
50 </div>
51
52 {include file="footer.tpl"}

```

Teraz wystarczy dodać następną zakładkę w menu strony — zmodyfikuj zatem plik `nav.tpl` i wstaw kod z listingu 42.4 pomiędzy linie 20 a 21.

LISTING 42.4	SMARTY	smarty\templates\nav.tpl
01	<li class="nav-item">	
02	Gra	
03		

Ćwiczenia dodatkowe

1. Udostępnij stronę w sieci domowej i zaproś kogoś do gry. Udałej zabawy!
2. Stan gry możesz przechowywać na serwerze w tak zwanych sesjach. Informacje przechowywane są w zmiennej superglobalnej `$_SESSION` i dostępne z poziomu kodu programu. Wejdź na stronę <https://www.php.net/manual/en/session.examples.php> i zapoznaj się z tym mechanizmem. Jak zmodyfikowałbyś kod programu, aby wykorzystać sesje?



Trochę porad na zakończenie

Skończyłeś wszystkie lekcje, więc możesz uznać, że podstawy programowania w języku PHP z wykorzystaniem języka SQL masz już opanowane. Zakładam, że przerobiłeś wszystkie ćwiczenia, również te dodatkowe, i możesz się zająć bardziej zaawansowanymi aspektami programowania.

Na zakończenie trochę porad, czyli co i jak dalej robić. Jesteś na etapie odkrywania tajemnic w programowaniu, dlatego powinieneś tworzyć bardziej zaawansowane programy niż zawarte w tej książce. Pamiętaj, że:

- proces nauki i zdobywania doświadczenia będzie trwał latami, taka jest prawda;
- ucz się na swoich błędach, z biegiem lat docenisz tę umiejętność;
- analizuj kod napisany przez innych programistów i staraj się go wykorzystać;
- nie przejmuj się krytyką, „nobody’s perfect” — czasami programiści mają odmienne pomysły na rozwiązanie konkretnego problemu, a wszystkie są poprawne (tylko który jest najlepszy?);
- opanowanie kilku języków programowania ułatwi Ci znalezienie fajnego zajęcia;
- cały czas rozwijaj swoje umiejętności, aby być gotowym na swoje pięć minut;
- pamiętaj o wypoczynku, najlepiej z dobrą książką o programowaniu!

Chciałbym polecić Ci następujące serwisy internetowe oraz książki, w których możesz znaleźć wiele ciekawych pomysłów, rozwiązań i porad innych programistów.

Bezpłatne skrypty do pobrania

- HotScripts (<https://www.hotscripts.com>) — jeden z najstarszych serwisów w sieci, udostępniający bezpłatnie skrypty w wielu językach programowania, m.in. PHP, C, C++, Python i JavaScript.

- PHPclasses (<https://www.phpclasses.org>) — podobnie jak HotScripts, jest to serwis z ponad 20-letnim stażem, oferujący darmowe skrypty w PHP.
- GitHub (<https://github.com>) — popularny serwis z tak zwanymi repozytoriami, wykorzystywany przez programistów do kontroli wersji kodu. Możesz znaleźć tutaj wiele ciekawych skryptów, które możesz dalej rozwijać i wykorzystać w swoich projektach.

Fora dla programistów

- Stack Overflow (<https://stackoverflow.com>) — społecznościowy serwis dla programistów, gdzie znajdziesz wiele odpowiedzi na pytania związane z szeroko pojętym programowaniem.
- Forum php.pl (<http://forum.php.pl>) — polskie forum programistów PHP, na którym znajdziesz wiele interesujących tematów związanych m.in. z bazami danych i serwerami WWW.

Książki

- Jest wiele powodów, dla których warto czytać książki, choćby stymulacja umysłu, pogłębianie wiedzy, wzbogacanie słownictwa czy poprawa zdolności do zapamiętywania informacji. Poniżej polecam najciekawsze pozycje.

Dla projektantów (front-endowców) i programistów (back-endowców):

- L. Lemay, R. Colburn, J. Kyrnin, *HTML, CSS i JavaScript. Wydanie VII* (<https://helion.pl/ksiazki/htcdk7>) — ponad 700 stron szczegółowych informacji i wskazówek dla projektantów stron WWW.
- H. Hayder, J. P. Maia, L. Gheorge, *Smarty. Szablony w aplikacjach PHP* (<https://helion.pl/ksiazki/smarty>) — kompleksowy opis technologii Smarty z licznymi przykładami. Pozycja zarówno dla początkujących, jak i doświadczonych programistów.
- B. Forta, *SQL w mgnieniu oka. Wydanie IV* (<https://helion.pl/ksiazki/sqldko>) — przyjazne kompendium wiedzy na temat języka SQL wraz z licznymi przykładami.

- B. Jakobus, J. Marah, *Bootstrap 4 dla zaawansowanych. Jak pisać znakomite aplikacje internetowe* (<https://helion.pl/ksiazki/boo4z2>) — wprowadzenie do Bootstrapa 4 i jego najużyteczniejszych elementów.
- Ł. Pasternak, *CSS3. Tworzenie nowoczesnych stron WWW* (<https://helion.pl/ksiazki/css3tw>) — kompletny opis najnowszego standardu arkuszy stylów kaskadowych (CSS3) wraz z licznymi przykładami.

Dla chcących pogłębić tajniki programowania w PHP:

- M. Rahman, *PHP 7. Algorytmy i struktury danych* (<https://helion.pl/ksiazki/php7a1>) — bardzo ciekawa (o ile nie jedyna) książka podejmująca tak obszernie temat struktur danych w języku PHP 7. Z książki dowiesz się, jak napisać kod implementujący listy, kolejki, drzewa i grafy.
- M. Zandstra, *PHP. Obiekty, wzorce, narzędzia. Wydanie V* (<https://helion.pl/ksiazki/phpob5>) — wszystko o programowaniu zorientowanym obiektowo w PHP 7.

Skorowidz

A

Apache NetBeans, 10, 24, 31
konfiguracja z PHP, 32
uruchamianie kodu PHP, 38

B

Bootstrap, 298, 305

C

Composer, 58, 285
komenda
install, 63
remove, 63
require, 63
update, 63
CSS, 283

D

Definicja funkcji, 122
anonimowej, 125
instrukcja
global, 133
return, 127
z argumentem, 123
zwrotnej, 125

E

Eclipse, 10, 24, 28
konfiguracja z PHP, 29
uruchamianie kodu PHP, 38
Exceptions, *Patrz* Wyjątki

F

Format heredoc, 73
Funkcja
abs, 116
array_column, 150
array_keys, 149
array_merge, 150
array_pop, 143
array_push, 143
array_sum, 150
array_unique, 150
asort, 148
checkdate, 182
cos, 116
count, 143
date, 182

date_default_timezone_set, 183
deg2rad, 117
explode, 71
fclose, 159
fdiv, 23, 129
fgetc, 160
fgets, 160
file_get_contents, 162
file_put_contents, 162
filesize, 159
flock, 167
fopen, 159
fread, 159
fseek, 166
function_exists, 179
funkcja gettype, 23
fwrite, 160
get_debug_type, 23
get_resource_id, 23
in_array, 143
include, 177
include_once, 177
is_file, 159
is_integer, 251
is_string, 251
isset, 178
key, 149
key_exists, 149
krsort, 147
ksort, 143, 147
log, 116
log10, 116
ltrim, 71
mb_convert_case, 101
mb_stripos, 94
mb_strlen, 94
mb_strpos, 94
mb_strrpos, 94
mb_strtolower, 101
mb_strtoupper, 101
mb_substr, 101
mkdir, 168
mktime, 119, 182
phpinfo, 51
pi, 112
preg_match, 251
print, 40
print_r, 73, 143
printf, 43, 115
rand, 182

require, 177
require_once, 177
rmdir, 168
rsort, 146
rtrim, 71
sin, 116
sort, 143, 146
sqrt, 112
str_contains, 71, 74
str_ends_with, 71, 74
str_replace, 101
str_starts_with, 71, 74
stream_socket_accept, 263
stream_socket_client, 265
stream_socket_recvfrom, 263
stream_socket_sendto, 263, 265
stream_socket_server, 263
stream_socket_shutdown, 265
strrev, 103
strspn, 94
strtotime, 182
tan, 116
time, 119, 182
trim, 71
ucfirst, 101
ucwords, 101
unlink, 168
var_dump, 149
var_export, 149
Funkcje sieciowe, 260

H

HTML, 290
szkielet dokumentu
hipertekstowego, 275
znaczniki, 276
HTML5, 277

J

JavaScript, 283, 305
JIT, 7, 14
Just In Time, *Patrz* JIT

K

Kodowanie UTF-8, 100
Komentarz
blokowy, 46
liniowy, 46

Konstrukcja warunkowa, 76
 if, 77
 if-else, 78
 if-elseif-else, 79
 match, 16, 84, 129
 operator
 porównania, 76
 trójargumentowy, 80
 switch, 83

M

MySQL, 24, 49, 214, 227,
 260, 298, 299, 321

N

Notepad++, 10, 24, 35
 uruchamianie kodu PHP, 38

O

OOP, 190
 Operator logiczny
 ! (NOT), 90
 AND, 88
 OR, 89
 Operatory
 arytmetyczne, 105
 bitowe, 105
 logiczne, 87, 105
 rzutowania, 107

P

PDO
 bindParam, 250
 bindValue, 243, 250
 execute, 243
 prepare, 243
 Pętla
 do-while, 136
 for, 135
 foreach, 150
 instrukcja
 break, 139, 150
 continue, 139, 150
 while, 136
 PHP 8
 atrybuty, 21
 deklaracja właściwości
 klasy, 18
 konstrukcja match, 16, 84,
 129
 mechanizm WeakMaps, 23
 nazwa argumentu, 15
 operacje na łańcuchach, 21
 operator zerowy, 20
 typy łączone, 17

PHP Composer, 58, 285
 PHP Data Objects, *Patrz* PDO
 PHPMailer, 62, 286, 316, 320
 phpMyAdmin, 54, 56
 Pliki
 odczytywanie danych, 161
 tryby otwarcia, 160
 zapisywanie danych, 160
 Programowanie zorientowane
 obiektowo, 190
 definicja klasy, 190
 destruktor klasy, 301
 dziedziczenie, 198
 enkapsulacja danych, 195
 instancje klasy, 192
 interfejsy, 205
 klasa abstrakcyjna, 205,
 206
 kompozycja, 200
 konstruktor klasy, 193
 modyfikator
 private, 205
 protected, 205
 public, 205
 wyrażenie
 abstract, 206
 extends, 198, 303
 final, 207
 static, 213

S

Serwer PHP, 270, 273
 Smarty, 62, 285, 286, 290,
 297
 instrukcja
 foreach, 308
 include, 304
 SQL, 214
 aktualizacja danych, 219
 filtrowanie danych, 228
 funkcje agregujące, 252
 instrukcja
 AS, 231
 ASC, 230
 BETWEEN, 230
 CREATE TABLE, 217
 DELETE, 219, 242
 DESC, 230
 FROM, 228
 INSERT, 218
 LIKE, 229, 246
 ORDER, 230
 SELECT, 218, 230, 242
 UPDATE, 219, 242
 WHERE, 219, 243

łączenie tabel, 230
 odczytywanie danych, 218
 tworzenie tabel, 216
 typy danych
 MySQL, 215
 SQLite, 215
 wstawianie danych, 218
 SQLite, 27, 214, 226, 227,
 243, 260, 298, 299, 321
 Stała, 66
 definicja, 67
 M_E, 118
 M_PI, 113
 magiczna, 53
 PHP_EOL, 67
 PHP_INT_SIZE, 188

T

Tablice, 143
 wymiar, 150
 Typ logiczny
 false, 105
 true, 105

V

Visual Studio Code, 10, 24, 33
 konfiguracja z PHP, 34
 uruchamianie kodu PHP, 38

W

Wyjątki, 170
 instrukcja throw, 170
 konstrukcja
 try-catch-finally, 170

X

XAMPP, 24, 25, 49, 269, 280,
 293
 konfiguracja serwera
 wirtualnego, 25

Z

Zmienna, 66
 \$_GET, 246
 \$_POST, 246, 315
 \$_SESSION, 331
 globalna, 132
 lokalna, 132
 notacja camelCase, 69
 superglobalna, 132
 środowiskowa PATH, 28
 tablicowa, *Patrz* Tablice